



Apuntes de Cátedra

Licenciatura en Ciencias de la Computación
Licenciatura en Sistemas de Información
Tecnatura en Programación Web

Departamento de Informática

2024



Programación Procedural

La asignatura Programación Procedural pertenece al área “Algoritmos y Lenguajes” y tiene como objetivo introducir los conceptos de la programación procedural, incluir construcciones estándar de programación, estrategias de resolución de problemas y estructuras fundamentales de datos.

En el desarrollo de la misma se promueve que el estudiante interprete conceptos básicos de diseño e implementación de los lenguajes imperativos, y por otro lado que realice prácticas intensivas en un lenguaje imperativo, a través de programas que ofrezcan soluciones óptimas a una amplia gama de problemas.

Teniendo en cuenta que en la materia correlativa de segundo año trabaja el paradigma orientado a objetos, en particular a través del uso de C#, se elige al lenguaje C para la discusión del modelo de computación procedural.

Así, través de lenguaje C se analiza la especificación e implementación de distintos tipos de datos, distintos tipos de operaciones y se desarrollan programas en el marco del diseño modular.

Estos Apuntes de Cátedra son el resultado de muchos años de trabajo de un equipo de cátedra comprometido con su trabajo, un especial agradecimiento a Mg. Adriana Valenzuela y Mg. Myriam Llarena por el invaluable aporte que realizaron a la cátedra Programación Procedural.

Redacción de Unidades teóricas a cargo de:

Dr. Mario Díaz

Lic. Laura Gutierrez

Compaginación:

Dr. Mario Díaz

Índice

PROGRAMACIÓN PROCEDURAL.....	3
PROGRAMA DE EXAMEN 2024.....	8
UNIDAD 1: LENGUAJES PROCEDURALES	10
INTRODUCCIÓN.....	10
LENGUAJE DE PROGRAMACIÓN.....	10
LENGUAJES IMPERATIVOS.....	11
DEFINICIÓN DE UN LENGUAJE DE PROGRAMACIÓN	11
TRADUCTORES Y COMPUTADORAS SIMULADAS POR SOFTWARE.....	12
<i>Simulación de software (interpretación)</i>	12
<i>Traductores</i>	13
<i>Distintos tipos de traductores</i>	14
<i>Traducción e Interpretación</i>	15
IMPLEMENTACIÓN DE LENGUAJES	15
CRITERIOS DE DISEÑO DE LENGUAJES DE PROGRAMACIÓN.....	16
MODELOS DE COMPUTACIÓN	17
PARADIGMA DE PROGRAMACIÓN	17
CLASIFICACIÓN DE LOS PARADIGMAS.....	17
VARIANTES DE LOS PARADIGMAS DE PROGRAMACIÓN	18
<i>Programación imperativa o procedural</i>	18
<i>Programación declarativa</i>	18
LENGUAJE C	19
ETAPAS EN LA OBTENCIÓN DE PROGRAMA EJECUTABLE EN LENGUAJE C	19
BIBLIOGRAFÍA	21
UNIDAD 2: OBJETO DE DATOS	22
INTRODUCCIÓN.....	22
OBJETO DE DATOS	22
VARIABLES Y CONSTANTES	23
TIEMPO DE VIDA.....	25
TIPOS DE DATOS.....	26
ESPECIFICACIÓN E IMPLEMENTACIÓN DE UN TIPO DE DATOS.....	26
TIPOS DE DATOS ELEMENTALES	27
DECLARACIONES	29
VERIFICACIÓN DE TIPOS	30
<i>Verificación de tipos y lenguaje C</i>	31
<i>Conversión de tipos</i>	31
CLASIFICACIÓN	33
TIPOS DE DATOS ENTEROS.....	33
<i>Operación de asignación</i>	34
<i>Valores lvalue y rvalue</i>	34
TIPOS DE DATOS REALES	36
TIPOS DE DATOS BOOLEANOS.....	36
TIPO DE DATOS CARACTER.....	37
TIPO DE DATO PUNTERO (APUNTADOR)	38

TIPOS DE DATOS ESTRUCTURADOS	45
ESPECIFICACIÓN DE TIPO DE DATOS ESTRUCTURADOS	46
REPRESENTACIÓN DE TIPOS DE DATOS ESTRUCTURADOS.....	47
DECLARACIONES Y VERIFICACIONES DE TIPO.....	49
CLASIFICACIÓN DE LOS TIPOS DE DATOS ESTRUCTURADOS	49
ARREGLOS UNIDIMENSIONALES	49
ARREGLOS BIDIMENSIONALES Y MULTIDIMENSIONALES	52
ARREGLOS Y PUNTEROS EN LENGUAJE C	53
CADENAS DE CARACTERES EN C	55
REGISTROS	57
ACCESO A LOS MIEMBROS DE UNA ESTRUCTURA.....	59
OPERACIONES SOBRE ESTRUCTURAS	61
PUNTEROS A ESTRUCTURAS.....	64
ACCESO A LOS COMPONENTES DE UNA VARIABLE STRUCT	64
BIBLIOGRAFÍA.....	66
UNIDAD 3: FUNCIONES.....	67
INTRODUCCIÓN	67
DEFINICIÓN DE TIPOS.....	67
LENGUAJE C Y TYPEDEF	69
SISTEMA DE TIPOS.....	70
VENTAJAS DE LA DEFINICIÓN DE TIPOS	70
SUBPROGRAMAS.....	70
ESPECIFICACIÓN DE UN SUBPROGRAMA	71
IMPLEMENTACIÓN DE UN SUBPROGRAMA	72
DEFINICIÓN, INVOCACIÓN Y ACTIVACIÓN DE SUBPROGRAMAS	73
FUNCIONES EN LENGUAJE C	75
DEFINICIÓN	75
INVOCACIÓN O LLAMADA A UNA FUNCIÓN	78
DECLARACIÓN DE UNA FUNCIÓN O PROTOTIPO DE FUNCIÓN	80
ORGANIZACIÓN DE LA MEMORIA EN C.....	81
EJECUCIÓN DE UN PROGRAMA EN C.....	82
TIPOS DE ALMACENAMIENTO	84
VARIABLES AUTOMÁTICAS	84
VARIABLES EXTERNAS	85
VARIABLES ESTÁTICAS.....	86
DECLARACIONES, BLOQUES Y ALCANCE	88
DECLARACIONES – SU IMPORTANCIA DURANTE LA TRADUCCIÓN.....	88
BLOQUES EN LENGUAJE C.	88
ALCANCE DE UN VÍNCULO EN LENGUAJE C:.....	88
APERTURA EN EL ALCANCE:	89
VISIBILIDAD DE UNA DECLARACIÓN:	89
PASAJE DE PARÁMETROS A UNA FUNCIÓN.....	90
PASAJE POR VALOR.....	90
PASAJE DE DIRECCIONES	92
PASAJE POR REFERENCIAS	93
FUNCIONES QUE DEVUELVEN MÁS DE UN RESULTADO.....	94
ARREGLOS COMO PARÁMETROS DE FUNCIONES	95
PASAJE CONSTANTE EN ARREGLOS	98

TRADUCCIÓN Y TABLA DE SÍMBOLOS EN LENGUAJE C	99
OPERACIONES CON LA TS	99
MANEJO DE LA TS EN PILA	100
BIBLIOGRAFÍA:.....	101
UNIDAD 4: RECURSIVIDAD.....	102
INTRODUCCIÓN.....	102
FUNCIONES RECURSIVAS.....	103
FUNCIONES RECURSIVAS QUE DEVUELVEN MÁS DE UN RESULTADO	109
RECURSIVIDAD USANDO ARREGLOS.....	110
RECURSIÓN VERSUS ITERACIÓN	113
UNIDAD 5: ESTRUCTURAS DINÁMICAS.....	114
INTRODUCCIÓN.....	114
EL MONTÍCULO O MONTÓN(HEAP)	114
MANEJO DEL MONTÍCULO EN LENGUAJE C	115
LENGUAJE C: VARIABLES DINÁMICAS SIMPLES	117
LENGUAJE C: ARREGLOS DINÁMICOS	119
ARREGLOS DINÁMICOS USADOS COMO CADENA DE CARACTERES	123
ARREGLOS DE CADENAS DINÁMICAS	124
LISTAS.....	127
IMPLEMENTACIÓN DE LISTAS	127
IMPLEMENTACIÓN DE LISTAS MEDIANTE ARREGLOS.....	127
IMPLEMENTACIÓN DE LISTAS MEDIANTE PUNTEROS	128
DEFINICIÓN Y DECLARACIÓN DE LAS DISTINTAS IMPLEMENTACIONES	129
MANIPULACIÓN DE LISTAS ENLAZADAS	130
<i>Creación</i>	<i>130</i>
<i>Inserción en una lista</i>	<i>130</i>
<i>Gestión de almacenamiento</i>	<i>132</i>
<i>Recorrido de la lista</i>	<i>134</i>
<i>Búsqueda de una componente de una lista</i>	<i>134</i>
<i>Modificación de una componente de la lista</i>	<i>135</i>
<i>Inserción de un nodo en cualquier lugar de la lista</i>	<i>137</i>
<i>Supresión de un elemento en una lista</i>	<i>139</i>
<i>Ordenamiento de una lista</i>	<i>141</i>
<i>Eliminación de todas las componentes de una lista</i>	<i>141</i>
LISTAS ENLAZADAS QUE ALMACENAN DATOS ESTRUCTURADOS	142
PROBLEMAS DE GESTIÓN DE ALMACENAMIENTO	143
<i>Referencias Desactivadas o Bamboleantes</i>	<i>143</i>
<i>Basura</i>	<i>143</i>
MANIPULACIÓN DE LISTAS CON FUNCIONES RECURSIVIDAD	144
ARREGLO DE LISTAS	145
BIBLIOGRAFÍA	147
UNIDAD 6: ARCHIVOS	148
INTRODUCCIÓN.....	148
DEFINICIÓN.....	148
ORGANIZACIÓN DE ARCHIVOS.....	148
CLASIFICACIÓN DE ARCHIVOS.....	149
VENTAJAS Y DESVENTAJAS DEL USO DE ARCHIVOS.....	150

DECLARACIÓN DE UN ARCHIVO EN EL LENGUAJE C	150
OPERACIONES BÁSICAS EN EL MANEJO DE ARCHIVOS	151
ARCHIVOS ORGANIZADOS SECUENCIALMENTE	153
ARCHIVOS DE CARACTERES.....	153
ARCHIVOS DE TEXTOS	155
ARCHIVOS SIN FORMATO	159
ARCHIVOS ORGANIZADOS SECUENCIALMENTE CON ACCESO DIRECTO.....	166
ELIMINACIÓN DE INFORMACIÓN DEL ARCHIVO.....	171
CÁLCULO DE LA LONGITUD DE UN ARCHIVO.....	173
BIBLIOGRAFÍA.....	173
ANEXO I.....	174
CONCEPTOS DE DISEÑO Y PROGRAMACIÓN RELEVANTES PARA UNA PRÁCTICA EFICIENTE	174
<i>La visión clásica de la programación estructurada: la programación sin goto</i>	<i>174</i>
<i>La visión moderna de la programación estructurada: La Segmentación</i>	<i>175</i>
PROGRAMACIÓN ESTRUCTURADA	176
CONCEPTOS DE LA PROGRAMACIÓN MODULAR	176
<i>Descomposición Modular</i>	<i>177</i>
PROGRAMACIÓN MONOLÍTICA.....	179
APLICACIÓN DE CONCEPTOS	180

Programa de Examen 2024

Unidad 1

Introducción. Lenguaje de programación. Algunas cuestiones de diseño e implementación: computadora distintos tipos. Traductores y computadoras simuladas por software. Modelos de computación. Lenguajes imperativos: Definición de un lenguaje de programación. Concepto de estado de máquina. Implementación de un lenguaje de programación. Eficiencia y Regularidad. Ejemplificación en lenguaje C.

Unidad 2

Objetos de Datos. Variables y Constantes. Tiempo de Vida. Enlace (ligadura) y Tiempo de enlace. Tipos de Datos. Especificación e Implementación.

a) Tipos de Datos Elementales. Especificación e Implementación. Representación de almacenamiento. Implementación de algoritmos y procedimientos que definen las operaciones. Declaraciones y Verificación de tipo. Verificación de tipos en lenguaje C. Conversión y coerción de tipos. Tipo de datos Entero. Semántica de la operación de Asignación: Valor l y Valor r. Números Reales de punto flotante. Enumeraciones. Tipos Booleanos. Tipos de datos caracteres.

Tipo Apuntador. Punteros a variables simples en lenguaje C. Operadores de apuntadores. Inicialización. Asignación de punteros. Ejercicios de Aplicación.

b) Tipos De Datos Estructurados: Introducción. Especificación de Tipo de Datos Estructurados. Implementación de Tipos de Datos Estructurados. Vectores. Implementación de Operaciones sobre Estructuras de Datos. Declaraciones y Verificaciones de Tipo. Arreglos Bidimensionales y Multidimensionales. Arreglos y Punteros en Lenguaje C.

Cadenas De Caracteres. Distintas formas. Cadenas de caracteres en C. Uso De Funciones de Cadena de la Biblioteca Estándar.

Registros: Especificación e Implementación. Operación de Selección de Componentes. Manejo de Struct en Lenguaje C. Punteros a Struct.

Registros Variantes: Implementación. Union y Struct en Lenguaje C. Ejercicios de Aplicación.

Unidad 3

Encapsulamiento a través de subprogramas. Subprogramas: Especificación e implementación de subprogramas. Definición, invocación y activación de subprogramas.

Concepto de función en C: Función main(). Declaración y definición de funciones. Organización de memoria: Almacenamiento Estático, Pila y Montículo. Ejecución de un programa en C.

Pasajes de parámetros: Pasajes de parámetros: valor, constante, por dirección. Variables referenciadas. Pasaje de parámetro por referencia. Arreglos como parámetros. Funciones que devuelven más de un valor. Ejercicios de aplicación.

Traducción y Tabla de Símbolos (TS): Compilación de un programa fuente. Lenguaje C como un lenguaje estructurado en bloque. Importancia de las declaraciones en C. Alcance de un vínculo en lenguaje C. Generación de la TS en un programa en C. Ejercicios de aplicación.

Clasificación de las variables según alcance y tiempo de vida: Variables automáticas, estáticas y externas. Ejercicios de aplicación.

Unidad 4

Recursividad. Funciones Recursivas en C. Definición. Caso base y caso general. Distintas combinaciones. Manejo de memoria (pila). Aplicaciones. Recursión e Iteración. Ejercicios de aplicación.

Unidad 5

Estructuras dinámicas. Almacenamiento estático y dinámico. Variables dinámicas. El montículo: Operaciones sobre el montículo o heap. Manejo del montículo en lenguaje C: funciones malloc y free. Variables dinámicas simples. Basura y referencias desactivadas. Arreglos dinámicos en lenguaje C. Arreglos dinámicos uni y bi-dimensionales. Mapa de memoria. Cadena de caracteres dinámicas. Arreglo de cadenas dinámicas. Ejercicios de Aplicación.

Listas: listas secuenciales y listas enlazadas. Manipulación de listas enlazadas: creación, inserción. Búsqueda, recorrido, modificación, supresión de elementos. Ordenamiento y eliminación de una lista. Gestión de almacenamiento. Problemas de gestión de almacenamiento: referencias bamboleantes y basura. como cola. Recursividad en listas. Ejercicios de Aplicación.

Unidad 6

Archivos. Concepto de archivo en C. Distintas clasificaciones. Archivos secuenciales: Archivos de caracteres y sin formato. Acceso secuencial de archivos: Funciones para la creación y utilización de archivos secuenciales. Acceso directo de archivos secuenciales. Funciones para el uso de archivos. Ejercicios de aplicación.

Dr. Mario Diaz

Prof. Titular Programacion Procedural

Unidad 1: Lenguajes Procedurales

Introducción

Para 1969 ya se habían contabilizado más de 120 lenguajes de programación, hoy en día hay quienes afirman que la lista supera los 2000. Wikipedia nombra y describe alrededor de 671¹.

Si bien un programador profesional usa efectivamente no más de tres lenguajes, en función de las posibilidades y las tendencias de la empresa o medio en que está inserto, es importante que se explore más profundamente en el diseño de los lenguajes y en su efecto sobre la implementación, por muchas razones, entre ellas:

- Mejorar la habilidad en el desarrollo de algoritmos eficaces.
- Mejorar el uso de lenguaje de programación que el profesional utiliza.
- Hacer posible una mejor elección del lenguaje de programación, cuando del profesional dependa.
- Facilitar el estudio de nuevos lenguajes
- Permitir el diseño de nuevos lenguajes.

Concepto de Computación

Desde un punto de vista amplio, que incluya la definición, diseño e implementación de lenguajes de programación, el **concepto de computación** se puede interpretar como "*cualquier proceso que puede ser realizado por una computadora*". No solo cálculos matemáticos sino también manipulación de datos, procesamiento de textos, almacenamiento y recuperación de la información.

Lenguaje de Programación

Para que exista comunicación, debe existir una comprensión mutua de cierto conjunto de símbolos y reglas del lenguaje. En un lenguaje natural, el significado de los símbolos se establece por la costumbre y se aprende mediante la experiencia.

Los lenguajes de programación tienen como objetivo la construcción de programas, normalmente escritos por personas.

Estos programas se ejecutarán sobre una computadora que realizará las tareas descritas.

La utilización de un lenguaje de programación requiere, por tanto, una comprensión mutua por parte de personas y máquinas.

*Un lenguaje de programación es un sistema notacional para describir computaciones en una forma legible tanto para la máquina como para el ser humano.*²

Algunas cuestiones de Diseño

En el diseño de lenguajes, la meta es lograr la potencia, expresividad y comprensión que requiere la legibilidad del ser humano, mientras se conservan la precisión y simplicidad necesarias para la traducción de máquina.

La *legibilidad* por parte del ser humano es un requisito complejo y sutil. Depende en gran parte de las posibilidades de abstracción que tiene un lenguaje de programación.

La *abstracción* permite concentrarse en un problema al mismo nivel de generalización, dejando de lado los detalles irrelevantes. La abstracción permite trabajar con conceptos y términos familiares al entorno del problema, sin tener que transformarlos a una estructura no familiar³.

Abstracciones procedimentales

Es una secuencia nombrada de *instrucciones* que tienen una función específica y limitada. (Ejemplo de esto es el uso de funciones en lenguaje C).

Abstracciones de datos

¹ Enlace Web https://es.wikipedia.org/wiki/Anexo:Lenguajes_de_programación - 01/08/2017.

² Louden, Kenneth C. (2004) Lenguajes de Programación Principios y Práctica. Editorial Thomson. México, D.F. Pág 3.

³ Ibidem. Pág 26.

Técnica de inventar *nuevos tipos de datos* que sean más adecuados a una aplicación y, por consiguiente, facilitar la escritura del programa. (Ejemplo: uso de struct en lenguaje C).

Abstracciones de control

Resume propiedades de la transferencia de control, es decir, la modificación de la trayectoria de ejecución de un programa en una determinada situación. (Ejemplo: for, if, while, etc son ejemplo de abstracciones de control en lenguaje C).

El principal objetivo de la abstracción en el diseño de lenguajes de programación *es el control de la complejidad*. Se controla la complejidad elaborando abstracciones que esconden los detalles cuando es apropiado. En general todos los mecanismos de abstracción se diseñan para que los seres humanos puedan entenderlos.

Lenguajes Imperativos

Un lenguaje procedural o imperativo es un lenguaje controlado por mandatos o instrucciones.

El proceso de ejecución de programas procedurales por la computadora, tiene lugar a través de una serie de estados, cada uno definido por el contenido de la memoria, los registros internos y los almacenes externos durante la ejecución.

El almacenamiento inicial de estas áreas define el **estado inicial** de la computadora. Cada paso en la ejecución transforma este estado en otro nuevo, a través de la modificación de alguna de estas áreas. Esta transformación se denomina **transición de estado**. Cuando termina la ejecución del programa, el **estado final** viene dado por el contenido final de las áreas antes mencionadas. Por lo tanto, la ejecución de un programa puede verse como una serie de transiciones de estados que realiza la computadora.

En este modelo de computación, un programa es un conjunto de enunciados y la ejecución de cada enunciado hace que cambie el valor de una o más localidad de almacenamiento en memoria, es decir que la memoria pase a un nuevo estado.

Si se considera a la memoria como un conjunto de cajas, la construcción de un programa consiste en construir los estados de máquina sucesivos (cánicas en cajas o valores en variables) que se necesitan para llegar a la solución.

Este tipo de modelo de computación, tiene características propias del modelo de Von Newmann, variables que representan valores de memoria y asignación que permite que el programa opere sobre dichos valores.

Existen otros paradigmas que no son tan dependientes del modelo de computadora de Von Newmann, y serán estudiados en cursos superiores.

El lenguaje C, lenguaje que se usará en este curso responde al paradigma imperativo.

Definición de un lenguaje de programación

Un lenguaje natural es un instrumento de comunicación utilizado de forma común entre personas. Para que exista comunicación, debe existir una comprensión mutua de cierto conjunto de símbolos y reglas del lenguaje.

Los lenguajes de programación tienen como objetivo la construcción de programas, normalmente escritos por personas. Estos programas se ejecutarán por una computadora que realizará las tareas descritas. El programa debe ser comprendido tanto por personas como por la computadora. La utilización de un lenguaje de programación requiere, por tanto, una comprensión mutua por parte de personas y máquinas. Este objetivo es difícil de alcanzar debido a la naturaleza diferente de ambos.

En un lenguaje natural, el significado de los símbolos se establece por la costumbre y se aprende mediante la experiencia. Sin embargo, los lenguajes de programación se definen habitualmente por una autoridad, que puede ser el diseñador individual del lenguaje o un determinado comité.



Para que el computador pueda comprender un lenguaje humano, es necesario diseñar métodos que traduzcan tanto la estructura de las frases como su significado a código máquina. Los diseñadores de lenguajes de programación construyen lenguajes que saben cómo traducir o que creen que serán capaces de traducir. Si las computadoras fuesen la única audiencia de los programas, estos se escribirían directamente en código máquina o en lenguajes mucho más mecánicos.

Sin embargo, el programador debe ser capaz de leer y comprender el programa que está construyendo y las personas no son capaces de procesar información con el mismo nivel de detalle que las máquinas.

Los lenguajes de programación son, por tanto, una solución de compromiso entre las necesidades del emisor (programador - persona) y del receptor (computadora - máquina).

Las declaraciones, tipos, nombres simbólicos, etc. son concesiones de los diseñadores de lenguajes para que los humanos podamos entender mejor lo que se ha escrito en un programa. Por otro lado, la utilización de un vocabulario limitado y de unas reglas estrictas son concesiones para facilitar el proceso de traducción.

La definición de un lenguaje de programación necesita una descripción precisa y completa, además de un traductor que permita aceptar el programa en el lenguaje en cuestión y que, lo ejecute directamente o bien lo transforme en una forma adecuada para su ejecución.

Dos partes se pueden distinguir en la definición de un lenguaje. La estructura o **sintaxis** y la **semántica** o significado.

La **sintaxis** estudia las reglas de formación de frases. Las reglas de sintaxis nos dicen cómo se escriben los enunciados, declaraciones y otras construcciones del lenguaje.

La **semántica**, sin embargo, hace referencia al significado de esas construcciones.

Traductores y computadoras simuladas por software⁴

La programación se hace más a menudo en un lenguaje de alto nivel “muy alejado” del lenguaje de máquina, (lenguaje muy cercano al del hardware).

Para que un lenguaje de programación sea útil debe tener un traductor, es decir un programa que acepte el programa que escribe el programador - en lenguaje de alto nivel - y que lo ejecute directamente o lo transforme en una forma adecuada para su ejecución.

Existen por tanto dos soluciones básicas para estas cuestiones: traducción e interpretación. Un traductor que produce otro programa equivalente pero adecuado para su ejecución se llama **compilador**, y un traductor que ejecuta el programa directamente se llama **intérprete**.

Simulación de software (interpretación)

En lugar de traducir los programas de alto nivel, a programas equivalentes en lenguaje máquina, se podrían simular a través de programas ejecutados en otra computadora anfitrión, una computadora cuyo lenguaje máquina sea de alto nivel. Esto se construye con software que se ejecuta en la computadora anfitrión, la computadora con lenguaje de alto nivel que de otra manera se podría haber construido en hardware. A esto se le conoce como una simulación por software (o interpretación) de la computadora con lenguaje de alto nivel en la computadora anfitrión.

Un **intérprete** es un software que recibe un programa en lenguaje de alto nivel, lo analiza y lo ejecuta. Para analizar el programa completo, va traduciendo sentencias de código y ejecutándolas si están bien, así hasta completar el programa origen.

⁴ Cueva Lovelle, Juan (1998) Conceptos Básicos de Procesadores de Lenguajes. Ed. SERVITEC. España. Capítulo 2. Pág. 8.

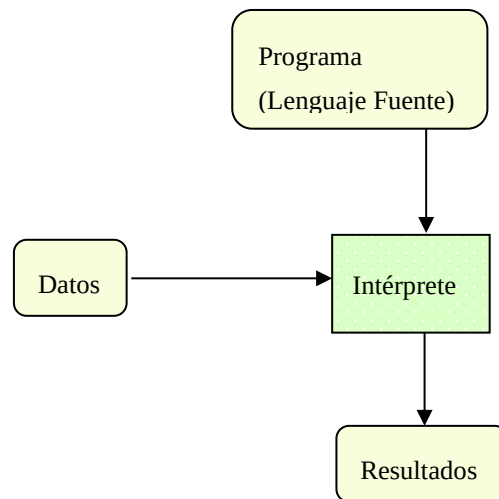


Fig. 3: Esquema general de un intérprete

Traductores

Por **Traductor** se denota a cualquier “procesador de lenguajes”⁵ que acepta programas en cierto lenguaje fuente (que puede ser de alto o bajo nivel) como entrada y produce programas funcionalmente equivalentes en otro lenguaje objeto.

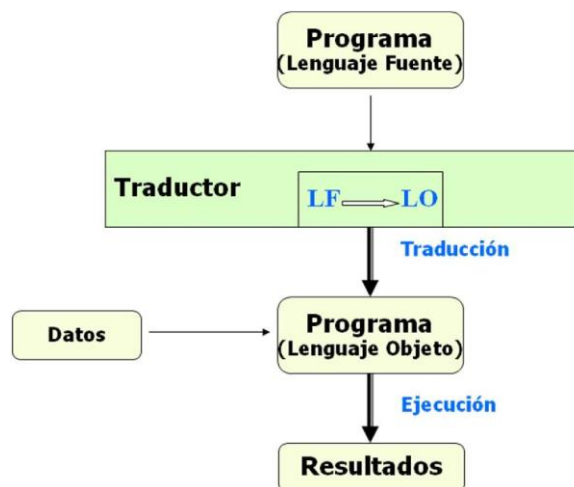


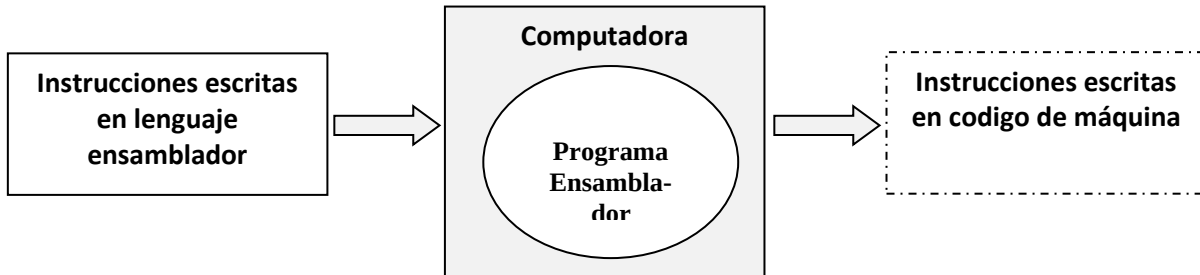
Fig. 1: Esquema general de un traductor⁶

⁵ Procesador de lenguaje es un nombre genérico que se aplica a traductores, compiladores, intérpretes, y otros programas que realizan operaciones con los lenguajes.

⁶ Pratt Terrence y Marvin Zelkowitz (1987) Lenguajes de Programación Diseño e Implementación. Editorial Prentice Hall. Hispano americana, S.A. 2° Edición. Capítulo 2. Pág. 41.

*Distintos tipos de traductores***Ensamblador**

El término ensamblador (del inglés assembler) se refiere a un tipo de programa que se encarga de traducir un archivo fuente escrito en un lenguaje ensamblador, a un fichero objeto que contiene código máquina, ejecutable directamente por el microprocesador.

**Compilador**

Para traducir las instrucciones de un programa escrito en un lenguaje de alto nivel a instrucciones de un lenguaje máquina, hay que utilizar un programa llamado **compilador**. Así pues, el compilador es un programa que recibe como datos de entrada el código fuente de un programa escrito por un programador, y genera como salida un conjunto de instrucciones escritas en el lenguaje binario de la computadora donde se van a ejecutar.

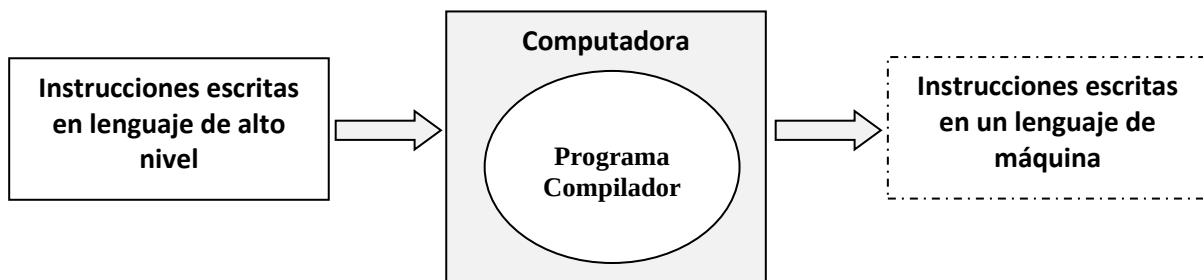
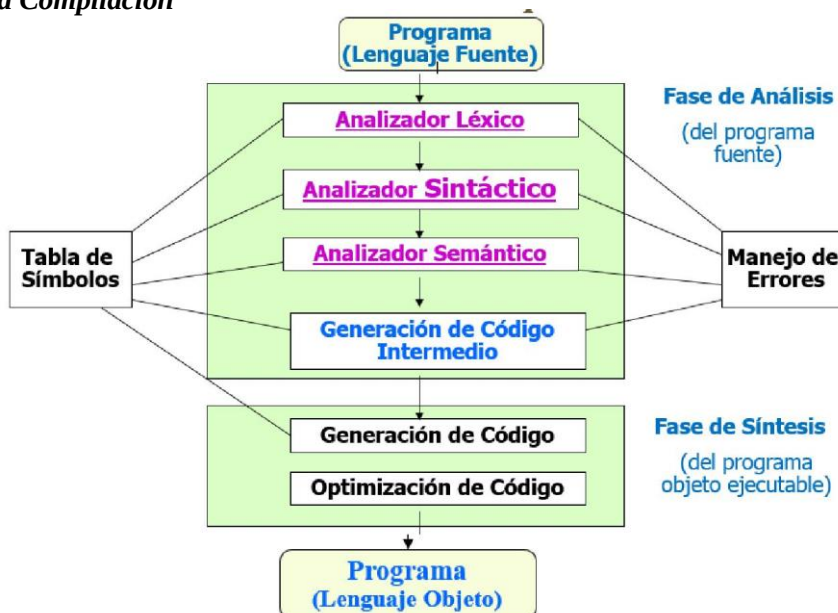
*Fases de la Compilación*

Fig. 2: Esquema general de un compilador⁷

⁷ Ibidem. Pratt. - Capítulo 3. Pág. 72.

Analizador léxico (o revisor) es un programa que lee la secuencia de caracteres del programa fuente, de izquierda a derecha y agrupa esas secuencias de caracteres en unidades con significado propio (componentes léxicos o “tokens” en inglés). Es decir que identifica el tipo de cada elemento léxico: identificador, número, operador, delimitador y adjunta una marca de tipo, para enviárselo al analizador sintáctico.

Analizador sintáctico o parsing identifica estructuras más grandes del programa como enunciados, declaraciones, llamadas a funciones, expresiones, etc., a partir de los elementos léxicos o tokens producidos por el analizador léxico. Analiza las relaciones estructurales entre los componentes léxicos, esto es semejante a realizar el análisis gramatical sobre una frase en lenguaje natural

Analizador semántico: programa que detecta la validez semántica de las sentencias aceptadas por el analizador sintáctico. Las tareas básicas a realizar son la verificación de tipos en asignaciones y expresiones, la declaración del tipo de variables y funciones antes de su uso, el correcto uso de operadores, el ámbito de las variables y la correcta llamada a funciones.

Se procesan las estructuras sintácticas reconocidas por el analizador sintáctico y comienza a tomar forma la estructura del código ejecutable

En este curso se observará al análisis semántico estático (en tiempo de compilación), utilizando la Tabla de Símbolos.

Traducción e Interpretación

Los traductores y los interpretes proporcionan ventajas diferentes, por ejemplo una ventaja de los traductores es que los códigos referentes a ciclos que se repiten gran cantidad de veces y son ejecutados muchas veces se traducen sólo una vez a diferencia de los interpretes que tienen que decodificarlo la misma cantidad de veces que se ejecute.

Una ventaja para la interpretación es que en caso de un error proporcionan más información acerca de donde fue la falla que en un código objeto traducido.

Por lo tanto, si el programa de entrada está en un lenguaje de alto nivel, el traductor procesará los enunciados del programa en el orden físico de entrada, una vez cada uno. El intérprete podrá procesar los enunciados siguiendo el orden lógico de control y algunos enunciados podrá procesarlos más de una vez y otros omitirlos (según que sean parte de un bucle o que el control no los alcance).

No siempre se dan por separado, la traducción y la interpretación. Generalmente se mezclan estos dos mecanismos; según cuál de ellos prevalezca se habla de **lenguajes compilados** cuando prevalece la traducción (ejemplo: C, C++, Pascal, Fortran, Ada, Cobol) o **interpretados** cuando predomina la simulación (ejemplo: LISP, PROLOG, SmallTalk, ML).

En general cuando hay enunciados repetitivos es mejor la traducción pues se decodifica una vez, aunque se ejecuten muchas veces. La desventaja es la pérdida de información acerca del programa.

Actividad 1: Investigue la bibliografía e indique claramente las ventajas y desventajas de la traducción y la simulación.

Implementación de lenguajes

Cuando se implementa un lenguaje de programación, las estructuras de datos y algoritmos que se utilizan en tiempo de ejecución de un programa escrito en ese lenguaje, definen un **modelo de ejecución** definido por la implementación del lenguaje.

Por ejemplo, para la implementación de las operaciones de suma de enteros y raíz cuadrada, el implementador puede optar por representar la suma de enteros a partir de la suma de enteros proporcionada directamente por el hardware, y para la operación de raíz cuadrada puede optar por una simulación por software.

De ahí que es importante entender que la implementación de un lenguaje está determinada por múltiples decisiones que debe tomar el implementador en función de los recursos de hardware y software disponibles en la computadora subyacente y los costos de su uso.

Criterios de diseño de lenguajes de programación

Al diseñar lenguajes de programación, a menudo es necesario tomar decisiones sobre las características que se incluyen de forma permanente, las características que no se incluyen (aunque existen mecanismos que facilitan su inclusión) y las que no se permiten. Estas decisiones pueden afectar al diseño final del lenguaje y posiblemente entrar en conflicto con otros aspectos del lenguaje.

A continuación se presenta una síntesis de criterios de diseño, extraídos de autores como Pratt y Loudon, Kenneth.

Existen dos principios importantes a tener en cuenta: la *eficiencia* y la *regularidad*.

Eficiencia

La **eficiencia** debe evaluarse en distintas dimensiones (código, traducción, etc.):

- **Eficiencia de código:** Hace referencia a un diseño del lenguaje que permite que el traductor genere un código ejecutable eficiente. Por ejemplo el uso de declaraciones anticipadas de las variables.
- **Eficiencia de traducción:** Hace referencia a un diseño del lenguaje que permite que el código fuente se traduzca con rapidez y con un traductor de tamaño razonable. Ejemplo: C standard permite un compilador de una sola pasada pues las variables se declaran antes de usarse. C++, el compilador debe realizar una segunda pasada para resolver referencias del identificador.
- **Eficiencia de implementación:** Hace referencia a la eficiencia con la que se puede escribir un traductor. Esto está relacionado con la eficiencia de la traducción y con la complejidad de la definición del lenguaje.
- **Eficiencia de la programación:** Hace referencia a la facilidad para escribir programas en el lenguaje. Entre otras, esto involucra la capacidad de expresión del lenguaje y la capacidad de darle mantenimiento al programa.

Regularidad

La **regularidad** es una cualidad que implica que hayan pocas restricciones en el uso de sus constructores particulares, pocas interacciones raras entre dichos constructores, y menos sorpresas en la forma en la que se comportan las características del lenguaje.

Distintos principios básicos miden la regularidad, entre otros la *generalidad*, la *ortogonalidad* y la *uniformidad*.

- **Generalidad:** Un lenguaje alcanza generalidad evitando casos especiales en cuanto a disponibilidad o uso de constructores.

La no generalidad está ligada a restricciones de los constructores con independencia del contexto en que estén.

En lenguaje C el operador de igualdad == carece de generalidad. Dos arreglos no pueden compararse a través de este operador, por ejemplo.

En cuanto a la longitud de los arreglos, lenguaje C admite generalidad pues maneja arreglos de longitud variable, mientras que esta generalidad no está presente en lenguaje Pascal, que admite sólo arreglos de longitud fija.

- **Ortogonalidad:** Se refiere al atributo de combinar varias características de un lenguaje de manera que la combinación tenga significado.
Ejemplo: Si un lenguaje provee de expresiones que pueden devolver un valor y también posee un enunciado condicional que compara valores; estas dos características son ortogonales si se puede usar dentro del enunciado condicional cualquier expresión.
- **Uniformidad:** Hace referencia a la consistencia de la apariencia y comportamiento de los constructores del lenguaje.

Las no-uniformidades son de dos tipos:

- a) dos cosas similares que no parecen serlo
- b) cosas no similares que parecen serlo o se comportan similarmente cuando no debieran.

Ejemplo de falta de uniformidad en lenguaje C los operadores & (and bit a bit) y && (and lógico) tienen una apariencia confusamente similar y producen resultados muy distintos.

Modelos de computación⁸

Entendemos por una computación a cualquier proceso que puede ser realizado por una computadora, con esto nos estamos refiriendo no solo a cálculos matemáticos sino que incluimos la manipulación de datos, el procesamiento de textos y el almacenamiento y recuperación de la información.

Estos modelos de computación dan lugar a distintas familias de lenguajes o paradigmas. En informática, es posible observar varias comunidades, cada una hablando su propio lenguaje y utilizando sus propios paradigmas. De hecho los lenguajes de programación suelen fomentar el uso de ciertos paradigmas y disuadir el uso de otros.

Paradigma de Programación⁹

Un paradigma de programación *"consiste en un método para llevar a cabo cómputos y la forma en la que deben estructurarse y organizarse las tareas que debe realizar un programa"*.

Se trata de una propuesta tecnológica adoptada por una comunidad de programadores, y desarrolladores cuyo núcleo central es incuestionable en cuanto que únicamente trata de resolver uno o varios problemas claramente delimitados; la resolución de estos problemas debe suponer consecuentemente un avance significativo en al menos un parámetro que afecte a la ingeniería de software. Representa un enfoque particular o filosofía para diseñar soluciones.

Los paradigmas difieren unos de otros, en los conceptos y la forma de abstraer los elementos involucrados en un problema, así como en los pasos que integran su solución del problema, en otras palabras, el cómputo.

Tiene una estrecha relación con la formalización de determinados lenguajes en su momento de definición. Es un estilo de programación empleado.

Un paradigma de programación está delimitado en el tiempo en cuanto a aceptación y uso, porque nuevos paradigmas aportan nuevas o mejores soluciones que lo sustituyen parcial o totalmente.

Clasificación de los Paradigmas

En general, la mayoría de paradigmas son variantes de los dos tipos principales de programación:

- Imperativa
- Declarativa.

En la **programación imperativa** se describe paso a paso un conjunto de instrucciones que deben ejecutarse para variar el estado del programa y hallar la solución, es decir, un algoritmo en el que se describen los pasos necesarios para solucionar el problema.

En la **programación declarativa** las sentencias que se utilizan lo que hacen es describir el problema que se quiere solucionar; se programa diciendo lo que se quiere resolver a nivel de usuario, pero no las instrucciones necesarias para solucionarlo. Esto último se realizará mediante mecanismos internos de inferencia de información a partir de la descripción realizada.



⁸ Ibidem. Louden. Capítulo 1. "Paradigmas de computación". Pág. 12 a 18.

⁹ https://es.wikipedia.org/wiki/Lenguaje_de_programaci%C3%B3n#Paradigma_de_programaci%C3%B3n

Variantes de los Paradigmas de Programación

Programación imperativa o procedural

Es el más usado en general, se basa en dar instrucciones al ordenador de cómo hacer las cosas en forma de algoritmos, en lugar de describir el problema o la solución. Las recetas de cocina y las listas de revisión de procesos, a pesar de no ser programas de computadora, son también conceptos familiares similares en estilo a la programación imperativa; donde cada paso es una instrucción. Es la forma de programación más usada y la más antigua, el ejemplo principal es el lenguaje de máquina. Ejemplos de lenguajes puros de este paradigma serían el C, BASIC o Pascal.



- **Programación orientada a objetos:** está basada en el imperativo, pero encapsula elementos denominados objetos que incluyen tanto variables como funciones. Está representado por C++, C#, Java o Python entre otros, pero el más representativo sería el Smalltalk que está completamente orientado a objetos.
- **Programación dinámica:** está definida como el proceso de romper problemas en partes pequeñas para analizarlos y resolverlos de forma lo más cercana al óptimo, busca resolver problemas en $O(n)$ sin usar por tanto métodos recursivos. Este paradigma está más basado en el modo de realizar los algoritmos, por lo que se puede usar con cualquier lenguaje imperativo.
- **Programación orientada a eventos:** la programación dirigida por eventos es un paradigma de programación en el que tanto la estructura como la ejecución de los programas van determinados por los sucesos que ocurran en el sistema, definidos por el usuario o que ellos mismos provoquen.

Programación declarativa

está basada en describir el problema declarando propiedades y reglas que deben cumplirse, en lugar de instrucciones. Hay lenguajes para la programación funcional, la programación lógica, o la combinación lógico-funcional. La solución es obtenida mediante mecanismos internos de control, sin especificar exactamente cómo encontrarla (tan solo se le indica a la computadora qué es lo que se desea obtener o qué es lo que se está buscando). No existen asignaciones destructivas, y las variables son utilizadas con transparencia referencial. Los lenguajes declarativos tienen la ventaja de ser razonados matemáticamente, lo que permite el uso de mecanismos matemáticos para optimizar el rendimiento de los programas. Unos de los primeros lenguajes funcionales fueron Lisp y Prolog.

- **Programación funcional:** basada en la definición los predicados y es de corte más matemático, está representado por Scheme (una variante de Lisp) o Haskell. Python también representa este paradigma.
- **Programación lógica:** basado en la definición de relaciones lógicas, está representado por Prolog.
- **Programación con restricciones:** similar a la lógica usando ecuaciones. Casi todos los lenguajes son variantes del Prolog.
- **Programación multiparadigma:** es el uso de dos o más paradigmas dentro de un programa. El lenguaje Lisp se considera multiparadigma. Al igual que Python, que es orientado a objetos, reflexivo, imperativo y funcional.⁴ Según lo describe Bjarne Stroustrup, esos lenguajes permiten crear programas usando más de un estilo de programación. El objetivo en el diseño de estos lenguajes es permitir a los programadores utilizar el mejor paradigma para cada trabajo, admitiendo que ninguno resuelve todos los problemas de la forma más fácil y eficiente posible. Por ejemplo, lenguajes de programación como C++, Genie, Delphi, Visual Basic, PHP o D5 combinan el paradigma imperativo con la orientación a objetos. Incluso existen lenguajes multiparadigma que permiten la mezcla de forma natural, como en el caso de Oz, que tiene subconjuntos (particularidad de los lenguajes lógicos), y otras características propias de lenguajes de programación funcional y de orientación a objetos. Otro ejemplo son los lenguajes como Scheme de paradigma funcional o Prolog (paradigma lógico), que cuentan con estructuras repetitivas, propias del paradigma imperativo.

- **Programación reactiva:** Este paradigma se basa en la declaración de una serie de objetos emisores de eventos asíncronos y otra serie de objetos que se "suscriben" a los primeros (es decir, quedan a la escucha de la emisión de eventos de estos) y *reaccionan* a los valores que reciben. Es muy común usar la librería Rx de Microsoft (Acrónimo de Reactive Extensions), disponible para múltiples lenguajes de programación.
- **Lenguaje específico del dominio o DSL:** se denomina así a los lenguajes desarrollados para resolver un problema específico, pudiendo entrar dentro de cualquier grupo anterior. El más representativo sería SQL para el manejo de las bases de datos, de tipo declarativo, pero los hay imperativos, como el Logo.

Lenguaje C

Para hablar del lenguaje C, es bueno recordar que los primeros lenguajes de programación se denominaban lenguajes de bajo nivel o lenguajes de máquina (ceros y unos). El uso de éstos implicaba un conocimiento exhaustivo del computador y un manejo preciso de las direcciones de memoria asociadas a las distintas operaciones y operadores. Para resolver los problemas que surgían de esta tediosa forma de programar, surgieron los lenguajes de programación ensambladores que usaban símbolos para expresar las distintas operaciones y las direcciones de memoria donde se almacenaban los valores de las variables.

Finalmente, surgen los lenguajes de alto nivel, que distan mucho del lenguaje de máquina y se asemejan más al lenguaje que utilizan las personas para comunicarse (el inglés).

Debido al notable abaratamiento de las computadoras alrededor de los años 70, fue evidente el progreso de la tecnología de compiladores, dando lugar al surgimiento de muchos lenguajes líderes sobre distintos dominios: comercial, científico, militar etc. Si bien muchos de ellos ya prácticamente no se usan, y muchos más se han desarrollado, en la actualidad el lenguaje C tiene la característica de estar aún vigente como consecuencia de las continuas revisiones que en él se han realizado y de su capacidad de reflejar los cambios que se produjeron en otras áreas de la computación.

El lenguaje C fue desarrollado en 1972 por Dennis Ritchie y Ken Thompson en los laboratorios Bell Telephone de AT&T. El lenguaje C comienza a desarrollarse como un lenguaje para escribir software y compiladores, específicamente para desarrollar el sistema operativo UNIX, pero su uso se ha generalizado y hoy en día muchos sistemas están escritos en C o en C++. Además, es un lenguaje que no está ligado a ningún sistema operativo ni máquina, lo que permite la portabilidad entre distintas plataformas. El lenguaje C es un lenguaje de propósitos generales y, si bien es un lenguaje de alto nivel, provee sentencias de bajo nivel que permite facilidades similares a las que se encuentran en los lenguajes ensambladores.

Etapas en la obtención de programa ejecutable en lenguaje C

Antes de ejecutar un programa en lenguaje C, existen ciertas etapas que se deben realizar, ya que todas ellas contribuyen a la obtención del programa ejecutable. Ellas son: edición, compilación, enlace, carga y ejecución.

Edición

Esta etapa consiste en la escritura de un programa en C, utilizando un programa editor. Este programa, que se conoce como **programa en código fuente**, deberá ser almacenado en el disco con un nombre que lo identifique para su posterior compilación. Los programas fuentes se almacenan o graban con extensión **.c** o **.cpp** según se utilice lenguaje C ó C++ (C plus plus).

Este es el **Preprocesador**, un editor de texto, que toma como entrada una forma ampliada de un lenguaje fuente y su salida es una forma estándar del mismo lenguaje fuente.



Compilación

El proceso de compilación consiste en la traducción del programa fuente a código binario, para que la computadora pueda interpretar las órdenes. Esto se logra dando la orden *compilar* del ambiente de desarrollo integrado (IDE) o bien desde la línea de órdenes.

Durante esta etapa, se detectan los posibles **errores sintácticos** cometidos por el programador, cuando no ha respetado las reglas de sintaxis del lenguaje. En este caso, es necesario volver a editar el programa, corregir los errores señalados y compilar nuevamente. Este proceso se repite hasta que no se detectan errores, obteniéndose el **programa en código objeto** que es almacenado en disco.

Se debe tener en cuenta que antes de la fase de traducción mencionada, el compilador invoca automáticamente a un programa *preprocesador* que obedece directrices especiales que indican ciertas operaciones que deben realizarse antes de la compilación. En general, esas operaciones consisten en la inclusión de otros archivos en el programa a compilar o en el reemplazo de símbolos especiales por textos de programa.

Enlace

El programa objeto no es ejecutable, esto se debe a que generalmente los programas contienen referencias a funciones definidas en otro lugar, una biblioteca estándar o en bibliotecas definidos por algún programador, por ejemplo. El enlazador es el que se encarga de vincular el código objeto con las bibliotecas, obteniendo así el **programa ejecutable**, que es almacenado en disco.

Carga

El **Cargador** es un procesador de lenguajes cuya entrada es un lenguaje objeto, un programa en lenguaje de máquina en manera reubicable; las modificaciones que realiza son a tablas de datos que especifican las direcciones de memoria donde el programa necesita estar para ser ejecutable.

El cargador toma el programa ejecutable del disco y los transfiere a memoria.

Finalmente la computadora, bajo el control de la UCP (Unidad Central de Procesamiento) toma cada una de las instrucciones y las ejecuta.

El siguiente es un esquema de las etapas descriptas:

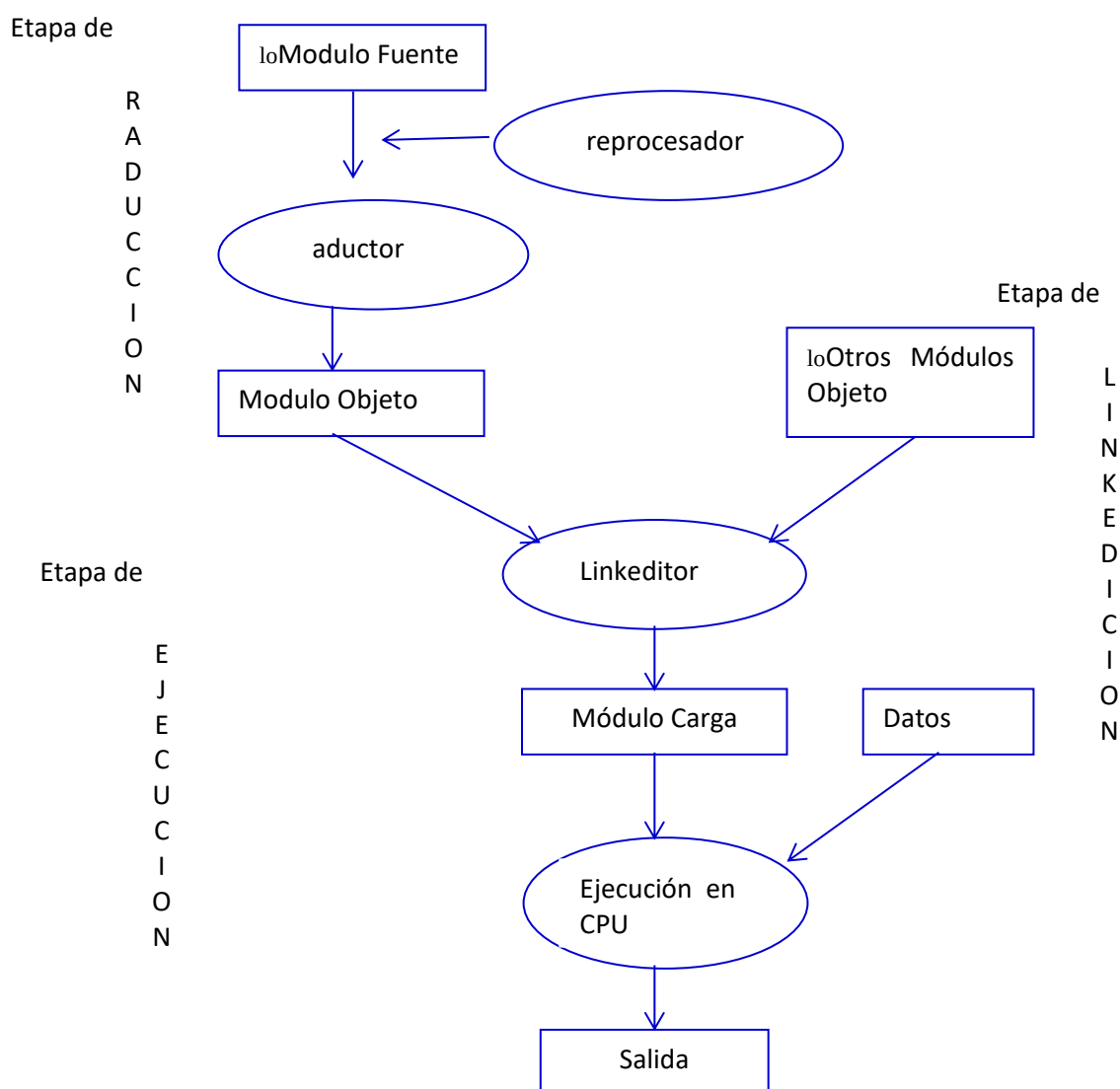


Fig. 4: Etapas para la obtención de un programa ejecutable en C

Bibliografía

- Cueva Lovelle, Juan (1998) Conceptos Básicos de Procesadores de Lenguajes. . Ed. SERVITEC. España.
- Fontela, Carlos (2003) Programación Orientada a Objetos. Técnicas Avanzadas de Programación. Editorial Nueva Librería. Buenos Aires.
- Louden, Kenneth C. (2004) Lenguajes de Programación Principios y Práctica. Editorial Thomson. México, D.F.
- Pratt Terrence y Marvin Zelkowitz (1987) Lenguajes de Programación Diseño e Implementación. Editorial Prentice Hall. Hispano americana, S.A. 2º Edición. México.

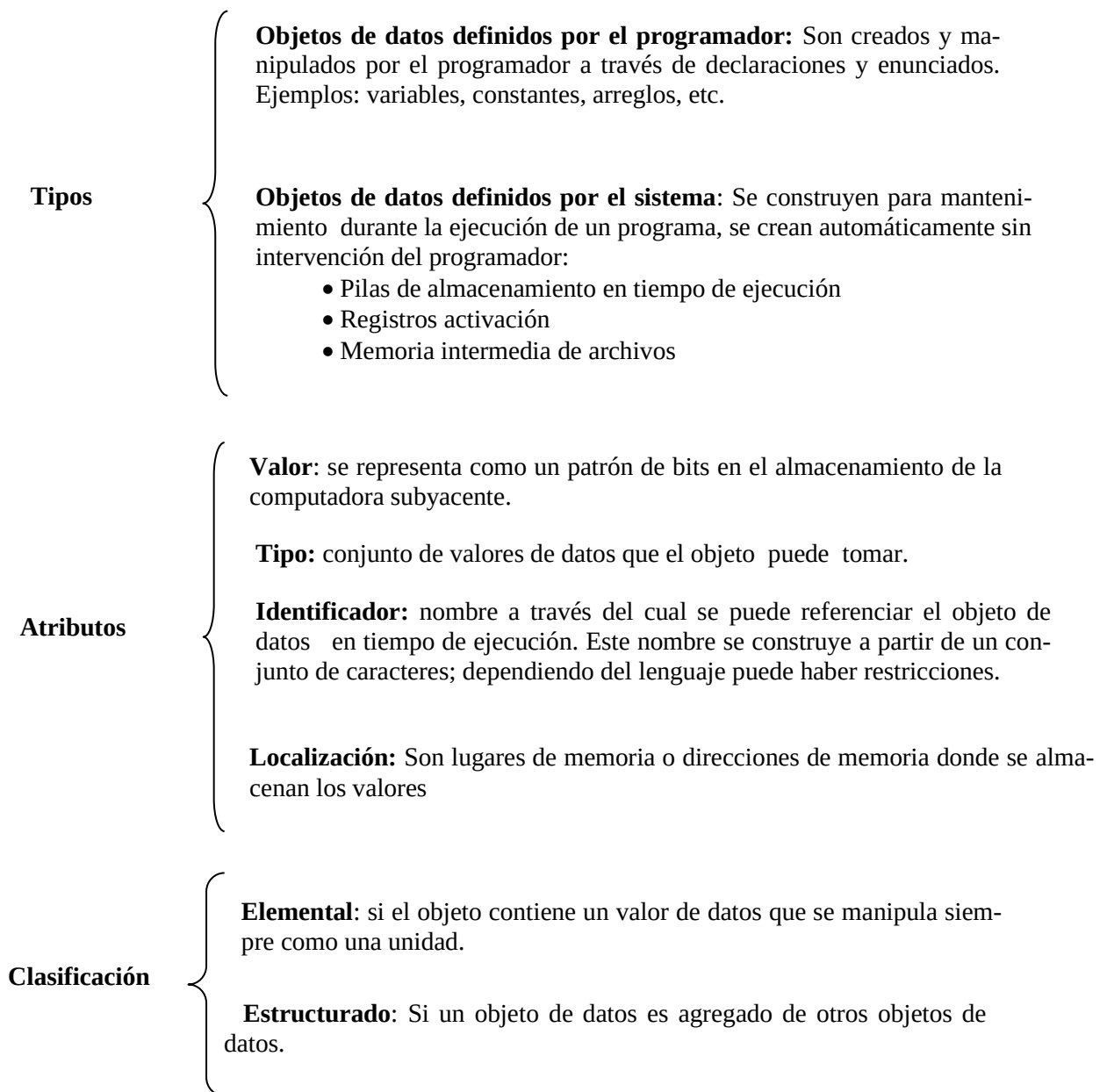
Unidad 2: Objeto de Datos

Introducción

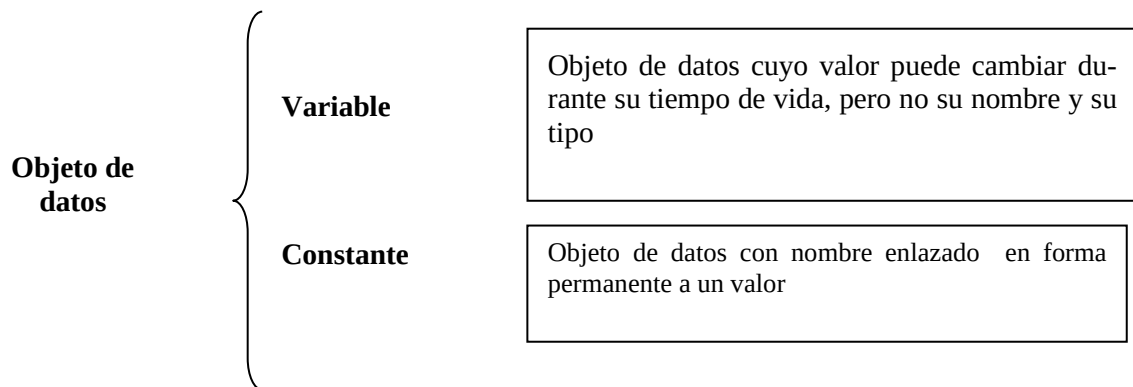
Si tenemos en cuenta que un programa se puede considerar como un conjunto de operaciones que se van a realizar sobre ciertos datos en un orden determinado, los puntos sobresalientes a tener en cuenta en el estudio de los lenguajes de programación son: datos, operaciones y control.

Objeto de Datos

Un **objeto de datos** es un recipiente que se usa para guardar y recuperar valores de datos



Variables y Constantes



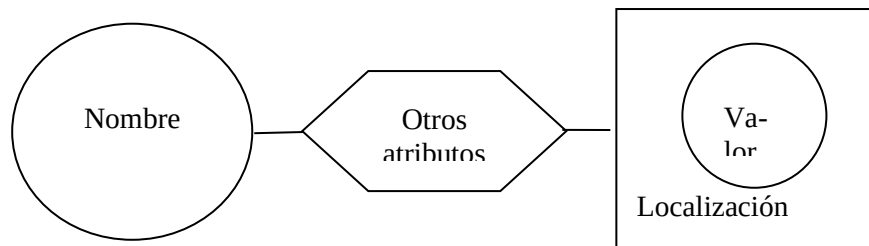
Almacenamiento de variables y constantes en memoria

Antes de usar una variable hay que declararla, al hacerlo se debe dar su nombre (identificador) y su tipo. Para el identificador se sugiere un nombre significativo, de modo de lograr una mejor legibilidad del programa.

Variable

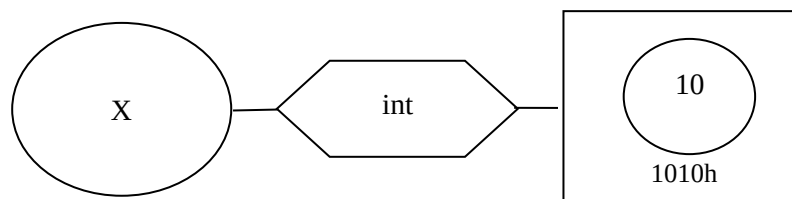
Una variable se puede especificar a través de sus atributos, que incluyen nombre, localización, valor y otros atributos como tipo de datos y tamaño.

Una representación esquemática con **diagramas sintácticos** (estos son una alternativa gráfica para la Forma de Backus-Naur (BNF)) es:



Ejemplo

int X=10;

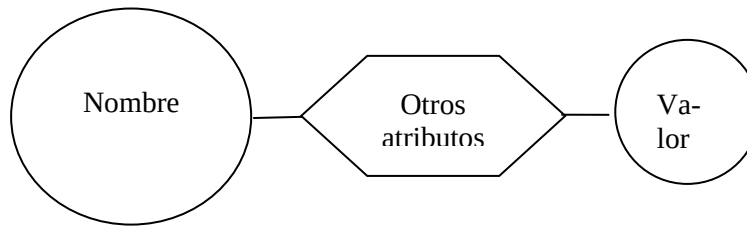


Si hacemos referencia a los atributos principales de una variable, nombre, localización y valor, podemos representarla de manera más sencilla como sigue:



Constante

Una constante es una entidad del lenguaje que tiene un valor fijo mientras existe dentro de un programa.



Una constante es similar a una variable, excepto que no tiene un atributo de localización. Esto no significa que una constante no se almacene en la memoria en algunas ocasiones.

En C las constantes se declaran con la directiva `#define`, esto significa que esa constante tendrá el mismo valor a lo largo de todo el programa.

El identificador de una constante así definida será una cadena de caracteres que deberá cumplir los mismos requisitos que el de una variable (sin espacios en blanco, no empezar por un dígito numérico, etc).

Ejemplo

```
#include <stdio.h>
#define PI 3.1415926
```

```
int main()
{
    printf("Pi vale %f", PI);
    return 0;
}
```

Lo cual mostrará por pantalla:

```
Pi vale 3.1415926
```

Hay lenguajes como C++ en los que el valor de las constantes se computa en tiempo de carga o al comienzo de la ejecución, por lo tanto su valor necesita ser almacenado en la memoria.

Ejemplo de constantes

```
#include<stdio.h>
main()
{ const int a=5;
  const int b=2 +a;

  printf("%d", b);
  getchar();
}
```

La diferencia entre el uso de **const** y el uso de **#define** está en que mediante **const** se declara una constante que tiene un tratamiento similar a una variable (por ejemplo, la constante es de un tipo de dato) mientras que mediante **#define** se indica que escribir el nombre especificado equivale a escribir el valor, con una correspondencia directa y sin tratamiento análogo al de una variable.

Ejemplo

```
const int JUGADORES = 5;           ...           #define JUGADORES 5
```

En la primera declaración se indica que JUGADORES es una constante de tipo int mientras que en la segunda se indica que donde aparezca en el código la palabra JUGADORES deberá ser reemplazada por 5 directamente. En general, usar **#define** supone que la compilación sea más rápida al no tener el compilador que realizar el tratamiento y verificaciones propias de variables. Por ello su uso resultará recomendable cuando existan ciertos valores numéricos que tengan un significado especial, valor constante y uso frecuente dentro del código. Las constantes definidas con `#define` se denominan constantes simbólicas.

Tiempo de Vida¹⁰

Durante la ejecución de un programa algunos objetos de datos existen desde el comienzo de la ejecución, otros pueden crearse dinámicamente durante la misma. Algunos objetos persisten durante toda la ejecución del programa, otros se destruyen durante la ejecución del mismo.

Por lo tanto, definimos como tiempo de vida de un objeto, al tiempo durante el cual el objeto puede usarse para guardar valores de datos.

Enlace (ligadura) y Tiempo de enlace

El **enlace o ligadura** de un elemento de programa a una característica o propiedad particular, hace referencia a la elección de una propiedad dentro de un conjunto de propiedades posibles.

El momento del programa durante el cual se realiza este enlace se denomina **tiempo de enlace o tiempo de ligadura**. Existen una gran variedad de tipos de enlace, algunos de ellos se presentan a continuación, a través de ejemplos en lenguaje C.

Ejemplo: Sea el enunciado $X=X+10$; podemos distinguir distintos tipos de enlaces:

1. La variable X puede tener asociados distintos tipos de datos, el conjunto de los posibles tipos de datos para X se fijan en **tiempo de definición del lenguaje**.

Por ejemplo, hay lenguajes como Pascal que admiten un tipo de dato boolean. Mientras que lenguaje C no lo admite, si lo admite C++. Esto se determina cuando se define el lenguaje.

2. Por otro lado, si el tipo de dato asociado a X fuese un tipo definido por el programador (tal cual lo admite Pascal o C), los posibles valores de X se fijan en **tiempo de traducción**.
3. La decisión de asignarle como nombre X a la variable, así como el tipo de dato de la misma, son enlaces elegidos por el programador y corresponden al tiempo de **traducción o compilación**.

Ejemplo: `int X;`

Hay lenguajes en los que el programador no especifica un tipo de datos para las variables, sino que las mismas van tomando el tipo de datos en función del valor que se le va asociando. En estos casos una misma variable puede ir tomando distintos tipos de datos en tiempo de ejecución. Ejemplo de esto lo brinda SmallTalk y Prolog.

4. El conjunto de los posibles valores que puede tomar la variable X se determina en **tiempo de implementación del lenguaje**.
5. La asignación $X=X + 10$, cambia el valor asociado a la variable X en **tiempo de ejecución**, asignándole como nuevo valor el valor que tenía más 10.
6. Además, el número o constante **10** se representa como una serie de bits en tiempo de implementación, mientras que su representación decimal en el programa se hace en **tiempo de definición del lenguaje**.
7. La elección del $+$ para representar la adición se hace en tiempo de definición del lenguaje, mientras que la determinación de cuál es la operación que se debe realizar se hace en **tiempo de compilación** (recordemos que el $+$ es un operador homónimo u homonímico, es decir el mismo operador puede usarse para distintos tipos de sumas, cada uno tiene una definición según el contexto).
8. La definición en detalle de la función $+$ se realiza en tiempo de implementación del lenguaje y ésta depende de la definición de adición del hardware subyacente.

¹⁰ Capítulo 5. Pág. 114 a 117. Kenneth Loudon.

Cuando los enlaces de un lenguaje se realizan antes de la ejecución se dicen que son enlaces tempranos o **ligaduras estáticas**¹¹, mientras que cuando se realizan en tiempo de ejecución hablamos de enlaces tardíos o **ligaduras dinámicas**. Las ventajas y desventajas de unos y otros giran en torno al conflicto entre eficiencia y flexibilidad.

De todos los tipos de enlaces expuestos, salvo los enlaces en tiempo de ejecución, todos los otros enlaces son **ligaduras estáticas**.

En lenguajes como C o Pascal, donde la eficiencia de ejecución es primordial, es común que la mayoría de los enlaces sean tempranos.

En lenguaje LISP, donde la flexibilidad es su objetivo, los enlaces se retardan hasta la ejecución para que los mismos puedan hacerse dependientes de los datos.

Es importante hacer notar, que el traductor debe conservar los enlaces de tal manera que se realice la operación apropiada durante la traducción y la ejecución. Esto es, el traductor debe conservar las ligaduras de modo que se den significados apropiados a los nombres en tiempo de traducción y ejecución.

Para esto, el traductor crea una estructura de datos, llamada **Tabla de Símbolos**. A grandes rasgos, se puede decir que esta estructura contiene una entrada por cada identificador diferente declarado en el programa fuente, además para cada identificador se introduce información adicional (tipo de variable, parámetro formal, nombre del subprograma, entorno de referencia, etc.).

Conforme se avanza en la traducción la Tabla de Símbolos va cambiando de modo que quedan siempre reflejadas la eliminación o inserción de enlaces.

En temas siguientes, una vez desarrollado el concepto de función retomaremos la tabla de símbolos, y mostraremos ejemplos que permitan ver el funcionamiento de la misma.

Tipos de datos

Tipo de Datos: En un programa, los objetos de datos como un flotante F, un arreglo A, una struct T, tienen un tipo de datos que está asociado al conjunto de valores que puede tomar dicho objeto. No obstante, para un lenguaje de programación podemos dar una definición más precisa:

Un tipo de datos es una clase de objetos de datos ligados a un conjunto de operaciones para crearlos y manipularlos.

De ahí, podemos hacer referencia a un tipo de datos como la clase de los arreglos, la clase de los flotantes, etc.

Los tipos de datos pueden ser: **Primitivos:** datos que están integrados al lenguaje o **Definidos por el programador:** el lenguaje provee recursos para definir nuevos tipos.

Especificación e implementación de un tipo de datos

Elementos básicos de una especificación de un tipo de datos

Atributos: distinguen a los objetos de datos de ese tipo. Por ejemplo, para un tipo de dato arreglo los atributos son: nombre, dimensiones, tipo de dato de las componentes, etc.

Valor: especifica los valores de datos que pueden tomar los objetos de este tipo de dato.

Operaciones: hacen referencias a las distintas formas de manipular los objetos de datos de ese tipo de datos.

¹¹Algunos autores hablan de enlaces, otros de ligaduras.

Elementos básicos de la **implementación** de un tipo de datos:

Representación de almacenamiento: hace referencia a la forma que se usa para representar los objetos de datos de un tipo de dato determinado de datos en memoria.

Algoritmos y procedimientos: definen las operaciones del tipo, en términos de manipulaciones de su representación de almacenamiento.

Tipos de datos elementales

Especificación

Tipo elemental de datos, hace referencia a una clase de objetos de datos que contienen **un solo valor**.

- Los atributos de un objeto de un tipo elemental de datos, como por ejemplo su tipo, se mantienen invariables durante su tiempo de vida.
- El conjunto de valores que puede tomar un objeto de un tipo de dato elemental está generalmente ordenado, hay un máximo, un mínimo y dados dos valores distintos del dato, uno es mayor que el otro.
- Las operaciones asociadas pueden ser primitivas o definidas por el programador. Las operaciones primitivas son operaciones que se especifican como parte de la definición del lenguaje, mientras que las definidas por el programador son subprogramas o procedimientos que el programador define sobre objetos de un tipo elemental de datos.

Representación de almacenamiento

Ordinariamente, representación de almacenamiento hace referencia al tamaño de bloque de memoria que se requiere, a la disposición de los valores y atributos dentro de ese bloque y no a su ubicación dentro de la memoria.

El almacenamiento está influido por la computadora subyacente que va ejecutar el programa. Por ejemplo, la representación de almacenamiento para valores enteros o reales generalmente utiliza la representación binaria que se usa en el hardware para esos valores. Por lo tanto, las operaciones básicas sobre datos de este tipo se pueden implementar a través de las operaciones de hardware. Caso contrario, las operaciones deben simularse por software, de manera mucho menos eficiente.

En lenguajes como C, Pascal, Fortran los **atributos** no forman parte del objeto de datos en tiempo de ejecución, es decir no se guardan en la representación de almacenamiento. Son determinados por el compilador. Esto permite mayor eficiencia en el uso del almacenamiento y mayor velocidad de ejecución, objetivos primordiales de estos lenguajes.

En lenguajes como Lisp y Prolog, los atributos forman parte del objeto de datos en tiempo de ejecución. Estos atributos se guardan como un descriptor.

Implementación de tipos de datos elementales

La implementación de un tipo elemental de datos hace referencia a la representación de almacenamiento para objetos de datos y valores de ese tipo, y a un conjunto de operaciones y procedimientos que definen las operaciones del tipo en términos de manipulaciones de la representación de almacenamiento.

Implementación de las operaciones

Implementación de **algoritmos y procedimientos** que definen las operaciones

- Como operación de hardware
- Simulada por software a través de subprogramas
- Simulada por software como una secuencia de código de línea

Ejemplos

- Como operación de hardware: Por ejemplo, si los números enteros usan la representación de hardware para enteros, entonces la suma puede implementarse usando la operación de suma integrada en el hardware.
- Simulada por software a través de subprogramas: Una operación de potenciación generalmente no es suministrada directamente como una operación de hardware. En este caso se puede implementar un subprograma que calcule esta operación.

Si los objetos de datos no se representan usando la representación de hardware, entonces las operaciones se deben simular comúnmente, a través de subprogramas que se suministran a través de bibliotecas.

Ejemplo

```
#include <stdio.h>
#include <math.h>

main()
{ int base, exponente, r;
  scanf("%d %d", &base, &exponente);
  r=pow(base, exponente);
  printf("\n Potencia: %d", r);
}
```

En este ejemplo en Lenguaje C, la función potencia (pow) está simulada por software y suministrada por el lenguaje en el archivo <math.h>.

-Lenguaje C también permite que ciertas operaciones puedan simularse por macros. Esto puede ser beneficioso cuando el subprograma es muy corto, directamente se reemplaza por él, cada vez que este aparece en el programa.

```
#include <stdio.h>
#include <conio.h>

#define maximo(A,B) A<B?B: A

main()
{
  int x=20, y=30,z;
  z= maximo(x,y);
  printf("\n el mayor es %d",z);
  getch();
}
```

Declaraciones

Una declaración es un enunciado escrito por el programador que sirve para comunicar al traductor información primordial para establecer ligaduras.

En general, podemos decir que las declaraciones contribuyen a:

- Ofrecer al traductor información que permite el almacenamiento para un objeto de datos, reduciendo el tiempo de ejecución del programa que se está traduciendo.
- Cuando hay operadores homónimos, las declaraciones permiten que el traductor, en tiempo de compilación, determinar cuál es la operación particular a la que se hace referencia el operador homónimo.

Nota: Los operadores +, *, -, /, etc. son operadores homónimos pues denotan operaciones genéricas que tienen distintas definiciones en función de los argumentos de las mismas.

Ejemplo

```
void main (void)
{ int A, B, C;
  float P, Q, R;
  A = B + C;
  P = Q + R;
  .....
}
```

En este ejemplo, el operador + se ha utilizado para realizar una suma de enteros y una de reales, operaciones que matemáticamente se realizan de manera distinta.

Las declaraciones de las variables permiten al traductor del lenguaje determinar en tiempo de compilación cual es la operación particular que designa el símbolo homónimo +, esto es, o suma de enteros o suma de flotantes.

Las declaraciones informan sobre el tiempo de vida de un objeto lo que hace más eficiente la gestión de almacenamiento durante la ejecución de un programa.

Por ejemplo, en **lenguaje C**, a las variables locales a un subprograma se les asigna espacio en un bloque de memoria en forma automática al entrar a la ejecución del subprograma. Al terminar ese bloque también en forma automática se libera. Sin embargo, para variables dinámicas (creadas con malloc), como no se declaran explícitamente en el subprograma, se les debe asignar almacenamiento en forma individual en un área de almacenamiento por separado, a un costo mayor porque el pedido lo hace el programador.

```
double calculo (double a)
{ return a * 1.20; }
```

```
void main (void)
{
  double sueldo,s;
  .....
  s=calculo(sueldo);
  int *p;
  p=malloc(30 * sizeof(int));
  .....
  free(p);
  .....
}
```

La declaración de la función **calculo** indica una operación que puede ser usada en el programa.

La variable **a** de tipo double, indica que para su almacenamiento se usará 8 bytes. Además por ser una variable local a la función se le asigna memoria automáticamente durante la ejecución del subprograma y automáticamente se libera al terminar la ejecución del mismo.

Las variables **sueldo** y **s**, son locales al main y por ser del tipo double necesitan para su almacenamiento 8 bytes.

La variable **p** es un puntero a entero, por lo tanto necesita para su almacenamiento 4 bytes.

Con la función malloc se habilitará el uso de un área de almacenamiento de $30 \times 4 = 120$ bytes.

La función free liberará el espacio generado por malloc.

El **propósito más importante** de las declaraciones es permitir la verificación estática de tipos.

Verificación de tipos

La verificación de tipos es el proceso que realiza el intérprete o el compilador para verificar que todas las construcciones en un programa tengan sentido en términos de los tipos declarados.

Ejemplo

Como la verificación de tipos implica, entre otras, comprobar que cada operación que un programa ejecuta recibe el número de argumentos apropiados y del tipo de datos correcto, el caso de la expresión $A = B + C$ si C es tipo `char`, las variables A y B son tipo `float`, entonces el compilador detectará un error de tipo de argumento en la operación.

La verificación de tipos se puede realizar en tiempo de ejecución (**verificación dinámica de tipos**) o en tiempo de compilación (**verificación estática de tipos**).

Verificación dinámica de tipos: Si la información de tipos se conserva y se verifica en tiempo de ejecución, la verificación es dinámica. Por definición los intérpretes llevan a cabo verificación dinámica. No obstante, existen algunos compiladores que proveen mecanismos para que la verificación de tipos se realice en tiempo de ejecución; tal es el caso del lenguaje LISP.

Para implementar este tipo de verificación se guarda una marca de tipo en cada objeto, el cual indica el tipo de datos asociado al mismo. Por lo tanto, cuando se realiza la verificación de tipos, lo primero que se efectúa es la verificación de marcas de tipo de argumentos. La operación se realiza si los tipos de argumentos son correctos, de lo contrario se indica un error. También, para cada operación, se deben anexar las marcas de tipo apropiadas a sus resultados, para que las operaciones subsiguientes se puedan verificar.

Este tipo de verificación es propia de lenguajes como LISP y PROLOG, ya que en ellos no se dan declaraciones de variables, y por lo tanto para dos variables A , y B , su tipo de datos puede cambiar durante la ejecución del programa.

Verificación estática de tipos: La verificación estática de tipos se realiza durante la traducción de un programa. En general, la información necesaria para esta verificación, es proporcionada por el programador a través de las declaraciones.

¿Cómo se realiza la verificación de tipos?

- El traductor recoge información de las declaraciones desde la tabla de símbolos, que contiene información de variables y operaciones.
- Después de reunir la información de tipos, se verifica cada operación que invoca el programa (verifica tipo de argumento válido). Si los argumentos son válidos, se determina el tipo de datos del resultado y esta información se guarda para verificar operaciones posteriores. Si la operación es homonimia, el nombre de la operación se puede reemplazar por el nombre de la operación específica de tipo particular que usa esos argumentos.
- Como la verificación estática incluye todas las operaciones que aparecen en cualquier subprograma, se verifican todas las rutas de ejecución. No se necesita una revisión adicional en busca de errores. Se evitan por lo tanto, marcas de tipo en los objetos en tiempo de ejecución, como en la verificación dinámica, se gana almacenamiento y tiempo de ejecución

Uno de los problemas más significativos respecto a los tipos de datos, es que no hay consenso entre los diseñadores de lenguajes respecto de hasta qué punto la información de tipos debe explicitarse y utilizarse para la verificación de tipos, antes de la ejecución de un programa.

Hay quienes ponen más énfasis, en una mayor flexibilidad en el uso de tipos, y que no procuran una verificación de tipos estricta, y hay quienes procuran la implementación de la verificación estricta de tipos en tiempo de traducción.

Un lenguaje **fuertemente tipificado** es por tanto, un lenguaje con una verificación de tipos estricta. O lo que es lo mismo, **un lenguaje de tipo fuertes** detecta en forma estática, los *errores de tipo de un programa*.

Verificación de tipos y lenguaje C

Los compiladores de C aplican una verificación estática de tipos durante la traducción, no obstante muchas inconsistencias en los tipos no causan errores de compilación, sino que son automáticamente eliminados, presentando o no un mensaje de advertencia.

La mayoría de los compiladores modernos, tienen configuraciones de nivel de error que aportan un tipificado más fuerte. C++ agrega una verificación de tipos más fuerte a C, pero bajo la forma de advertencias en vez de errores, con fines de compatibilidad con C. Este tipo de mensaje no impide la ejecución. Ignorar estas advertencias puede ser un desatino peligroso (Stroustrup, 1994, Página 42).

Conversión de tipos

Cuando en la verificación de tipos hay una discordancia entre el tipo real de un argumento y el tipo esperado, el lenguaje puede:

1. marcar el error y ejecutar una acción de error apropiada
2. se puede realizar una conversión de tipos.

Una conversión de tipos es una operación, de la forma:

Conversión: tipo1 $\longrightarrow$ tipo2

Esta conversión toma un objeto de un tipo y produce el objeto de datos correspondiente de un tipo distinto.

Conversión implícita o coerciones¹²: son invocadas automáticamente en ciertos casos de discordancia. El principio básico que gobierna las coerciones es no perder información.

Ejemplo

```
void main(void)
{
    char c;
    int i;
    float f;
    double d, result;
    result = (c / i) + (f * d) - (f + i)
    ....
}
```

Si durante la verificación de tipos de una operación como la planteada en el ejemplo anterior, ocurre una discordancia entre los tipos de los operandos y/o del tipo de resultado esperado, el compilador busca la operación de conversión para que de **manera automática** se realicen los cambios al tipo de dato ade-

¹² K. Loudon, en el libro Lenguajes de Programación las denomina coerciones. Pág. 211

cuado. En este caso, estamos frente a una **conversión implícita** o **coerción**. En C, las coerciones son reglas.

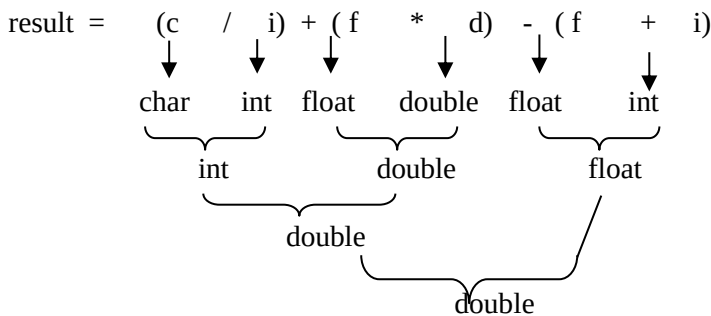
Por ejemplo, en expresiones aritméticas el lenguaje C permite utilizar operandos de distintos tipos en una expresión, teniendo en cuenta las siguientes reglas:

a- Si los dos operandos son de tipo coma flotante con distinta precisión, el operando de menor precisión se transforma a la precisión del otro operando y el resultado se expresa en esta precisión (la más alta). Por ejemplo entre un float y un double, el float se convierte a double y el resultado será double.

b- Si un operando es de tipo coma flotante y el otro es un char o un int, el char/int se convertirá al tipo coma flotante y el resultado se expresará en este tipo. Por ejemplo una operación entre un int y un double tendrá como resultado un double.

c- Si ninguno de los operandos es de tipo coma flotante, el compilador convierte los char y short int a int antes de evaluar, salvo que en la expresión aparezca un long int. En este caso los operandos y el resultado se convertirán a long int.

Para el ejemplo anterior, la conversión implícita es la siguiente:



En todos los casos la conversión se ha realizado de tal forma que el tipo de datos final puede contener toda la información que se convierte sin perder información. En este caso la conversión es de **ensanchamiento o extensión**.

No obstante, si la variable **result** fuese de tipo **int**, entonces habría una conversión de un tipo real a un entero, y en tal caso puede ser que se pierda información. Mientras el valor real 15.0 es igual al entero 15, no sucede lo mismo con el real 15.20, el cual no es representable como entero. Este real será convertido al entero 15, perdiendo información. Este tipo de coerción se llama de **estrechamiento o restricción**.

Conversión explícita o forzada: conjunto de funciones integradas que el programador puede llamar para realizar una conversión de tipos.

En C, se puede forzar a una expresión a ser de un tipo específico, usando el operador unario **cast**.

Formato: **(tipo) expresión** donde **tipo**, es el tipo al que se desea forzar el resultado

Ejemplo

```
#include <stdio.h>

void main(void)
{
    int x=7;
    printf("\n %f", x/2);
}
```


Para este programa la salida es 3.0, sin embargo si forzamos a x a un tipo flotante, tal cual lo expresa el siguiente programa:

```
void main(void)
{ int x=7;
  printf("\n %f", (float) x/2);
}
```

La salida es 3.50.

En estos casos de conversión explícita, para la verificación estática de tipos, se inserta código adicional en el programa compilado para invocar la operación de conversión en el punto apropiado durante la ejecución.

El lenguaje Pascal casi no provee coerciones, salvo excepciones, todas las discordancias son tratadas como errores.

En C, las coerciones son la regla. Cuando se encuentra una discordancia de tipo, el compilador busca una operación de conversión apropiada para insertarla en el código compilado y de ese modo proveer el cambio de tipo adecuado. Sólo si no se encuentra una conversión posible, la discordancia se presenta como un error.

En otros capítulos, se presentarán nuevos ejemplos donde revisaremos los distintos tipos de coerciones.

Clasificación

Los tipos de datos elementales (también llamados primitivos) son los tipos de datos originales de un lenguaje de programación, esto es, aquellos que proporciona el lenguaje y con los que se puede construir estructuras de datos y tipos de datos abstractos. Estos se pueden clasificar de la siguiente manera:

- Carácter
- Entero
- Real
- Booleano o Lógico
- Puntero o Apuntador

Algunos lenguajes no proporcionan los tipos de datos booleanos y puntero. A continuación se describirá cada uno de ellos.



Tipos de datos enteros

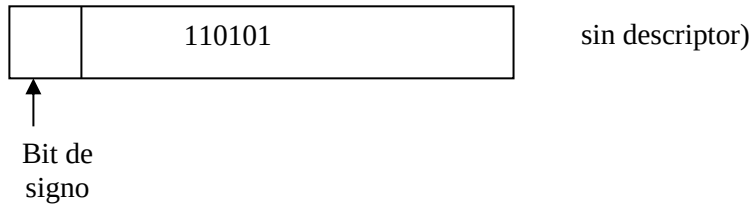
Especificación: Un objeto de datos de tipo entero tiene como atributo solo su tipo. Los valores que puede tomar un objeto de datos es un subconjunto ordenado de números enteros, que tiene límites inferior y superior. El máximo valor es elegido por el implementador de manera que refleje el valor entero máximo representable en el hardware subyacente.

Por lo tanto, los valores posibles varían en un intervalo como:

[-maximo-entero , maximo-entero]

Implementación: La implementación de un tipo de dato entero usa la representación de almacenamiento definida por el hardware y las operaciones primitivas relacionadas a enteros.

En lenguaje C (también el lenguaje Pascal) las representaciones de almacenamiento carecen de descriptor puesto que ambos lenguajes ofrecen declaraciones y verificación estática de tipos.



Otros lenguajes incluyen descriptores en tiempo de ejecución, tal es el caso de SmallTalk .

Operaciones sobre enteros: Las operaciones típicas que un lenguaje de programación define sobre objetos de datos enteros son las siguientes: Operaciones aritméticas, Operaciones relacionales, Operación de asignación.

Además, en lenguaje C, los enteros representan valores booleanos, lo cual permite operaciones de bit, tales como & (and), | (or) y <<(desplazamiento) entre otras. Estas operaciones no serán trabajadas en este curso.

Operación de asignación

Dentro de las operaciones antes destacadas, resulta importante entender **la asignación**. La asignación es una operación de casi todos los tipos de datos elementales, y es la operación básica para cambiar el valor de una variable. Ejemplos de asignación son:

X=2+8; para X entera

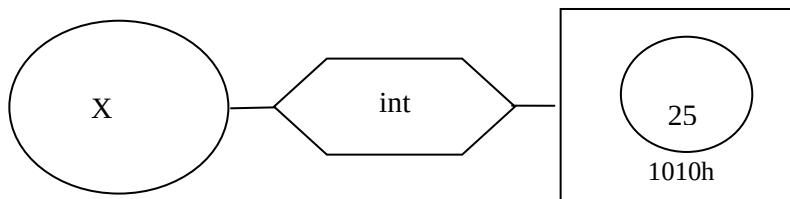
X=Y; para X e Y enteras.

Valores lvalue y rvalue

Para comprender de manera más clara como funciona esta operación, se analiza el concepto de **valor l** y **valor r** de una variable.

Dado que una variable tiene una localización y un valor almacenado en dicha localización, para distinguir claramente a ambos, se denomina **valor r** de una variable al valor que contiene dicha variable, y **valor l** se refiere a la localidad de la variable.

Supongamos que la variable X tiene almacenado un valor 25, su representación esquemática es:



Por lo tanto **el valor l (X) es 1010h**, mientras que **el valor r(X) es 25**.

Luego de la operación X= 35, cambia el valor r (X) por 35.

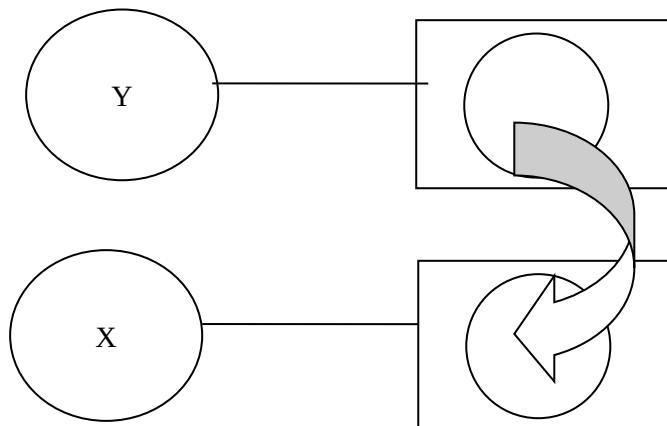
El **l** representa la izquierda (left), aludiendo a que un **valor l** o **lvalue** debe estar a la izquierda de una operación de asignación. Del mismo modo la **r** representa la derecha (right), aludiendo a que un **valor r** o **rvalue** debe estar a la derecha de una operación de asignación.

Como puede inferirse una constante no tiene **lvalue**.

La semántica de una operación de asignación puede cambiar en distintos lenguajes. Se analiza a continuación la operación de asignación en lenguaje Pascal y en lenguaje C.

Esquemáticamente:

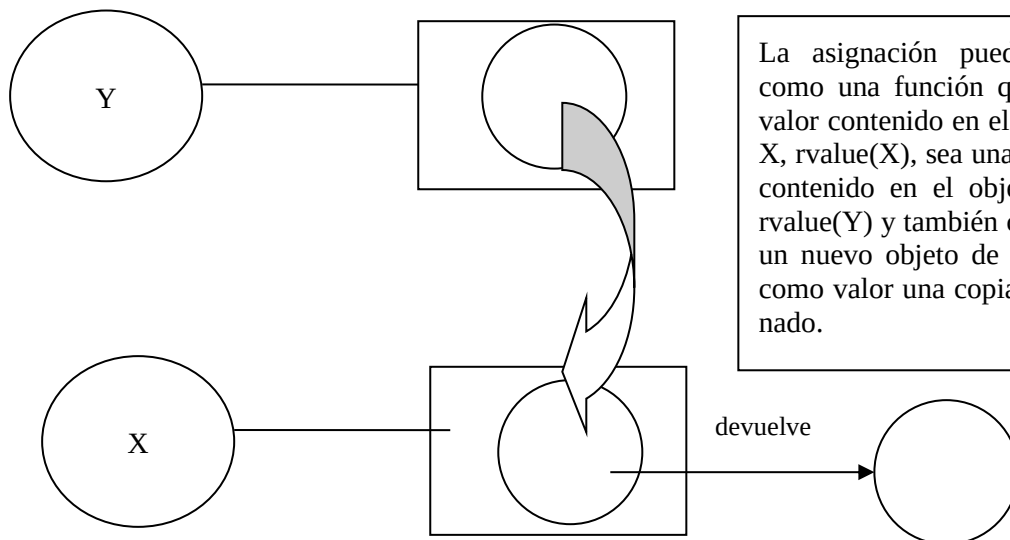
En lenguaje Pascal, la sintaxis de la asignación: **X:=Y**



La asignación se interpreta como una función que hace que el valor contenido en el objeto de datos X es una copia del valor del objeto de datos Y, no devolviendo esta función ningún resultado explícito.

De ahí que la asignación es un efecto colateral de la función.

En lenguaje C, la sintaxis de la asignación es: **X=Y**



La asignación puede interpretarse como una función que hace que el valor contenido en el objeto de datos X, $rvalue(X)$, sea una copia del valor contenido en el objeto de datos Y $rvalue(Y)$ y también crear y devolver un nuevo objeto de datos que tiene como valor una copia del valor asignado.

Por lo tanto, si en lenguaje C se tiene la siguiente asignación:

X= Y + 3*2;

La operación que se realiza sería la siguiente:

- 1) Computar el valor l de la variable X
- 2) Computar el valor r de la expresión $Y + 3*2$, es decir, computar el valor r de Y y finalmente computar el valor r de toda la expresión.
- 3) Asignar el valor r calculado al objeto de datos del valor l computado.
- 4) Devolver el valor r del objeto de datos antes asignado.

Esta manera de interpretarse la semántica de la operación de asignación, se infiere que las asignaciones múltiples en lenguaje C son válidas.

Ejemplo

```
int a=10, b, c;
c=b=a*2;
```

Tipos de datos reales

Especificación: Un objeto de este tipo puede tomar cualquier valor real dentro de una serie ordenada de valores, y limitada por un valor mínimo y un valor máximo determinado por el hardware. Cabe tener en cuenta que los valores no están distribuidos uniformemente en ese intervalo, como en el caso de los enteros.

La precisión en cuanto al número de dígitos para la representación decimal puede ser especificada por el programador tanto en **Pascal** como en **C**.

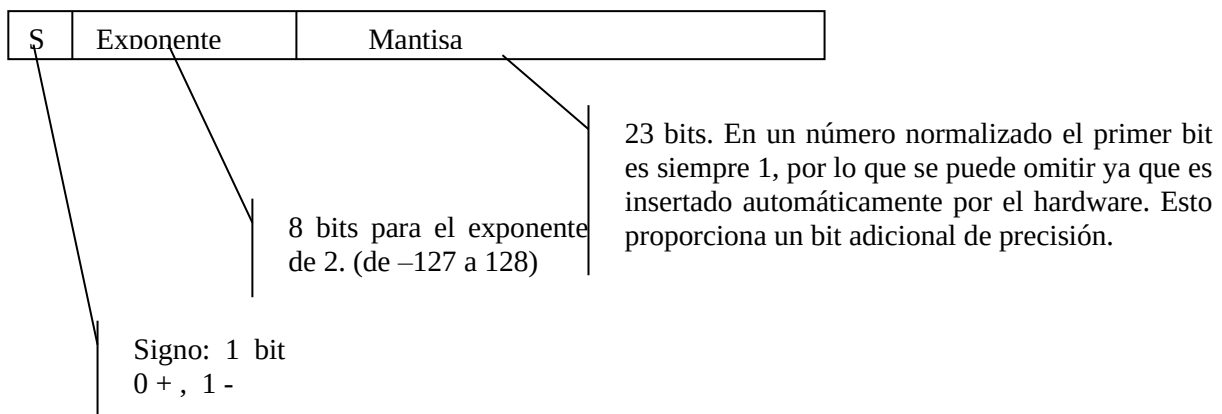
Las operaciones relacionales y aritméticas provistas para enteros, son válidas para reales, aunque algunas operaciones booleanas están restringidas debido a cuestiones de redondeo.

Implementación: Para representar valores reales de punto flotante, utiliza la misma representación de hardware, la que emula a la notación científica.

La norma **754 de IEEE** (1985) se ha convertido en la definición aceptada para la implementación de números reales de punto flotante de precisión simple o doble.

Utiliza 32 o 64 bits para números de punto flotante. Para números de punto flotante de precisión doble se agrega a la mantisa una palabra más (mantisa extendida).

La representación de número normalizado es la siguiente:

**Tipos de datos booleanos**

Especificación: Este tipo representa a objetos de datos que pueden tomar uno de dos valores posibles: True o False. En Lenguaje Pascal, el tipo boolean se considera como un tipo enumerado definido por el lenguaje, donde los valores simbólicos son TRUE y FALSE.

Esto es, Type boolean=(FALSE, TRUE);

Operaciones que soportan: {

- Conjunción
- Disyunción (inclusive)
- Negación

Implementación: Si bien se necesita un solo bit para su representación (0 o 1), como los bit no son individualmente direccionables, se usa un byte o una palabra que es una unidad direccionable.

Hay dos maneras distintas de representar estos valores dentro de la unidad de almacenamiento:

1. Se usa un solo bit del byte para representar con 0 o 1 según sea *verdadero* o *falso*. Los bits restantes se ignoran. (*generalmente se usa el bit de signo*)
2. Un valor 0 en toda la unidad de almacenamiento representa *falso*, cualquier otro valor representa *verdadero*.

En lenguaje Pascal: Utiliza un byte False se representa con 0, True se representa con 1.

Lenguaje C no provee de tipos boolean pero se usan objetos del tipo de dato entero para representar valores verdadero o falso. En general con 0 se representa un valor falso, cualquier valor distinto de 0 representa verdadero, aunque lo óptimo es usar el 1 como verdadero para no tener problemas cuando se realizan operaciones de bits como el &.

Podría suceder por ejemplo:

Bandera=7; en binario es: 0000111

!Bandera; en binario es: 1110000 seguiría siendo verdadero.

El lenguaje C estandar no admite este tipo, pero C++ admite el tipo booleano (bool) y por lo tanto la siguiente declaración es válida:

bool Band = false; // Band es una variable booleana, el valor verdadero se escribe true.

Tipo de datos caracter

Especificación: Este tipo de dato toma como valor un solo carácter. El conjunto de posibles valores se corresponde generalmente con el conjunto de caracteres que maneja el hardware y el Sistema Operativo subyacente. Este conjunto, de caracteres tiene un orden, lo cual resulta importante cuando se construyen cadenas de caracteres. Es casualmente este orden el que permite el orden alfabético entre cadenas y las operaciones referidas a las mismas.

Pascal y C no poseen cadenas de caracteres, sino que se construyen a partir de tipo de dato estructurado arreglo.

En C, el tipo carácter es un subtipo del tipo entero. Cada carácter ocupa un byte y puede ser manipulado como un entero.

Respecto al tipo de dato caracter, prácticamente todas las computadoras utilizan para representar caracteres el código ASCII extendido (Código estándar Americano para el Intercambio de Información). Este código es una extensión del ASCII standard (128 caracteres), y si bien no es un código standard ofrece la posibilidad de trabajar con 256 caracteres. Estos caracteres están ordenados de 0 a 255, lo que permite la comparación de caracteres entre sí. Los dígitos están ordenados en su propia secuencia numérica y las letras dispuestas en orden alfabético precediendo las mayúsculas a las minúsculas.

En general, todas las computadoras manipulan los siguientes caracteres:

Conjunto de letras minúsculas: a..z. (excepto ch, ñ, ll)

Conjunto de letras mayúsculas: A..Z. (excepto CH, Ñ, LL)

El conjunto de dígitos decimales: 0..9.

Carácter de espacio en blanco.

Caracteres especiales: +, -, %, ... @

Signos de puntuación: , ; :

Una variable de tipo caracter puede tomar uno de los valores de este conjunto, el que se debe escribir entre apóstrofes para evitar confundirlo con el nombre de una variable, operador o número.

Ejemplo

```
char sexo
```

```
sexo='m'
```

En el ejemplo, sexo es una variable de tipo carácter que se utiliza para representar el sexo de un atleta; el carácter 'm' es un valor constante que se asigna a la variable sexo para indicar que el atleta es varón.

Implementación: Generalmente los objetos de datos de tipo carácter son manejados por el hardware y el Sistema Operativo (S.O) subyacente, debido a su uso en la entrada y salida de información. Generalmente la implementación de un lenguaje usa la misma representación de almacenamiento para caracteres que el hardware. Si se usa una cadena distinta a la que soporta el hardware (distinta generalmente en el orden de los caracteres) la implementación del lenguaje debe proveer las conversiones apropiadas a la representación del nuevo conjunto o implementar las operaciones relacionales en las cual incide la secuencia de ordenación.

Tipo de dato puntero (apuntador)

Un objeto del tipo apuntador, también llamado tipo de referencia o de acceso, posee la dirección o localidad de otro objeto (valor L del objeto) o puede contener un apuntador nulo es decir a ninguna dirección.

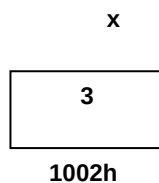
No todos los lenguajes proveen este tipo de datos, pero entre los tipos de datos elementales que provee el lenguaje C, el tipo de datos puntero adquiere relevancia al momento de trabajar con gestión de memoria de manera dinámica. Esto es, a través del uso de punteros el programador puede, en tiempo de ejecución crear variables y destruirlas cuando las considere innecesarias.

En lenguaje C, los punteros se utilizan para manejar estructuras de datos tales como arreglos y listas enlazadas. También son útiles para devolver resultados a través de los parámetros de una función. En capítulos posteriores se analizan estos temas.

Punteros a variables simples

Cuando se declara una variable, el sistema se encarga de reservar el espacio de memoria necesario para almacenar un valor del tipo declarado.

Así por ejemplo, dada la siguiente declaración `int x;` el sistema reserva 4 bytes para la variable entera x. Con `x=3` por ejemplo, se almacena el valor 3 en el espacio reservado a esa variable.



Se debe distinguir entre la dirección de memoria de una variable y su contenido.

El siguiente esquema de memoria representa esta situación:

1000h		
1001h		
1002h	3	x
1003h		
:		
1005h		
Dirección de memoria	Contenido	Identificador

En el caso presentado, la variable x está almacenada en la dirección de memoria 1002h y su valor o contenido es 3.

En la memoria de la computadora, cada dato almacenado ocupa varias celdas contiguas, dependiendo del tipo de dato de la variable declarada. Así, un entero ocupa 2 ó 4 bytes consecutivos según la representación de almacenamiento de entero definido por el hardware, un carácter 1 byte, etc.

Las celdas adyacentes de memoria están numeradas en forma consecutiva desde el principio al final de la misma. Este número, conocido como dirección de memoria se representa por lo general en sistema hexadecimal; F96 y EC2 son ejemplos de direcciones válidas de memoria. Por razones didácticas se simbolizará de ahora en más las direcciones con números decimales consecutivos, seguidos o no de la letra h de hexadecimal, así 1002h ó 1002 serán direcciones válidas.

Declaración de Punteros

Si se quiere definir un puntero p que contenga la dirección de memoria de una variable entera, los pasos a seguir son los siguientes:

Declarar las variables

`int *p, x;` Esto se lee: p es un puntero a un entero, x es una variable entera

Se debe notar que en la declaración, el nombre de una variable puntero es precedido por un asterisco.

Asignar al puntero la dirección de memoria de la variable

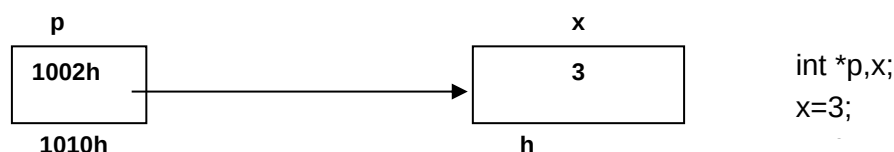
Una forma de asignar un valor a una variable puntero es a través del uso del operador &, que devuelve la dirección de una variable.

Por lo tanto la asignación,

`p=&x;` guarda en p la dirección de la variable entera x.

Como el puntero contiene en este caso la posición de otra variable, se suele decir que la variable x está apuntada por el puntero p.

Si la variable x tiene asignado el valor 3, lo expuesto se representa:



Un puntero a una variable toma como valor, el valor L de la variable apuntada.

La siguiente tabla muestra el estado de la memoria después de la ejecución de las sentencias:

1000h		
1001h		
1002h	3	x
1003h		
:		
1005h		
:		
.		
1010	1002h	p
Dirección de memoria	Contenido	Identificador

Como el puntero p es también una variable, se almacena en memoria, en este caso en la posición 1010h.

Se debe tener en cuenta que, en la mayoría de los compiladores una variable puntero ocupa 4 bytes.

Formato:

En forma general, la declaración de un puntero es:

< tipo > * < nombre variable >

Donde: nombre variable es el identificador de la variable puntero, y tipo es el tipo de dato al que apunta el puntero.

Ejemplo

```
int x , *p;  
float puntajes[15] , *punt ;
```

Se declara la variable p y punt como punteros a variables entera y real respectivamente.

Es bueno destacar que una variable puntero es un tipo de dato simple, que puede apuntar a cualquier variable cuyo tipo sea simple o estructurado, tal como Arreglo, Struct, etc.

Operadores de punteros

Existen dos operadores que permiten trabajar con apuntadores: * y &.

Operador de dirección &

Este operador se utiliza para obtener la dirección de memoria de una variable.

&x se expresa “la dirección de memoria de x”.

Operador de indirección *

Este operador se aplica a una variable puntero y se utiliza para obtener el contenido de la posición de memoria apuntada por un puntero, es conocido también con el nombre de operador indirección o contenido.

***p** se expresa “el contenido de lo apuntado por p”.

Por tanto, en el ejemplo considerado, para acceder al valor de la variable x se puede utilizar directamente su nombre **x** o la expresión ***p**. Esto es, **x** es equivalente a ***p**.

NOTA: El operador * es unario, por lo que el compilador puede distinguir su aplicación respecto del operador * que representa la operación producto.

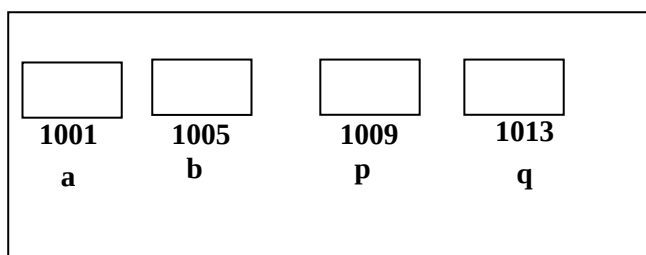
Ejemplo

El siguiente esquema de memoria permitirá visualizar los distintos pasos del algoritmo que se muestra:

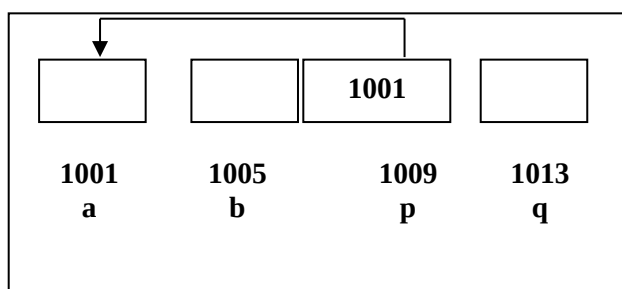
```
#include <stdio.h>  
void main (void )  
{  
  int a,b ,*p,*q;           1  
  p=&a;                     2  
  *p=8;                     3  
  q=&b;                      4  
  *q=23;                    5  
  printf(“%4d %4d \n”,a,b);  6  
  *p=*q+2;                  7  
  printf(“%4d %4d %4d %4d \n”,*p,*q,a,b);  8  
  p=q;                      9  
  printf(“%4d %4d %4d %4d \n”,*p,*q,a,b); 10  
}
```


Paso 1

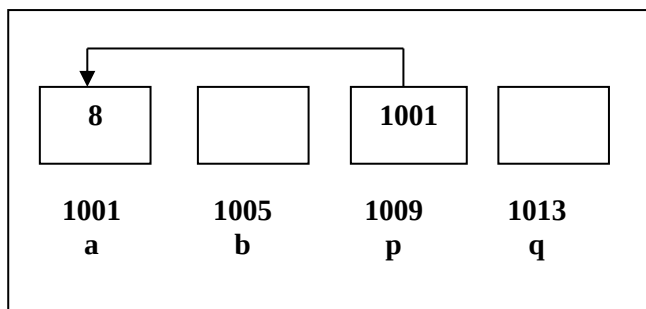
Se asigna espacio de memoria para las variables **a**, **b**, **p** y **q**.

**Paso 2**

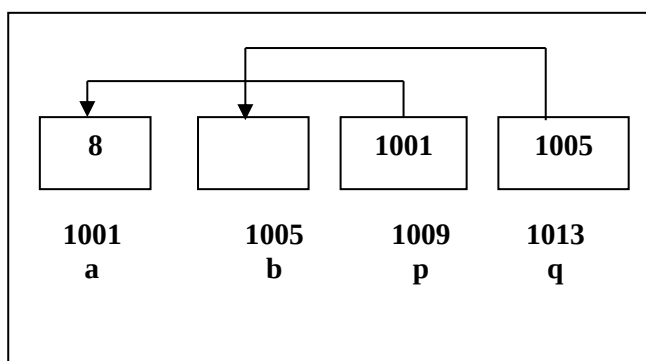
Se asigna al puntero **p**, la dirección de memoria de **a**, **p** apunta a la variable **a**.

**Paso 3**

Se asigna el valor 8 a la variable apuntada por **p**, ahora la variable **a** vale 8.

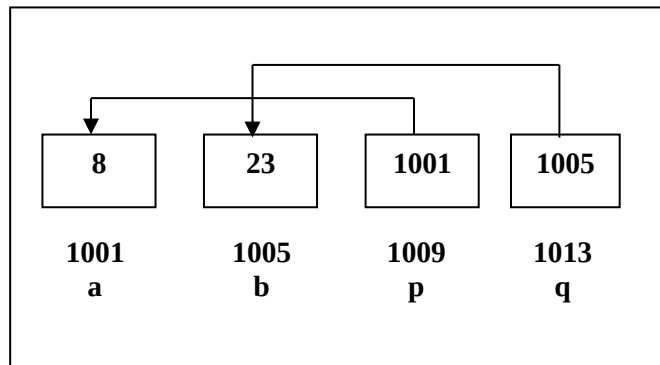
**Paso 4**

Se asigna al puntero **q** la dirección de memoria de la variable **b**.



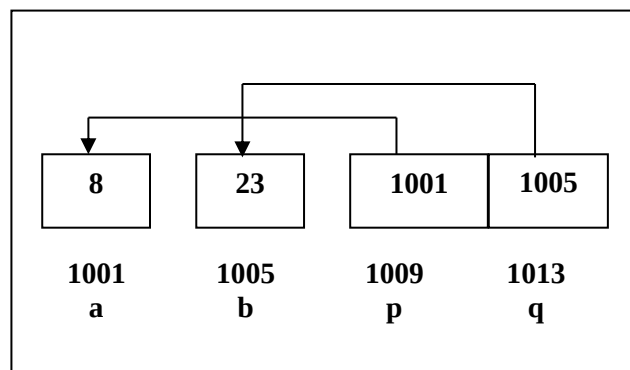
Paso 5

Se asigna el valor 23 a la variable apuntada por **q**,
b vale entonces 23.

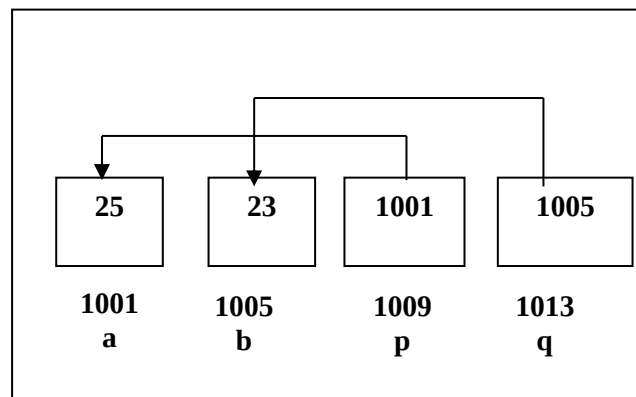
**Paso 6**

Muestra en pantalla el contenido de las variables **a** y **b**.

Salida: 8 23

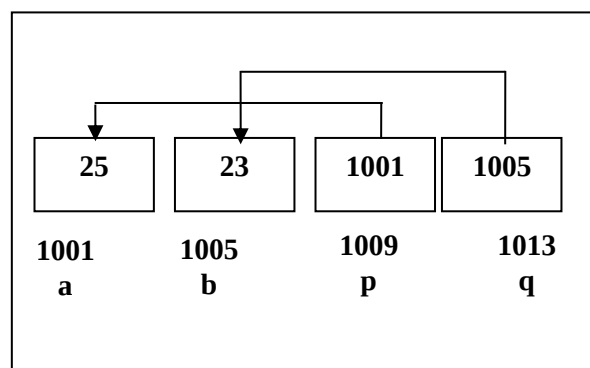
**Paso 7**

A la variable apuntada por **p** se le asigna el resultado de incrementar en 2 el valor de la variable a la que apunta **q**.
 La variable **a** vale ahora 25.

**Paso 8**

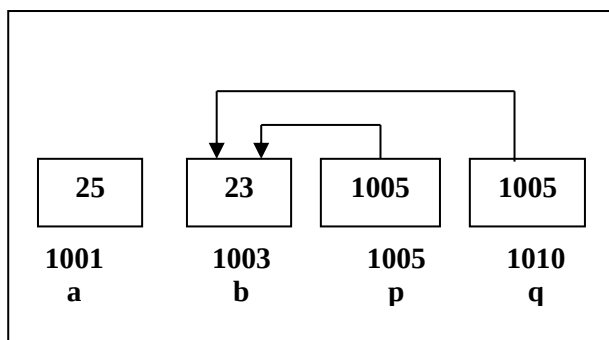
Muestra en pantalla el contenido de lo apuntado por **p**, **q** y el de las variables **a** y **b**.

Salida: 25 23 25 23

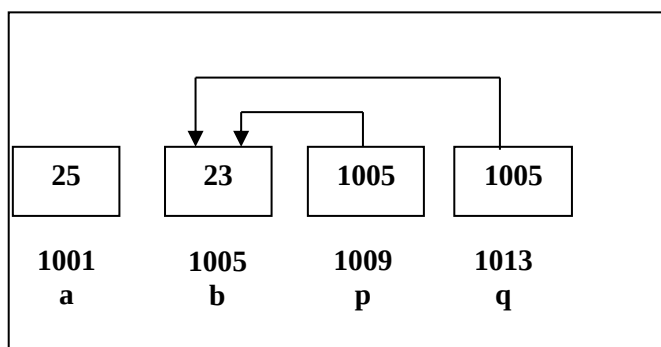


Paso 9

Al puntero **p** se le asigna el valor del puntero **q**. Ambos apuntan ahora a la misma dirección de memoria, la variable **b**.

**Paso 10**

Se muestra por pantalla los contenidos de lo apuntado por los punteros **p** y **q** y el contenido de las variables **a** y **b** respectivamente.

**Salida:**

23 23 25 23

Asignación de valores a punteros

Existen tres formas para la asignación de valores a variables punteros: a través del operador **&**, por medio de otro puntero, o con una dirección de memoria constante.

- Por medio del operador &**

Como se ha visto, este operador permite asignar al puntero la dirección de memoria de una variable.

Ejemplo

```
char c , * punt;
```

```
punt = &c;
```

- Por medio de otro puntero**

A un puntero se puede asignar el valor de otro puntero que apunte al mismo tipo de dato.

Ejemplo

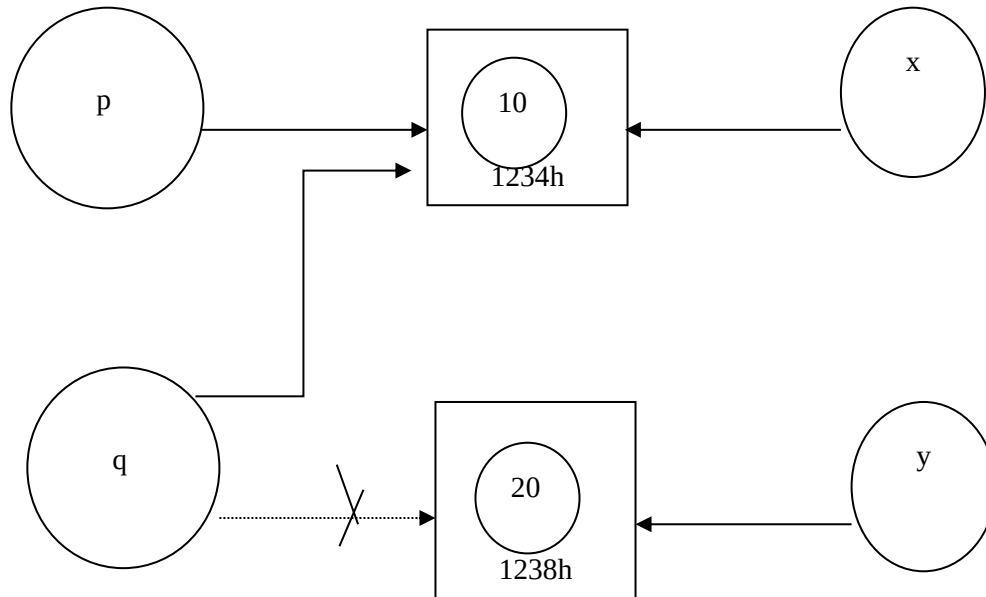
```
int x=10, y=20, *p , *q;
```

```
q=&y;
```

```
p=&x;
```

```
q=p;
```

En este caso, al puntero **q** se le asigna el valor del puntero **p**, por lo tanto ambos punteros apuntan a la variable **x**.



Como puede verse, la asignación de punteros en lenguaje C tiene una semántica distinta de la antes vista. La asignación entre punteros produce que se copien las direcciones en vez de los valores apuntados. Este tipo de asignación se llama asignación por compartición, y como se verá más adelante, provoca efectos colaterales no deseados en la asignación de objetos de datos estructurados del tipo struct, cuando estos incluyen campos o miembros de tipo apuntador.

Existen otros casos en los cuales a un puntero se puede asignar otro puntero que resulta de invocar a una función. Por ejemplo, la función malloc, devuelve como resultado un puntero a un bloque de memoria solicitado en forma dinámica. Este resultado puede ser asignado a otro puntero. Esto se trata con más profundidad en la unidad 5.

Con una dirección de memoria constante

Cuando esto ocurre, al puntero se le asigna directamente la dirección de memoria a la cual va a apuntar. Por ejemplo si se asigna a un puntero la dirección de la memoria de vídeo o la parte baja de la memoria en donde se guardan datos referidos al sistema operativo como los muestra el siguiente ejemplo.

Ejemplo

```
int *pvar;
```

```
pvar=24A3;
```

Inicialización de variables punteros

Al igual que una variable de cualquier tipo, el lenguaje C no inicializa los punteros cuando se declaran, por lo tanto es necesario inicializarlos antes de usarlos. Los punteros pueden ser inicializados a NULL o a una dirección determinada.

NULL es una constante simbólica definida en el archivo de cabecera stdio.h. Se dice que un apuntador con el valor NULL apunta a ninguna dirección, esto es a ningún dato válido.

Un puntero puede ser inicializado con la dirección de una variable al momento de ser declarado, siempre que la variable esté previamente declarada.

En caso que no haya sido inicializado, un puntero quedará apuntando a una dirección de memoria desconocida en donde pueden existir datos. La alteración de estos datos puede producir resultados inesperados.

Comparación de punteros

Los punteros, al igual que las demás variables, se pueden comparar. La comparación es posible siempre que los punteros apunten al mismo tipo de datos.

Los operadores usados en la comparación, son los operadores relacionales conocidos:

== != <> <= >=

La comparación de punteros es válida siempre y cuando respete una lógica. Existen tres formas para comparar punteros:

- Dos punteros entre sí.
- Un puntero con la dirección de memoria de una variable, expresada por medio del operador &.
- Un puntero con una dirección de memoria dada directamente.

Ejemplo

```
#include <stdio.h>
void main (void )
{
    int *pa, *pb , x;
    :
    if ( pa ==&x )          /* comparación de un puntero con una dirección de variable */
        printf("El puntero está apuntando a la variable x");

    if ( pb != pa )         /* comparación entre dos punteros */
        printf("Los punteros apuntan a posiciones distintas");

    if ( pa >f041 )         /* comparación con una dirección determinada */
        printf ("El puntero esta direccionado después de la posición f041");
}
```

Punteros NULL y void

Los punteros nulo (NULL) y genérico (void), son dos tipos de punteros muy utilizados.

El puntero NULL no direcciona ningún dato válido en memoria, mientras que el puntero void o puntero genérico es un puntero que apunta a cualquier tipo de datos. Dicho de otro modo, contiene la dirección de un dato de un tipo no especificado.

Este tipo de datos apuntador, será estudiado con más detalle cuando se trabaje con asignación dinámica de memoria.

Tipos de datos estructurados

Un objeto de datos que está construido por un agregado de otros objetos de datos, llamados componentes, se conoce como un objeto de datos estructurado o una estructura de datos. Algunos objetos de datos estructurados los define el programador y otros los define el sistema durante la ejecución del programa. Son ejemplos de estructuras de datos: arreglos, registros, cadenas de caracteres, listas, conjuntos y archivos.

Dado un grupo de tipos básicos como int , double y char, todo lenguaje ofrece diversas maneras para construir tipos más complejos, basándose en los tipos básicos; estos mecanismos se conocen como **constructores de tipo** y los tipos creados se conocen como **tipos definidos por el usuario**. Uno de los constructores de tipos más comunes es el arreglo.

Especificación de tipo de datos estructurados

Los atributos principales que se contemplan para especificar estructuras de datos son:

Número de componentes: Las estructura pueden ser de tamaño **fijo** o tamaño **variable**. Cuando es de **tamaño fijo** el número de componentes es invariable durante su tiempo de vida (arreglos y registros). Si el número de componentes cambia durante su tiempo de vida la estructura es de tamaño variable. Estas estructuras usan generalmente un puntero o apuntador para vincular las componentes. (Ej. Pilas, Listas, Archivos, etc.)

Tipo de componente: Si la estructura tiene componentes del mismo tipo de datos, se dice que es **homogénea** (Ejemplo Arreglos y Archivos). Caso contrario se dice que son estructuras **heterogéneas** (Ejemplo Registros).

Nombres que se debe usar para seleccionar una componente: Las estructuras de datos necesitan un mecanismo de selección para identificar componentes individuales de la estructura. En el caso de los arreglos, el nombre que lo identifica puede ser definido por el programador, mientras que una componente individual se identifica través de un subíndice. Para el caso de registros, generalmente el nombre que lo identifica lo define el programador. Para archivos por ejemplo, sólo se puede acceder a un componente particular, por ejemplo el componente apuntado en un momento determinado.

Organización de las componentes

1. Organización Unidimensional: serie lineal de componentes. (Ejemplo: arreglo unidimensional, registro, cadena de caracteres, listas, etc.)
2. Organización Multidimensional: Esta organización es similar a una serie lineal de componentes, donde cada componente es una estructura de datos de un tipo similar.
(Ejemplo: arreglo multidimensional, registro cuyos componentes son registros, listas cuyos componentes son listas, etc.)

Operaciones sobre estructuras de datos

Hay operaciones que son propias de las estructuras, algunas de las cuales son importantes de destacar. En muchas ocasiones, el procesamiento de una estructura implica la recuperación de cada una de las componentes de la estructura. El acceso a una componente puede hacerse de dos maneras distintas:

- **Operación de Selección Directa:** permite el acceso arbitrario a una componente de una estructura.
- **Operación de Selección Secuencial:** en este caso, el acceso a una componente se realiza en un orden predeterminado.

Por ejemplo, al trabajar con un arreglo en lenguaje C,
`int a[10];` la selección directa de la tercera componente se hace a través de la operación de subindización
`a[2],`

y a través de la subindización y el uso del `for`, se puede seleccionar una serie de componentes
`for(i=0; i<10, i++)`
`.....a[i];`

Operaciones sobre una estructura de datos completa

Forman un conjunto limitado de operaciones, dependiendo del lenguaje. Por ejemplo, lenguaje Pascal provee un tipo de datos estructurado Set (conjunto) y dispone para este tipo de datos operaciones de union (+), intersección (*) y diferencia de conjuntos.

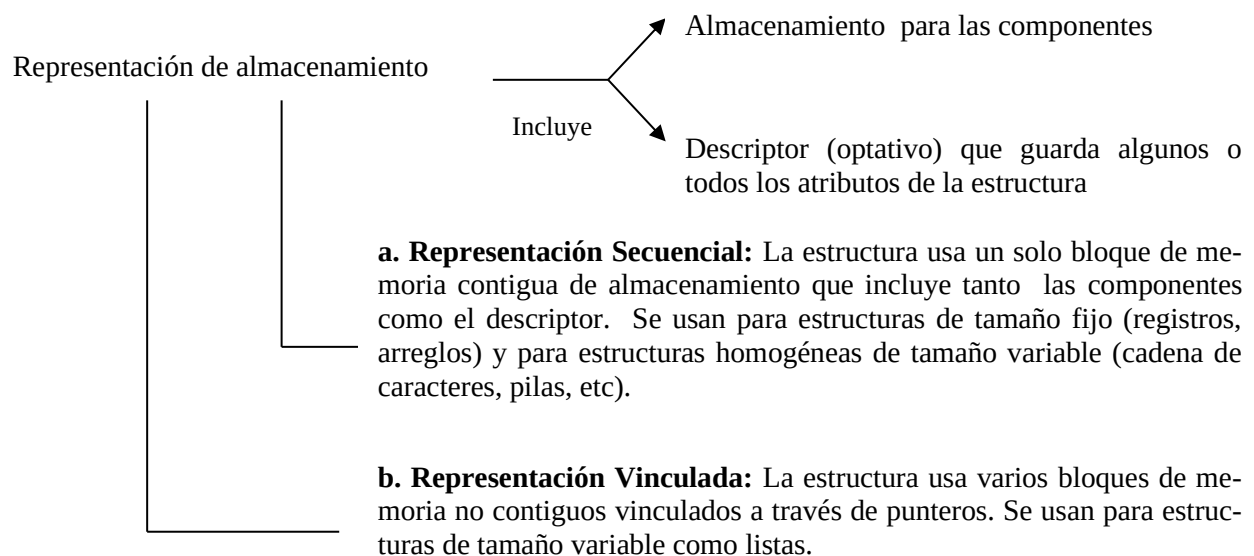
En este sentido, lenguaje C, provee la operación de asignación de registros, como operación sobre una estructura de datos completa. Otras operaciones, como la asignación de arreglos de igual dimensión, la suma de arreglos de igual dimensión, etc. deben ser definidas a través de funciones.

En lenguaje C, como se analizará más adelante, la asignación entre registros es válida siempre y cuando las componentes no sean punteros.

- *Inserción/ Eliminación de componentes:* Estas operaciones tienen un gran impacto sobre la representación de almacenamiento y la gestión de almacenamiento.
- *Creación/destrucción de estructuras de datos:* también tiene impacto sobre la gestión de almacenamiento.

Representación de tipos de datos estructurados

Se deben atender cuestiones que afectan fuertemente la elección de representaciones de almacenamiento: la selección eficiente de componentes de una estructura de datos y la gestión global eficiente del almacenamiento de memoria.



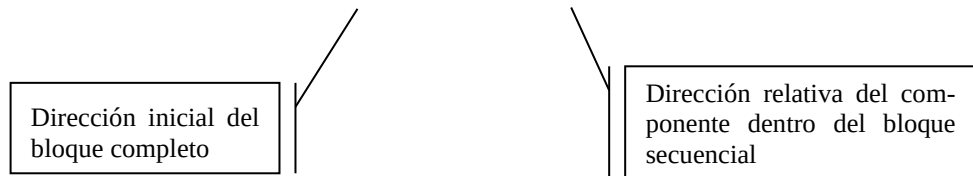
Importante: Casi todo el hardware carece de recursos para descriptores de estructuras de datos, manipulación de representaciones vinculadas, gestión de almacenamiento de estructura de datos y facilidades para el manejo de archivos externos. Por lo tanto las estructuras de datos y las operaciones que se realizan con ellas, deben simularse por software en la implementación del lenguaje de programación. Acá se observa una gran diferencia con los tipos de datos elementales en los cuales generalmente la representación de almacenamiento y las operaciones son manejadas por hardware.

Implementación de operaciones sobre estructuras de datos

Es importante la eficiencia tanto en la selección directa como en la selección secuencial

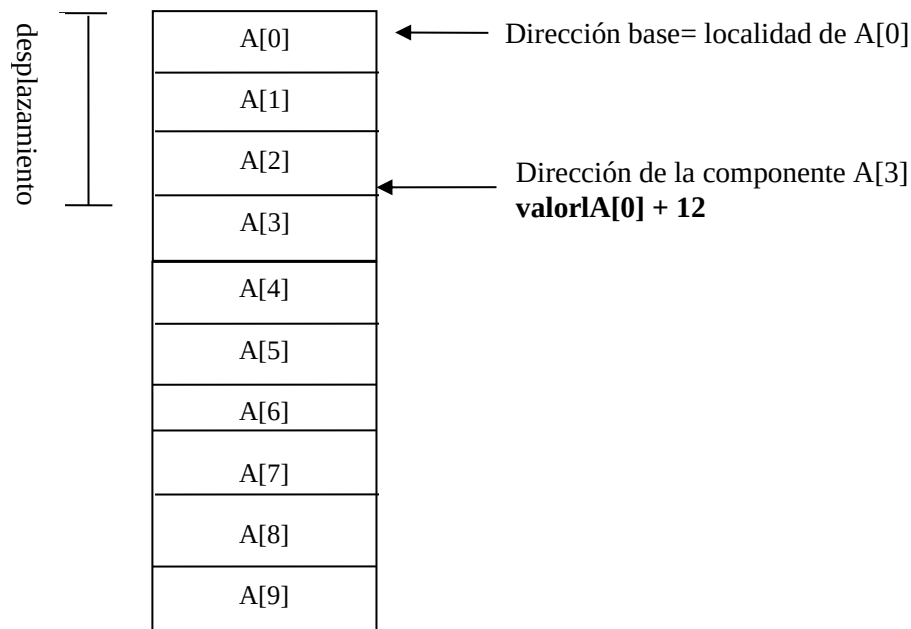
a) Representación secuencial: Para estructuras con representación secuencial la selección directa de un componente se calcula de la siguiente fórmula:

$$\text{Dirección de la componente} = \text{Dirección base} + \text{Desplazamiento}$$



Ejemplo

Sea en el lenguaje C, el arreglo `int A[10]`; se almacena de la siguiente forma:



Acceso a la componente A[3] en un arreglo de enteros en lenguaje C

b) Representación Vinculada: En este caso la selección directa implica seguir una cadena de punteros desde el primer bloque de almacenamiento de la estructura hasta la componente deseada. Este tipo de representación se analiza cuando se estudian listas enlazadas por punteros.

Declaraciones y verificaciones de tipo

Los conceptos y consideraciones a tener en cuenta sobre declaraciones y verificación de tipo para estructuras de datos, son similares a las expuestas para objetos de datos de tipo simple, no obstante las verificaciones son más complejas, pues son más los atributos a verificar.

Las declaraciones aportan varios atributos de la estructura. Por ejemplo, en la siguiente estructura,

int A[10][15]; los atributos son:

- tipo de datos: es un arreglo
- cantidad de componentes: 150
- dimensiones: 2
- tipo de datos de cada componente: entero

Cuando se hace la verificación de tipos la ruta a seguir para la verificación de una componente suele ser compleja. Además, aunque la verificación de tipo sea correcta en tiempo de compilación, nada nos asegura que un determinado desplazamiento sea válido en la estructura. En tiempo de ejecución, en alguna operación puede querer accederse a una componente inexistente y si por razones de eficiencia esto no se válida, entonces se produce un error de verificación de tipos.

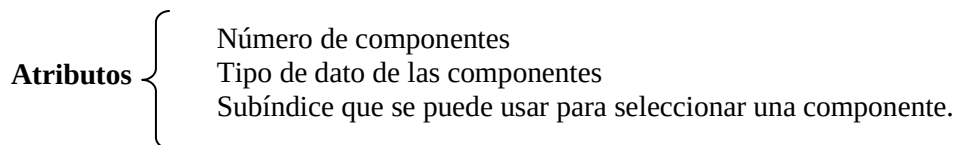
Clasificación de los tipos de datos estructurados

Los tipos de datos estructurados se clasifican en dos grandes grupos:

- **Tipos estáticos:** aquellos en los que la cantidad de componentes de la estructura esta predeterminada. Esto significa que al declararse un variable de este tipo, se reserva un espacio de memoria fijo que no podrá modificarse durante la ejecución del programa. Entre estos se encuentran: Arreglo (uni, bi y multidimensional), Registro y Archivo¹³.
- **Tipos dinámicos:** estos tipos tienen una cantidad de componentes que no está predeterminada al momento de su declaración. Esto significa que al declararse una variable de este tipo, se está reservando un espacio de memoria que se podrá modificar (aumentar o disminuir) durante la ejecución del programa. Entre estos se encuentran Listas, Árbol, Grafo, etc.¹⁴

Arreglos Unidimensionales

Un arreglo unidimensional (también llamados vectores) es una estructura de datos formada por un número fijo de componentes del mismo tipo de datos, organizadas como una serie lineal simple.



El arreglo es uno de los constructores de tipo más comunes.

La definición `int a[10]` crea una variable cuyo tipo es un “arreglo de enteros”.

El constructor arreglo toma el tipo base `int`, así como un tamaño (rango) y construye un tipo de datos nuevo. Esta construcción, que no tiene asociada un nombre (tipo anónimo), puede considerarse como una construcción implícita de un nuevo conjunto de valores, que se describe indicando la forma en que los valores pueden ser manipulados y representados.

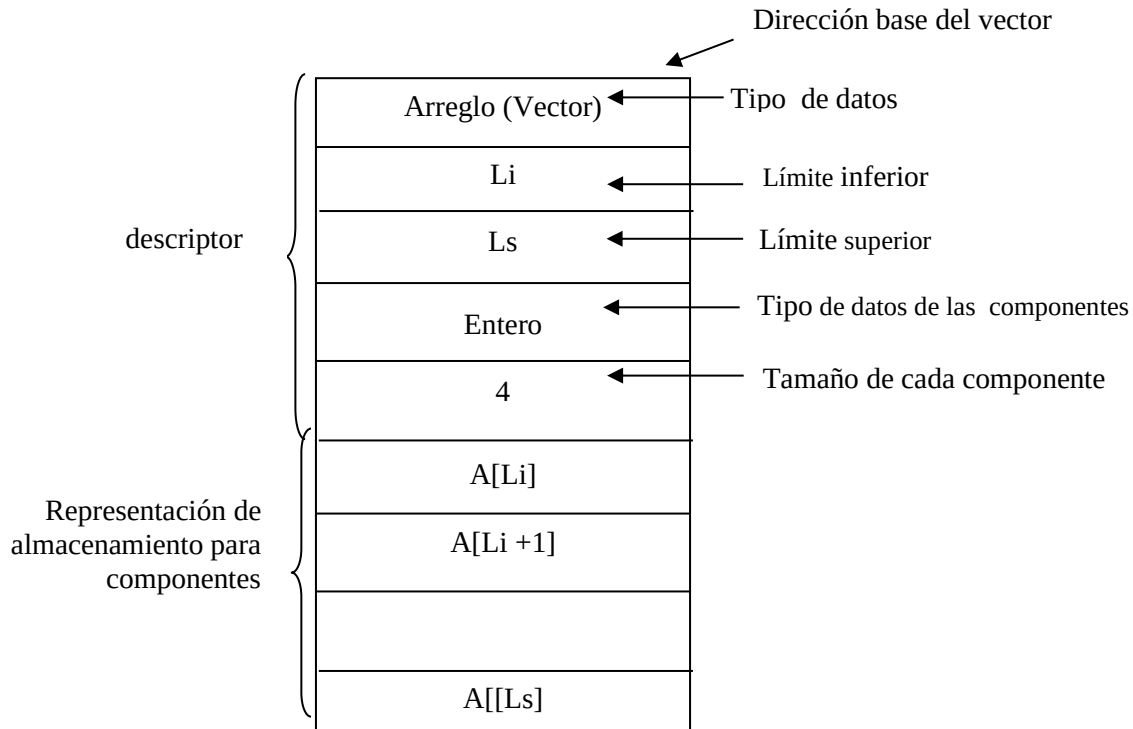
¹³ Tipo archivo se verá en la unidad 6.

¹⁴ En este curso solo se verá el tipo arreglo dinámico y lista, que se desarrollará en la unidad 5

Dado que el nombre de un tipo es de gran importancia para la verificación de tipos, para darle un nombre se utilizará typedef, como se verá más adelante.

Para los arreglos unidimensionales (vectores), la **representación secuencial de almacenamiento es la apropiada**. Se puede incluir un descriptor con algunos o todos los atributos, si esta información no se conoce hasta el tiempo de ejecución. En este descriptor también se puede guardar los límites inferior y superior del intervalo de subíndices que pueden usarse para acceder a un componente.

Una representación de almacenamiento completa de un arreglo unidimensional (vector) es la siguiente:



En general, Li y Ls se guardan si se quiere realizar verificación de subíndices. Los otros atributos no se guardan en tiempo de ejecución, sólo se necesitan durante la traducción para la verificación de tipos y para establecer la representación de almacenamiento.

Operaciones sobre arreglos (vectores):

La operación de selección de una componente se llama subindización y se escribe con el nombre del arreglo seguido del subíndice que se desea seleccionar.

Puesto que los vectores son de tamaño fijo no se incluyen operaciones de inserción ni de eliminación.

En lenguaje C los subíndices son enteros, lenguaje Pascal por lo contrario, admite una gran variedad de subíndices que no tienen porqué ser enteros o subrango de enteros.

Hay lenguajes que permiten ciertas operaciones entre vectores, pero éstas pueden requerir almacenamiento temporal, y la gestión de este almacenamiento puede aumentar la complejidad y el costo de ejecución del programa.

Importante: La operación de selección de un componente o valor de datos de un objeto se debe distinguir de la operación de *referenciamiento*.

Para `int V[10];` la selección de `V[5]` incluye dos pasos, a saber:

1. operación de *referenciamiento* que devuelve la dirección de V, valor L o Lvalue de V.
2. operación de selección que devuelve un apuntador a la localidad de ese componente

Implementación

En general es sencilla, debido a la homogeneidad de las componentes y el tamaño fijo de la estructura.

La homogeneidad

↓
Implica que la estructura y el tamaño de cada componente es el mismo

el tamaño fijo

↓
Implica que el número de componentes y la posición de cada una de ellas es invariante a lo largo del tiempo de vida del vector

Fórmula de acceso para una componente

Un arreglo en memoria siempre se almacena en posiciones contiguas a partir de la dirección base de comienzo del arreglo, la cual se obtiene cuando se declara una variable de este tipo (valor L).

El lenguaje C no necesita descriptores para los arreglos puesto que los índices siempre comienzan en cero, además el resto de la información no se necesita en tiempo de ejecución, ya que la misma se enlaza durante la traducción.

Pascal, permite definir arreglos donde los índices pueden tomar, por ejemplo, un subconjunto de valores enteros. En este lenguaje, el ámbito de subíndices deberá fijarse en tiempo de compilación de modo que puedan llevarse a cabo todos los cálculos de dirección en esta etapa, y por lo tanto no necesite descriptores durante la ejecución del programa.

En lenguaje C, **int a[10]**, declara un arreglo a de 10 componentes de tipo entero, donde los índices varían de 0 a 9.

En lenguaje Pascal, **a: Array [-3..6] of integer;** declara un arreglo a de componentes de tipo entero, donde los índices varían de -3 a 6.

Cálculo de la dirección en la que comienza a almacenarse un determinado elemento de un arreglo unidimensional:

Sea α la dirección del primer componente del vector, entonces la dirección de la componente $A[I]$ se calcula:

$$\text{Dir}(A[I]) = \text{ValorL}(A[I]) = \alpha + (I - L_i) * \text{Tamaño de la componente}$$

En lenguaje C, $L_i=0$, si llamamos E al tamaño de la componente, la dirección de la componente i-ésima se calcula con la siguiente fórmula: $\text{ValorL}(A[I]) = \alpha + I * E$

Como puede inferirse, en lenguaje C, el Valor L ($A[I]$) es una constante que se puede calcular en tiempo de traducción y esto hace el acceso a vectores mucho más rápido.

Luego, para el ejemplo en lenguaje C **int a[10]**, si la dirección α es 1000h, entonces la dirección de la componente $A[3]$, se puede calcular como:

$$\text{ValorL}(A[3]) = 1000 + 3 * 4 = 1012h$$

Arreglos bidimensionales y multidimensionales

A partir del concepto de vector, como un arreglo de una dimensión, puede definirse vectores de dos dimensiones (filas y columnas), de tres dimensiones (planos, filas, y columnas), y así sucesivamente.

Especificación de un arreglo multidimensional: La especificación difiere respecto a la de un arreglo unidimensional, en que para cada dimensión debe especificarse un intervalo de subíndices.

int A[10][20] declara en lenguaje C, un arreglo bidimensional de 10 filas y 20 columnas. Los índices varían de 0..9 para las filas, y de 0..19 para las columnas. A este tipo de arreglos se les llama tabla o matriz.

Var A: array[3..12, -5..14] of integer; declara en lenguaje Pascal, un arreglo bidimensional de 10 filas y 20 columnas. Los índices varían de 3..12 para las filas, y de -5..14 para las columnas.

Implementación

En general, un arreglo de cualquier dimensión está organizado o tiene una representación en *orden por filas*. Esta representación implica que el arreglo se divide en subvectores para cada elemento del intervalo del primer subíndice, luego cada uno de estos subvectores se divide en subvectores para cada elemento del intervalo del segundo subíndice, y así sucesivamente.

Para el caso de un arreglo bidimensional int A[10][20], el arreglo A es un vector que tiene 10 subvectores de 20 componentes cada uno de ellos. A es un vector de vectores.

Una representación menos usada es la de orden por columnas, en la cual la matriz es considerada una sola fila de columnas.

El **tamaño** que ocupa el arreglo en memoria es el producto del número de sus elementos por el tamaño de cada uno de ellos.

En int A[10][20]; el tamaño es 4 byte * 200 elementos = 800 bytes

En el caso de un arreglo bidimensional, las componentes también se seguirán almacenando en posiciones contiguas de memoria y la forma en la que están almacenados esos elementos va a depender de que el arreglo se almacene fila a fila (orden por filas) o columna a columna, que también se conoce como almacenamiento de orden por columnas.

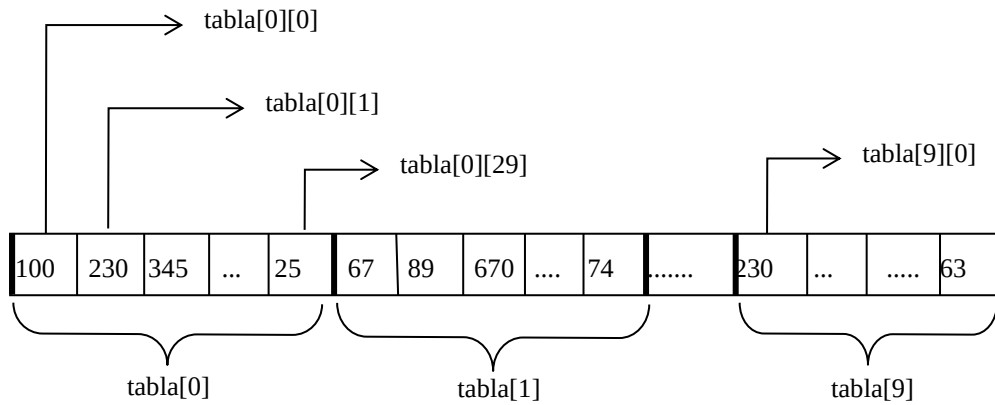
Por ejemplo, la siguiente tabla muestra el total de votos escrutados para 10 partidos políticos en 30 distritos.

		Distritos					
		0	1	2			29
Partido	0	100	230	345			25
	1	67	89	670			74
	2						
	3						
	8	811					125
	9	230					63

int Tabla[10] [30] permite declarar un arreglo bidimensional en lenguaje C, para almacenar la información de la tabla.



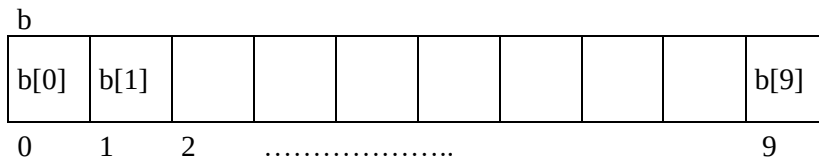
La información se almacenará en posiciones contiguas de la siguiente forma:



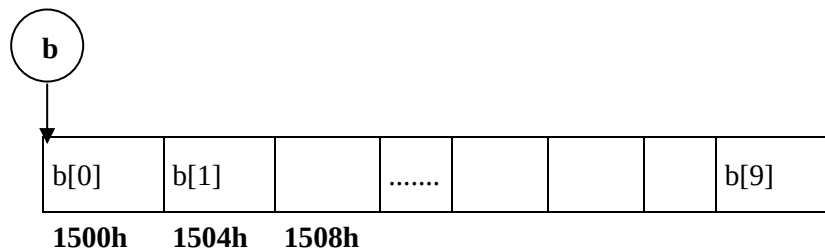
Arreglos y punteros en lenguaje c

En C existe una fuerte relación entre punteros y arreglos.

Hasta ahora para definir un arreglo **b** de 10 componentes enteras, hemos usado la expresión `int b[10]`, que permite reservar un bloque fijo de 40 bytes, cuya representación interna es la siguiente:



Teniendo en cuenta que el nombre de un arreglo es un puntero constante que contiene la dirección del primer elemento del mismo, **b almacena una constante, la dirección de b[0]**.



Por lo tanto la dirección del primer elemento del arreglo puede expresarse como **&b[0]** o simplemente **b**, la dirección del segundo elemento es **&b[1]** ó **b+1**

En general la dirección del elemento ubicado en la posición **i** del arreglo, puede expresarse como **&b[i]** ó **b+i**

Esta estrecha relación entre indexación en un arreglo y aritmética de punteros, permite acceder a la dirección de un elemento de un arreglo de dos maneras diferentes:

- escribiendo el elemento precedido por un **&**. Por ejemplo **&b[i]**.
- escribiendo el nombre del arreglo más una cantidad entera **i**. Por ejemplo **b+i**. Esta última expresión representa la dirección del elemento que está **i** posiciones después del primero. El valor de **i** se conoce como desplazamiento (offset).

Si **&b[i]** y **b+i** representan la dirección del elemento del arreglo ubicado en la **i**-ésima posición, entonces el acceso al contenido de esas direcciones se realiza con **b[i]** o ***(b+i)**

Por lo tanto, las siguientes expresiones son equivalentes:



Actividad 1

Si **b** un arreglo de 10 componentes enteras, completar con expresiones equivalentes.

***(b+6)** equivale a

&b[6] equivale a

***(b+2) +16** equivale a

Ejemplo: Carga y lista un arreglo

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
const int N=5;
```

```
void carga (int x[], int lim)
```

```
{ int i;
```

```
    printf("\nCarga el arreglo con %d elementos\n ", lim );
```

```
    for(i=0; i < lim; i++)
```

```
        scanf("%d",&x[i]); // equivalente scanf("%d",x+i);
```

```
}
```

```
void lista(int x[], int lim)
```

```
{int i;
```

```
    printf("\n Componente Valor Direccion \n" );
```

```
    for(i=0; i<N;i++)
```

```
        printf("\n x[%2d] %10d %10x ",i ,x[i], x+i);
```

```
}
```

```
int main(void)
```

```
{ int a[N],i;
```

```
    carga(a, N);
```

```
    lista (a, N);
```

```
    for(i=0; i<N;i++)
```

```
        *(a+i)=*(a+i)*3;
```

```
    lista (a, N);
```

```
    printf("\n Valor del puntero %x ",a );
```

```
    printf("\n La direccion del primer elemento del arreglo es % x", &a[0] );
```

```
    getch();
```

```
}
```

Cadenas de caracteres en C

Una cadena de caracteres es un objeto de datos compuesto de una serie de caracteres.

Especificación y sintaxis: Existen distintos tratamientos de las cadenas, dependiendo del lenguaje¹⁵.

Cadena de longitud fija declarada en el programa.

Cadena de longitud variable hasta un límite máximo declarado en el programa.

Cadena de longitud no determinada.

En el caso del lenguaje C, el uso de cadenas de caracteres es muy particular. Este lenguaje no provee un tipo de datos cadena de caracteres. Al igual que en Pascal, utiliza arreglos de caracteres para almacenar cadenas, estableciendo por convención que el carácter nulo “\0” sigue al último carácter de la cadena. Es decir, cuando se guarda una cadena de caracteres en un arreglo, el traductor anexa el carácter nulo a la cadena.

El siguiente ejemplo muestra la forma en que se almacena una cadena de caracteres en un arreglo.

```
void main(void)
{
    char cad[13];
    gets(cad); /* suponer que por teclado se ingresa la cadena LUNES 31 */
    puts(cad); /* muestra LUNES 31 */
}
```

En memoria, el almacenamiento de la cadena es el siguiente:

0	1	2	3	4	5	6	7	8	9	11	12
L	U	N	E	S		3	1	\0			

Para cadenas que responden a los casos 1 y 2 citados, la asignación de almacenamiento para cada objeto cadena de caracteres se realiza en tiempo de traducción. Para cadenas de longitud no determinada, donde las cadenas pueden tener cualquier longitud, y la misma puede variar durante la ejecución del programa, la asignación de almacenamiento es dinámica, por lo que debe hacerse en tiempo de ejecución. El lenguaje C también provee la posibilidad de crear cadenas dinámicas, como se analizará más adelante.

Uso de funciones de cadena de la biblioteca estándar – Lenguaje C

Para manipular cadenas, se utilizan funciones que se encuentran declaradas en el archivo de cabecera <string.h>, de la biblioteca estándar de C.

Las funciones cadena tratan a las cadenas como apuntadores a tipos de datos char, por lo tanto tienen argumentos declarados de la siguiente forma: char *s o bien const char *s lo cual significa que la función espera una cadena que puede o no modificarse.

Cuando se invoca a una función de este tipo, se puede especificar como argumento, el identificador de la variable declarada como arreglo de caracteres o como puntero a un carácter. En el caso que el argumento sea un arreglo de caracteres, en realidad C pasa la dirección del primer elemento del arreglo de caracteres.

¹⁵ Pratt, Terence y Marvin Zelkowitz. Lenguajes de Programación: Diseño e implementación.

La siguiente tabla muestra algunas de las funciones más usadas para manipular, comparar y buscar elementos en cadenas, con su correspondiente formato.

Función	Formato y Aplicación
strlen	Size_t strlen(const char * origen) Devuelve la longitud de la cadena
Asignación y concatenación de Cadenas	
strcpy	char * strcpy(char * destino, const char * origen) Copia la cadena origen en la cadena destino, el resultado es la cadena destino a la que agrega el carácter nulo.
strncpy	char * strncpy(char * destino, const char * origen, size_t n) Copia n caracteres de la cadena origen en la cadena destino, con el objeto de realizar truncamiento o relleno de caracteres.
strcat	char * strcat(char * destino, const char * origen) Agrega la cadena origen al final de la cadena destino, devuelve la cadena destino.
strncat	char * strncat(char * S1, const char * S2, size_t n) Agrega los n primeros caracteres de S2 a la cadena S1, devuelve S1.
Comparación de Cadenas	
strcmp	int strcmp(const char * S1, const char * S2) Compara alfabéticamente S1 y S2. El resultado es un número Positivo si S1 > S2 Negativo si S1 < S2 Cero si S1 = S2
stricmp	int stricmp(const char * S1, const char * S2) Es igual a strcmp, pero sin distinguir entre letras mayúsculas y minúsculas.
strncmp	int strncmp(const char * S1, const char * S2, size_t n) Compara S1 con la subcadena formada por los n primeros caracteres de S2. Al igual que strcmp devuelve un valor que puede ser positivo, negativo o cero.
Búsqueda de Caracteres y cadenas	
strrchr	char * strrchr(const char * S, int c) Devuelve un puntero a la última ocurrencia del carácter c en S , devuelve NULL si c no está en S . La búsqueda se hace en sentido inverso, desde el último al primer carácter.
strstr	char * strstr(const char * S1, const char * S2) Busca la cadena S2 en S1 y devuelve un puntero a los caracteres donde se encuentra S2 o NULL si no lo encuentra
Conversión de Cadenas	
strlwr	char * strlwr(char * S) Convierte las letras mayúsculas de la cadena S a minúsculas.
strupr	char * strupr(char * S) Convierte las letras minúsculas de la cadena S a mayúsculas.
Conversión de Cadenas a Números	
atoi	int atoi(const char * S) Convierte una cadena en un valor entero, devuelve cero en caso que no pueda realizar la conversión.
atol	long atol(const char * S) Convierte una cadena en un valor entero largo (long).
atof	double atof(const char * S) Convierte una cadena en un valor double en coma flotante.

NOTA

- Las funciones que permiten convertir cadenas en números se encuentran en la biblioteca `stdlib.h`
- El tipo **size_t** es equivalente, dependiendo del sistema, al tipo `unsigned long` o `unsigned int`.
- Como se observa en la tabla, algunos argumentos de las funciones aparecen precedidos de la palabra **const**, esto permite identificar rápidamente cuales parámetros son de entrada y cuáles de salida.

Así por ejemplo, en la función `strcpy`:

char * strcpy(char * destino, const char * origen)

La cadena origen sólo se utiliza para obtener los caracteres a copiar, ella no cambia, es la cadena destino la que se modifica. Aquellos datos que se utilizan como parámetros de entrada llevan la palabra `const`, no así los de salida.

- En general, las funciones manipulan los primeros caracteres de una cadena. Sin embargo, a veces es necesario trabajar con los últimos caracteres, en estos casos se recurre al uso de punteros, como lo muestra el ejemplo siguiente:

El siguiente algoritmo muestra como a partir de una cadena que contiene el nombre y apellido de una persona, se extrae la cadena que contiene sólo su apellido.

```
char c1 [30]="Darío Pérez" ;
char c2[30];
char *p= c1;
p+= 6;          /* apunta a Pérez*/
strcpy(c2,p);
puts(c2);
```

Registros

Especificación: Un registro es una estructura de datos compuesta por un número fijo de componentes de distinto o igual tipo. Por lo tanto, es una estructura de datos de *longitud fija*.

Diferencias entre un arreglo(vector) y un registro:

Los componentes de un registro pueden ser heterogéneos u homogéneos

Los componentes se designan con nombres simbólicos en lugar de indizarse con subíndices.

Los componentes de un registro se llaman campos o miembros.

Operación básica: La operación básica es la operación de selección.

Ejemplo en lenguaje C:

Declaración de tipo

```
struct empleados
{
    char nom[10];
    int edad;
}
```



Declaración de variable

empleados e;

En este caso se ha declarado o definido **el tipo de datos empleados** y es posible definir variables de ese tipo, en este caso **e**

e.nom indica la operación de selección del campo nom

Atributos de un registro

Número de componentes (en el ejemplo anterior, la struct empleados tiene 2 componentes)

Tipo de datos de cada componente (arreglo de caracteres y entero)

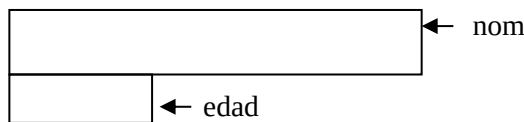
Selector que se usa para nombrar cada componente (en el ejemplo anterior nom y edad)

Implementación: Para la implementación de un registro se usa una representación secuencial de almacenamiento.

Las componentes individuales pueden requerir descriptores para indicar su tipo de datos u otros atributos, pero ordinariamente no se necesita un descriptor del registro en tiempo de ejecución.

La selección de una componente se implementa con facilidad pues los nombres de los campos se conocen en tiempo de traducción, en lugar de que se realicen los cálculos que en tiempo de ejecución, requieren los arreglos.

Para el ejemplo anterior, la representación en memoria es la siguiente:

**Variables del tipo struct en lenguaje C**

Una vez definido el tipo de dato struct (registro), se deben declarar las variables de la misma manera que se declaran las variables de tipos predefinidos.

El formato general de la **definición de un tipo struct** es:

struct <identificador><var 1>,...,<var n>;

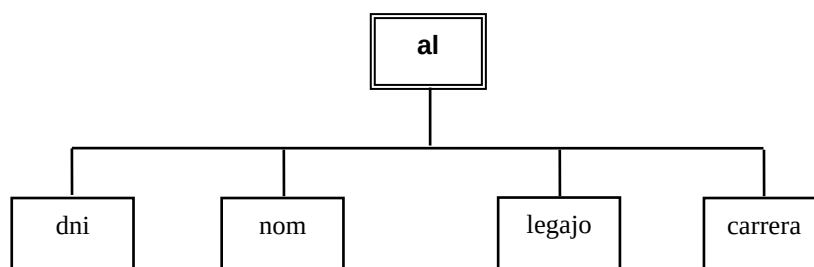
Si por ejemplo definimos la siguiente estructura:

```
struct alumno
{
    int dni;
    char nom[20];
    int legajo;
    char carrera;
};
```

la declaración es: **struct alumno al;**
esto se lee la variable **al** es del tipo **struct alumno**



La variable **al** identifica a toda la estructura y se puede representar gráficamente en función de sus miembros, como sigue:



También es posible declarar la variable así :

```

struct alumno
{
    int dni;
    char nom[20];
    int legajo;
    char carrera;
} al;
  
```

Ejemplo

La siguiente estructura permite almacenar información de un determinado partido político:

```

struct partido_politico
{
    int numero;
    int votos [19];
};
  
```

Como se puede observar los miembros de una estructura pueden ser variables simples o estructuradas.

Acceso a los miembros de una estructura

Los miembros de una estructura se procesan generalmente en forma individual. El acceso a cada miembro de una estructura se realiza utilizando el operador de miembro de estructura (**.**) también conocido como *operador punto*, de la siguiente manera:

variable . miembro

En esta expresión, también conocida como *selector de miembro de una struct*, **variable** es el identificador de la variable de tipo estructura y **miembro** es el nombre de uno de los campos de la misma.

Para la estructura:

```

struct alumno
{
    int dni;
    char nom[20];
    int legajo;
    char carrera;
};
  
```

```

struct alumno al;
  
```

al. legajo y **al.carrera** hacen referencia al legajo y carrera de un alumno respectivamente.

Para la estructura:

```
struct partido_politico
{
    int numero;
    int votos [19];
};
struct partido_politico m;
```

m. numero refiere al número del partido político y **m.votos[5]** refiere a los votos obtenidos en el sexto distrito.

Anidamiento de struct

Si un miembro de una struct es otra struct, se está haciendo referencia a un anidamiento de structs.

Ejemplo

La siguiente estructura permite almacenar los datos de la cuenta corriente de un cliente de una determinada empresa.

struct fecha

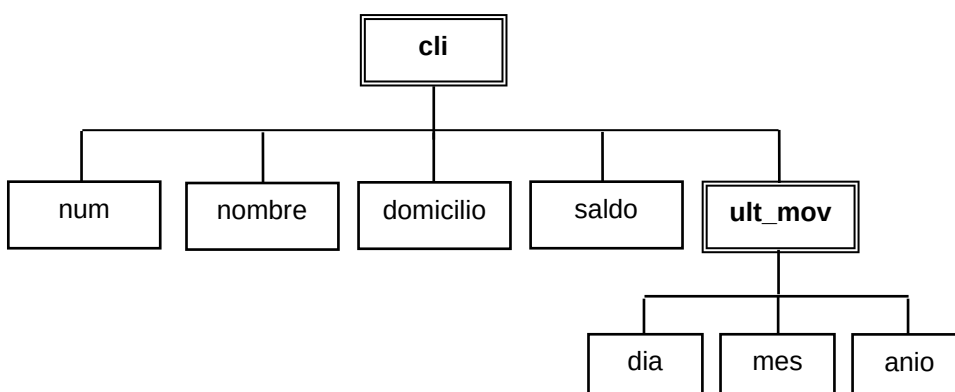
```
{
    int dia;
    int mes;
    int anio;
};
```

struct cliente

```
{
    int num;
    char nombre[20];
    char domicilio[30];
    float saldo;
    struct fecha ult_mov;
};
```

```
struct cliente cli;
```

Gráficamente



En este ejemplo, una estructura ha sido definida como miembro de otra estructura. En estas situaciones, la definición de la estructura interna debe preceder a la definición de la estructura externa que la contiene.

Actividad 2

Considerando la estructura del ejemplo, indicar como se accede al día, mes y año del último movimiento de un cliente.

```
struct fecha
{
    int dia;
    int mes;
    int anio; }

struct cliente
{
    int num;
    char nombre[20];
    char domicilio[30];
    float saldo;
    struct fecha ult_mov;
}
```

```
struct cliente cl;
```

Operaciones sobre estructuras

Las operaciones de **lectura y escritura** se deben efectuar individualmente para cada uno de sus miembros.

Para asignar valores a un miembro de una struct debe tenerse en cuenta que, una vez accedido mediante el selector, es tratado como cualquier otra variable del mismo tipo.

Sea **cl** una variable de tipo **struct cliente**, son correctas las siguientes expresiones:

```
scanf("%f",&cl.saldo);
scanf("%d",&cl.ult_mov.dia);
gets(cl.nombre);
gets(cl.domicilio);
```

para ingresar respectivamente al saldo, día del último movimiento, nombre y domicilio de un cliente.

Sea **p** una variable de tipo **struct partido_politico**, son correctas las siguientes expresiones:

```
printf("%d", p.numero);

for( i=0;i<19;i++)
    printf("%d", p.votos[i]);
```

para mostrar el número de un partido político y el total de votos obtenidos en cada uno de los 19 distritos.

Asignación de estructuras

La asignación entre estructuras está permitida si ambas son del mismo tipo y no contienen ningún miembro del tipo puntero. Entonces, dada la declaración:

```
struct alumno
{
    int dni;
    char nom[20];
    int legajo;
    char carrera;
};
```

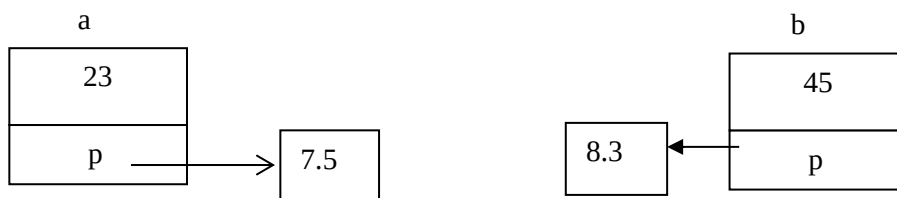
```
struct alumno a, b;
```

la asignación **a = b;** equivale a asignar el contenido de cada miembro de la variable b, a los miembros de la variable a.

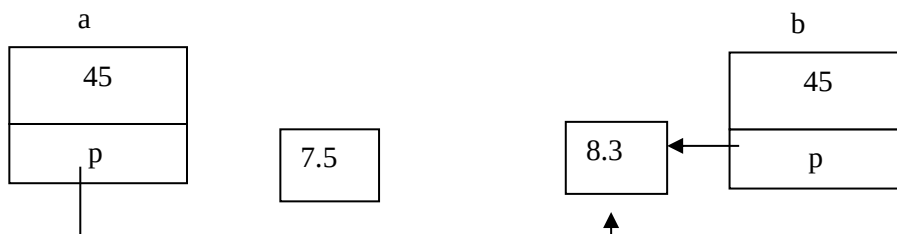
Para la declaración

```
struct datos
{
    int e ;
    float *p ;
};
struct datos a, b;
```

Supongamos en que en memoria se almacenan los siguientes valores para cada uno de los componentes de a y b.



La asignación **a=b;** provoca la siguiente representación:



Esto provoca un efecto colateral no deseado, ya que en el caso que se modifique el valor apuntado por p desde el registro a, esa modificación afectaría al valor original apuntado por p desde el registro b.

De ahí que, cuando un tipo struct define una estructura donde uno de sus componentes es un puntero, la asignación tal cual se la interpreta en el capítulo 2, no es una operación válida.

Actividad 3: Realizar el seguimiento de los siguientes algoritmos y escribir conclusiones.

a)

```
typedef struct
{
    char nom[10];
    int edad;
} empleados;

main()
{
    empleados a,b;
    strcpy(a.nom, "carlos");
    a.edad=27;
    b=a;
    a.edad=30;
    printf("Registro a nombre %s edad %d", a.nom, a.edad);
    printf("\n Registro b nombre %s edad %d", b.nom, b.edad);
    getch();
}
```

b)

```
typedef struct
{
    char nom[10];
    int* edad;
} empleados;

main()
{
    empleados a,b;
    int e=27;
    strcpy(a.nom, "carlos");
    a.edad=&e;
    b=a;
    *(a.edad)= 30
    printf("\n Registro b nombre %s edad %d", b.nom, *(b.edad));
    getch();
}
```

Observaciones:

El operador punto (.), pertenece al grupo de mayor precedencia, por lo que tiene prioridad sobre los operadores monarios (++,-,etc).

Así la expresión, **++variable.miembro** es equivalente a **++(variable.miembro)**, siendo miembro una variable numérica.

Esto significa que el operador ++ se aplica al miembro de la estructura y no a la estructura completa.

La expresión **&variable.miembro** es equivalente a **&(variable.miembro)**.

Esto significa que la expresión accede a la dirección del miembro de la estructura.

La asociatividad del operador '.' es de izquierda a derecha. Así, la expresión del tipo **variable.miembro.submiembro** es equivalente a **((variable.miembro).submiembro)**

Punteros a estructuras

Así como un puntero puede apuntar a un entero, a un char, también puede hacerlo a variables estructuradas, en particular a una variable struct.

Un puntero a una variable struct contiene la dirección de comienzo de la estructura o valor L de la estructura.

Así, para la definición de tipo,

struct alumno

```
{ char nombre[20];
  int nota;
};
```

se puede declarar las variables,

struct alumno a, *pstrc ;

donde a es una estructura del tipo alumno y pstrc es un puntero a una variable del tipo alumno.

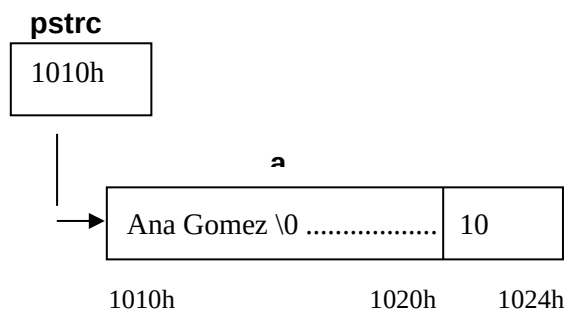
Las sentencias,

```
strcpy(a.nombre, "Ana Gomez");
a.nota=10;
```

asignan valores a cada miembro de la estructura.

Si el bloque de almacenamiento asignado en memoria para la estructura a, comienza en la dirección 1010h, entonces la asignación **pstrc=&a;** guarda en pstrc la dirección 1010h.

Gráficamente se pueden representar las variables:



Acceso a los componentes de una variable struct

Cuando se usan punteros a struct, el acceso a sus campos también se puede realizar utilizando el operador flecha -> (se forma con el signo menos seguido del símbolo mayor).

Por lo tanto, para acceder a través de un puntero a los miembros de una variable struct se pueden usar las expresiones:

(* puntero) . miembro	ó	puntero -> miembro
-----------------------	---	--------------------

Por lo tanto, para la estructura definida:

pstrc->nombre	o	(*pstrc).nombre,	acceden al miembro <i>nombre</i> de a.
pstrc->nota	o	(*pstrc).nota,	acceden al miembro <i>nota</i> de a.

Ejemplo

El siguiente algoritmo muestra la forma de cargar y mostrar una estructura a través del uso de punteros.

```
# include <stdio.h>
struct alumno
{   char nombre[20];
    int nota;
};

void main(void)
{struct alumno pers,*pstrc ;
  pstrc=&pers;
  printf("\n ingrese nombre ");
  gets( pstrc->nombre);      // equivale a gets((*pstrc).nombre)
  printf("\n Ingrese nota");
  scanf("%d",&(pstrc->nota));    // equivale a scanf("%d",  & (*pstrc).nota)
  puts( pstrc->nombre);
  printf(" obtuvo %d puntos", pstrc->nota);
  getchar();}
```

Para acceder a submiembros de una estructura, se deben usar los operadores -> y punto.

Dadas las siguientes definiciones y declaraciones:

```
struct fecha
{ int dia, mes, anio;
} ;
```

```
struct egresado
{char nombre[20];
  float promedio;
  struct fecha egreso; } ;
```

```
struct egresado egre, *p;
```

Realizada la asignación p=&egre;

Una de las formas de acceder al día, mes y año de egreso de un alumno egresado es:

```
p->egreso.dia
p->egreso.mes
p->egreso.anio
```

Actividad 4

Dada la siguiente estructura, que contiene los datos correspondientes a un cliente de una entidad bancaria:

```
struct domicilio;
{ char calle[30];
  int nro;
  cadena localidad;
} ;
```

```
struct cuenta
{
    int nrocli;
    char tipo;
    char nombre[30];
    float saldo;
    struct domicilio domi;
    float movi[10];
};
```

struct cuenta *pcli

Se pide:

Escribir la sentencia que permita mostrar el nombre de la calle correspondiente al cliente.

Si en el arreglo se han almacenado los 10 últimos movimientos realizados por el cliente, escribir la sentencia que permita mostrar la última operación realizada.

Bibliografía

- Lenguajes de Programación Diseño e Implementación. Pratt Terrence y Marvin Zelkowitz. Editorial Prentice Hall. Hispano americana, S.A. Segunda Edición. 1987.
- Programación Orientada a Objetos. Técnicas Avanzadas de Programación. Fontela, Carlos. Buenos Aires. Editorial Nueva Librería. 2003..
- Lenguajes de Programación Principios y Práctica. Loudon, Kenneth C. Mexico, D.F. Editorial Thomson. 2004.

Unidad 3: Funciones

Introducción

Así como un lenguaje base suministra un tipo de dato primitivo entero y operaciones sobre enteros que hacen innecesario que el programador se preocupe de los detalles de la representación subyacente de enteros como serie de bits, del mismo modo se puede definir un tipo nuevo de datos de más alto nivel y un conjunto de operaciones sobre dicho tipo de dato, sin que el programador necesite ocuparse de los detalles de la implementación del mismo.

El objetivo actual en los diseños de lenguajes de programación es que los programadores puedan crear tipos nuevos de datos y operaciones sobre ese tipo, de modo que pueda ser usado como un tipo de dato provisto por el lenguaje.

Existen distintos mecanismos básicos para crear tipos nuevos de datos y operaciones sobre esos tipos. Es propósito de este apartado, entender que construcciones proveen los lenguajes procedurales, en particular C, para definir nuevos tipos de datos y describir la funcionalidad de los mismos.

Definición de tipos

Una *definición de tipos* proporciona un *nombre de tipo* junto con una declaración que describe la estructura de una clase de objetos de datos. Así, el nombre del tipo se convierte en el nombre de esa clase de objetos de datos, y cuando se necesita trabajar con un objeto de datos particular basta con proporcionar el nombre del tipo en lugar de repetir la descripción completa de la estructura de datos¹⁶.

Dado un grupo de tipos básicos como int, char y float, todo lenguaje ofrece una variedad de formas para construir tipos más complejos, basándose en los tipos básicos. Estos mecanismos se conocen como *constructores de tipos*. Las *struct*, *union* y los *arreglos* son ejemplos de constructores de tipo en lenguaje C.

Los tipos creados por los constructores de tipos se llaman *tipos de datos definidos por el usuario*.

Ejemplo:

```
struct alumno
{   char nombre[20];
    int nota;
};
struct alumno arre[20];
int Tabla[10][30];
```

Los nuevos tipos creados con constructores no tienen automáticamente un nombre. Los nombres son importantes no sólo para documentar el uso de tipos nuevos sino para la verificación de tipos, de la cual ya se ha hablado antes.

Para asignar nombres a los nuevos tipos de datos se utiliza una *declaración de tipos*, en algunos lenguajes se denomina *definición de tipos*.

En lenguaje Pascal por ejemplo, si se necesita trabajar con varios registros que tienen una misma estructura, entonces el programador puede dar la definición de un nuevo tipo de datos a través de la palabra clave *Type*:

¹⁶ El concepto de definición de tipos fue evolucionando y las primeras definiciones de tipos surgen con Pascal en 1970.

Type partido_politico = record

```
Numero: integer;  
Votos: array[1..19] of integer;  
End;
```

seguida de la declaración **Var P1, P2: partido_politico;**

De este modo la definición de la estructura de un objeto de datos del tipo *partido_politico* se da una sola vez, en lugar de repetirla tres veces para P1, P2 y P3.

En lenguaje C, la situación es algo distinta, los nuevos tipos de datos sólo se proveen a través de la construcción **struct**, luego:

```
struct partido_politico {  
    int Numero;  
    int Votos[19];  
};
```

también con:

```
typedef struct {  
    int Numero;  
    int Votos[19];  
} partido_politico;
```

define un nuevo tipo de datos de nombre *partido_politico*.

La declaración de tres objetos de datos de esa estructura se realiza de la siguiente manera:

struct partido_politico P1, P2, P3;

Algunos compiladores aceptan directamente esta declaración:

partido_politico P1, P2, P3;

Una definición de tipo permite separar la *definición* de la estructura de un objeto de datos, de los puntos en los cuales se va a *declarar* el objeto.

Por ejemplo, en lenguaje C:

```
struct complejo                    // definición de la estructura  
{  
    float real;  
    float imaginario;  
}  
void main(void)  
{  
    int a, b;  
    :  
    complejo c1, c2;            // definición del punto donde se van a crear los objetos c1, c2  
    :  
}
```

Lenguaje C y typedef

En varias ocasiones puede ser útil definir nuevos nombres para tipos de datos, estos nombres o 'alias' nos hace más fácil la declaración de variables y parámetros. Para esto C dispone de la palabra clave **typedef**, cuyo formato es:

typedef <tipo> <identificador>;

Por ejemplo:

typedef int entero,

significa que se ha dado un nuevo nombre al tipo de datos int, lo cual permite declarar variables **entero** a, b como si entero fuera un tipo de dato.

El siguiente cuadro muestra el uso de la construcción typedef en lenguaje C.

En Lenguaje C:

```
typedef struct
{
    float real;
    float imaginario;
}complejo;
```

Luego la declaración de variables es:

complejo c1, c2;

En Pascal *complejo* define un nuevo tipo de datos, no obstante, existen diferencias entre el **type** de Pascal y el **typedef** de C.

En lenguaje C la construcción **typedef** no crea un nuevo tipo de datos. El efecto producido por **typedef** es el de una sustitución por macro.

En nuestro ejemplo, cuando en el compilador encuentra la declaración de variables *complejo c1,c2*; se sustituye *complejo* por **struct complejo**.

Cuando se necesite trabajar con un objeto de datos con esa estructura, basta con proporcionar el nombre definido "complejo", en lugar de repetir su declaración.

Par el caso de arreglos, typedef permite que la declaración de una variable **arre** como un arreglo de 20 enteros, pueda ser usada como si fuera un tipo de dato. Lenguaje C utiliza el *typedef* de la siguiente forma.

typedef int arre[20];

Luego la declaracion de variables es:

arre a,b ; //esta es la declaración de a y b como arreglos de 20 componentes enteras.
arre es usado como si fuera un tipo de datos.

Otros ejemplos de uso de typedef:

typedef char cadena[28];

```
main()
{
    cadena nombre="Blanca Flores";
    :
}
```

Sistema de tipos¹⁷

Los métodos utilizados para la construcción de tipos, el algoritmo de equivalencia de tipos, y las reglas de inferencia y corrección de tipos, se conocen de manera colectiva como *sistema de tipos*.

Si la definición de un lenguaje de programación especifica un sistema de tipos completo que pueda aplicarse estáticamente y que garantiza que todos los errores de corrupción se detectarán lo antes posible, entonces se dice que el lenguaje es *fuertemente tipificado*.

Pascal es un lenguaje fuertemente tipificado pese a que existen algunas fallas. No obstante, lenguaje C es un lenguaje que incluso tiene más fallas por eso a veces se lo conoce como un lenguaje débilmente tipificado. El lenguaje C++ ha eliminado muchas de las fallas de C, pero por razones de compatibilidad sigue siendo un lenguaje que no cumple con una tipificación fuerte.

Ventajas de la definición de tipos

1. Simplificación la estructura del programa.
2. Modificación más eficiente. Si se desea realizar cambios en la estructura definida, basta hacerlo una vez en la definición, en contraposición a lo que significaría la presencia de varias declaraciones del mismo tipo de datos.
3. En el uso de subprogramas, facilita el pasaje de argumentos, pues evita repetir la descripción del tipo de datos. En Pascal, esto es un requisito, pues los argumentos deben estar acompañados de un tipo de datos definido anteriormente. En lenguaje C, es una facilidad que se provee.

Subprogramas

La modularización, es una técnica usada en programación para hacer códigos más cortos y eficientes, consiste en reducir un problema complejo, en partes simples y sencillas (subproblemas), para luego buscar la solución por separado, de cada uno de ellos.

En C, se conocen como subprogramas o funciones aquellos trozos de códigos utilizados para dividir un programa con el objetivo de que cada uno realice una tarea determinada para resolver una parte del problema.

El uso de subprogramas definidos por el programador permite dividir un programa en un cierto número de componentes más pequeñas, cada una de éstas con un propósito específico y determinado.

Los subprogramas representan una operación abstracta definida por el programador.

Se sugiere el uso de subprogramas en distintas situaciones, a saber:

- Cuando un programa debe realizar reiteradamente una determinada tarea, el conjunto de sentencias que realiza dicha tarea se puede definir como un subprograma, al que se accederá las veces que sea necesario. Cada vez que se invoque ese subprograma, se podrá trabajar con un conjunto de datos distintos.
- Para lograr una mayor claridad y legibilidad en el programa principal, es óptimo definir subprogramas que realicen tareas específicas, aún cuando las mismas no se invoquen de manera repetida.

Por ejemplo:

```
#include <stdio.h>
int factorial (int a)
{
    int i, f=1;
    for(i=1; i <=a; i++)
        f*=i;
    return f;
}
```

¹⁷ Para entender mejor este concepto se sugiere leer Pág. 179 - Lenguajes de Programación. Louden. K.

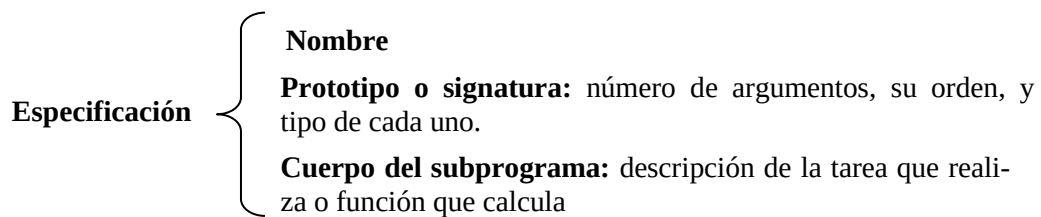
```

void main(void)
{
    int c;
    int m,n;
    printf(" ingrese m y n \n");
    scanf("%d",&m);
    scanf("%d", &n);
    c = factorial(m) / (factorial(n) * factorial(m-n)) ; // uso de la función factorial (invocación)
    printf("Combinaciones %d", c) ;
    getchar();
}

```

En este ejemplo, la función factorial es invocada tres veces.

Especificación de un subprograma



Ejemplo en lenguaje C:

```
#include <stdio.h>
```

typedef int vector[10]; → permite el uso del identificador vector como si fuera un tipo de datos

int escalar (vector a, vector b) → Operación producto escalar de vectores

```

{
    int e=0,i;
    for(i=0; i < 10; i++)
        e+=a[i]*b[i];
    return e ;
}

```

void carga(vector x) → Operación de carga del vector

```

{
    int i;
    printf(" \n ingrese las componentes del vector");
    for(i=0; i<10; i++)
        scanf("%d",&x[i]);
    return;
}

```

```

void main(void)
{
    vector v1, v2;
    carga(v1);
    carga(v2);
    printf(" el producto escalar es %d ", escalar(v1,v2));
    getchar();
}

```

En general, en los lenguajes procedurales se utilizan las siguientes denominaciones para los subprogramas:

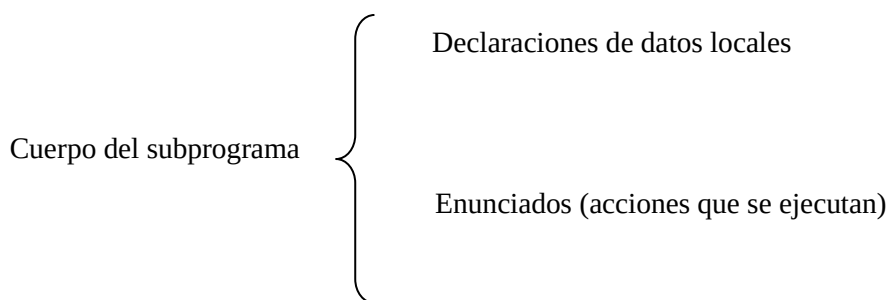
Si devuelve un resultado	→	función o subprograma
Si devuelve más de un resultado o modifica sus argumentos	→	procedimiento o subrutina

Implementación de un subprograma

Un subprograma es un conjunto de sentencias que realiza una tarea específica. Para su implementación se utilizan estructuras de datos y operaciones provistas por el lenguaje.

Un subprograma es una operación de la capa de una computadora virtual, construida por el programador.

La implementación está definida por el cuerpo del subprograma.



En **algunos lenguajes procedurales** el cuerpo puede incluir otras definiciones de subprogramas también encapsuladas, **en lenguaje C no es posible**.

En ambos casos, **la verificación de tipos es estática**, pues se provee el tipo de datos de los argumentos y del resultado.

En lenguaje C los subprogramas son funciones.

Es importante notar, que algunos lenguajes también proveen coerción de argumentos para transformarlos de manera automática al tipo de datos correspondiente.

Caso 1. Transforma un argumento int a float

```

float calculo( float a)
{
    int e=5; /* Declaraciones de datos locales */
    a= a +e; /* Enunciados (acciones que se ejecutan) */
    return a; /*Resultado*/
}

main()
{
    int n;
    scanf ("%d", &n);
    printf ("\n %f", calculo(n)); /* Invocación*/
    getch ();
}

Salida: 11.000000 para un valor de n=6
  
```


Caso 2. Transforma automáticamente un float a int

```

void calculo(int a)
{
    /* No hay declaraciones de datos locales */
    printf("Valor del argumento %d", a); /* Enunciados (acciones que se
                                         ejecutan) */
    return (); /*no hay resultado, es opcional colocar la sentencia
               return*/
}

main()
{
    float n=23.40;
    calculo (n); /* Invocación*/
    getch ();
}
  
```

Salida: Valor del argumento 23

Definición, invocación y activación de subprogramas

Uno de los problemas principales en casi todos los lenguajes, es el diseño de recursos para definir e invocar subprogramas.

La **definición de un subprograma** es una propiedad estática de lenguaje; en tiempo de traducción es la única información que está disponible. Por ejemplo: tipo de variables de los argumentos, de las variables locales, etc.

Un subprograma, desde el punto de vista de la programación, se define como un proceso que recibe valores de entrada (llamados parámetros) y el cual retorna un valor resultado. Se pueden invocar (ejecutar) desde cualquier parte del programa, es decir, desde otra función, desde la misma función o desde el programa principal, cuantas veces sea necesario.

La **activación de un subprograma** se genera, en tiempo de ejecución cuando se lo llama o invoca. Al terminar la ejecución, la activación se destruye.

Por lo tanto, la definición es una plantilla, que permite generar activaciones en tiempo de ejecución.

Dada la definición de la función cálculo:

```

float calculo (float f, int i)
{
    const float por=10.7

    float g;
    int A[10];
    .....
    .....
}
  
```

La traducción de la definición da como resultado una **plantilla**, a partir de la cual se puede construir una activación particular de la función cada vez que se ejecuta el subprograma.

Por cada activación esta *plantilla completa* se podría crear en una nueva área de memoria, sin embargo la plantilla se divide dos partes con el fin de ahorrar memoria.

La plantilla se divide en una parte fija y otra dinámica:

El **segmento de código** es la **parte estática** compuesta por las constantes y el código ejecutable generado a partir de los enunciados del *cuerpo de la función*.

El **registro de activación** es la **parte dinámica**, compuesta por:

- Parámetros
- Resultados de la función
- Datos locales
- Punto de retorno
- Áreas temporales de almacenamiento necesarias para mantenimiento
- Vinculaciones para referencias de variables no locales

El registro de activación siempre tienen la misma estructura por cada activación, lo que cambia son los valores de los datos almacenados.

Los registros de activación se crean cada vez que se invoca un subprograma y destruyen cada vez que el mismo concluye con un retorno.

El tamaño y estructura del registro de activación se determina en tiempo de traducción, es decir, el compilador determina cuantos componentes tendrá el registro de activación y la posición de cada uno de ellos en la estructura.

Por lo tanto, para crear un nuevo registro de activación, solo hace falta conocer el tamaño del bloque, no la estructura interna en detalle, pues los desplazamientos se computan durante la traducción y en tiempo de ejecución solo se requiere la dirección base para acceder a un componente del registro.

Prólogo – Epílogo

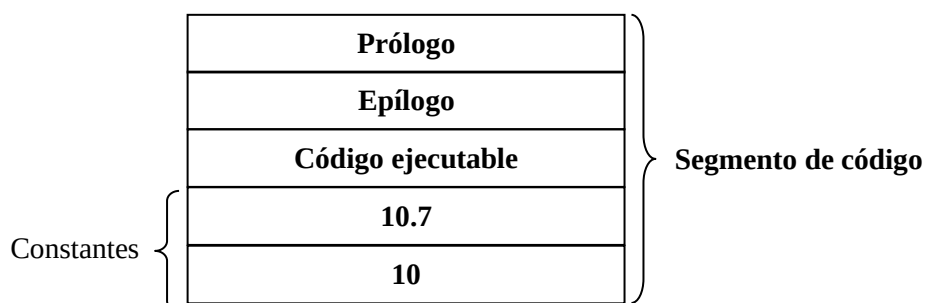
El **prólogo** es un bloque de código que el traductor introduce al comienzo del segmento de código, para permitir tareas de creación del registro de activación, transmisión de parámetros, creación de vínculos y actividades similares de mantenimiento.

El **epílogo**, al igual que el prólogo, es un conjunto de instrucciones que el traductor inserta al final del bloque de código ejecutable, para llevar a cabo acciones que permitan devolver resultados y liberar el almacenamiento destinado al registro de activación.

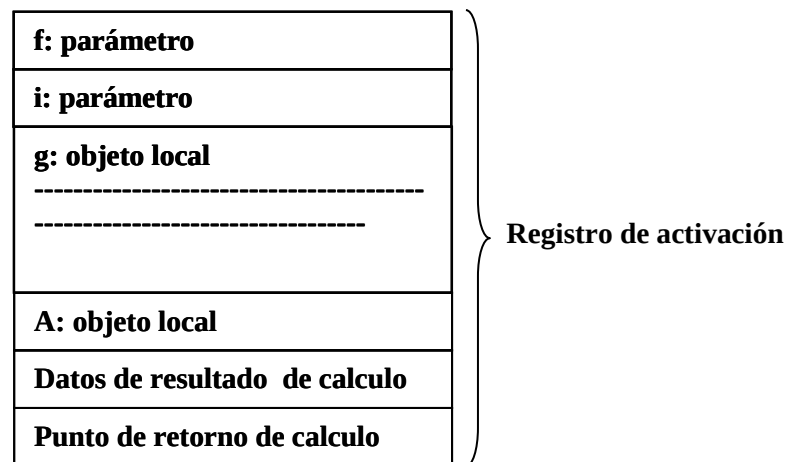
Por lo tanto, la estructura de una activación de la **función calculo**, antes definida es:

```
float calculo (float f, int i)
{
  const float por=10.7
  float g;
  int A[10];
  .....
  .....
}
```

En Memoria Estática (Área de Almacenamiento Dinámico):



En Memoria de Dinámica (Área Pila):



Funciones en lenguaje C

Así como el lenguaje C proporciona funciones de biblioteca para realizar distintas operaciones o cálculos frecuentes, también permite al programador definir y construir funciones que realicen tareas específicas.

El uso de funciones definidas por el programador permite dividir un programa en un cierto número de componentes más pequeñas, cada una de éstas con un propósito específico y determinado. Es por ello que se dice que un programa en C se puede *modularizar* a través del uso correcto de funciones. Al trabajar modularmente, descomponiendo un programa en varias funciones, se logra programas más fáciles de codificar y depurar.



Definición

Una función es un conjunto de sentencias que realiza una determinada tarea, que retorna como resultado cero o un valor.

Por ejemplo la función `getch()`, es una función predefinida que devuelve un único valor, un carácter.

```
char car;  
car=getch();
```

La función `clrscr()` es una función predefinida que no devuelve ningún valor.

Un programa en C está formado por una o más funciones, siendo *main* la función principal por donde se inicia la ejecución del programa.

Al invocar una función desde algún punto del programa, se ejecutan las sentencias que forman parte de ella. Una vez que finaliza la ejecución de esa función, el control se devuelve al punto desde donde fue invocada, es decir, el control vuelve al `main` (programa principal) o a la función que la llamó.

Esquemáticamente:

```

void main (void)
{
  int p;
  leer();
  p=calcular();
}

leer();
calcula();

```

Si un programa contiene varias funciones, sus definiciones pueden aparecer en cualquier orden. Generalmente, una función usará o procesará la información que le es transferida desde el punto del programa donde es invocada o llamada. Esa información es comunicada a la función a través de identificadores llamados **argumentos** o **parámetros**. El resultado de la función se devuelve a través de la sentencia **return**.

La definición de una función consta de dos partes:

- **Cabecera** (la primera línea): incluye las declaraciones de los argumentos
- **Cuerpo** de la función.

La *primera línea* de la definición de una función contiene la especificación del tipo del valor devuelto, seguido del nombre o identificador de la función, y opcionalmente un conjunto de argumentos o parámetros, separados por comas y encerrados entre paréntesis. Cada argumento debe ir precedido por su declaración de tipo.

Si la función no incluye parámetros, el identificador de la función puede ir seguido de un par de paréntesis vacíos o con la palabra reservada *void*.

La primera línea de la definición o cabecera de una función, puede indicarse con el siguiente formato:

Formato:

<tipo de dato> <identificador>(< tipo1 arg1, tipo2 arg2, . . . , tipon argn>)

arg1, arg2, ..., argn, se denominan **argumentos formales** o **parámetros formales** e identifican a los datos que recibe la función en el momento de su invocación o llamada.

tipo1, tipo2, ..., tipon, representan los tipos de datos asociados a cada parámetro formal.

Al momento de invocar la función, los datos que se comunican a la función reciben el nombre de **argumentos reales**, **parámetros actuales** o **parámetros reales**, ya que se refieren a la información que realmente se transfiere. Cabe aclarar que los nombres de los parámetros formales y los nombres de los parámetros reales no necesariamente deben coincidir.

El cuerpo de la función se encierra entre llaves { } y contiene el conjunto de sentencias que se deben ejecutar cada vez que se la invoca.

Los parámetros formales y las variables declaradas dentro de una función, son locales a la función, pues no son reconocidos fuera de ella.

Cuando se usan funciones es importante tener en cuenta que:

- los parámetros actuales deben coincidir en tipo, orden y cantidad con los parámetros formales.
- el cuerpo de la función debe incluir al menos una sentencia *return* para devolver cero o un valor al punto de invocación o llamada. Es la sentencia *return* la que permite que se devuelva el control al punto de llamada o invocación sino se coloca, de modo implícito se realiza esta tarea en el punto donde se encuentra la llave de cierre de la función.

La instrucción *return* se especifica de la siguiente manera:

Formato:

return expresión;

expresión: se coloca si la función retorna un valor, si la función es tipo *void* no se coloca.

El valor de expresión, si existe, se devuelve a la función que llama. Si expresión se omite, el valor devuelto de la función es indefinido. El valor de expresión, si está presente, se evalúa y después se convierte al tipo devuelto por la función. Si la función se declaró con el tipo de valor devuelto *void*, una instrucción *return* que contiene una expresión genera una advertencia y la expresión no se evalúa.

Si no aparece ninguna instrucción *return* en la definición de función, el control vuelve automáticamente a la función de llamada después de que se ejecute la última instrucción de la función llamada. En este caso, el valor devuelto de la función llamada es indefinido. Si no se necesita un valor devuelto, se debe declarar la función para que tenga el tipo de valor devuelto *void*.

Si la función no retorna un resultado, la sentencia *return* puede no aparecer.

Ejemplo :

El siguiente código corresponde a la definición de la *función calcula*, que devuelve el perímetro de un terreno, cuyas dimensiones son recibidas en los parámetros formales.

```
float perimetro( float largo, float ancho) /*largo y ancho son parámetros formales*/
{
float perim;                               /* perim es una variable local a la función perimetro */
perim = 2 * ( largo + ancho) ;
return perim ;
}
```

Ejemplo

El siguiente código corresponde a la definición de la *función sumatoria*, que realiza el cálculo de:

$$\sum_{N=a}^{N=b} 2N + 1$$

para valores de *a* y *b* recibidos como parámetros formales. En este caso, la función no devuelve un resultado.

```
#include <stdio.h>
void sumatoria ( int a, int b )
{
int i, s=0;
for (i=a; i<=b; i++)
    s+=2 * i + 1;
printf(" la suma es %d ", s);
}
```

Ejemplo

La siguiente definición corresponde a la *función cabecera*, que imprime el membrete de una factura. En este caso, no recibe información y no devuelve un resultado.

```
#include <stdio.h>
void cabecera(void)
{
    printf("    EMPRESA UNION S.A  \n");
    printf("    Perú  123 Sur- Te: 02644378990  \n");
    printf("    San Juan  \n");
    return;
}
```

En casos como los dos últimos ejemplos, puede omitirse la sentencia `return` en la definición de una función, aunque esto no es una manera aconsejable de programar. En estos ejemplos, al llegar al final del cuerpo de la función, se devuelve el control al punto de llamada de la función sin retornar resultado alguno.

Como puede observarse, se coloca la palabra reservada *void* como especificador de tipo del resultado de la función y la sentencia `return` vacía.

También pueden definirse funciones que incluyan varias sentencias `return`, conteniendo cada una de ellas, una expresión distinta.

Ejemplo

La siguiente es la definición de una función que devuelve como resultado 'p', 'n' o 'c', según que el argumento recibido sea un número positivo, negativo o nulo.

```
char signo ( int num)
{
    if ( num > 0)
        return 'p';
    else
        if ( num < 0 )
            return 'n';
        else
            return 'c';
}
```

Invocación o llamada a una función

La llamada o invocación a una función produce la ejecución de las sentencias del cuerpo de la misma, hasta que encuentra la sentencia `return` o la llave de final de cuerpo.

Se puede llamar o invocar a una función indicando su nombre seguido de la lista de argumentos encerrados entre paréntesis y separados por coma.

Formato:

<identificador de la función> (<arg1,arg2, . . . ,argn>)

`arg1,arg2, . . . , argn` se denominan **argumentos reales, parámetros reales o actuales**.

Estos argumentos deben corresponderse con los argumentos o parámetros formales que aparecen en la primera línea de la definición de la función. Si la llamada de la función no necesita argumentos, a continuación del identificador de la función se coloca un par de paréntesis vacíos.

La invocación a una función puede formar parte de una expresión cuando esta retorna un valor.

Ejemplo

Construir un programa en C que utilice una *función maximo* para determinar el mayor valor entre tres números enteros.

```
#include <stdio.h>
#include <conio.h>
int maximo ( int x, int y )           /* definición de la función maximo */
{
    int z;
    z = ( x >= y )? x:y ;
    return z;
}

void main (void )
{
    int a , b , c , d ;
    printf ( " \n a = " );           /* ingreso de las tres números enteros */
    scanf ( " %d " , &a );
    printf ( " \n b = " );
    scanf ( " %d " , &b );
    printf ( " \n c = " );
    scanf ( " %d " , &c );
    d = maximo (a, b );              /*primera invocación a la función maximo*/
    printf ( " \n el máximo de los tres números es: %d", maximo( c , d ) );
                                    /*segunda invocación a la función maximo*/
    getch();
}
```

En este ejemplo, se invoca a la función maximo desde dos lugares diferentes en el main:

- la primera invocación se efectúa en una asignación y los parámetros reales son las variables a y b.
- la segunda invocación se realiza en una escritura y los parámetros reales son las variables c y d.

Ambas invocaciones podrían reemplazarse por la siguiente línea:

```
printf ( " \n el máximo de los tres números es:%d", maximo ( c, maximo( a , b ) ) );
```

En este caso, una de las invocaciones de la *función maximo* es un argumento para la otra llamada, evitando así el uso de la variable d.

Ejemplo

Para dos números naturales distintos, el siguiente programa escribe un mensaje diciendo si uno de ellos es divisor del otro.

```
#include <stdio.h>
#include <conio.h>
int divisor (int x, int y)
{
    int res;
    ((x<y)&& ((y%x)==0))? res=-1:((x>y)&& ((x%y)==0))? res=1: res= 0;
    return res;
}
```

```
void main(void)
{
    int a,b,r;
    printf(" ingrese dos números naturales distintos\n");
    scanf("%d", &a);
    scanf("%d", &b);
    r=divisor(a,b);
    if (r== -1)
        printf("%d es divisor de %d", a,b);
    else
        if (r== 1)
            printf("%d es divisor de %d", b,a);
        else
            printf("no hay divisores");
    getch();
}
```

ACTIVIDAD 1

Construir una función que permita calcular la suma de todos los números menores o iguales a un número entero determinado. Codificar el programa principal con la invocación a la función.

Declaración de una función o prototipo de función

En los ejemplos realizados hasta ahora, se puede observar que la definición de una función siempre ha precedido a la función main. De esta forma, al compilar el programa, la función está definida antes de la primera invocación o llamada.

Sin embargo, el main puede aparecer antes de la definición de alguna función, es decir, la invocación de una función precede a su definición. Esta situación confundiría al compilador, salvo que se le alerte que la función a invocarse será definida más adelante en el programa. Con esta finalidad se utilizan los **prototipos de funciones**.

La forma general del **prototipo** de una función es:

<tipo de dato> <identificador>(< tipo1 arg1, tipo2 arg2, . . . , tipon argn>);

Puede observarse, que el prototipo de una función se corresponde con la primera línea de la definición de la misma, pero finaliza con punto y coma.

No es necesario declarar en ningún lugar del programa los parámetros o argumentos formales del prototipo de una función, pues éstos son argumentos ficticios que sólo se reconocen dentro del prototipo. Incluso, pueden omitirse los nombres de tales parámetros, ya que lo obligatorio es indicar el tipo de datos de cada parámetro formal.

Una función puede definirse en cualquier parte del programa, pero antes de su invocación debe estar presente al menos su declaración o prototipo, a veces llamada declaración por adelantado o declaración *forward*. Esta declaración informa al compilador que será accedida antes de que sea definida.

Ejemplo

En este ejemplo, la función divisor se define después de su invocación, por lo tanto su prototipo se incluye en la función main.

```
#include <stdio.h>
#include <conio.h>
void main(void)
{
    int divisor (int x, int y);    /* prototipo de la función divisor */
    int a,b,r;
    printf(" ingrese dos números naturales distintos\n");
    scanf("%d", &a);
    scanf("%d", &b);
    r=divisor(a,b);
    if (r== -1)
        printf("%d es divisor de %d", a,b);
    else
        if (r== 1)
            printf("%d es divisor de %d", b,a);
        else
            printf("no son divisbles" );
    getch();
}

int divisor (int x, int y)
{
    int res;
    ((x<y)&& ((y%x)==0))? res=-1:((x>y)&& ((x%y)==0))? res=1: res= 0;
    return res;
}
```

Organización de la memoria en C

Para la ejecución de un programa, el lenguaje C, al igual que otros lenguajes imperativos, usa una división tripartita de la memoria: estas tres áreas corresponden a:

- Un área de almacenamiento estático
- Un área asignada como pila (stack)
- Un área asignada como montículo (heap)

Almacenamiento Estático: La asignación de esta área de memoria se realiza en tiempo de traducción, y permanece fija durante la ejecución de un programa. En esa área se almacenan los segmentos de códigos de las funciones y otros datos del sistema.

Pila (stack): La pila es un bloque secuencial de memoria que se asigna en tiempo de ejecución. El lenguaje C utiliza la **pila** para gestionar las llamadas a las funciones. En este área se almacenan los registros de activación de las funciones,

La asignación de memoria en la pila se realiza secuencialmente a partir de un extremo, mientras que la liberación de la misma se realiza en orden inverso.

Generalmente, la pila crece hacia abajo, ocupando así, direcciones altas de memoria.

Cuando se invoca una función, el registro de activación correspondiente se almacena en la pila (se *apila*). Una vez que finaliza la ejecución de la misma, ese área se libera (el registro se *desapila*), retornando el control a la función llamante.

Montículo (heap): Un montículo es un bloque de almacenamiento dentro del cual se asignan o liberan segmentos de memoria. Sobre este bloque de almacenamiento se profundiza cuando se trabajan variables y estructuras dinámicas.

Los espacios destinados a la pila y al montículo se ubican en extremos opuestos de la memoria para que puedan crecer de manera conveniente.

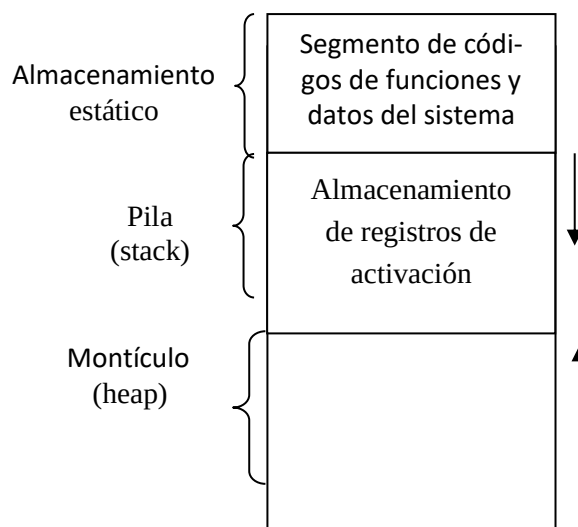


Fig. 1 Organización de la memoria durante la ejecución de un programa

Ejecución de un programa en C

Cuando se inicia la ejecución de un programa, en tiempo de compilación se generan los segmentos de códigos de las distintas funciones que el programa contiene.

En la pila, el primer registro de activación que se almacena corresponde al main, ya que la ejecución se inicia por esta función.

Cuando se invoca a una nueva función, se “apila” su correspondiente registro de activación, almacenándose la información necesaria en sus campos. Una vez finalizada la ejecución, ese registro de activación se “desapila” y queda sólo el registro de activación del main en la pila.

Ejemplo

El siguiente ejemplo, analiza la ejecución de un programa que calcula el perímetro de un terreno.

```
float perimetro ( float xl, float xa )
{
    float perim;
    perim = 2 * ( xl + xa );
    return perim;
}
```

```
void main(void)
{
    float largo, ancho;
    printf("ingrese el largo y el ancho");
    scanf("%f", &largo);
    printf("\n");
    scanf("%f", &ancho);
    printf("\n");
    printf("el perímetro del terreno es: %f", perimetro( largo,ancho ));
}
```

Al iniciarse la ejecución del programa, se genera el registro de activación del *main*. Este registro no reserva espacio para los parámetros ni para el resultado de la función *main*.

Al encontrar la invocación de la función *perímetro*, se genera otro registro de activación, en el cual se destina espacio para guardar: los parámetros formales *xa* y *xl*, la variable local *perim*, almacenamiento temporal, el resultado de la función y la dirección de retorno.

La figura 2 muestra el estado de la memoria en esta situación, para los valores *largo*=23.5 y *ancho*=13.6.

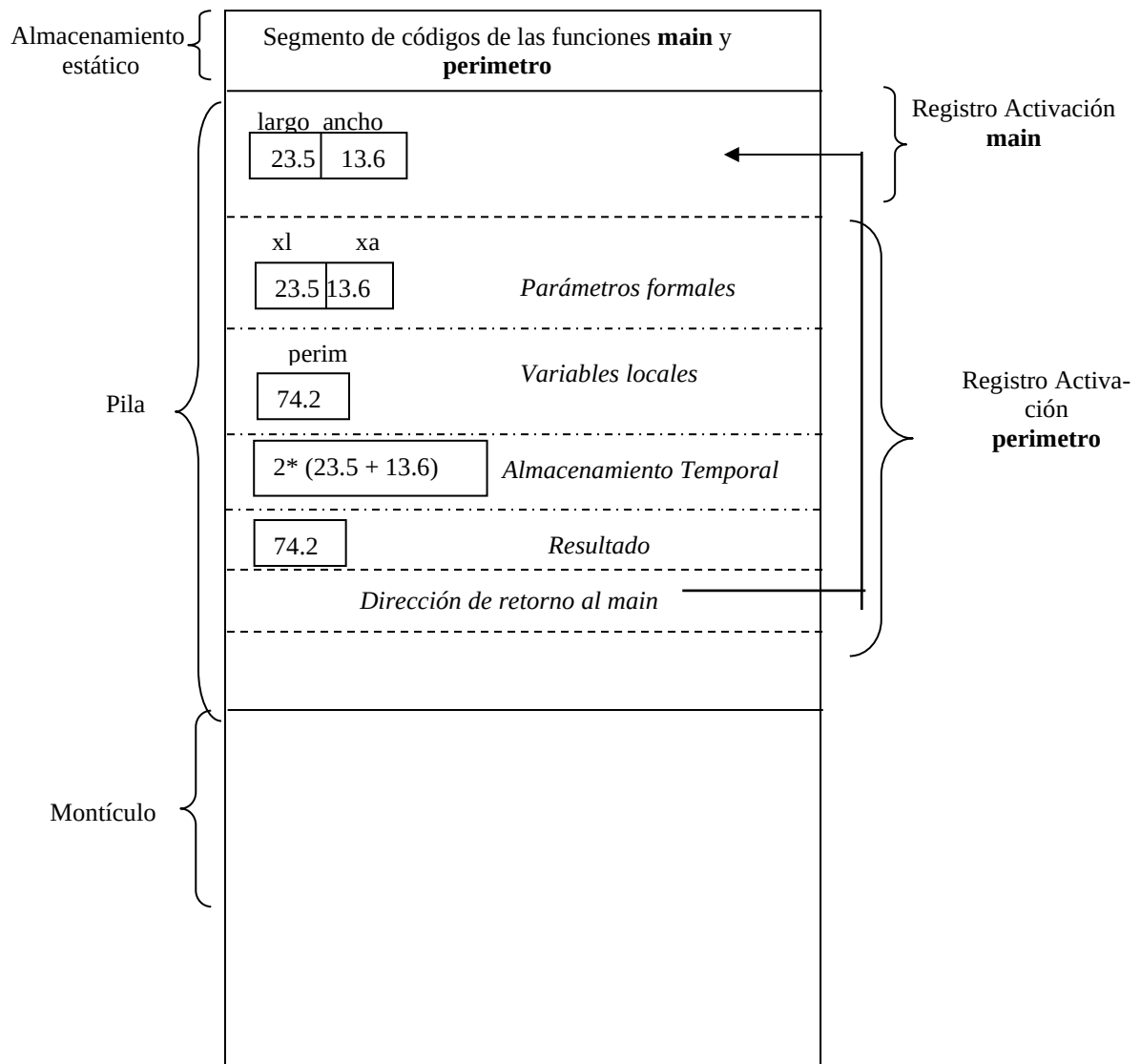


Fig 2. Organización de la memoria durante la ejecución de la función *perímetro*

Salida: el perímetro del terreno es: 74.2

Cuando termina la ejecución de la función *perímetro*, el sistema procede a desapilar el registro de activación asociado a ella en la pila y el control es devuelto al punto de llamada en el *main*. Una vez finalizado el programa, el sistema desapila el registro de activación del *main*.

Más adelante, cuando se trabaje el concepto de recursividad, se analizará el modo en que varios registros de activación de una misma función se pueden apilar consecutivamente.

Tipos de almacenamiento

Existen dos formas de clasificar variables: por su *tipo de datos* y por su *almacenamiento*.

El tipo de datos hace referencia al conjunto de posibles valores que puede tomar una variable. Por ejemplo una variable puede ser de un tipo de dato *int*, *float*, *char*, etc.

El tipo de almacenamiento hace referencia a la permanencia o tiempo de vida de la variable y a su alcance dentro del programa en el cual se reconoce.

Respecto a la permanencia, se puede observar que algunas variables tienen una existencia breve, otras son creadas y destruidas de manera repetida, y otras existen durante toda la ejecución del programa. El tiempo de vida de un objeto es la duración de asignación en memoria.

Hay tres tipos de almacenamiento: *estático* (para variables globales y estáticas), *automático* (para variables locales) y *dinámico* (para variables dinámicas).

La palabra reservada **auto** (automática) se utiliza para declarar variables de persistencia automática. Estas variables se crean al generarse el registro de activación de la función en la cual están declaradas, existen mientras dicho registro está activo, y se destruyen cuando se sale de ese bloque.

Variables Automáticas

Las variables locales de una función, aquellas declaradas en la lista de parámetros o en el cuerpo de la función, por lo regular tienen una persistencia automática. La palabra reservada **auto** declara en forma explícita a las variables de persistencia automática.

Respecto a su alcance, el mismo está restringido a la función en la cual se declaran. De ahí que todas las variables definidas dentro de una función no necesitan llevar el prefijo **auto** pues se asumen automáticas. Las variables automáticas se almacenan en la pila en el registro de activación asociado a la función.

Ejemplo

```
void fucion1( void)
{  doble d;
   int x ;
   ...
}
void fucion2 (void)
{  int y; float d ;
   ...
}
void main(void)
{  char z;
   ...
}
```

En este ejemplo, las variables automáticas de cada una de sus funciones son:

main: z

funcion1: d, x

funcion2: y, d

Como puede inferirse, las variables automáticas definidas en funciones diferentes serán independientes unas de otras, incluso si tienen el mismo nombre.

Se pueden asignar valores iniciales a las variables automáticas. Tales valores se reasignarán cada vez que se entre en la función, pues los valores de las variables automáticas se pierden cuando se sale de la función. Si una variable automática no es inicializada, su valor inicial será impredecible.

Variables Externas

Las variables externas difieren de las automáticas pues su alcance no se restringe a una función. Una vez definida una variable externa, toda función puede tener acceso a ella a través de su nombre.

Al trabajar con variables externas es necesario distinguir entre **definir y declarar** la variable.

Una variable externa debe definirse una sola vez, fuera de cualquier función. Esto reserva su espacio de almacenamiento en el área de almacenamiento estático. Además su definición puede incluir la asignación de un valor inicial.

Una variable externa también debe declararse en cada función que desee tener acceso a ella. Una **declaración** de variable externa tiene que comenzar con el especificador de tipo de almacenamiento **extern**. Su declaración no reserva espacio de memoria.

El nombre de la variable externa y su tipo de datos tienen que coincidir con su correspondiente definición de variable externa que aparece fuera de la función.

Si una función altera el valor de una variable externa, este valor está accesible por cualquier otra función que la utiliza.

Bajo ciertas circunstancias, la declaración de una variable externa puede omitirse. Tal es el caso de que la definición de la variable externa ocurra dentro del archivo fuente, antes de main. Se las llama también variables globales.

ACTIVIDAD 2

Realizar el seguimiento de los siguientes algoritmos para $j = 30$ y mostrar las salidas correspondientes.

a)

```
#include <stdio.h>
#include <conio.h>
int i=32;           /* definición de i como una variable externa y global */

void main(void)
{
    int j;           /* j es una variable automática de main */
    clrscr();
    printf("ingrese un entero ");
    scanf("%d",&j);
    int cambia(int j); /* declaración de la función */
    printf("\n Cambio realizado en j en el subprograma: %d",cambia(j));
    printf("\n Valor actual de j: %d",j);
    printf("\n Valor actual de i: %d",i);
    getch();
}

int cambia(int j)           /* j es una variable automática de cambia */
{
    j=j * 2;
    i=i * 2;
    return j;
}
```

En este ejemplo, la definición de la variable externa precede a la definición de la función, por lo que no es necesaria su declaración.

```

b)
#include <stdio.h>
#include <conio.h>

int cambia(int j)                                /* j es una variable automática de cambia */
{
    extern i;                                    /* i es declarada como externa */
    j=j*2;
    i=i*2;
    return j;
}

int i;                                            /* i es definida como externa */
void main(void)
{
    int j;                                       /* j es una variable automática de main */
    clrscr();
    printf("ingrese un entero ");
    scanf("%d",&i);                             /* ingreso 20 */
    printf("\n ingrese un entero ");
    scanf("%d",&j);                             /* ingreso 30 */
    printf("\n Cambio realizado en j en subprograma: %d",cambia(j));
    printf("\n Valor actual de j: %d",j);
    printf("\n Valor actual de i: %d",i);
    getch();
}

```

En este ejemplo la definición de la variable externa es posterior a la definición de la función **cambia**. Luego en la función **cambia**, la variable **i** debe declararse como externa.

ACTIVIDAD 3

¿Qué modificaciones debería realizar en el inciso b) de la actividad anterior, si la definición de la variable externa **i** se realiza después del **main()**?

Variables Estáticas

Las variables estáticas tienen el mismo alcance que las variables automáticas, son locales a la función en la que están declaradas. Sin embargo, respecto a su tiempo de vida, las variables estáticas retienen sus valores durante toda la vida del programa. Como consecuencia de esto, al invocar nuevamente a la función, se puede acceder al valor de la variable correspondiente a la última invocación. Esta característica permite a las funciones mantener información permanente a lo largo de toda la ejecución del programa. Las variables **static** se almacenan en memoria, en el área de almacenamiento estático, a continuación del segmento de código de la función.

La declaración de una variable estática deberá empezar con la especificación del tipo de almacenamiento, para lo cual se utiliza el prefijo **static**.

Las variables estáticas pueden usarse de la misma manera que las otras variables, sin embargo, no pueden ser accedidas fuera de la función en que están definidas.

En la declaración de variables estáticas se pueden incluir valores iniciales. Las reglas asociadas a la asignación de estos valores son esencialmente las mismas que las asociadas con la inicialización de variables externas, aunque las variables estáticas se definen localmente dentro de una función.

En general:

- Los valores iniciales deben ser expresados como constantes, no como expresiones.
- Los valores iniciales se asignan a sus respectivas variables al comienzo de la ejecución del programa. Las variables retienen estos valores a lo largo de toda la vida del programa, salvo que se le asignen valores diferentes en el curso de la ejecución.
- A todas las variables estáticas cuyas declaraciones no incluyan valores iniciales explícitos, se les asignará el valor cero. De esta manera, las variables estáticas tendrán siempre valores asignados.

Ejemplo

En el siguiente programa muestra el uso de variables estáticas

```
float promedio (int xa)
```

```
{
    static int sum=0;
    static int cont=0;
    sum +=xa;
    cont ++;
    return (float) sum / cont ;
}
```

```
void main (void)
```

```
{
    float prom;
    int a;
    printf("\n ingrese un entero \n");
    scanf("%d", &a);
    while (a!=100)
    {
        if (a>0)
            prom=promedio (a);
        printf("\n ingrese un entero \n");
        scanf("%d", &a);
    }
    printf("\n promedio positivos %f",prom);
    getch();
}
```

En este programa sum y cont son variables estáticas de la función promedio, por lo tanto retienen su valor después de cada ejecución de misma.

ACTIVIDAD 4

Realizar el seguimiento del programa anterior para el siguiente lote de prueba: 2, 8, -4, 5,-6,100.

Declaraciones, Bloques y Alcance¹⁸

Declaraciones – Su importancia durante la traducción

Como se sabe, una declaración es un enunciado del programador que sirve para comunicar al traductor distinta información, primordial para establecer ligaduras.

Las declaraciones se utilizan para construir **en tiempo de traducción** la Tabla de Símbolos (TS), contribuyendo de ese modo a uno de los propósitos más importantes de las mismas: **permitir la verificación estática de tipos**.

Bloques en lenguaje C.

Un bloque es una secuencia de declaraciones seguidas por una secuencia de enunciados, y rodeados por marcadores sintácticos como son las llaves o pares inicio-terminación.

En lenguaje C, los bloques se conocen como enunciados compuestos y aparecen como el cuerpo de funciones en la definición de las mismas. Además, es posible también anidar bloques, como se muestra a continuación:

```
void p (void)
{  doble r;           /* el bloque de la función p*/
  .....
  {
    int x, y;          /* un bloque anidado */
    x=2 ;
    y=3 + x ;
  }
  .....
  .....
}
```

El lenguaje C también admite **declaraciones externas o globales** fuera de cualquier enunciado compuesto, como por ejemplo:

```
int x=10;             /* x es externa para todas las funciones y también es global*/

void main(void)
{
  float f;
  int g;              /* f y g  están asociadas al bloque del main*/
  .....
  .....
}
```

Alcance de un vínculo en lenguaje C:

Las declaraciones vinculan varios atributos a un nombre, el alcance de este vínculo es la región del programa donde este vínculo se mantiene.

En el lenguaje C, lenguaje estructurado en bloques, el alcance de un vínculo queda limitado al bloque donde aparece la **declaración asociada**, y a los bloques contenidos en el interior. Este alcance se llama **alcance léxico**.

¹⁸ Lenguajes de Programación Principios y Prácticas. Kenneth Loudon. Pág. 118.

El **alcance de una declaración** en lenguaje C, se extiende desde el punto siguiente a la declaración, hasta el final del bloque en el cual ésta se encuentra.

```
int x;
void p(void)
{  doble d;
  ...
}
void q(void)
{  int y;
  ...
}
void main(void)
{  char z;
  ...
}
```

En este ejemplo, las declaraciones de la variable **x**, y de las funciones **p**, **q**, **main** son globales, es decir tienen alcance global. Las declaraciones de **d**, **y**, **z** son **locales** respecto a las funciones p, q, y main respectivamente. Por lo tanto sus declaraciones son solamente válidas para dichas funciones.

Apertura en el alcance:

Una característica de la estructura de bloques es que las declaraciones en los bloques anidados toman precedencia sobre declaraciones anteriores.

```
int x;
void p(void)
{  doble d;
  int x ;
  ...
}
void q(void)
{  int y;
  ...
}

void main(void)
{  char z;
  ...
}
```

En este ejemplo, **la declaración de x** en la función **p** tiene precedencia sobre la **declaración global de x** del comienzo del bloque. Por lo tanto, dentro de la función p, la variable global x no puede ser accedida. Se dice que **la variable global x tiene una apertura en el alcance**. En este punto cabe agregar el concepto de visibilidad.

Visibilidad de una declaración:

La visibilidad incluye únicamente aquellas regiones de un programa donde las ligaduras de una declaración son aplicables.

En síntesis, en el ejemplo anterior la variable global x tiene alcance dentro de la función p, ya que la ligadura sigue existiendo, no obstante la variable global x está oculta.

ACTIVIDAD 5: Realice el seguimiento del siguiente algoritmo y analice el alcance de las distintas declaraciones.

```
int b=10;
void mostrar(int x, int y)
{
    printf ("\n en este ejercicio hemos sumado %d + %d y obtuvimos este resultado:%d",x,y,x + y);
    printf ("\n Valor de b :%d",b);
}

void main(void)
{
    int a=10,b=5;
    printf ("\n a + b antes del bloque %d", a+b);
    {
        int j=5,a=3;
        printf ("\n a + b dentro del bloque %d", a+b);
        j++;
        printf ("\n j dentro del bloque %d", j);
    }
    printf ("\n a + b al salir del bloque %d", a+b);
    mostrar(a,b);
    getchar();
}
```

Pasaje de parámetros a una función

Un programa se comunica con sus funciones a través de los parámetros. Existen distintas formas de pasar parámetros a una función.

Pasaje por valor

Este tipo de pasaje es uno de los más comunes. En este mecanismo los parámetros reales se evalúan al momento de la llamada a la función y se transforman en los valores que toman los parámetros formales durante la ejecución de la función.

El paso por valor es el mecanismo por omisión de lenguajes como C++ y Pascal, siendo el único mecanismo de pasaje de parámetros de C y Java. En todos estos lenguajes **los parámetros formales se consideran como variables locales** que toman como valor inicial el valor de los parámetros reales.

Los parámetros formales pueden cambiar sus valores a través de asignaciones sin que estos cambios afecten los valores de los parámetros reales.

La desventaja es que al pasar un parámetro por valor, se produce una duplicación del área de memoria.

Ejemplo

```
int factorial ( int x )
{
    int f=1;
    while (x)
    {
        f*=x;
        x--;
    }
    return f;
}
```

```

int main(void)
{
    int a;
    int fac;
    printf("\n Ingrese un valor para a: ");
    scanf("%d", &a);
    fac=factorial(a);
    printf("\n El factorial de %d es %ld", a,fac);
    getch();
}

```

Salida:

Ingrese un valor para **a**: **9**

El factorial de **9** es **362880**

Estos resultados muestran que dentro de main, **a** no se ha modificado, aunque se haya modificado el valor correspondiente a **x** en la función factorial.

La ventaja del pasaje por valor es que se protege el valor del parámetro real de posibles alteraciones dentro de una función.

Como puede inferirse, el pasaje por valor ofrece la posibilidad de proporcionar como parámetro real una expresión o valor constante, en lugar de que éste sea necesariamente una variable.

La figura 3 muestra la organización de la memoria durante la ejecución de la función factorial anterior.

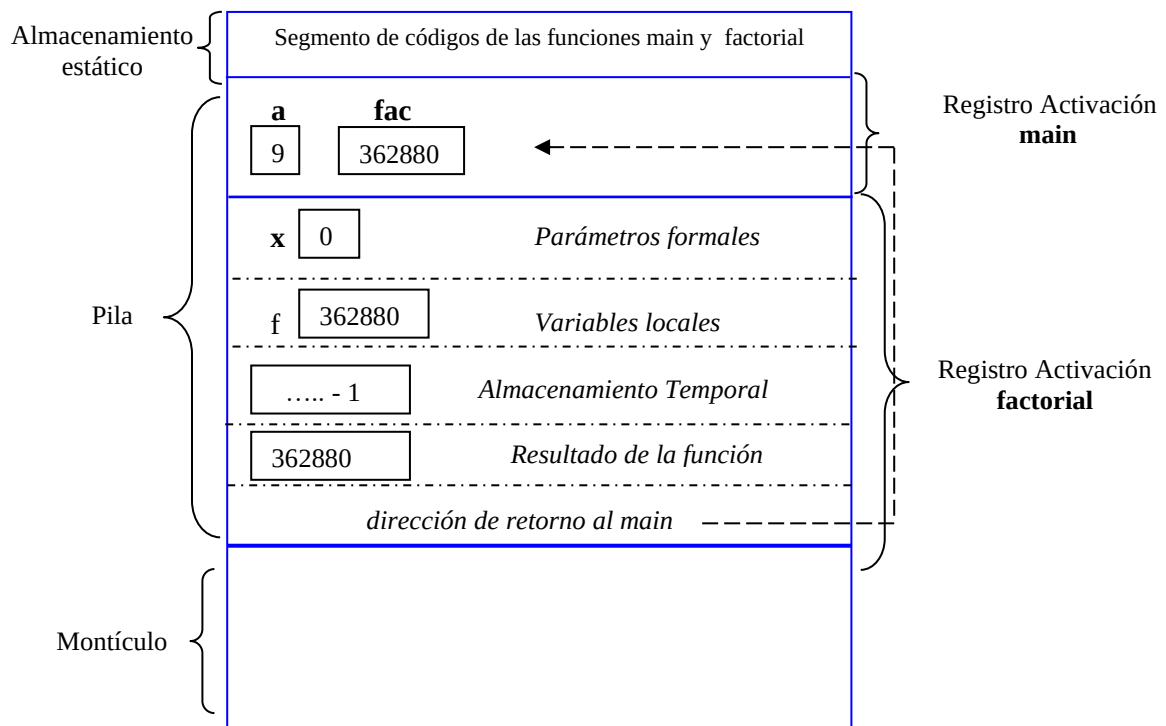


Fig. 3 Organización de la memoria antes de terminar la ejecución de la función factorial (pasaje por valor)

Nótese en este algoritmo que no hace falta definir una variable para la iteración, en su lugar se va decrementando el parámetro formal **x** hasta llegar a 1.

Pasaje de direcciones

El paso de pasajes de parámetros por valor no implica que no puedan ocurrir cambios fuera de la función. Si el parámetro es un puntero, esto es una dirección, puede usarse para cambiar la memoria apuntada por fuera de la función. Otra ventaja que ofrece el pasaje de parámetros por dirección es retornar más de un resultado desde la función.

En C, para pasar un parámetro por dirección, se utiliza el operador de dirección **&**, al momento de invocar a la función. Luego, el operador de indirección ***** debe utilizarse en la función para acceder al valor almacenado en esa dirección.

Retomemos la función factorial, pero de tal modo que la variable **a** se pasa por dirección: en los parámetros formales.

Ejemplo

```
int factorial ( int * x )
{ int f=1;
  while (*x)
  {
    f*=*x;
    --(*x);
  }
  return f;
}
```

```
int main (void )
{ int a;
  long int fac;
  printf (" \n Ingrese un valor para a: ");
  scanf("%d", &a);
  printf("Valor de a antes de invocar a la función factorial %d\n \n ", a);
  fac=factorial(&a);
  printf("Valor de a después de invocar a la función factorial %d\n \n ", a);
  printf (" \n El factorial es %ld", fac);
  getch();
}
```

Salida:

Ingrese un valor para a: **9**

Valor de a antes de invocar a la función factorial **9**

Valor de **a** después de invocar a la función factorial **0**

El factorial es **362880**

Como el operador **&** permite obtener la dirección de una variable, al definir la función factorial se debe indicar que el parámetro es un puntero y acceder indirectamente a través de éste a la variable **a**.

Más adelante, cuando se analiza el pasaje de arreglos como parámetros, se vuelve a trabajar con pasaje de direcciones.

La figura 4 muestra la organización de la memoria durante la ejecución de la función factorial anterior.

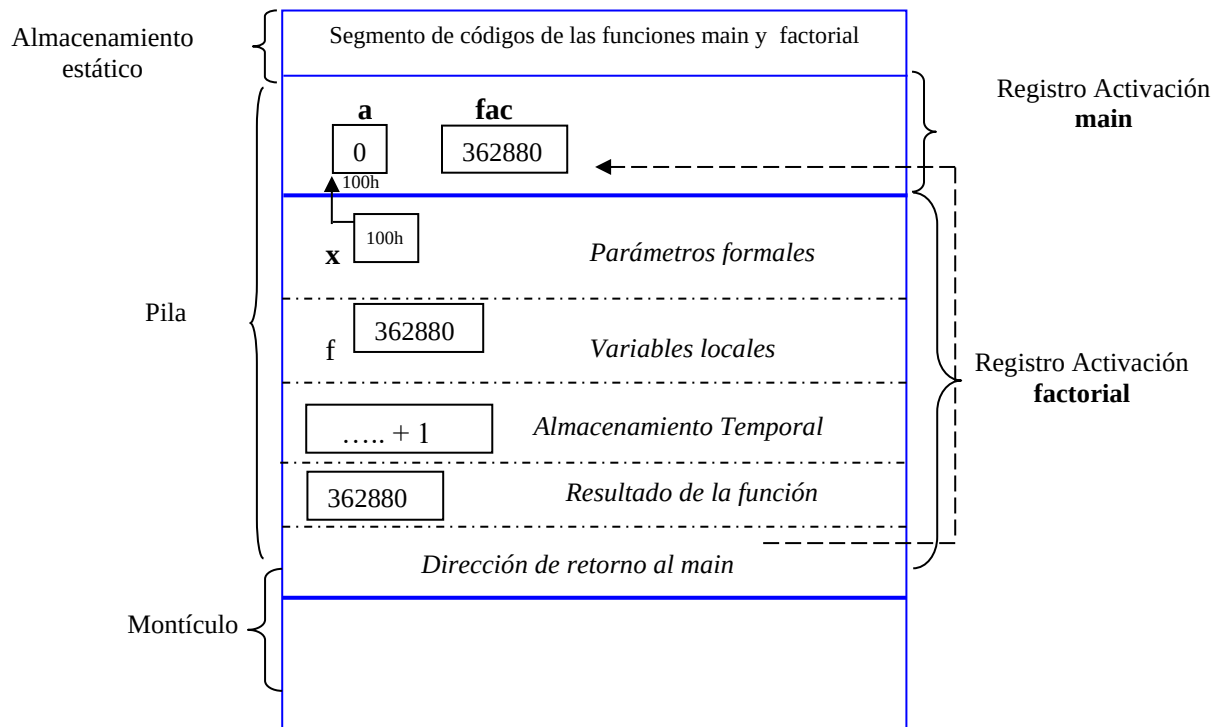


Fig. 4 Organización de la memoria antes de terminar la ejecución de la función factorial (pasaje de dirección)

Pasaje por referencias

Para realizar un pasaje por referencia el argumento a pasar debe ser una variable con una dirección asignada (si el paso fuese por valor el argumento puede ser una variable, una expresión o constante).

Esto es así por cuanto el pasaje por referencia no pasa la ubicación de la variable, por lo que el parámetro formal se transforma en un alias del parámetro actual de modo que cualquier cambio que se realiza en el parámetro formal se siente en el parámetro actual.

Esto puede interpretarse como que a una misma área de memoria se asignan dos nombres distintos.

Lenguajes como C++ y Pascal permiten el uso de pasaje por referencias.

En el caso de C++ , para indicar este tipo de pasajes se debe colocar el operador & a continuación del tipo de datos del parámetro formal.

Ejemplo : La función factorial antes definida, puede redefinirse usando pasaje por referencias:

```
int factorial ( int &x )
```

```
{ int f=1;
  while (x)
  { f*=x;
    x--;
  }
  return f;
}
```

```

void main (void)
{ int a;
  int fac;
  printf (" \n Ingrese un valor para a: ");
  scanf("%d", &a);
  printf("Valor de a antes de invocar a la función factorial %d\n \n ", a);
  fac=factorial(a);
  printf("Valor de a despues de invocar a la función factorial %d\n \n ", a);
  printf (" \n El factorial es %ld", fac);
  getch();
}

```

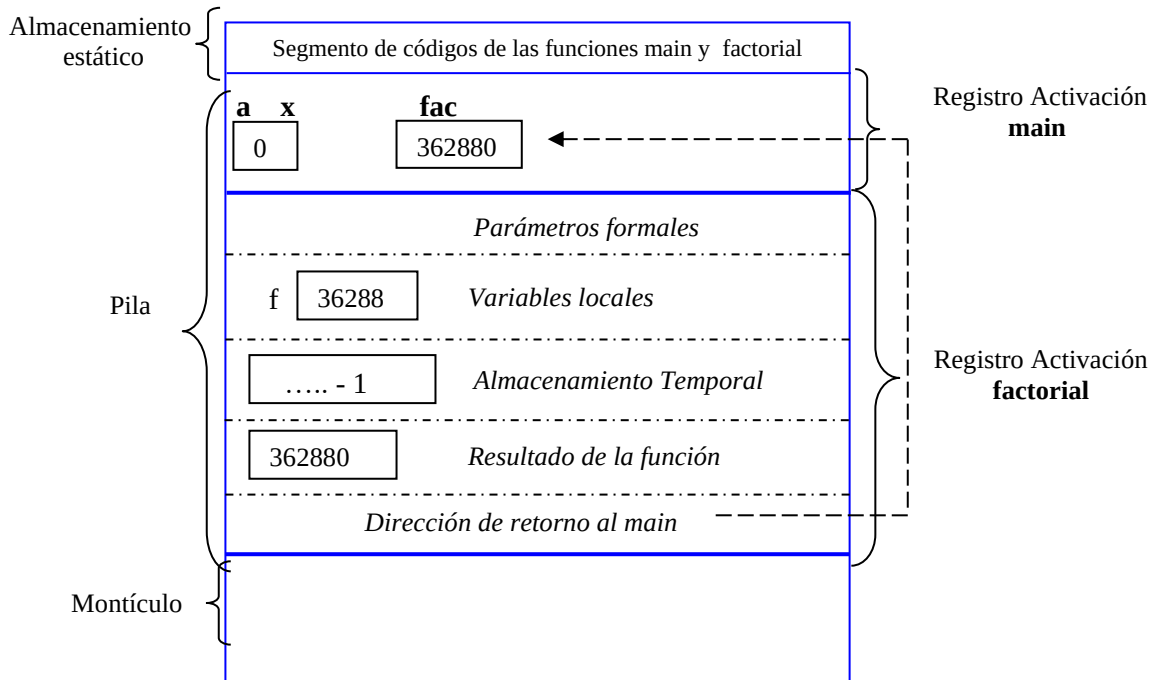


Fig. 5 Organización de la memoria antes de terminar la ejecución de la función factorial (pasaje por referencia)

Salida:

Ingrese un valor para a: **9**

Valor de a antes de invocar a la función factorial **9**

Valor de **a** después de invocar a la función factorial **0**

El factorial es **362880**

En este caso, **x** hace referencia a la variable **a**. Esto implica que no hay duplicación de memoria y cualquier modificación que se efectúe en el parámetro formal **x** afectará o se verá reflejado en el parámetro real **a**.

Observación: En lenguaje C estándar todas las llamadas son por valor, las llamadas por referencia se las puede simular haciendo pasaje por dirección.

No obstante en este curso se usará también los pasajes por referencias provistos por C++.



Funciones que devuelven más de un resultado

Recordemos que una función devuelve cero o un resultado. Hay situaciones en las cuales es necesario que la función devuelva más de un resultado.

Por ahora, la forma de lograr resolver este problema, es a través del pasaje de parámetros por direcciones o referencias.

Ejemplo

Supongamos que se necesita definir una función que retorna la suma de los números pares y la suma de los números impares, menores que un valor ingresado por teclado.

En este caso, uno de los resultados será devuelto por la función y el otro en un parámetro, a través de un pasaje de referencias.

```
int acumula_pares_impares (int &si, int xnum)
{
    int sp=0;
    while (xnum)
    {
        if ((xnum % 2)==0)
            sp+=xnum;
        else
            si+=xnum;
        xnum--;
    }
    return sp;
}

void main (void)
{
    int sumai=0, num;
    printf("ingrese un número \n"); scanf("%d", &num);
    printf(" suma de pares= %d \n", acumula_pares_impares (sumai, num));
    printf(" suma de impares= %d \n", sumai );
    getch();
}
```

En este ejemplo, sumai es una variable del main que almacenará la suma de los impares, para ello se pasa por referencia.

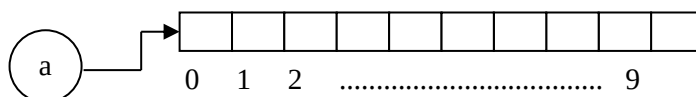
ACTIVIDAD 6

- Modificar el programa anterior de manera que el pasaje de la variable sumai se realice por dirección. ¿Qué conclusiones se pueden inferir si compara ambos tipos de pasajes?
- Modificar el programa anterior de manera que los dos resultados sean devueltos por la función a través de los parámetros.

Arreglos como parámetros de funciones

Como se sabe, el nombre de un arreglo es una *dirección constante* que apunta a la primera componente del mismo.

Para el arreglo **int a[10]**; la siguiente gráfica permite entender lo expuesto.



Por lo tanto, al pasar un arreglo como parámetro de una función, no se pasan los valores de las componentes del mismo, sino la dirección de la primera componente del arreglo. El parámetro formal se transforma entonces en un puntero al primer elemento.

De ahí, si se realiza alguna modificación a cualquier componente del arreglo, esa modificación será reconocida en todo el ámbito del arreglo.

Los prototipos de funciones que incluyen arreglos como argumentos, se pueden especificar de distintas formas:

- Colocando el nombre del arreglo en cuestión seguido de un par de corchetes vacíos o con el tamaño respectivo; todo esto precedido por el tipo de datos de las componentes del arreglo.
- Prescindiendo del nombre del arreglo, pero colocando un par de corchetes vacíos o indicando el tamaño del arreglo, precedidos por el tipo de datos de las componentes del arreglo.

Ejemplo

El siguiente programa muestra la forma de trabajar los arreglos como parámetros de funciones y las distintas formas de especificar el prototipo de las funciones.

```
#include <stdio.h>
#define n 20
```

```
void carga( float arre[n] )           /*carga del arreglo*/
{int i;
  for (i=0; i < n ; i++)
  { printf("ingrese valor \n");
    scanf(" %f", &arre [i]);
  }
  return ;
}
```

```
void main (void )
{
  float datos [n];
  carga(datos);    //pasaje del nombre de un arreglo como parámetro
  :
}
```

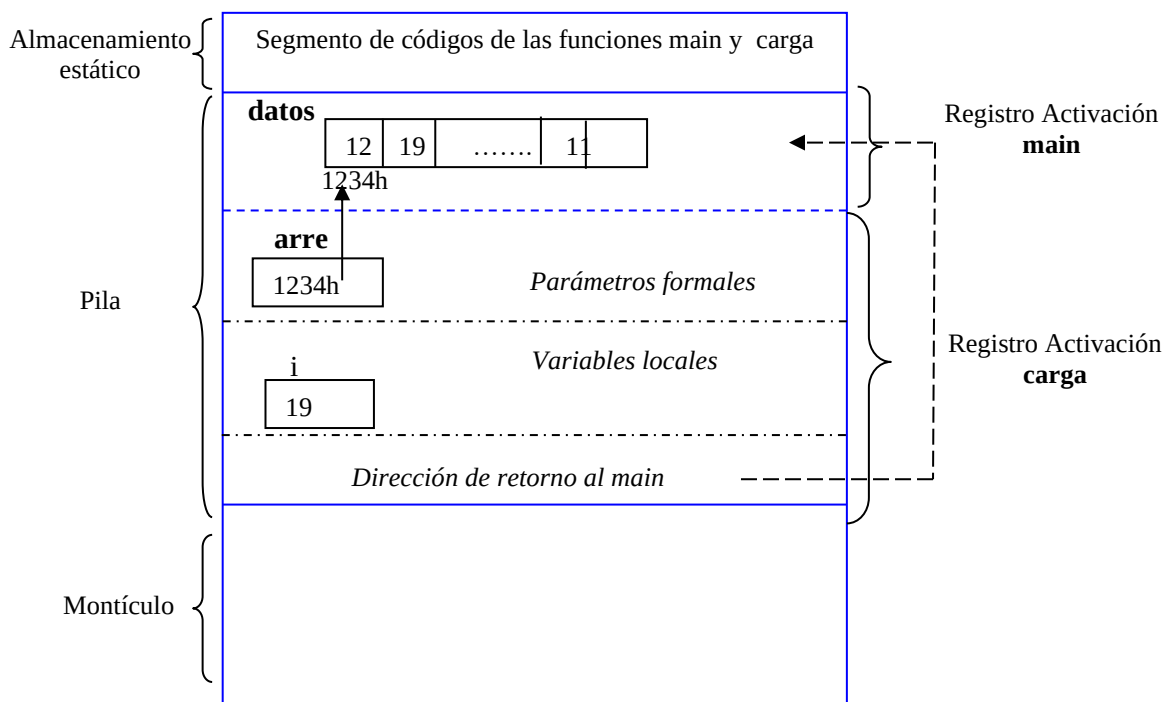


Fig. 6 Organización de la memoria antes de terminar la ejecución de la función carga

Observación: Como puede observarse, cuando se pasa un arreglo a una función se realiza un pasaje por valor de una dirección, la dirección de la primer componente del arreglo.

Otras formas de definir la función carga

a) Una buena práctica al momento de usar arreglos es enviar como parámetro no solo el nombre, sino también el tamaño.

```
void carga( float arre[], int n )           /*carga del arreglo*/
{ int i;
  for (i=0; i < n ; i++ )
  { printf("ingrese valor \n");
    scanf(" %f", &arre [i]);
  }
  return ;
}
```

```
void mostrar( float arre[], int lim )
{ int i;
  for (i=0; i < lim ; i++ )
    printf("\n %f", arre[i]);
}
```

```
void main (void )
{
  float datos[5];
  float precios[30];
  carga(datos,5);
  mostrar(datos, 5);
  carga(precios, 30);
  .....
  getch();
}
```

En este ejemplo puede verse claramente la ventaja de enviar el tamaño del arreglo como parámetro.

b) Además, como el nombre del arreglo es un puntero, puede recibirse como un puntero y el tamaño del arreglo.

```
void carga( float * arre, int n )       /*carga del arreglo*/
{ int i;
  printf("\n Ingrese las %d componentes del arreglo \n", n );
  for (i=0; i < n ; i++ )
  { printf("ingrese valor \n");
    scanf(" %f", &arre [i]);
  }
  return ;
}
```

```
void mostrar( float * arre, int lim )
{ int i;
  for (i=0; i < lim ; i++ )
    printf("\n %f", arre[i]);
}
```

```
void main (void )
{
    float datos[5];           // define el arreglo datos
    float arreglo[3];         // define el arreglo de nombre arreglo
    carga(datos,5);
    carga(arreglo,3);
    printf("\n datos del primer arreglo\n ");
    mostrar(datos, 5);
    printf("\n datos del segundo arreglo\n ");
    mostrar(arreglo, 3);
    getch();
}
```

ACTIVIDAD 7

Realizar un programa en C que, a través del uso de funciones, permita:

- Cargar un arreglo de N componentes enteras.
- Dado un valor entero, indicar si ese valor está o no en el arreglo. Si el valor está en el arreglo devolver la posición, de lo contrario devolver -1.
- Mostrar en el programa principal la cantidad de componentes pares del arreglo y la suma de las mismas.

Pasaje constante en arreglos

Dado que los arreglos son siempre pasados a las funciones a través del pasaje de una dirección, resulta difícil controlar las modificaciones de sus componentes. Para evitar estas modificaciones, C proporciona un calificador especial de tipo **const**.

En lenguaje C, los parámetros usados por una función pueden declararse como constantes (**const**) al momento de la declaración de la función. Un parámetro que ha sido declarado como constante significa que la función no podrá cambiar el valor del mismo (sin importar si dicho parámetro se recibe por valor o por referencia).

Ejemplo :

```
#include < stdio.h >
```

```
void intenta_modificar ( const int [ ] );
```

```
void main (void )
{
    int a [ ] = { 10 , 20 , 30 };
    intenta_modificar ( a );
    printf ( " %d %d %d \n", a [0], a [1], a [2 ] );
}
```

```
void intenta_modificar ( const int b [ ] )
{
    b [0] = 2;  /*error*/
    b [1] = 2;  /*error*/
    b [2] = 2;  /*error*/
}
```

Cada uno de los tres intentos por modificar las componentes del arreglo, en la función da como resultado el siguiente error de compilación: **cannot modify a const object**

Traducción y Tabla de Símbolos en Lenguaje C¹⁹

La tabla de símbolos es una estructura de datos que se crea en tiempo de traducción del programa fuente. Es como un diccionario de variables, debe darle apoyo a la inserción, búsqueda y cancelación de nombres (identificadores) con sus atributos asociados, representando las vinculaciones con las declaraciones.

Aunque su nombre parece indicar una estructuración en una tabla no es necesariamente ésta la única estructura de datos utilizada, también se emplean árboles, pilas, etc.

Los símbolos se guardan en la tabla con su nombre y una serie de atributos opcionales que dependerán del lenguaje y de los objetivos del procesador. Este conjunto atributos almacenados en la TS para un identificador determinado se define como registro de la tabla de símbolos (symbol-table record).

La lista siguiente de atributos no es necesaria para todos los compiladores, sin embargo cada uno de ellos se puede utilizar en la implementación de un compilador particular:

- Nombre de identificador.
- Dirección en tiempo de ejecución a partir de la cual se almacenará el identificador si es una variable. En el caso de funciones puede ser la dirección a partir de la cual se colocará el código de la función.
- Tipo del identificador. Si es una función, es el tipo que devuelve la función.
- Número de dimensiones del array, o número de miembros de una estructura o clase, o número de parámetros si se trata de una función.
- Tamaño máximo o rango de cada una de las dimensiones de los arrays, si tienen dimensión estática.
- Tipo y forma de acceso de cada uno de los miembros de las estructuras, uniones o clases. Tipo de cada uno de los parámetros de las funciones o procedimientos, etc.

Operaciones con la TS²⁰

Las dos operaciones que se llevan a cabo generalmente en las TS son la inserción y la búsqueda.

Inserción y búsqueda

En lenguaje C, la operación **de inserción** se realiza cuando se procesa una declaración, ya que una declaración es un descriptor inicial de los atributos de un identificador del programa fuente.

Si la TS está ordenada, es decir los nombres de las variables están por orden alfabético, entonces la operación de inserción llama a un procedimiento de búsqueda para encontrar el lugar donde colocar los atributos del identificador a insertar. En tales casos la inserción lleva tanto tiempo como la búsqueda.

Si la TS no está ordenada, la operación de inserción se simplifica mucho, aunque también la operación de búsqueda se complica ya que debe examinar toda la tabla.

En las operaciones de inserción se detectan los identificadores que ya han sido previamente declarados, emitiéndose, a su vez, el correspondiente mensaje de error.

Ejemplo en lenguaje C: *multiple declaration for 's'* si s ya estaba en la TS

En las operaciones de **búsqueda** se detectan los identificadores que no han sido declarados previamente emitiéndose el mensaje de error correspondiente.

Ejemplo en lenguaje C: *Undefined simbolo 'x,'* si x es una variable que desea usarse pero no se declaró.

Set y Reset

Otras operaciones realizadas sobre una tabla de símbolos SET (activar un bloque) y RESET (desactivar el bloque). La operación de set se utiliza cuando el compilador detecta el comienzo de un bloque o función en el cual se pueden declarar identificadores locales o automáticos. La operación complementaria reset, se utiliza cuando se detecta el final del bloque o función.

¹⁹ Lenguajes de Programación Principios y Prácticas. Kenneth Loudon. Pág. 127 - 131

²⁰ Lenguajes y Sistemas Informáticos: Tabla de Símbolos. Universidad de Oviedo. Pág. 9 a 16; 37 – 41.

Como consecuencia del uso de bloques anidados, la organización más simple para soportar la Tabla de Símbolos en los lenguajes estructurados en bloques, es una **PILA** (LIFO Last Input First Output).

Existen tablas de símbolos con estructura de árbol implementadas en pilas, Tablas de símbolos con estructura hash implementadas en pilas, pero éstos temas son objeto de estudio en años superiores.

Para comprender el manejo de la TS en lenguaje C podemos considerar una tabla de símbolos como un conjunto de nombres, cada uno de los cuales tiene una pila de declaraciones asociados con ellos, de manera que la declaración en la parte superior de la pila es aquella cuyo alcance está actualmente activo.

A la entrada de un bloque todas las declaraciones se procesan y se agregan las vinculaciones correspondientes a las TS; a la salida de un bloque se eliminan todas las ligaduras proporcionadas por las declaraciones, restaurando cualquier vínculo anterior que pudiera haber existido.

Podemos ver que la tabla de símbolos cambiará conforme se avanza en la traducción, para reflejar las inserciones o eliminaciones de ligaduras dentro del programa que se está traduciendo.

Manejo de la TS en PILA

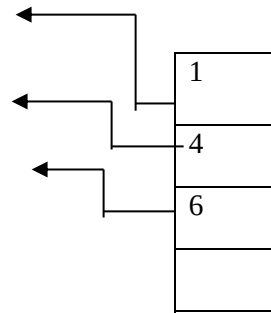
La organización más simple, desde el punto de vista conceptual, de una tabla de símbolos de un lenguaje estructurado en bloques es la pila. En este tipo de organización los registros que contienen los atributos de los identificadores se van colocando unos encima de otros según se van encontrando las declaraciones de las variables del texto fuente. Una vez que se lee el fin del bloque, todos los registros de los identificadores declarados en el bloque se sacan de la pila, pues dichos identificadores no tienen validez fuera del bloque.

Resulta importante entender el funcionamiento de la función SET y RESET. Para esto se usa una pila auxiliar, de tal modo que la operación SET guarda en esta pila un índice de bloque (BLOCK INDEX) que representa el lugar donde se encuentra almacenado el primer registro del bloque, y que al momento de la operación ocupa la parte superior (TOP) de la pila.

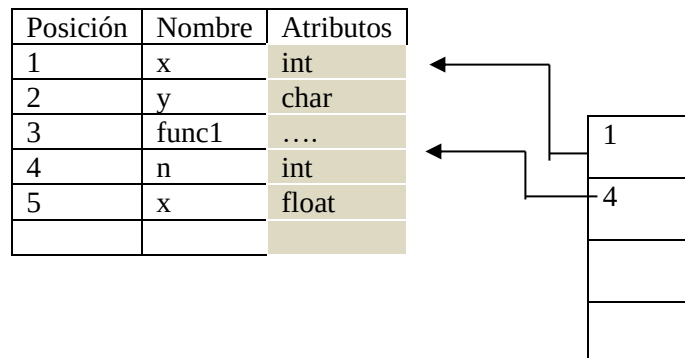
```
int x;
char y;
float func1(int n)
{ float x=10.50;
  x*=2;
  { .....
    int y[10]; /1*****/
    char z
  }
  .../2*****/
}
void func2(void)
{ int y;
  ...
}
void main(void)
{ char x; /3*****/
  ...
}
```

Volviendo al ejercicio anterior, en la línea marcada con **/*****/**, la tabla de símbolo y la pila auxiliar tiene la siguiente información.

Posición	Nombre	Atributos
1	x	int
2	y	char
3	func1	
4	n	int
5	x	float
6	y	Arreglo
7	Z	char



En la línea marcada con /2****/, la operación RESET saca de la pila todos los registros de las variables que se acaban de compilar en el bloque finalizado, para esto el BLOCK INDEX indica hasta que registro ha de sacar desde la parte superior de la tabla de símbolos.



ACTIVIDAD 8: Muestre la tabla de símbolos y la pila auxiliar en la línea marcada con /3****/.

Bibliografía:

Louden, Kenneth (2004) Lenguajes de Programación Principios y Práctica. Editorial Thomson. Mexico.
 Universidad de Oviedo (2006) Lenguajes y sistemas informáticos. Tablas de símbolos.
 Deitel, Harvey M. y Deitel, Paul J. (2003) Como programar en C, C++ . Pearson Educación. Cuarta Edición. Buenos Aires.
 Programación en C. Byron S. Gottfried. (1997): McGraw-Hill. Madrid - Buenos Aires.
 Joyanes Aguilar, Luís (2001) Programación en C - Metodología, algoritmos y estructura de datos. McGraw-Hill. Madrid - Buenos Aires.
 Documentos y Guías de trabajos Prácticos propuesto por el equipo de cátedra.

Unidad 4: Recursividad

Introducción

La recursividad es una alternativa a la iteración, es un proceso mediante el cual se puede definir una función en términos de sí misma.

La recursividad en programación es una técnica fundamental que permite a una función llamarse a sí misma para resolver un problema.

Generalmente, la solución recursiva a un problema planteado suele resultar menos eficiente en cuanto al tiempo de ejecución y al almacenamiento en memoria. A pesar de esta dificultad, en muchos casos si el algoritmo está definido recursivamente; se elige la solución recursiva por ser más simple y natural que la solución iterativa correspondiente, permitiendo así, reducir su complejidad y ocultar detalles de programación.

La recursividad es una de las formas de control más importantes en la programación debido a que los procedimientos recursivos son la forma más natural de representar muchos algoritmos. Es importante tener en cuenta que un procedimiento o una función recursiva debe cumplir con dos propiedades generales para no dar lugar a un ciclo infinito con las sucesivas llamadas:

- Cumplir una cierta condición o criterio base del que dependa la llamada recursiva.
- Cada vez que el procedimiento o función se llamen así mismos, directa o indirectamente debe estar más cerca del incumplimiento de la condición de que depende la llamada.

Tipo de recurrencias

Directa. Este tipo de recursividad es la más común y se presenta cuando una función se llama así misma una o varias veces. (este modelo se desarrollará en los contenidos de la materia).

Ejemplo

Enunciado: Calcular el factorial de un número natural.

int factorial (int i)

```
{
    if ( i == 0 )
        return ( 1 )
    else
        return ( i * factorial ( i - 1 ) );
}
```

Indirecta. Este tipo de recursividad se presenta cuando una función es llamada de manera indirecta, es decir, a través de otra función, generando llamadas recursivas.

Ejemplo

Enunciado: Realizar un programa que solicite un número y muestre en pantalla si el número es par o impar. Para este ejercicio se debe crear una o varias funciones recursivas que determinen si el número es par o impar.

int par(int n)

```
{
    if (n == 0)
        return 1;
    return impar(n-1);
}
```

int impar(int n)

```
{
    if (n == 0)
        return 0;
    return par(n-1);
}
```

Funciones Recursivas

En el desarrollo temático de la unidad se trabajará solamente con funciones recursivas con recurrencia directa.

El factorial de un número natural, está definido iterativamente de la siguiente manera:

$$y \begin{cases} n! = n * (n-1) * (n-2) * \dots * 2 * 1, & \text{si } n > 0 \\ n! = 1, & \text{si } n = 0 \end{cases}$$

Ese mismo cálculo puede expresarse de manera recursiva, así:

$$y \begin{cases} n! = n * (n-1)!, & \text{si } n > 0 \\ n! = 1, & \text{si } n = 0 \end{cases}$$

A continuación, se presenta en lenguaje C la solución recursiva:

```
# include < stdio.h >
int factorial (int i ); /* declaración de la función factorial */
int main ( )
{
    int n;
    printf ("\n n = ");
    scanf ("%d", n);
    printf ("\n n! = %d\n", factorial ( n ));
}

int factorial (int i )
{
    if ( i == 0 )
        return ( 1 )
    else
        return ( i * factorial ( i - 1 ));
}
```

Puede observarse que la función factorial se llama así misma de manera recursiva, con un argumento real (i - 1), que decrece en cada llamada recursiva. Estas llamadas recursivas finalizan cuando el parámetro real toma el valor 0.

Cuando se ejecute este programa, se invocará a la función factorial sucesivamente, una vez desde main y (i - 1) veces desde dentro de sí misma, aunque no sea advertido por el programador.

Al construir una función recursiva, se debe asegurar la existencia de:

Un caso base (puede haber más de uno), que permite detener la invocación sucesiva de la función; caso contrario se tendrían una serie infinita de invocaciones sucesivas.

Uno o más casos generales, que permiten que la función se invoque a sí misma con valores de parámetros que cambian en cada llamada; acercándose cada vez más al caso base.

Cuando se ejecuta una función recursiva, las sentencias que aparecen después de cada invocación no se resuelven inmediatamente, éstas quedan pendientes. Por cada invocación recursiva, se genera y se apila en el stack o pila un registro de activación. Luego, esos registros de activación se van desapilando en el orden inverso a como fueron apilados, ejecutándose además aquellas sentencias que quedaron pendientes.



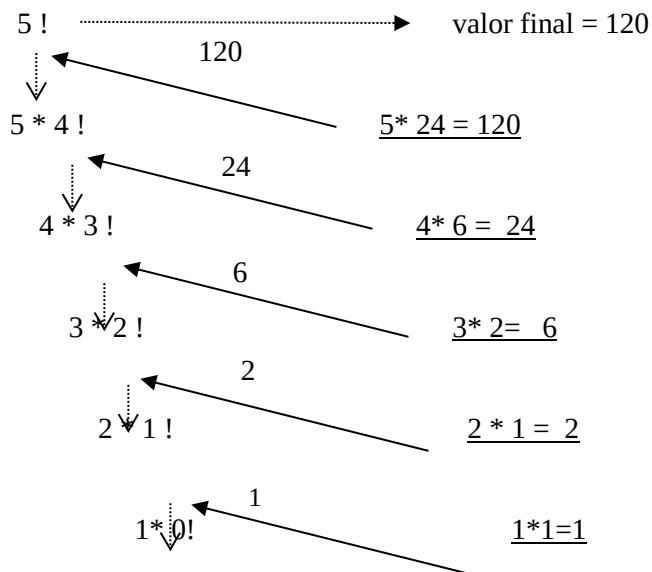
Retomando el ejemplo de la función factorial recursiva, las llamadas sucesivas de la función serán:

$$\begin{aligned} n! &= n * (n-1)! \\ (n-1)! &= (n-1) * (n-2)! \\ (n-2)! &= (n-2) * (n-3)! \\ &\dots\dots\dots \\ &\dots\dots\dots \\ 2! &= 2 * 1! \\ 1! &= 1 * 0! \\ 0! &= 1 \end{aligned}$$

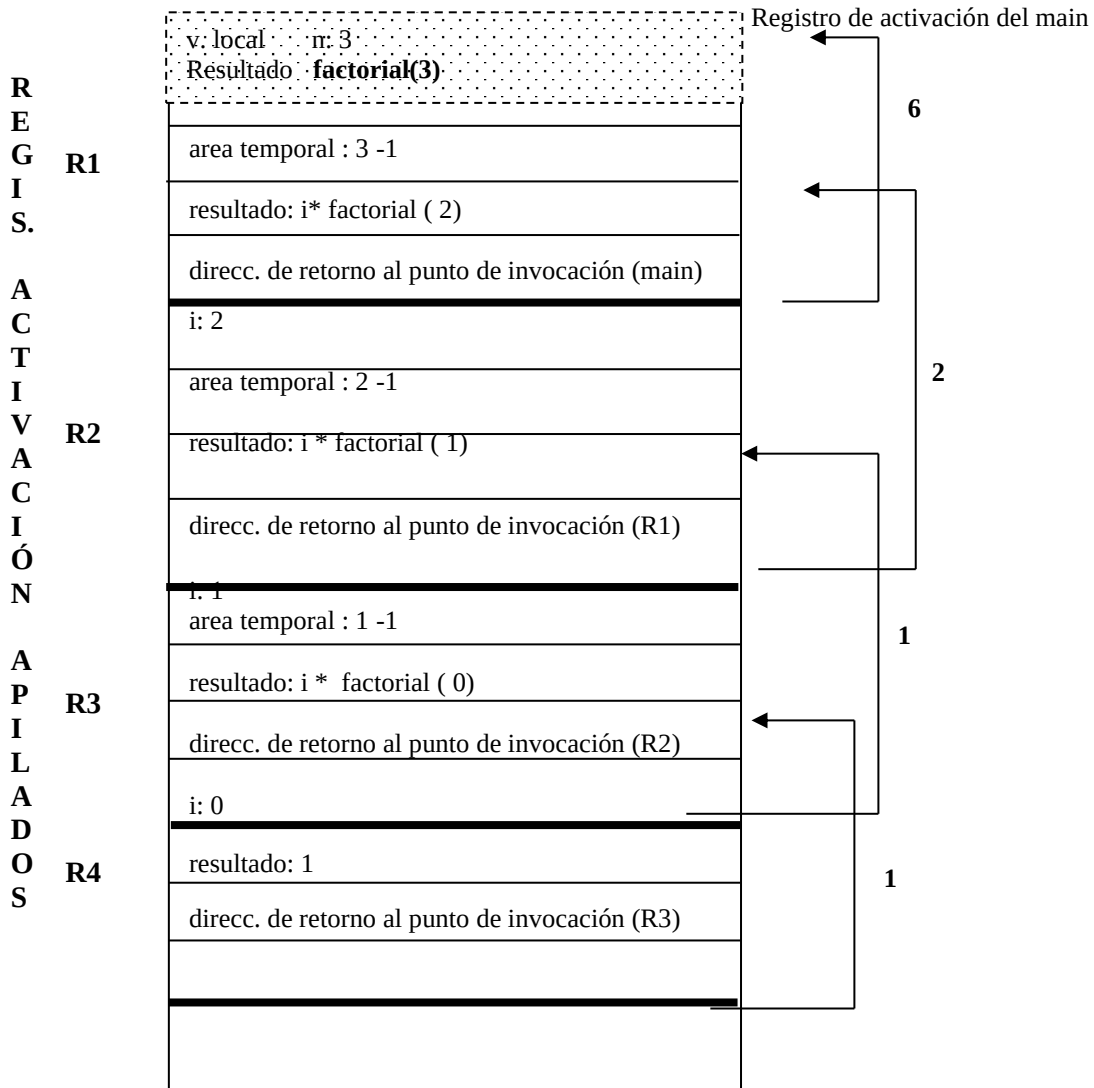
Luego, los valores de retorno se devolverán en el orden inverso:

$$\begin{aligned} 1! &= 1 \\ 2! &= 2 * 1 = 2 \\ 3! &= 3 * 2 = 6 \\ 4! &= 4 * 6 = 24 \\ &\dots\dots\dots \\ &\dots\dots\dots \\ n! &= n * (n-1)! \end{aligned}$$

Por ejemplo, al calcular 5!, se realizan sucesivas llamadas recursivas y finalmente, al encontrar el caso base, se retornan los valores obtenidos en cada llamada:



El orden inverso de ejecución es una característica típica de todos los algoritmos recursivos. Gráficamente, lo que sucede internamente en la memoria al invocar a la función factorial, por ejemplo para $n = 3$, es:



Al invocar a **factorial(3)**, se apila el primer registro de activación **R1**; en el cual se guarda el *parámetro actual* $n = 3$, la *dirección de retorno*, el *resultado de la función* y también un *área temporal de almacenamiento* para los cálculos intermedios. Como en el cálculo de factorial(3) está involucrado el cálculo de **factorial(2)**, se apila el registro de activación **R2**, que almacena el parámetro actual $n = 2$, el punto de retorno, el resultado y un área temporal de almacenamiento para los cálculos intermedios.

Como en el cálculo de **factorial(2)** está involucrado el cálculo de **factorial(1)**, se apila el registro de activación **R3**, que almacena el parámetro actual $n = 1$, el punto de retorno, el resultado y un área temporal de almacenamiento para los cálculos intermedios.

Del mismo modo, como en el cálculo de factorial(1) está involucrado el cálculo de **factorial(0)**, se apila el registro de activación **R4**, que almacena el parámetro actual $n = 0$, el punto de retorno y el resultado (en este caso no hace cálculos intermedios). Finalmente, al llegar al caso base, este resultado se transfiere al registro de activación **R3**, retornando al punto de invocación; desapilándose **R4**.

El resultado obtenido en **R3**, se transfiere al punto de invocación en **R2**, desapilándose **R3**.

El resultado obtenido en **R2**, se transfiere al punto de invocación de **R1**, desapilándose **R2**.

El resultado obtenido en **R1**, retorna al punto del programa desde donde fue invocada factorial(3); desapilándose **R1**.

Al invocar una función recursiva, se apilan sucesivamente cada uno de los registros de activación, correspondientes a las distintas invocaciones. Al llegar al caso base, comienzan a desapilarse cada uno de esos registros de activación, resolviendo tareas pendientes, si es que existen.

Cabe aclarar que al ejecutar una función recursiva, ésta genera un número considerable de auto invocaciones; se corre el riesgo de ocupar gran cantidad de memoria en la pila.

Si una función recursiva posee variables locales, se creará un conjunto diferente de variables por cada llamada. Obviamente, los nombres de esas variables locales serán siempre los mismos; sin embargo, las variables representarán un conjunto diferente de valores cada vez que se invoque la función.

Tanto la recursión como la iteración son lógicamente equivalentes; esto es, todo algoritmo recursivo puede escribirse de manera iterativa y viceversa.

Generalmente, la recursividad presenta mayor grado de simplicidad y síntesis, al momento de codificar. La desventaja se presenta normalmente en tiempo de ejecución, debido a la cantidad de memoria utilizada.

ACTIVIDAD 1: Completar el programa con una función recursiva “divisor” que recibe dos números naturales distintos a y b, $a > b$. La función devuelve un valor 1 al programa principal para que se escriba un mensaje diciendo que b es divisor de a, caso contrario cuando b no es divisor de b devuelve cero(0).

Hacer el mapa de memoria para cada una de las soluciones planteadas con el lote de prueba $x=7$, $y=2$.

```
#include <stdio.h>
#include <conio.h>
```

```
int divisor (int x, int y) //completar
{
    ----
    ----
    ----
}
```

```
main(void)
{
    int a,b,r;
    printf("Ingrese dos números naturales distintos el primero mayor que el segundo\n");
    scanf("%d %d",&a, &b);
    if (divisor(a,b)==1)
        printf("%d es divisor de %d", a,b);
    else
        printf("no hay divisores");
    getch();
}
```

ACTIVIDAD 2: Para el siguiente programa:

- Defina una función “Binario” que transforme un número natural en el número binario correspondiente.
- Graficar el mapa de memoria al invocar a Binario (9).
- ¿Puede generalizar el algoritmo para sistemas de base 3, 6, etc.? ¿Qué modificaciones deberían hacerse?

```
#include <conio.h>
#include <stdio.h>

void Binario(int n) //completar
{
    -----
    -----
}

void main()
{
    int n;
    printf("\n Introduzca un entero positivo: ");
    scanf("%d", &n);
    printf("\n El Numero Decimal %d en binario es ", n);
    Binario(n);
    getch();
}
```

ACTIVIDAD 3

Teniendo en cuenta la siguiente definición de una función recursiva:

```
void listar ( int n)
{
    if ( n != 0 )
    { printf (" %d", n);
      listar ( n - 1 );
      printf (" %d", n);
    }
    else
        printf (" el número es cero");
    return;
}
```

- 1) Construir en C, el programa que pueda invocarla.
- 2) Realizar el seguimiento
- 3) Graficar el mapa de memoria al invocar a **listar (4)**.

Ejemplo : La siguiente función calcula, recursivamente, la suma de los números pares menores o iguales que un valor ingresado por teclado.

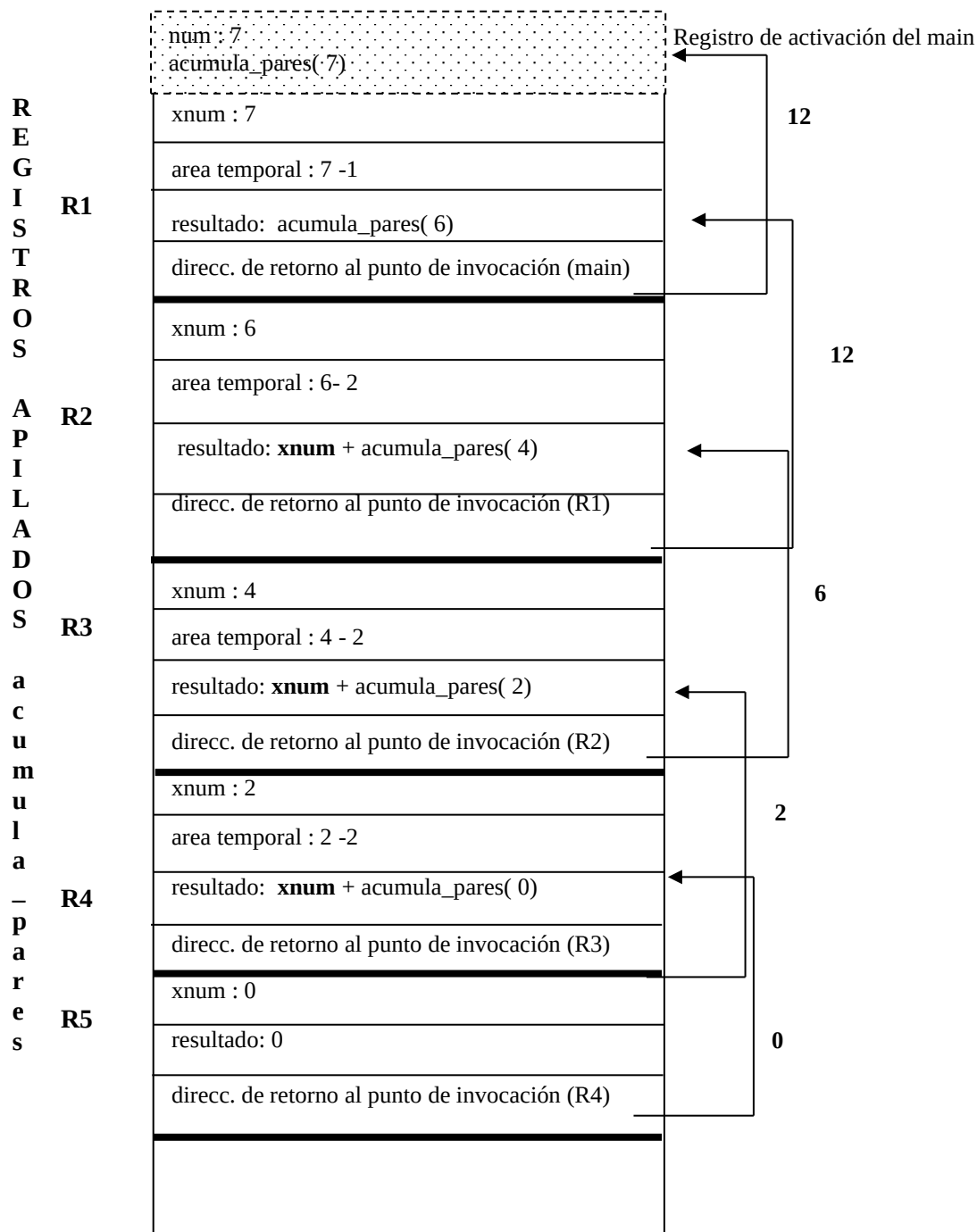
```
#include <stdio.h>
#include <conio.h>
int acumula_pares ( int xnum)
{
    if (xnum)
        if ((xnum % 2) == 0)
            return xnum + acumula_pares( xnum - 2);
        else return(acumula_pares( xnum - 1));
    else return (0);
}
```

En este caso, la función presenta tres sentencias return que se corresponden con el caso base y los dos casos generales.

La invocación puede verse en el siguiente algoritmo:

```
int main()
{ int num;
  scanf("%d", &num);
  printf("\n suma de pares menores que: %d es %d", num, acumula_pares(num-1));
  getch();
}
```

A continuación se muestra en forma gráfica cómo se apilan los distintos registros de activación en la llamada de la función para un valor de num igual a 7.



Puede observarse que el valor del parámetro actual **num** no se modifica, a la salida de la ejecución de la función `acumula_pares`.

Funciones recursivas que devuelven más de un resultado

Modifiquemos el programa anterior de modo que la función retorne al programa principal los resultados de la suma de los números pares y de la suma de los números impares menores o iguales que un número leído desde teclado.

Ejemplo : Realizar el seguimiento, y mostrar la organización de la memoria cuando se ejecuta `acumula_pares_impares` con `xnum = 5`.

```
int acumula_pares_impares ( int xnum, int &imp)
{
    if (xnum)
        if ((xnum % 2) == 0)
            return xnum + acumula_pares_impares ( xnum - 1,imp);
        else
            { imp+= xnum;
              return (acumula_pares_impares ( xnum - 1,imp));
            }
    else return (0);
}
int main()
{ int num, impares=0;
  scanf("%d", &num);
  printf("\n suma de pares menores que: %d es %d", num, acumula_pares_impares(num-1,impares));
  printf("\n suma de impares impares es : %d:", impares);
  getch();
}
```

ACTIVIDAD 4: Realizar el seguimiento de la siguiente función recursiva.

- Analice qué realiza para los siguientes lotes de prueba [a:18, b:4]; [a:23, b:5]; [a:21, b:3]
- Realice el mapa de memoria para el primer lote de prueba.

```
int funcion (int x, int y, int &r)
{
    if ((x-y)<0)
        return 0;
    else
        {
            r= x-y;
            return 1 + funcion(x - y, y, r);
        }
}

int main (void )
{ int a,b,c=0;
  printf("\n INGRESE DOS VALORES\n ");
  scanf("%d %d", &a, &b);
  printf("resultados %d --- %d ", funcion(a,b,c), c);
  getch();
}
```

Recursividad usando Arreglos

Dentro de las estructuras que se pueden procesar recursivamente se encuentran los arreglos. A continuación, se analizan algunos ejemplos:

Ejemplo

Calcular recursivamente la suma de las componentes de un arreglo.

Se declara y se inicializa el arreglo arre:

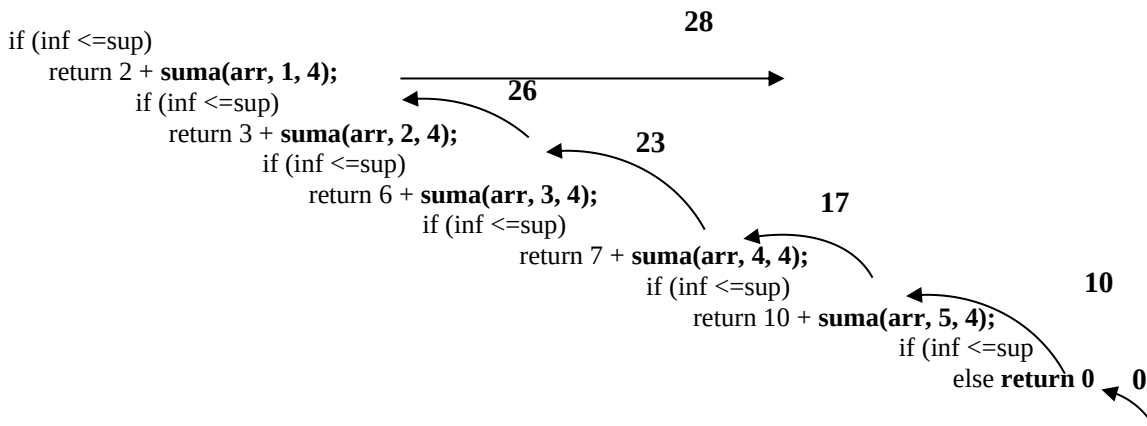
```
int arre[5]={2, 3, 6, 7, 10}
```

se define una función **suma** que reciba el arreglo arre, el límite inferior (0) y el límite superior (4).

La invocación desde el programa principal será **suma(arre, 0,4)** y el prototipo de la función es:

```
int suma (int arr[], int inf, int sup);
```

Esquema de la solución:



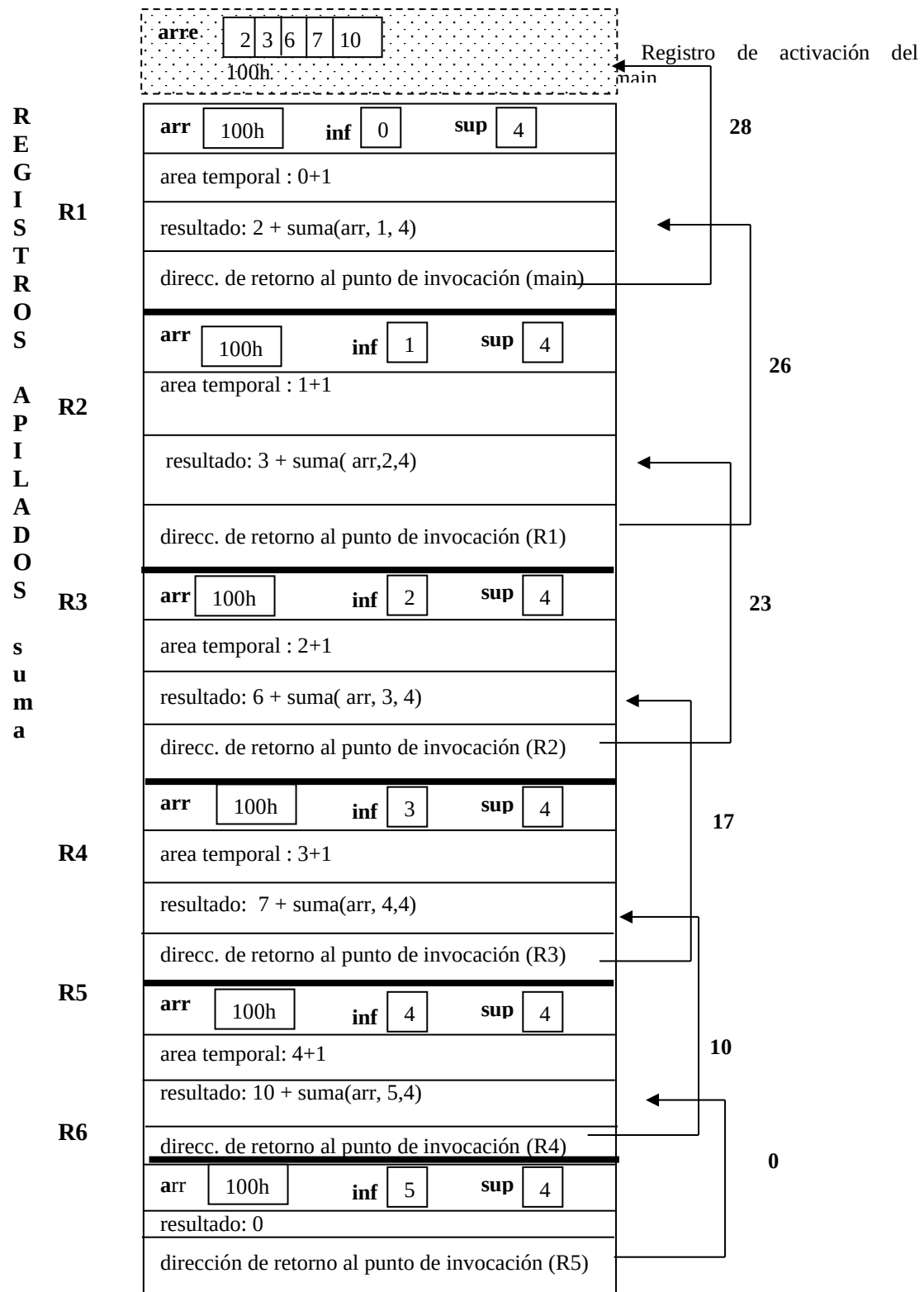
Código en lenguaje C

```
#include<stdio.h>
#include<conio.h>
int suma(int arr[], int inf, int sup)
{
    if (inf <=sup)
        return arr[inf] + suma(arr, inf+1, sup);
    else return 0;
}

int main()
{ int arre[5]={2, 3, 6, 7, 10} ;
  printf("\n Suma de las componentes %d", suma(arre, 0,4));
  getch();
}
```



Mapa de Memoria



ACTIVIDAD 5

Modifique el algoritmo anterior de modo que la función devuelva la suma de las componentes y el total de componentes mayores a 5.

Ejemplo : El siguiente programa usa las funciones recursivas **carga**, **busq_sec_rec** y **escalar**, para realizar las siguientes tareas:

- 1- Cargar dos vectores de 5 componentes enteras positivas
- 2- Dado un valor, indicar en cuál de los vectores se encuentra
- 3- Calcular el producto escalar de los dos vectores

```
void carga (int arr[], int i, int n)
{
    if (i!=n)
    {
        printf ("\n ingrese el valor de la posición %d:", i);
        scanf ("%d",arr+i);
        carga(arr, i+1,n);
    }
    else printf ("\n el arreglo esta cargado \n");
}

int busq_sec_rec ( int arr[], int xi, int n, int elem)
{
    if( xi==n )
        return -1;
    else if (arr[xi]==elem)
        return 1;
    else return busq_sec_rec ( arr, xi+1,n,elem);
}

// esta función recibe la última posición del arreglo
int escalar (int x[],int y[],int j)
{
    if (j >= 0)
        return x[j]*y[j]+ escalar(x,y,j-1);
    else return 0;
}

int main(void)
{
    int a[n], b[n], valor;
    printf ("\n Carga del primer vector \n"); carga(a,0);
    printf ("\n Carga del segundo vector \n"); carga(b,0);
    printf ("\n Ingrese valor a buscar en los vectores: ");
    scanf ("%d",&valor);
    if (busq_sec_rec(a,0,valor)==1)
        printf ("\n El valor está en el primer vector\n");
    else printf ("\n El valor NO está en el primer vector\n");
    if (busq_sec_rec(b,0,valor)==1)
        printf ("\n El valor está en el segundo vector\n");
    else printf ("\n El valor No está en el segundo vector \n");
    printf ("\n Producto escalar de los vectores = %d", escalar(a,b,n -1));
    getch();
}
```


NOTA

- La función **busq_sec_rec** recibe como parámetros formales el arreglo, la posición del primer elemento, el tamaño y el elemento a buscar. En esta función el recorrido del arreglo se inicia en la posición 0.
- La función **escalar** recibe dos arreglos de igual dimensión y la posición del último elemento de los arreglos. En este caso el recorrido se realiza desde la última componente del arreglo hasta la primera componente (posición 0), por eso no hace falta enviar como parámetro la posición del primer elemento.

ACTIVIDAD 6

- Construya una función recursiva que invierta los elementos de un arreglo.
- Construya la función recursiva búsqueda binaria

ACTIVIDAD 7

Realice el seguimiento del siguiente programa cuando se ejecuta la función **muestra**, y el mapa de memoria correspondiente.

```
void muestra(int arr[], int inf, int sup)
{
    if (inf <= sup)
    {
        muestra(arr, inf+1, sup);
        printf("\n %d", arr[inf]);
    }
    else return;
}

int main(void)
{
    int arre[5]={2, 3, 6, 7, 10};
    muestra(arre, 0,4);
    getch();
}
```

Recursión versus Iteración

La solución recursiva de ciertos problemas simplifica mucho la estructura de los programas. Sin embargo, existe una desventaja, en la mayoría de los lenguajes de programación las llamadas recursivas a funciones tienen un costo de tiempo mucho mayor con respecto a las soluciones iterativas. Se puede, por tanto, afirmar que la ejecución de un programa recursivo va a ser más lenta y menos eficiente que el programa iterativo que soluciona el mismo problema, aunque, a veces, la sencillez de la estructura recursiva justifica el mayor tiempo de ejecución.

Tanto la recursión como la iteración se basan en una estructura de control:

la iteración utiliza una estructura de repetición

la recursión utiliza una estructura de selección.

Tanto la recursión como la iteración implican repetición:

la iteración utiliza la repetición de manera explícita

la recursión consigue la repetición mediante repetidas llamadas a una misma función.

La recursión y la iteración involucran una prueba de terminación:

- la iteración termina cuando falla la condición de continuación del ciclo
- la recursión termina cuando se reconoce un caso base.



Unidad 5: Estructuras Dinámicas

Introducción

Hasta ahora los tipos de datos simples y estructurados con que se ha trabajado son de almacenamiento fijo o estático. Esto significa que el sistema se encarga de asignar a cada variable el área que ocupará en tiempo de compilación, la que permanecerá fija durante toda la ejecución de la función que la declara. No se podrá utilizar por tanto esas posiciones de memoria para otros fines, durante dicha ejecución. En el caso de variables globales, el almacenamiento se mantiene fijo durante toda la ejecución del programa y se usa para ellas la zona de memoria que llamamos de **Almacenamiento Estático**. Para las variables locales, el almacenamiento que se les asigna corresponde a la **Pila (Stack)**.

Existen situaciones en las que es más apropiado utilizar variables o estructuras que puedan crearse en tiempo de ejecución, de modo que se permita liberar su espacio de almacenamiento cuando no se las necesite así como modificar su tamaño durante la ejecución del programa. A este tipo de variables y estructuras se las denomina **dinámicas**.

El uso de punteros o apuntadores permite el manejo de objetos de datos dinámicos.

La diferencia entre objetos de datos creados en tiempo de compilación y los creados en tiempo de ejecución es que:

- El objeto creado en tiempo de ejecución no necesita tener nombre
- Estos objetos se pueden crear en cualquier punto durante la ejecución de un programa, no sólo al entrar al subprograma.

Se puede sintetizar diciendo que una variable dinámica se crea y se libera por petición, durante la ejecución del programa y no en forma automática como las de almacenamiento estático.

El Montículo o Montón(Heap)

Cuando el tamaño de un objeto a colocar en memoria puede variar en tiempo de ejecución, no es posible su ubicación en memoria estática, ni en la pila. Son ejemplos de este tipo de objetos los arreglos dinámicos, las listas enlazadas, las cadenas de caracteres de longitud variable, etc. Para manejar este tipo de objetos el compilador debe disponer de un área de memoria de tamaño variable y que no se vea afectada por la activación o desactivación de subprogramas. Este trozo de memoria se llama **montículo o montón** (traducción de *heap*).

Las operaciones básicas que se realizan sobre el montículo son:

Alojamiento: Se demanda un bloque de memoria para almacenar un objeto de un cierto tamaño.

Desalojo: Se indica que ya no es necesario conservar un objeto de dato, la memoria que ocupa debe quedar libre para ser reutilizada en caso necesario por otros objetos.

Según sea el programador o el propio sistema el que las invoque, estas operaciones pueden ser explícitas o implícitas respectivamente. En caso de alojamiento explícito el programador incluye en el código fuente una instrucción que demanda una cierta cantidad de memoria para la ubicación de un dato (en C mediante *malloc*, en PASCAL mediante la instrucción *new*, etc.).

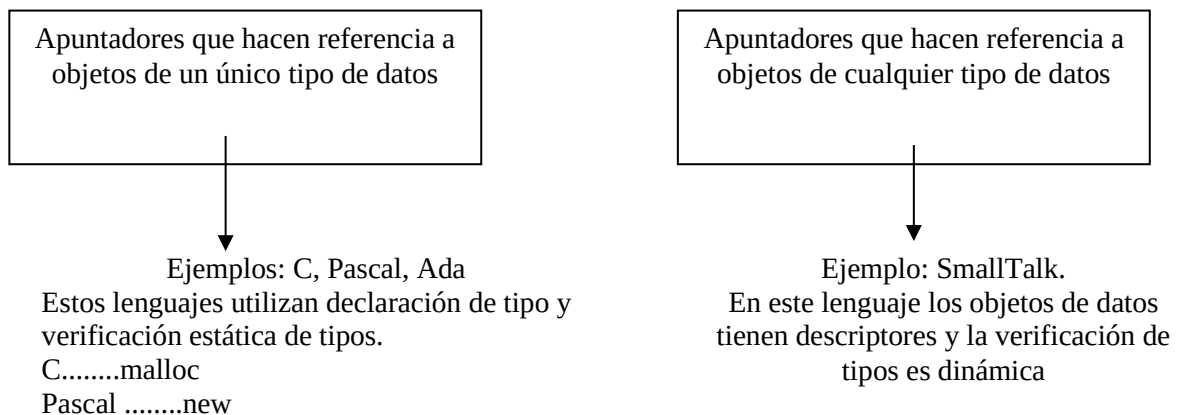
La cantidad de memoria requerida puede ser calculada por el compilador en función del tipo de dato correspondiente al objeto que se desea alojar, o puede ser especificado directamente por el programador. El resultado de la función de alojamiento es por lo general un puntero a un trozo contiguo de memoria dentro del montículo que puede usarse para almacenar el valor del objeto. Los lenguajes de programación imperativos utilizan por lo general alojamiento y desalojo explícitos.

La gestión del montículo requiere técnicas adecuadas que optimicen el espacio que se ocupa o el tiempo de acceso, o bien ambos factores.

Como los apuntadores permiten definir objetos de datos de tamaño variables, el lenguaje debe poseer las siguientes características:

1. Un tipo elemental de datos apuntador, también llamado tipo de referencia o de acceso (un tipo puntero).
2. Una operación de creación de objetos de tamaño fijo. La operación devuelve el valor L del bloque de almacenamiento creado y ese valor L se convierte en el valor R de una variable del tipo apuntador.
3. Una operación de desreferenciar para objetos del tipo apuntador, la cual permite acceder al valor del objeto al cual apunta.

El tipo apuntador se puede tratar de dos maneras:



IMPLEMENTACION: Existen dos formas de representación de almacenamiento para valores de una variable apuntador:

Direcciones Absolutas: El apuntador contiene la dirección de memoria real del bloque de almacenamiento del objeto de datos.

Direcciones Relativas: El apuntador contiene el desplazamiento del objeto respecto a la dirección base de un bloque determinado de almacenamiento más grande que contiene al objeto (montículo).

Manejo del Montículo en Lenguaje C

FUNCIONES MALLOC Y FREE

El lenguaje C provee funciones que permiten el manejo de memoria de forma dinámica. La función **malloc** (**M**emory **ALLO**Cation: asignación de memoria) permite asignar espacio en tiempo de ejecución y la función **free** permite liberarlo. Los prototipos de estas funciones se encuentran en el archivo de cabecera `<alloc.h>` para el caso de C++ y `<malloc.h>` para el caso de DEV C++.

La función **malloc** solicita al sistema operativo un bloque de memoria en el montículo, del tamaño especificado en el argumento. Su resultado es la dirección de comienzo del bloque asignado o la constante **NULL**, si no hay espacio disponible.



Formato: (void *) malloc (tamaño del bloque)

donde *tamaño*, hace referencia a la longitud en bytes del bloque.

La función malloc devuelve un puntero a void o puntero genérico, por esto se debe realizar la conversión forzada de tipo, utilizando el operador cast, que permite especificar el tipo de dato al que debe apuntar el puntero.

Por ejemplo si se debe solicitar espacio para almacenar 10 componentes enteras, la función malloc es:
int *p;

$$p = (\text{int } *) \text{ malloc}(10 * \text{sizeof}(\text{int}))$$

Op. Cast
Pedido de Memoria
Tamaño del Bloque

malloc devuelve la dirección de comienzo del bloque en un puntero a void, por ese motivo se debe hacer una conversión explícita de tipo (cast) para convertirlo en un puntero a entero.

El Sistema Operativo maneja una tabla de direcciones de memoria que indica el espacio de memoria ocupado (para que un programa no modifique direcciones de memoria que no le corresponden, corrompiendo la memoria de otros procesos / programas / información).

malloc indica que se va a cargar en memoria una estructura y por ello el sistema operativo registra en su tabla de direcciones de memoria que un bloque ha sido ocupado, y envía a malloc la dirección de inicio de ese bloque.

La función free, libera bloques asignados previamente por malloc. De esta manera existe más espacio de memoria disponible para posteriores asignaciones.

Formato: free (variable puntero);

Por ejemplo si se desea liberar el espacio antes solicitado, la función free es:

free(p);

free indica que el bloque de memoria asignado a través de malloc se ha dejado de usar, con esta información el Sistema Operativo registra en su tabla de direcciones de memoria que dicho bloque ha sido desocupado. La variable p conserva el valor que tenía antes de free, es decir no cambia su contenido, por este motivo es que para evitar errores de acceso, resulta ser una buena práctica inicializar la variable después de su liberación (p = NULL;).



Lenguaje C: Variables dinámicas simples

El siguiente ejemplo muestra las sentencias necesarias para generar una variable dinámica de tipo entero.

Ejemplo 1: Almacenar en una variable dinámica un número entero

```
#include <alloc.h>
void main(void)
{
    int * n;                                1
    n = (int *) malloc (sizeof (int))      2
    *n=12;                                3
    :
    free(n);                                4
}
```

En la sentencia 1 se declara un puntero a entero, luego en la sentencia 2, se solicita espacio para almacenar en el montículo una variable dinámica entera y la dirección obtenida como resultado, se guarda en el puntero n.

Mediante la sentencia 3, se asigna un valor a la variable apuntada por el puntero.

La sentencia 4 permite liberar el espacio reservado a la variable dinámica apuntada por n.

La figura 1 muestra la organización de la pila y el montículo, antes de invocar a la función free.

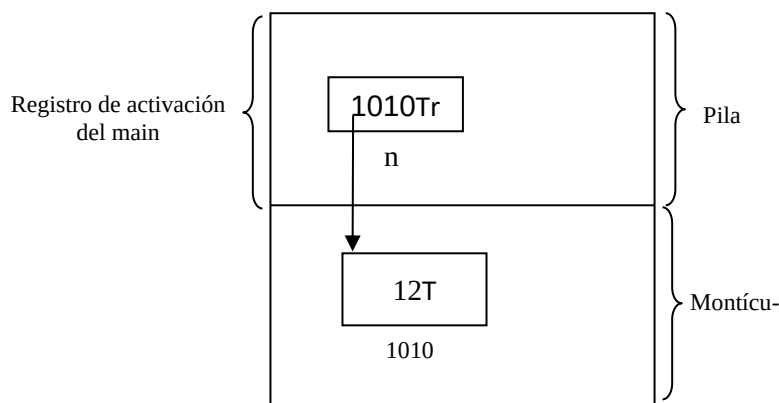


Fig. 1 Organización de la pila y el montículo cuando se crea una variable dinámica

La variable dinámica se almacena en el montículo y se libera cuando ya no se necesita, quedando ese espacio libre para posteriores asignaciones.

¿Cómo sería la gráfica de memoria después de ejecutar `free(n)`? Exactamente igual, solo cambiaría la Tabla de direcciones de memoria del Sistema Operativo.

Un problema a tener en cuenta

El problema principal de los apuntadores y los objetos de datos construidos por el programador es la asignación de memoria asociada a la operación “crear”.

El **lenguaje C** utiliza para manejo de funciones y recursividad una gestión de almacenamiento basada en pilas. Cuando se agrega punteros con la operación `malloc`, es necesaria una ampliación considerable de la estructura global de la gestión de almacenamiento.

Para objetos de tamaño fijo, el tiempo de vida se *inicia* con la asignación de una localidad de almacenamiento para el objeto (enlace del objeto al bloque de almacenamiento) y *termina* cuando se elimina el enlace del objeto al bloque de almacenamiento. Cuando se crea el objeto, es decir al inicio de su tiempo

de vida, se crea una *ruta de acceso* al objeto, para que las operaciones del programa puedan tener acceso al objeto de datos. La ruta de acceso se logra asociando al objeto un identificador o nombre o bien a través de un puntero al objeto el cual se guarda en otro objeto conocido y de fácil acceso.

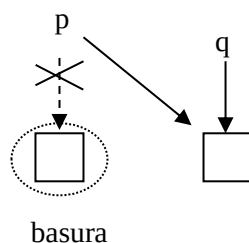
Puede haber varias rutas de acceso adicionales, por ejemplo cuando se pasa un objeto como argumento en un subprograma o cuando se crean varios punteros al objeto. Por lo tanto, existen varios problemas que se originan por la acción conjunta del tiempo de vida y las rutas de acceso a una estructura de datos, a saber:

Basura: En el contexto de la gestión de almacenamiento en montículos, un elemento basura es aquel que está disponible para nuevo uso pero no se halla en la lista de espacios libres de la Tabla de direcciones de memoria del Sistema Operativo, es decir no se ha ejecutado una sentencia `free`, por ello se ha vuelto inaccesible. Si se pierde la ruta de acceso a una estructura (por ejemplo a causa de una sobre escritura por asignación) pero la estructura misma no ha sido liberada para que se recupere el almacenamiento, entonces la estructura se convierte en basura.

Si la basura se acumula, el almacenamiento disponible se reduce de manera gradual hasta que el programa puede ser incapaz de continuar por falta de espacio libre.

Ejemplo en lenguaje C:

```
int *p, *q;
q=(int *) malloc(sizeof (int));
p=(int *) malloc(sizeof (int));
....
p=q;
....
free(p);
....
```



Referencias desactivadas o bamboleantes: Si se destruye un objeto creado dinámicamente (se libera el almacenamiento usando `free`) antes de haber eliminado todas las rutas de acceso a él (se hicieron asignaciones de ese objeto a otras variables), toda ruta de acceso remanente (esas otras variables) se convierte en una referencia desactivada. Por lo tanto, si luego se usa esa ruta para hacer una asignación podría modificarse el almacenamiento de algún otro objeto o provocar errores en la integridad del sistema de gestión de almacenamiento.

Ejemplo 2

```
void modifica(int *&p)
{
    int *x;
    x=(int*)malloc(sizeof(int));
    *x=25;
    p=x;
    free(x);
}
```

```
void main(void)
{
    int * n;
    modifica (n);
    :
}
```

El uso de **p** después de ejecutar la función `modifica` puede acarrear problema al apuntar a una dirección de memoria que ha sido liberada con `x`.

Lenguaje C: Arreglos dinámicos

El arreglo, tal como se ha visto hasta el momento, es un ejemplo de una estructura de almacenamiento fijo. El espacio reservado cuando se declara la estructura, no puede modificarse durante su tiempo de vida.

Cuando la asignación de almacenamiento es estática, las direcciones de almacenamiento para todos los datos se generan en tiempo de compilación, esto hace eficiente a este mecanismo de gestión de memoria, por cuanto no se gasta tiempo durante la ejecución.

Sin embargo, en muchos casos, la utilización de estructuras de almacenamiento fijo no conduce a la solución más eficiente del problema desde el punto de vista del manejo de memoria. Por ejemplo, si se define un arreglo de 80 componentes y durante la ejecución del programa sólo se ocupan 30, se están desperdiciando 50 lugares. Por el contrario, podría ocurrir que este espacio reservado resultara insuficiente.

Como el nombre del arreglo es un puntero constante al primer elemento del mismo, también se puede definir un arreglo como una variable puntero, en lugar de hacerlo de la forma convencional.

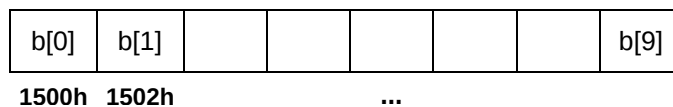
Un arreglo de 10 componentes enteras se puede declarar entonces de dos maneras:

int b[10]; ó utilizando una variable puntero **int *b;**

Estas dos declaraciones difieren en cuanto al almacenamiento de memoria.

Con la definición convencional **int b[10]**, el sistema reserva un *bloque fijo y consecutivo* de diez celdas de memoria, que permanece durante toda la ejecución de la función en la que el arreglo está declarado. Por esto también se dice que el almacenamiento es estático.

Gráficamente:



Si se analiza la segunda declaración **int *b**, surge una primera pregunta ¿es posible, a partir de un puntero a entero, reservar espacio para almacenar diez números?

Como se ha visto, la función **malloc** solicita al sistema operativo, en tiempo de ejecución, la asignación de un bloque de memoria en el montículo del tamaño especificado en su argumento.

En el ejemplo considerado, se debe solicitar espacio para almacenar 10 componentes enteras, de la siguiente manera:

b = (int *) malloc(10 * sizeof(int));

El puntero **b** recibe la dirección de comienzo del bloque o la constante **NULL** si no hay espacio disponible.

Este modo de creación de arreglos tiene dos momentos:

Declaración de la variable de acceso (nombre del arreglo): **int *b;**

Asignación dinámica de memoria: **b = (int *) malloc (10 * sizeof(int));**

A diferencia la declaración estática que hace ambos pasos en un mismo tiempo.

La siguiente, es una síntesis de las diferencias entre un arreglo estático y un arreglo dinámico, en cuanto a la declaración, manipulación y almacenamiento.

Arreglo Estático

```
void carga(int x[], int n)
```

```
{ int i;
  for(i=0; i<n; i++)
    scanf("%d",&x[i]);
}
```

```
void main(void)
```

```
{int b[10];
```

```
carga(b, 10);
```

```
....
```

```
}
```

Arreglo Dinámico

```
void carga (int *x, int n)
```

```
{ int i;
  for (i=0; i<n; i++)
    scanf("%d",&x[i])
}
```

```
{ int *b;
```

```
b=(*int) malloc(sizeof(int)*10);
```

```
carga(b,10);
```

```
.....
```

```
free(b);
```

```
}
```

Como se observa en la figura 2, el arreglo estático se genera en la pila, y está disponible durante toda la ejecución de la función que lo declara. El arreglo dinámico se genera en el montículo, por lo tanto su almacenamiento puede liberarse cuando se considere adecuado.

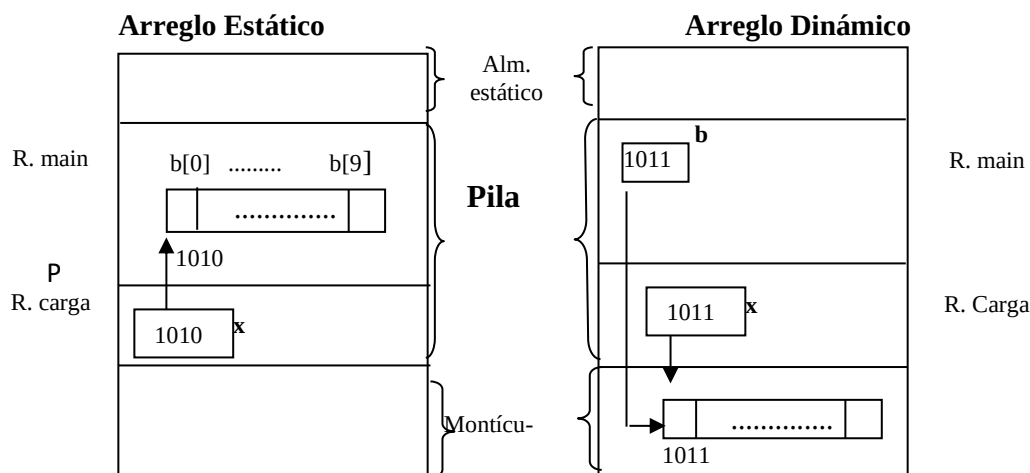


Fig. 2 Organización de la memoria cuando se genera un arreglo estático y un arreglo dinámico

Ejemplo 3

El Instituto Provincial de la Vivienda ha implementado un sistema que consta de 5 planes de pago distintos, con el fin de que los adjudicatarios de sus viviendas puedan cancelar sus deudas. Por cada uno de los 5 planes, se ingresa en forma ordenada la cantidad de adjudicatarios adheridos y por cada uno de ellos el monto adeudado.

Se necesita realizar un programa, que utilizando de manera óptima funciones, informe:

Para cada plan:

- Monto total adeudado por los adjudicatarios adheridos al mismo.
- Monto promedio adeudado.
- Cantidad de usuarios cuyo monto adeudado es superior al promedio calculado.
- El o los números de planes con mayor cantidad de adjudicatarios

Un análisis de la situación propuesta, plantea la necesidad de generar las siguientes estructuras: un arreglo manejado en forma dinámica para cada uno de los planes, que almacene la deuda de los usuarios adheridos al mismo y un arreglo estático de 5 componentes que contenga la cantidad de usuarios adheridos a cada plan.

El siguiente código resuelve el problema planteado.

```
#include <stdio.h>
#include <alloc.h>
#include <ctype.h>
#define N 5

void carga( float *t, int xcu)
{
    printf("\n Ingrese la deuda de los %3d Usuarios ",xcu);
    for( int i=0; i<xcu; i++)
        scanf("%f",&t[i] );
    return;
}

float total( float *t, int cu)
{
    float prom=0;
    for(int i=0; i<cu;i++)
        prom=prom+ t[i];
    printf("\n Total adeudado %5.2f ", prom );
    prom= prom/cu;
    return prom;
}

int cantidad( float *t, int cu, float prom)
{
    int c=0;
    for( int i=0; i<cu; i++)
        if (t[i]>prom)
            c++;
    return(c);
}

void maximo(int *tot, int N)
{
    int max=0, i;
    for(i=0; i < N; i++)
        if (tot[i] > max)
            max=tot[i];
    printf("\n Planes con mayor cantidad de adjudicatarios" );
    for( i=0; i < N; i++)
        if (tot[i] == max )
            printf("\n  %3d ",i+1 );
    return;
}
```

```

void main(void)
{ int i,cu,totu[5];
  float *monto, prom;
  for(i=0; i<N;i++)
  { printf("\n Ingrese cantidad de adjudicatarios del plan %3d ",i+1 );
    scanf("%d",&cu);
    totu[i]=cu;
    monto = (float *) malloc(cu * sizeof(float)); /* solicita espacio para almacenar la deuda
    carga(monto, cu);                             de los adjudicat. de un plan*/
    prom=total(monto, cu);
    printf("\n Promedio adeudado por Adjudicatarios del plan %3d es %5.2f ", i+1, prom );
    printf("\n Total adjudic. con monto superior a promedio %4d",cantidad(monto, cu,prom));
    free(monto); // libera el espacio asignado a un plan, cuando no se necesita
  }
  maximo(totu,5);
}

```

La figura 3 muestra el manejo de memoria cuando se invoca la función carga. Suponiendo que el programa principal ejecuta sólo el primer plan y que dicho plan tiene 8 adjudicatarios.

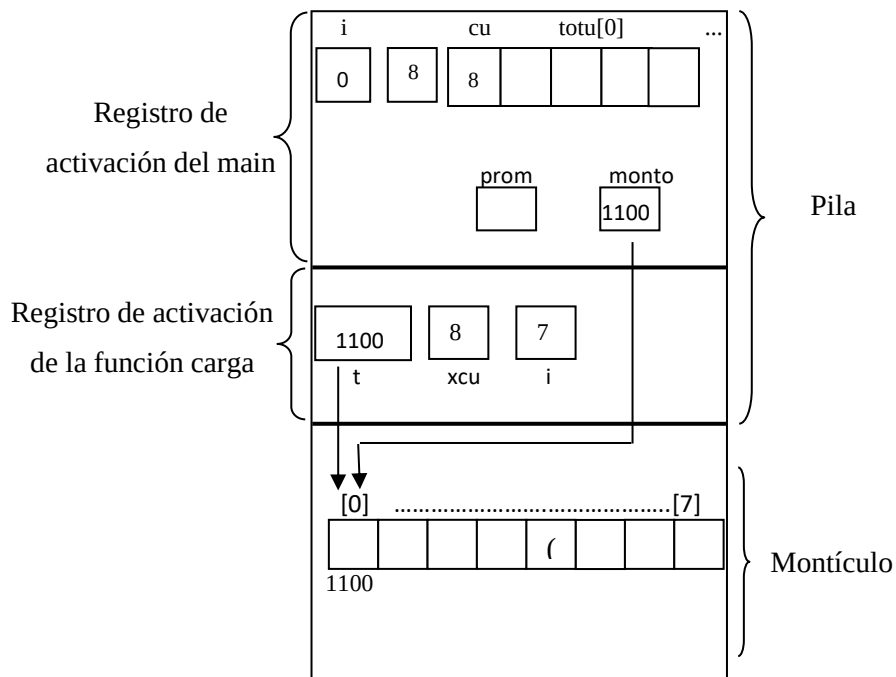


Fig. 3 Organización de la memoria cuando se ejecuta la función carga

En el montículo se reserva espacio para un único arreglo unidimensional por cada plan, cuyo tamaño varía según un valor ingresado desde teclado. El espacio reservado es liberado cada vez que se termina de procesar la información del plan correspondiente.

ACTIVIDAD 1

Codificar nuevamente la función carga, suponiendo que en ella se solicita el espacio de memoria para almacenar el arreglo monto.

1. ¿Qué cambios realizó en la función?
2. Muestre la organización de la memoria cuando se ejecuta la función carga

ACTIVIDAD 2

Defina la estructura de datos adecuada que le permita resolver la siguiente situación problemática:

El Instituto Provincial de la Vivienda ha implementado un sistema que consta de 5 planes de pago distintos, con el fin de que los adjudicatarios de sus viviendas puedan cancelar sus deudas. Por cada uno de los 5 planes, se ingresa en forma ordenada la cantidad de adjudicatarios adheridos y por cada uno de ellos el DNI y monto adeudado. Se pide:

1. Realice la carga de la información
2. Para un adjudicatario cuyo DNI se ingresa por teclado, indicar el número de plan al cual se adhirió y el monto adeudado.
3. Muestre el esquema de memoria, después de ejecutar la función que carga los datos

ACTIVIDAD 3

Se ingresa el registro y la nota obtenida (valor entero entre 1 y 10) por los alumnos que rindieron en la última mesa de examen de la materia Programación Procedural. Escribir un programa en C que permita:

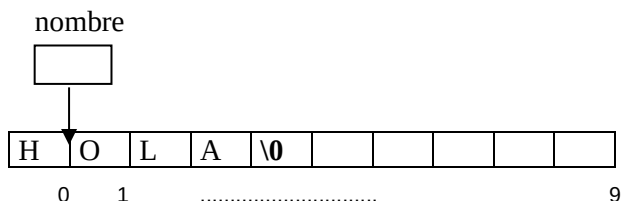
- Definir una función carga que almacene toda la información de la mesa de examen. Por teclado se ingresa la cantidad de alumnos que rindieron el examen.
- Mostrar la nota promedio y el número de registro de quienes obtuvieron en el examen una nota mayor o igual al promedio
- Construir una función que usando la menor y la mayor nota obtenidas, indique la cantidad de alumnos que obtuvieron cada nota entera comprendida en ese rango.
- Realizar el mapa de memoria después de ejecutar la función carga.

Arreglos dinámicos usados como cadena de caracteres

Como se ha visto, la estructura soporte de una cadena de caracteres es un arreglo del tipo char, que se caracteriza por finalizar con el carácter '\0'.

Por lo tanto, siendo char nombre[10] un arreglo, su nombre no es más que la dirección de memoria del primer carácter del arreglo.

Si la cadena de caracteres almacenada es "HOLA", gráficamente se representa:



Del mismo modo que a partir de un puntero a un entero, es posible generar un arreglo dinámico de enteros; a partir de un puntero a un carácter puede generarse un arreglo dinámico de caracteres.

char *nombre; declara a nombre como un puntero a un carácter. El espacio para almacenar la cadena se asigna dinámicamente en tiempo de ejecución, según el tamaño de la misma, evitando de ese modo el sobredimensionamiento de la memoria.

El siguiente ejemplo, muestra la carga de una cadena de caracteres como un arreglo dinámico.

```
void main(void)
{
    char *nombre, *aux;
    int l;
    aux=(char *) malloc( 30* sizeof(char)); // solicita memoria para la variable aux, con un tamaño
    suficientemente grande
    puts("Ingrese Nombre:\n");
    gets(aux);
    l = strlen(aux)+1;
    nombre=(char *) malloc( l* sizeof(char)); //solicita memoria necesaria para almacenar la variable
```

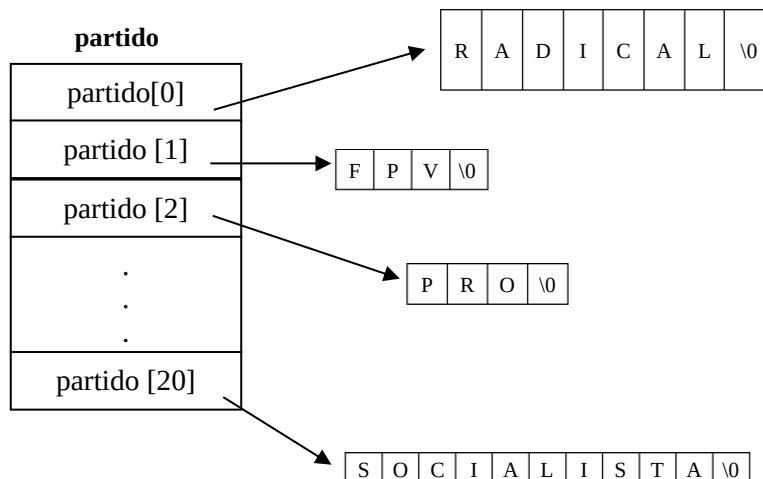

La declaración en este caso es:

```
char * partido[21];
```

donde **partido** es un arreglo de 21 punteros a char.

Cada uno de los 21 punteros contendrá la dirección del primer carácter de una cadena. Esto permite, a diferencia de la declaración anterior, que cada cadena ocupe solo la memoria requerida.

En este caso el almacenamiento es:



El formato para definir un arreglo de punteros a caracteres es el siguiente:

```
char * <identificador arreglo> [tamaño]
```

Ejemplo 4

El siguiente programa muestra una forma de leer y escribir, un conjunto de cadenas de caracteres de distinta longitud.

```
#include<stdio.h>
#include<malloc.h>
#include<string.h>
```

```
void carga(char *nom[21], int cfil)
{ char *aux;
  int i, l;
  aux=(char*) malloc(30*sizeof(char)); // solicita memoria para la variable aux
  puts(" Ingrese Nombres:\n");
  for(i=0; i<cfil; i++)
  { printf("\n Partido %d: ", i+1);
    gets(aux);
    l = strlen(aux)+1;
    nom[i] =(char *) malloc( l* sizeof(char));
    strcpy(nom[i],aux);
  }
  free (aux);
}
```

```
void muestra(char *nom[cfil])
{   printf("\nPartidos Ingresados: ");
    for (int i=0; i<cfil; i++)
        puts(nom[i]);
}
```

```
int main(void)
{
    char* nomb[21];
    carga(nomb,21);
    muestra(nomb,21);
}
```

ACTIVIDAD 4

Suponga que cuenta con los nombres y edad de 50 empleados de una empresa y desea:

a - Mostrar el nombre del empleado de mayor edad.

b - Dado un carácter, decir la cantidad de veces que aparece en los nombres de los empleados

Completar el siguiente programa con las funciones que le permitan resolver la problemática planteada.

```
#include <stdio.h>
#include <alloc.h>
#include <string.h>
typedef struct
{   char *nom;
    int edad;
} empleados;
.... carga (.....)
.. maximo(...)
```

```
void mostrar(empleados *x,int n, char r)
{
    int i,j,l,c=0;
    for(i=0; i<n; i++)
    {   l=strlen( x[i].nom);
        for(j=0;j<l ; j++)
            if (x[i].nom[j]== r)
                c++ ;
    }
    printf("\ncantidad de letras %c es %d \n",r,c);
}
```

```
void libera (empleados *x,int n)
{
    int i;
    for(i=0; i<n; i++)
    {   free( x[i].nom);
        // x[i].nom= NULL;
    }
}
```

```
void muestra (empleados *x,int n)
{
    int i;
    for(i=0; i<n; i++)
        puts( x[i].nom);
}
```

```
int main()
{
    char e;
    empleados *a;
    int n = 50;
    carga(a,n);
    printf("\ mayor edad %s ", maximo(a,n));
    printf(" \n ingrese un caracter %c",e);
    e = getchar();
    mostrar(a,n,e);
    muestra(a,n);
    libera(a,n);
    muestra(a,n);
}
```

Indicar si es necesario colocar la sentencia `x[i].nom= NULL` en la función libera.

Listas

Una lista es una estructura de datos compuesta por una serie de objetos de datos vinculados entre sí, donde se almacenan datos del mismo tipo, con la característica que puede contener un número indeterminado de elementos y que, mantienen un orden explícito (es decir que bajo un criterio se organizan sus posiciones en la estructura).

Las listas son estructuras de datos dinámicos, por tanto, pueden cambiar de tamaño durante la ejecución del programa, aumentando o disminuyendo el número de nodos.

Definiremos como lista a un conjunto de elementos del mismo tipo, que puede crecer o contraerse sin restricciones, todos los elementos son accesibles y se puede insertar y suprimir un elemento en cualquier posición de la misma.

Implementación de listas

Para implementar listas podemos utilizar una estructura de arreglo o bien utilizando variables del tipo apuntador.

Implementación de listas mediante arreglos

La implementación de listas mediante arreglos recibe el nombre de **listas secuenciales**. En este caso, los elementos se almacenan en celdas contiguas de memoria. Esta representación permite recorrer con facilidad una lista y agregarle nuevos elementos al final.

Insertar un elemento en la mitad de la lista, obliga al desplazamiento de una posición a todos los elementos que siguen al nuevo, para concederle espacio. De la misma manera, la eliminación de un elemento, excepto el último, requiere desplazamientos para llenar el vacío formado.

La mayor desventaja de las listas secuenciales es tener que asignar suficiente espacio de memoria, de manera de poder almacenar todos los elementos de la lista cuya cantidad no se conoce a priori. Este espacio permanece asignado, aún cuando la lista use una cantidad menor o incluso ninguno.

Además, redimensionar la lista cambiando el tamaño del arreglo cuando está completo, resulta ser una tarea lenta y tediosa (crear otro arreglo y copiar). Por otro lado, el no asignar más memoria introduce la posibilidad de desborde.

Este tipo de implementación tiene sus ventajas y desventajas, dependiendo de las operaciones que se quieran realizar sobre las listas.

Esta implementación se verá en años superiores al igual que otras implementaciones.

Implementación de listas mediante punteros

Las verdaderas posibilidades que ofrece la creación de variables dinámicas, se ponen de manifiesto al poder crear varios elementos de un mismo tipo de dato que se relacionen entre sí.

Para hacer posible la relación entre ellos, cada elemento deberá contener además de su información intrínseca, un puntero a otro elemento de estructura similar.

Los elementos se crean y/o liberan durante la ejecución del programa de acuerdo a los requerimientos del problema planteado.

Esta implementación de una lista recibe el nombre de **listas enlazadas**, permite almacenar los elementos en celdas de memoria no contiguas, de esta manera desaparece el problema de los costosos desplazamientos de elementos para insertar y eliminar componentes. No obstante, hay que pagar el precio de un espacio adicional para los apuntadores.

En esta representación cada elemento (nodo) de una lista debe tener dos partes:

- una contiene la información de cualquier tipo de dato simple o estructurado excepto FILE
- la otra parte, contiene un enlace o puntero que indica la posición del siguiente elemento.

Por lo tanto, cada elemento (nodo) tiene la estructura de tipo *struct*.

Estructura de un elemento (nodo) de una lista

Datos	Puntero
-------	---------

Es un tipo de dato autoreferenciado porque contienen un puntero o enlace a otro dato del mismo tipo. Las listas pueden concatenarse entre sí o dividirse en sublistas.

NOTA

En adelante se referirá a lista enlazada simplemente como lista, y a los elementos de la lista como nodos.

Ejemplo 5

a) A continuación se muestra gráficamente como se representa la siguiente lista de números enteros, por medio de un arreglo de longitud N.

3 4 -8 9 15 23

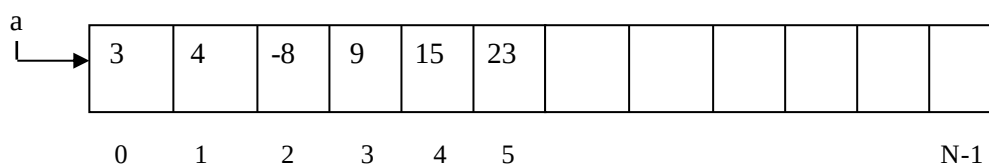


Fig. 5 Lista como arreglo de enteros

En este ejemplo, hemos supuesto que el número máximo de componentes a almacenar es N. Si la cantidad de componentes es superior, entonces para evitar el desborde de memoria, el arreglo debería redimensionarse.

b) Punteros: A continuación se muestra gráficamente una forma de representar una lista de números enteros por medio de punteros.

3 4 -8 9 15 23

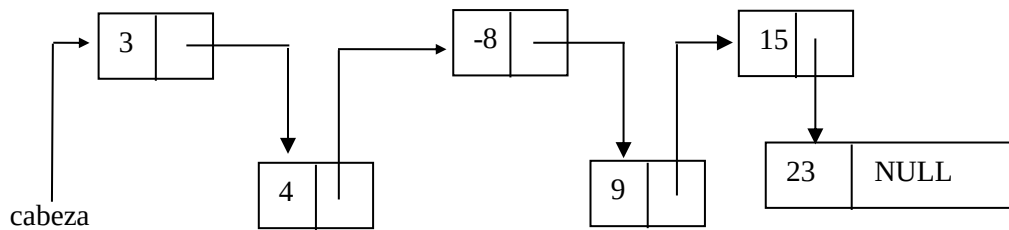


Fig. 6 Lista enlazada de números enteros

Se puede observar que en cada nodo, el campo enlace apunta al siguiente nodo de la lista excepto el último nodo que contiene NULL, para indicar el final de la lista. Como anteriormente se explicó, este valor de apuntador NULL no hace referencia a un lugar de memoria y se puede asignar a cualquier variable apuntador. Como puede inferirse, es ilegal una referencia al contenido de una variable puntero que tiene asignado NULL.

Existe la necesidad de un puntero que contenga en todo momento la dirección del primer elemento de la lista, denominado *cabeza de la lista*. Resguardar la cabeza de la lista garantiza el acceso a la misma.

Definición y declaración de las distintas implementaciones

A continuación se muestra las definiciones de tipo y declaraciones de variables para los dos tipos de implementación de listas de enteros.

a) Lista implementada a través de un arreglo estático

```
int arre[10];
```

b) Lista implementada a través de un arreglo dinámico

```
int * arre, longitud;
```

c) Lista implementada a través de punteros

```
struct nodo
{
    int nro;
    struct nodo *sig;
};
typedef struct nodo * puntero;
```

Luego en la declaración de variables:

```
puntero cabeza;
```

Analizando la definición del registro nodo, se puede observar que es recursiva ya que uno de sus componentes (campo *sig*) es una referencia a sí misma, es decir, una referencia al mismo registro.

De este modo, cada nodo se usa para construir la lista de datos, y cada uno mantendrá la relación con el siguiente. Además, para acceder a un nodo de la estructura sólo se necesita el puntero a ese nodo.

Manipulación de listas enlazadas

Las operaciones básicas necesarias para manipular listas enlazadas son:

- Creación
- Inserción
- Recorrido
- Búsqueda
- Modificación
- Supresión
- Ordenamiento
- Eliminación

Creación

La creación consiste en definir la lista vacía, utilizando la constante NULL.

Inserción en una lista

Una vez creada la lista, se puede insertar elementos en cualquier posición como se explica a continuación:

- Al comienzo de la lista: la inserción se hace al principio de ella, es decir se inserta por la cabeza de la lista. Es la opción más utilizada debido a que su codificación es muy sencilla.
- Al final de la lista: el nuevo elemento se coloca a continuación del último elemento de la lista.
- Entre otros nodos: el nuevo elemento se inserta en el medio de la lista.

Ejemplo 6

Supongamos creada una lista de números enteros (vacía), insertar una componente en la lista.

```
#include <alloc.h>
```

```
struct nodo
```

```
{
```

```
    int nro;
```

```
    struct nodo *sig;
```

```
};
```

```
typedef struct nodo * puntero;
```

```
void crear (puntero &xp)
```

```
{
```

```
    xp=NULL;
```

```
}
```

```
void insertar (puntero & xp)
```

```
{
```

```
    puntero nuevo;
```

```
    nuevo = (puntero) malloc(sizeof(struct nodo));
```

```
    printf("\n Ingrese el nuevo valor : ");
```

```
    scanf ("%d", &nuevo->nro);
```

```
    nuevo->sig = xp;
```

```
    xp = nuevo;
```

```
    return;
```

```
}
```

```
void main (void)
```

```
{
```

```
    puntero cabeza;
```

```
    crear (cabeza);
```

```
    insertar (cabeza);
```

```
    getch();
```

```
}
```

Tanto en la función `crear` como `insertar`, el parámetro formal **xp** es una referencia al parámetro actual **cabeza**, por lo tanto todo cambio producido en **xp** afecta a **cabeza**. De ahí, la variable **cabeza** guarda la dirección de la primera componente de la lista.

Ejemplo 7

Supongamos creada una lista de números enteros (vacía), insertar varias componentes. El ingreso de componentes finaliza con cero.

```
#include <alloc.h>
struct nodo
{
    int nro;
    struct nodo *sig;
};
typedef struct nodo *puntero;

void crear (puntero &xp)
{
    xp=NULL;
}

void insertar (puntero & xp)
{
    int dato;
    puntero nuevo;
    printf("\n Ingrese el nuevo valor (0 para terminar): ");
    scanf("%d",&dato);
    while (dato != 0)
    {
        nuevo =(puntero) malloc(sizeof(struct nodo));
        nuevo->nro = dato;
        nuevo->sig = xp;
        xp = nuevo;
        printf("\n Ingrese el nuevo valor (0 para terminar): ");
        scanf("%d",&dato);
    }
    return;
}

void main(void)
{
    puntero cabeza;
    crear(cabeza);
    insertar (cabeza);
}
```

La lista así creada tiene los elementos en un orden inverso a la forma en que se han introducido. Por ejemplo, para el lote: 3, 4, -8, 9, 15, 23, 0, la lista queda almacenada de la siguiente forma.

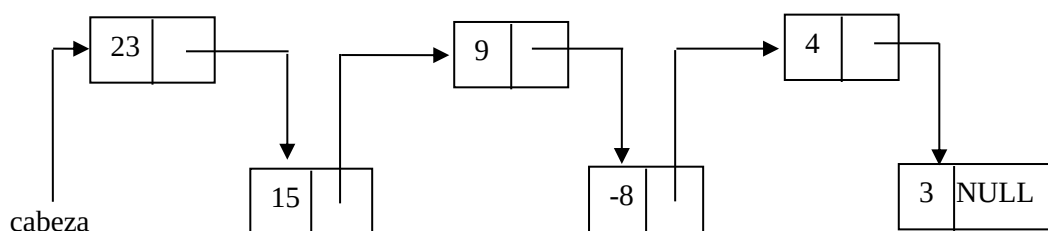


Fig. 7 Lista enlazada de enteros

ACTIVIDAD 5

Definir una función que permita crear la lista anterior, pero con sus componentes en el mismo orden en el que ingresan.

Gestión de almacenamiento

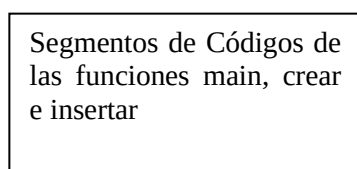
Si en un programa se declaran variables de tipo puntero, cuyo espacio de almacenamiento se asigna dinámicamente a través de la sentencia *malloc*, el sistema debe controlar no sólo el almacenamiento estático y dinámico basado en pila, sino también el almacenamiento dinámico basado en el montículo.

Para mostrar la gestión del almacenamiento, considérese el siguiente programa que genera una lista de números enteros, para el lote de prueba anterior.

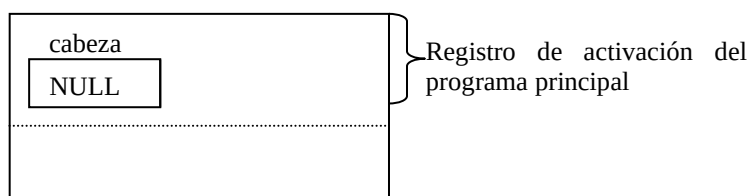
Para interpretar el estado de la memoria durante la ejecución del programa, consideraremos dos situaciones:

1) El estado en que se encuentra la memoria, antes de invocar la función **insertar(cabeza)**

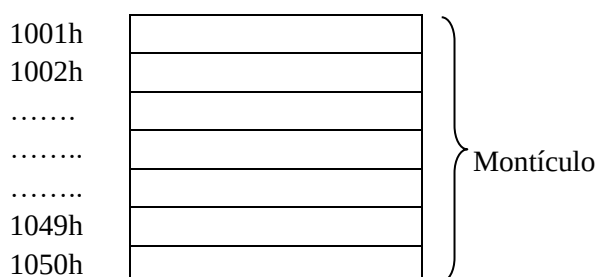
Almacenamiento estático



Almacenamiento dinámico: pila



Almacenamiento dinámico: montículo



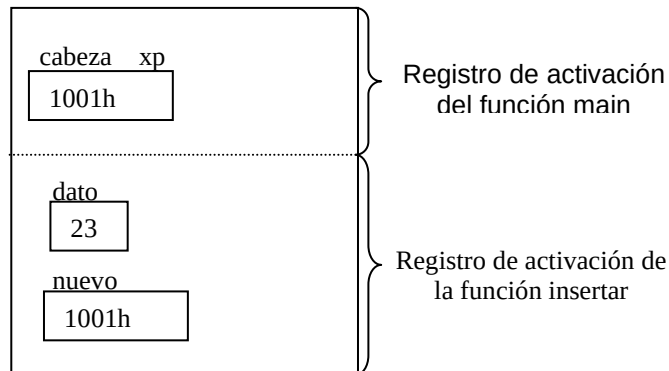
Como puede observarse el montículo está vacío, pues aún no se inició la generación de la lista. El valor NULL de cabeza, en el registro de activación del main, indica que la lista aún está vacía.

2) Una vez que **insertar(cabeza)** es invocado para generar la lista, el estado de la memoria justo antes de la sentencia return, es decir antes de transferir el control al programa principal, es el siguiente:

Almacenamiento estático:

Segmentos de Códigos de
las funciones main, crear,
insertar

Almacenamiento dinámico: pila



Nuevamente se observa que la función insertar recibe por parámetro una referencia de la variable cabeza, es decir que **xp** no es otra cosa que un alias del lugar de memoria referenciado por cabeza.

Almacenamiento dinámico: montículo

	nro	sig
1001h	23	1040h
.....		
1012h	9	1032h
.....		
1006h	4	1050h
1007h		
1032h	-8	1006h
1040h	15	1012h
1050h	3	NULL

Montículo

NOTA: la inserción en lista responde al siguiente algoritmo:

Inicio Bucle
Pedido de memoria
Carga de datos en el nuevo nodo
Enlace
Fin del Bucle

Recorrido de la lista

Para mostrar los elementos de una lista, se la debe recorrer a través de un puntero vaya apuntado sucesivamente a cada uno de los elementos, comenzando desde la cabeza, y mostrando entonces el campo de datos de cada elemento. Este proceso termina cuando se llega al final de la lista.

En general, si **p** apunta a un nodo, para avanzar al próximo se debe realizar la siguiente asignación:

p = p->sig;

Ejemplo 8

La siguiente función permite recorrer una lista como la generada en el ejemplo anterior.

```
void recorre (puntero xp)
{ printf("\n Listado de datos ");
  while (xp != NULL)
  {
    printf("\n %d ",xp->nro);
    xp = xp->sig;
  }
}
```

Cuando la función “recorre” procese la última componente de la lista, la asignación **p=p->sig;** guardará en **p** el valor **NULL** y la iteración terminará.

En el programa principal, la invocación a esta función se realiza a través de la ejecución de la sentencia:

recorre(cabeza); /*invocación a la función que recorre la lista */

NOTA

Se puede observar que los elementos de la lista se muestran en el orden inverso al que ingresaron.

NOTA: El recorrido en lista responde al siguiente algoritmo:

```
Inicio Bucle
    Se procesa el nodo apuntado
    Se avanza al siguiente nodo
Fin del Bucle
```

ACTIVIDAD 6

Analizar la función **recorre** y explicar qué resultados obtiene si el parámetro formal **xp** se pasa como una referencia.

Búsqueda de una componente de una lista

Para buscar una componente específica en la lista, se la debe recorrer desde el principio hasta encontrar la componente o hasta llegar al final de la lista si es que la componente buscada no pertenece a la misma.

Ejemplo 9

En este algoritmo, se busca un elemento en una lista enlazada.

```
#include <alloc.h>
```

```
struct nodo
```

```
    { int nro;
      struct nodo *sig;
    };
```

```
typedef struct nodo * puntero;
```

```
void crear (puntero &xp)
```

```
{ xp=NULL;
}
```

```

void insertar (puntero & xp)
{ int dato;
  puntero nuevo;
  printf("\n Ingrese el nuevo valor (0 para terminar): ");
  scanf("%d",&dato);
  while (dato != 0)
  { nuevo =(puntero) malloc(sizeof(struct nodo));
    nuevo->nro = dato;
    nuevo->sig = xp;
    xp = nuevo;
    printf("\n Ingrese el nuevo valor (0 para terminar): ");
    scanf("%d",&dato);
  }
  return;
}

```

```

void busca (puntero xp, int num)
{
  while((xp != NULL )&& (xp->nro != num))
    xp = xp->sig;
  if (xp == NULL) printf(" El número no está en la lista");
  else printf(" Se encontró el número en la lista");
}

```

```

void main(void)
{
  puntero cabeza;
  int nn;
  crear(cabeza);
  insertar (cabeza);
  printf ("\n Ingrese el número a buscar: ");
  scanf ("%d", &nn);
  busca (cabeza, nn);
}

```

La función *busca* define el parámetro **xp** por valor, para preservar el valor original de la variable que contiene la dirección del primer nodo de la lista.

Modificación de una componente de la lista

Para modificar el valor de algún dato de un nodo de la lista, se debe recorrer desde el comienzo hasta encontrar el componente correspondiente.

Ejemplo 10

En este algoritmo se busca en la lista un número leído previamente y, si se encuentra, se cambia por otro.

```

#include <alloc.h>
struct nodo
{ int nro;
  struct nodo *sig;
};
typedef struct nodo *puntero;

void crear (puntero &xp)
{ xp=NULL;
}

```

```

void insertar (puntero & xp)
{ int dato;
  puntero nuevo;
  printf("\n Ingrese el nuevo valor (0 para terminar): ");
  scanf("%d",&dato);
  while (dato != 0)
  {
    nuevo =(puntero) malloc(sizeof(struct nodo));
    nuevo->nro = dato;
    nuevo->sig = xp;
    xp = nuevo;
    printf("\n Ingrese el nuevo valor (0 para terminar): ");
    scanf("%d",&dato);
  }
  return;
}

void modifica(puntero xp)
{ int viejo, nuevo;
  printf("\n Ingrese el número que quiere cambiar ");
  scanf ("%d", &viejo);
  printf("\n Ingrese el nuevo número ");
  scanf ("%d", &nuevo);
  while ((xp != NULL) && (xp->nro != viejo))
    xp = xp->sig;
  if (xp != NULL)
    xp->nro = nuevo;
  else
    printf("\n el número no está en la lista");
  return;
}

void main(void)
{
  puntero cabeza;
  int nn;
  crear(cabeza);
  insertar (cabeza);
  modifica (cabeza);
}

```

ACTIVIDAD 7

En el ejemplo anterior, analizar el pasaje de parámetros de las funciones `modifica`. Extraer conclusiones.

NOTA: Las funciones de búsqueda y modificación responden al algoritmo señalado para recorrer la lista:

Inicio Bucle

Se procesa el nodo apuntado (si es el nodo buscado se informa o se modifica y se sale del bucle)

Se avanza al siguiente nodo (si no se encontró el buscado)

Fin del Bucle

Inserción de un nodo en cualquier lugar de la lista

Hasta ahora, las inserciones de nodos en la lista se han realizado por la cabeza. Esto es, cada nodo que se inserta pasa a ser el primero de la lista.

En muchas aplicaciones es conveniente insertar un dato en cualquier lugar de la lista, dependiendo de los requerimientos. Por ejemplo, en la lista antes creada, podría incluirse un nodo al final de la lista de enteros (ver figura 7)

Inserción de un nodo al final de una lista

Ejemplo 11

En el siguiente programa se utiliza la función **insertarAlFinal** para agregar un nodo al final de la lista generada.

```
#include <alloc.h>
struct nodo
{
    int nro;
    struct nodo *sig;
};
typedef struct nodo *puntero;

void insertarAlFinal (puntero &xp)
{
    puntero p, nuevo, anterior;
    nuevo =(puntero) malloc(sizeof(struct nodo));
    printf("\n Ingrese el nuevo valor: ");
    scanf("%d",&nuevo->nro);
    nuevo->sig = NULL;
    if (xp == NULL)          /* controla si la lista está vacía */
        xp = nuevo;
    else
    {
        p = xp;              /* resguarda a xp de las siguientes modificaciones */
        while (p != NULL)
        {
            anterior = p;
            p = p->sig;
        }
        anterior->sig = nuevo;
        printf("El nuevo elemento ha sido insertado al final de la lista.");
    }
    return;
}

void main(void)
{
    puntero cabeza;
    crear(cabeza); // ídem a anterior
    insertar (cabeza); // ídem a anterior
    insertarAlFinal (cabeza);
}
```

Inserción de un nodo cualquier lugar de la lista

Una de las ventajas del uso de listas enlazadas a través de punteros es la flexibilidad de las mismas para aumentar o reducir su tamaño. La inserción de nodos en este tipo de estructura es mucho menos compleja que en el caso de listas soportadas por arreglos.

Por ejemplo, partiendo de una lista de enteros generada como en la figura 7, supóngase que se desea insertar un nodo en la posición inmediata anterior al nodo que tiene almacenado el número 9.

Así, si en la lista se quiere insertar una componente que tenga el número 12 entonces debe crearse un nodo con el número 12 e insertarse antes del nodo que contiene a 9. Esto implica realizar el cambio de punteros necesarios de modo que el nodo que contiene al número 15 apunte al nuevo nodo generado (que contiene el número 12) y éste al nodo correspondiente al número 9.

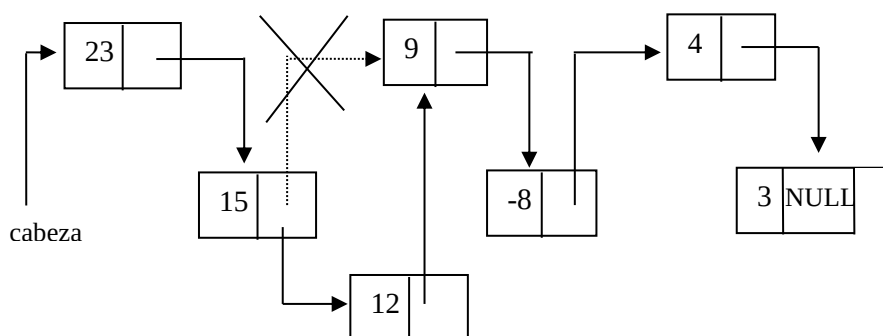


Fig. 8 Inserción del nodo que contiene el número 12

El siguiente código, resuelve la situación planteada:

void insertarAdentro (puntero &xp, int xnum)

```
{ puntero p, nuevo, anterior;
  nuevo =(puntero) malloc(sizeof(struct nodo));
  nuevo->nro=xnum;
  if (xp == NULL)          /* caso 1: la lista está vacía */
  { xp = nuevo;
    nuevo->sig = NULL;
  }
  else
  if (xp->nro == nuevo->nro) /* caso 2: la inserción debe hacerse al comienzo */
  { nuevo->sig = xp;
    xp=nuevo;
  }
  else
  { p = xp;          /* resguarda a xp de las siguientes modificaciones */
    anterior=xp;     /* caso 3: la inserción debe hacerse adentro o al final */
    while ( (p != NULL) && (nuevo->nro > p->nro))
    { anterior = p;
      p = p->sig;
    }
    anterior->sig = nuevo;
    nuevo->sig = p;
    printf("\n El elemento ha sido insertado en el lugar que corresponde");
  }
}
```

xnum: valor de la nueva componente

El programa principal es el siguiente:

```
void main (void)
{
    int num;
    puntero cabeza;
    crear(cabeza);
    insertar (cabeza);
    printf("ingrese número que desea agregar");
    scanf("%d", &num);
    insertarAdentro (cabeza, num);
}
```

ACTIVIDAD 8

Realizar un programa que mediante funciones permita:

- Generar una lista de números enteros positivos, ordenada en forma ascendente. Validar la entrada.
- Escribir un mensaje diciendo si el número del último nodo de la lista es par.

NOTA: el algoritmo de inserción en lista visto anteriormente, también responde a estas dos formas:

Inicio Bucle

Pedido de memoria

Carga de datos en el nuevo nodo

Enlace (se busca la posición en la lista o se va al final si es por cola)

Fin del Bucle

Supresión de un elemento en una lista

Para eliminar una componente en una lista, previamente se debe realizar el encadenamiento de la componente anterior a la que hay que eliminar con la siguiente a la misma.

Suponiendo la lista de enteros generada en la figura 7, la figura 9 muestra el cambio de punteros que se debe realizar para eliminar la componente que contiene el valor 9.

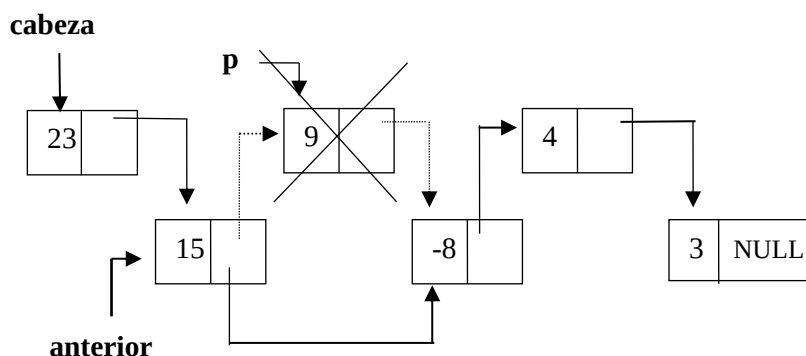


Fig. 9 Eliminación de la componente con el valor 9

Puede observarse, que para eliminar la componente, es necesario ubicar la dirección de ese nodo (**p** en el gráfico) y guardar la dirección del nodo anterior. Luego se enlaza el nodo anterior al que se desea eliminar con el siguiente al mismo. Esto es, el nodo que contiene el valor 15 debe apuntar al nodo que contiene el valor -8. Una vez realizado el cambio de punteros, debe liberarse el espacio de memoria apuntado por **p** mediante la función free.

ACTIVIDAD 9**Investigar las siguientes situaciones:**

¿Qué sucede si el número a suprimir es el primero de la lista?

¿Qué sucede si el número a suprimir aparece repetido más de una vez

El siguiente código corresponde a la función que suprime una componente de una lista enlazada.

void suprimir (puntero &cab)

```
{
puntero anterior, p;
int dato;
printf("\n Ingrese el número a suprimir ");
scanf("%d",&dato);
if (cab->nro == dato)
{
    p=cab;
    cab=cab->sig;
    free (p);    /*eliminación de la cabeza de la lista */
}
else
{
    p=cab;
    anterior=cab;
    while ((p != NULL) && (p->nro != dato))
    {
        anterior = p;
        p=p->sig;
    }
    if (p != NULL)
    {
        anterior->sig = p->sig;
        free(p);
        printf("\n El número fue eliminado de la lista");
    }
    else
        printf("\n No se encuentra el número en la lista");
    return;
}
}
```

NOTA: el algoritmo de supresión, también responde a lógica del recorrido:

Inicio Bucle

Se procesa el nodo apuntado (si es el nodo buscado se acomodan los enlaces, se libera el nodo y se sale del bucle)

Se avanza al siguiente nodo (si no se encontró el buscado, se resguarda la posición actual en anterior para luego acomodar los enlaces)

Fin del Bucle

Ordenamiento de una lista

Así como para arreglos, el proceso de ordenamiento por un campo determinado también puede aplicarse a listas. Se debe tener en cuenta que al momento de realizar un cambio de componentes, éste debe afectar sólo a la parte de información y no a los campos que contienen los punteros enlaces, de no ser así, se produciría un desencadenamiento de la misma.

ACTIVIDAD 10

A continuación se presenta el código de una función que ordena en forma ascendente una lista de números enteros.

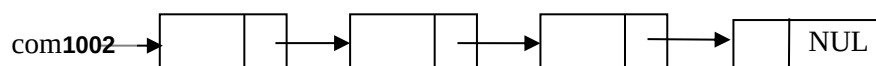
Realizar el seguimiento de la función ordena tomando como lote de prueba, una lista de cinco elementos y una lista vacía.

void ordena (puntero cab)

```
{
    puntero k, cota, p;
    int aux;

    cota = NULL;
    k = NULL;
    while (k != cab)
    {
        k = cab;
        p = cab;
        while (p->sig != cota)
        {
            if (p->nro > p->sig->nro)
            {
                aux = p->sig->nro;
                p->sig->nro = p->nro;
                p->nro = aux;
                k = p;
            };
            p = p->sig;
        }
        cota = k->sig;
    }
}
```

Eliminación de todas las componentes de una lista



Este proceso consiste en ir recorriendo la lista desde el comienzo, e ir eliminando de a una componente por vez, a medida que se avance a lo largo de ella.

Ejemplo 12

El siguiente ejemplo muestra la función que elimina todas las componentes de una lista.

void elimina (puntero & cab)

```
{
puntero p;
while ( cab != NULL )
{
    p = cab;
    cab = cab->sig;
    free (p);
}
return;
}
```

Listas enlazadas que almacenan datos estructurados

Hasta ahora se ha presentado ejemplos de listas enlazadas donde los datos almacenados son simples. A continuación se presenta un ejemplo donde los datos a almacenar son estructurados.

Ejemplo 13

Generar una lista con los datos de los empleados de una empresa: legajo y sueldo. El ingreso de información termina con legajo cero(0).

Mostrar el legajo de los empleados almacenados en la lista, cuyo sueldo es superior a 1000.

```
#include <stdio.h>
#include <alloc.h>
typedef struct
{
    int legajo;
    float sueldo;
} empleado ;

struct nodo
{
    empleado dato;
    struct nodo * sig;
};
typedef struct nodo *puntero;

void insertar (puntero &xco)
{
    puntero nuevo;
    int num;
    printf("\n Ingrese legajo: ");
    scanf ("%d", &num);
    while( num )
    {
        nuevo = (puntero) malloc(sizeof(struct nodo));
        nuevo->dato.legajo = num;
        printf("\n Ingrese sueldo : ");
        scanf ("%f", &nuevo->dato.sueldo);
        nuevo->sig = xco;
        xco = nuevo;
        printf("\n Ingrese legajo : ");
        scanf ("%d", &num);
    }
    return;
}
```

```

void listar (puntero xco)
{ printf("\n Listado de empleados con sueldo mayor de 1000 ");
  while (xco != NULL)
  {
    if (xco->dato.sueldo) > 1000
      printf("\n %d ",xco->dato.legajo);
    xco = xco->sig;
  }
}
void main(void)
{ puntero comienzo;
  comienzo=NULL;
  insertar(comienzo);
  listar(comienzo);
}

```



ACTIVIDAD 11

En el algoritmo anterior, agregar una función recursiva que para un número de legajo conocido, devuelva el sueldo. Si el empleado no está en la lista, la función devuelve 0.

Problemas de gestión de almacenamiento

El hecho de que el programador pueda gestionar memoria, esto es, pedir bloques de memoria y liberarlos cuando considere conveniente, puede crear distintos problemas. Como sabemos la función `free` se usa para liberar el espacio de memoria asignado dinámicamente, a través de la función `malloc`, a una variable. De esta manera, al finalizar el tiempo de vida de una variable, el bloque de memoria asignado a la misma se recupera para ser resignado oportunamente cuando se requiera. Sin embargo se suelen suscitar problemas tales como la generación de referencias bamboleantes o la creación de basura.

Referencias Desactivadas o Bamboleantes

Sea `p` un puntero a una variable dinámica, una llamada a `free(p)` hace ilegal una referencia posterior a `p->`, aunque el valor de `p` queda intacto (ya que con `free` solo se modifica la Tabla de direcciones de memoria del Sistema Operativo).

No obstante algunos compiladores no detectan la ilegalidad, por lo tanto a través de `p` se puede acceder a los datos en esa dirección, provocando errores impredecibles, tales como la modificación del almacenamiento asignado previamente a otra variable dinámica. El puntero `p` recibe el nombre de referencia desactivada o bamboleante.

Por todo lo expresado, es una buena “regla de pulgar”, después de liberar el espacio ocupado por una variable dinámica colocar en `NULL` el puntero asociado a ella.

En el ejemplo anterior, la función elimina, libera el espacio ocupado por todas las componentes de la lista y por lo tanto se la recorre hasta que la cabeza de la lista (`cab`) tome el valor `NULL`.

Basura

Otra característica peligrosa asociada a los punteros y el uso incorrecto de la sentencia `free`, es la generación de basura. Se llama basura, al espacio de memoria que no puede ser accedido porque se han perdido todas las referencias a él.

Por ejemplo, supongamos tener generada una lista en la que **cabeza** es la dirección de la primer componente de ésta. Si por alguna circunstancia se hiciera **free(cabeza)**; se liberaría la memoria de la primera componente de la lista, perdiéndose la dirección del segundo nodo y, por lo tanto, todo el resto de la lista. Las celdas que siguen a la primera quedarán como basura, es decir, queda inutilizado todo el espacio de memoria ocupado por ellas. Esto puede provocar que el sistema no pueda seguir operando por falta de espacio cuando en realidad si lo tiene. En estos casos, el sistema invoca a una rutina llamada colector de basura para recuperar el espacio perdido.

NOTA

La recolección de basura (en inglés Garbage Collection - GC) es un mecanismo que proporciona recuperación de memoria automática para bloques de memoria no utilizados. El motor de GC se encarga de reconocer que ya no se usa un bloque particular de memoria asignada y lo vuelve a poner en el área de memoria libre. GC fue presentado por John McCarthy en 1958, como el mecanismo de gestión de memoria del lenguaje LISP. Desde entonces, los algoritmos GC han evolucionado. Varios idiomas se basan nativamente en GC.

Lenguaje C no incluye en forma nativa recolección de basura. Existe la biblioteca GC Boehm-Demers-Weiser (BDW), un paquete que permite a los programadores C y C ++ incluir administración automática de memoria en sus programas. La biblioteca BDW es una biblioteca de acceso libre que proporciona programas C y C ++ con capacidades de recolección de basura.

Manipulación de Listas con Funciones Recursividad

Las listas enlazadas son estructuras ideales para mostrar las ventajas del uso de funciones recursivas. Un ejemplo de ello, es la posibilidad que este mecanismo ofrece para acceder a los datos de la lista en el mismo orden en que ingresaron, para el caso de listas enlazadas generadas por inserción de nodos por la cabeza.

ACTIVIDAD 12

Analizar la siguiente función, realizar el seguimiento con el Lote de prueba: 3, 4, -8, 9, 15, 23, 0

```
#include <alloc.h>
```

```
void crear(puntero &xp)
```

```
{  
    xp=NULL;  
}
```

```
void listar (puntero xp)
```

```
{  
    if (xp != NULL)  
    {  
        listar(xp->sig);  
        printf("\n %d ",xp->nro);  
    }  
    else  
        printf(" \n Listado de lista en el orden de ingreso de los datos");  
}
```

```
void main(void)
```

```
{  
    crear(cabeza);  
    insertar (cabeza)  
    listar (cabeza);  
}
```


Arreglo de Listas

Ejemplo 14

La Facultad de Ciencias Exactas organizó el Congreso de Informática y necesita contar con un programa que le permita administrar la información relativa a los 10 tutoriales que se proponen en dicho evento. Para ello, el programa deberá permitir, a través de un menú de opciones, los siguientes ítems:

- Ingresar los datos correspondientes a cada tutorial, a saber: número de tutorial (1-10), título.
- Registrar las inscripciones, ingresando el DNI del inscripto y el número de tutorial al que desea asistir.
- Eliminar alguna inscripción, en cuyo caso se ingresarán los mismos datos que en el ítem anterior.
- Dado el número o el título de un tutorial, mostrar los datos específicos del mismo, incluido la cantidad inscriptos.
- Dado el DNI de una persona, informar el/los tutoriales (número y título) en los que se inscribió.

NOTA

Para resolver este ejercicio, se presentan las siguientes definiciones de las estructuras de los datos a utilizar:

struct nodo

```
{
    int dni;
    struct nodo * sig;
};
```

struct datos /* corresponde a cada tutorial */

```
{
    int numero;
    cadena titulo;
    struct nodo * ins; /* puntero a una lista de inscriptos*/
};
typedef struct nodo * inscriptos;
```

```
typedef char cadena[30];
```

void main(void)

```
{
    int i, n, op;
    long d;
    datos tutorial[fila]; /* arreglo de tutoriales */
    printf("\n ***** Cargar de los tutoriales C\n *****");
    for (i=0;i<fila;i++)
    {
        tutorial[i].numero=1;
        printf("\n Ingrese nombre del tutorial %d :",i+1);
        gets(tutorial[i].titulo);
        crear(tutorial[i]);
    }
    printf("\n ***** MENU DE OPCIONES *****\n");
    do
    {
        clrscr();
        printf("\n 1- Realizar una inscripción: ");
        printf("\n 2- Eliminar una inscripción: ");
        printf("\n 3- Mostrar Datos de un tutorial: ");
        printf("\n 4- Mostrar tutoriales en los cuales se inscribió una persona : ");
        printf("\n 5- Mostrar tutoriales con Datos de Inscriptos : ");
        printf("\n 6- Salir del MenúRealizar una inscripción: ");
```

```

scanf("%d", &op);
switch (op)
{
    case 1: { printf("\n Ingrese numero del tutorial :");
              scanf("%d", &n);
              printf("\n Ingrese DNI :");
              scanf("%d", &d);
              insertar(tutorial[n-1].ins,d);
              break;
            }
    case 2: /*** Completar ***/; break;
    case 3: /*** Completar ***/; break;
    case 4: /*** Completar ***/; break;
    case 5: listar(tutorial) ;
}
} while (op !=6);
getche();
}

```

A continuación se definen las funciones necesarias para responder a la opción 1.

void crear (datos &d)

```

{
d.ins=NULL;
}

```

void insertar (inscriptos &xp, int DNI)

```

{
inscriptos nuevo;
nuevo = (inscriptos) malloc(sizeof(struct nodo));
nuevo->dni=DNI;
nuevo->sig = xp;
xp = nuevo;
return;
}

```

void listar(datos t[])

```

{
    int i;
    inscriptos xp;
    for(i=0; i< fila; i++)
    {
        printf("\n **** Datos de Inscriptos en el Tutorial:%3d ****\n", i+1);
        printf("\n TITULO: "); puts(t[i].titulo);
        printf("\n INSCRIPTOS \n ");
        xp= t[i].ins;
        while (xp!= NULL)
        {
            printf("\n %d",xp->dni);
            xp = xp->sig;
        }
    }
}

```

ACTIVIDAD 13: Escribir las funciones necesarias para dar respuesta a los otros ítems del menú principal.

Bibliografía

- Apuntes de Cátedra Programación Procedural
- Aho Alfred V., John E. Hopcroft, Jeffrey D. Ullman (1988) "Estructura de datos y algoritmos publicación". Addison-Wesley Iberoamericana: Sistemas Técnicos de Edición. México, DF.
- Braunstein y Gioia (1987) "Introducción a la Programación y a la Estructura de Datos" - Eudeba
- Criado Clavero, M^a Asunción.(2006) "Programación en Lenguajes Estructurados". Alfaomega. Rama. México.
- Deitel, H M y Deitel, P J (2003) "Como programar en C/C++" Pearson Educación – Hispanoamericana
- Joyanes Aguilar A y Zahonero Martínez (2001) "Programación en C. Metodología, estructuras de datos y objetos". Mc Graw - Hill.
- Joyanes Aguilar Luis (2004) "Algoritmos y Estructuras de Datos. Una Perspectiva en C". Segunda Edición Mc Graw Hill.

Unidad 6: Archivos

Introducción

Hasta ahora los tipos de datos simples y estructurados con que se ha trabajado son de almacenamiento interno. A menudo se necesita guardar datos en forma permanente. La memoria RAM, o memoria principal de la computadora es volátil, por ello, toda información contenida en ella se pierde si se apaga la computadora ya que necesita energía para mantener los datos.

Si se necesita almacenar los datos para que perduren más de una sesión de trabajo, hay que recurrir al almacenamiento secundario, que aunque más lento que la memoria principal, permite almacenarlos en **forma permanente**.

Los lenguajes de programación proporcionan estructuras que permiten el almacenamiento permanente de la información, en el caso de C se denominada FILE.

Esta estructura permite gestionar a través del sistema operativo, los medios necesarios para almacenar información en dispositivos de almacenamiento secundario en forma de archivo de datos. Entre los dispositivos de almacenamiento, secundarios o auxiliares se puede citar por ejemplo disco rígido, discos externos, pendrive, SD, CD, etc.

Los archivos son utilizados por empresas, reparticiones del gobierno, usuarios particulares, etc. ya que en todos los casos deben resguardar información que debe ser accedida cuando se necesite. Los datos a su vez deben conservar su integridad, es decir, lo que se guarda en el archivo no debe sufrir alteración, por la importancia que ellos revisten. Como ejemplos del uso de archivos se cita:

Archivos médicos con historial de pacientes.

Archivos de legajos del personal de una institución

Archivos sismológicos que permiten guardar la actividad sísmica de una región.

Archivos que almacenan stock de mercaderías.

Archivos con notas de alumnos de una carrera.

Archivos de legislaciones provinciales y/o nacionales.

Definición

Un archivo es una estructura compuesta por datos almacenados en forma organizada en un dispositivo de almacenamiento secundario. Para su correcta manipulación, esta estructura es identificada por un nombre.

La transferencia de información entre la memoria RAM y el almacenamiento secundario se realiza a través de una unidad llamada componente.

Organización de Archivos

El término organización hace referencia a la disposición de los componentes del archivo en el dispositivo físico. Esta organización es determinada en el momento de su creación o utilización.

Organización Secuencial: Un archivo tiene organización secuencial, cuando las componentes que lo conforman se emplazan en posiciones físicas contiguas en el soporte de almacenamiento.

Organización Directa: Un archivo tiene organización directa, cuando las componentes que lo conforman ocupan posiciones físicas relacionadas mediante una clave. Esta clave puede ser un dato de la componente o bien el resultado de un cálculo matemático.

La organización adecuada para un archivo en particular está determinada por las características operacionales físicas del dispositivo de almacenamiento y por los requerimientos de la aplicación.

Clasificación de archivos

A los archivos se los puede clasificar según diferentes criterios:

Por la dirección del flujo de datos

- Archivos de entrada: el programa *lee* los datos desde el archivo.
- Archivos de salida: el programa *escribe* los datos en el archivo.
- Archivos de entrada / salida: los datos pueden ser *escritos o leídos* desde el archivo.

Según el tipo de componente

- **Archivos de texto:** En estos archivos cada símbolo se representa por un byte (ASCII, EBCDIC) y no están permitidos ciertos valores en los bytes ya que tienen un significado especial. Por ejemplo, el valor hexadecimal 0x1A marca el fin de archivo. Si se abre un archivo en modo texto, no será posible leer más allá de ese byte, aunque el archivo sea de mayor longitud.
- **Binarios:** Estos archivos son utilizados para almacenar valores enteros, en coma flotante, imágenes, archivos ejecutables etc. Están permitidos todos los valores para cada byte, los archivos binarios son secuencias de ceros y unos. Trabajar con archivos binarios optimiza el procesamiento de la información en memoria, aunque la visualización de la información debe hacerse desde el programa. En estos archivos, la detección del final del mismo, depende del soporte y del sistema operativo. Los archivos binarios se pueden trabajar de dos maneras: **con formato**, utilizando funciones que interpretan directamente el formato de cada parte de la componente y **sin formato**, utilizando funciones que toman la componente como un bloque de bytes para que el programa interprete cada parte del bloque.

Según el tipo de acceso

El término acceso hace referencia a la técnica que se utiliza para leer o grabar componentes de un archivo. Hay diversas maneras de acceder a las componentes de un archivo: en forma secuencial, directa o aleatoria, por índice y dinámico. En este curso, se considerará tan solo el acceso secuencial y el acceso directo.

- **Acceso secuencial:** Este tipo de acceso permite como única alternativa recorrer el archivo componente a componente, comenzando por la primera hasta llegar a la deseada.
- **Acceso Directo:** Este tipo de acceso permite acceder a una componente en forma directa a través de su posición en el soporte de almacenamiento secundario. Hay dispositivos que permiten acceso directo a una componente determinada, sin tener que recorrer secuencialmente los que lo preceden, tal es el caso de discos, disquete, o CD, los que permiten organización directa y/o secuencial.

Según la longitud de registro

- **Archivos de longitud variable:** Cada registro del archivo puede ser de longitud diferente. La unidad de almacenamiento que se denomina componente, es el byte. La aplicación debe conocer el tipo y longitud de cada dato almacenado en el archivo, para leer y/o escribir las componentes necesarias en cada ocasión. Por ejemplo, en el caso de archivos de texto, se usa como marca de final de registro el carácter especial salto de línea o de retorno de línea.
- **Archivos de longitud constante:** Los datos se almacenan en forma de registro de tamaño constante, es decir que la componente tiene un tamaño constante. C dispone de funciones de librería adecuadas para manejar este tipo de archivos.

Es posible crear archivos combinando las categorías mencionadas, así se puede hablar de archivos secuenciales de texto con longitud de registro variable, que son los típicos archivos de texto. Archivos de acceso aleatorio binarios de longitud de registro constante con organización secuencial, normalmente usados en bases de datos. Existen otras combinaciones menos usadas, tales como archivos con organización secuencial y de acceso secuencial binarios de longitud de registro constante, usados generalmente para cintas de resguardo o los archivos con organización directa de rápido acceso como son los índices de las bases de datos.

Ventajas y desventajas del uso de archivos

- Los datos almacenados en un archivo pueden ser usados por distintos programas, en distintos procesos y momentos.
- La capacidad de almacenamiento de la memoria secundaria es superior a la de la memoria principal, permitiendo de esta manera guardar grandes volúmenes de información.
- El uso de archivos permite tener en memoria principal sólo aquella parte de datos que necesita ser accedida por el programa en un momento dado, sin necesidad de tener toda la información en memoria principal.
- El tamaño de los archivos no es fijo y está limitado sólo por la cantidad de memoria secundaria disponible.

El acceso a los datos de un archivo es mucho más lento que los datos residentes en memoria principal. Los tiempos de procesamiento de datos de la memoria central son tiempos electrónicos (el tiempo de acceso de las memorias RAM están en el orden de los nanosegundos), mientras que el proceso de datos almacenados en periféricos incluye tiempos mecánicos (los tiempos de acceso de los disquetes están en el orden de los milisegundos).

Declaración de un archivo en el lenguaje C

Para manipular información de entrada / salida se proporciona una memoria intermedia que funciona como un canal entre el programa y el dispositivo de E/S, en esta unidad hacemos referencias solo a dispositivos de almacenamiento de datos como E/S.

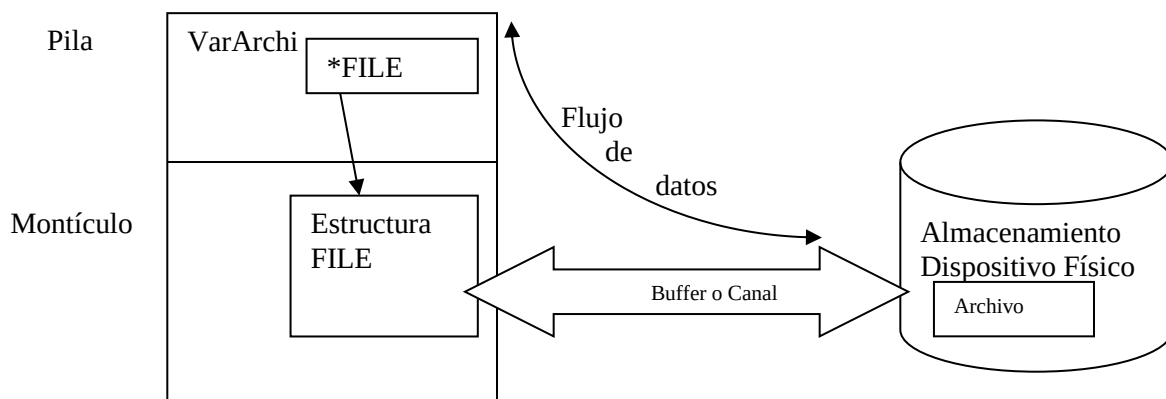
La librería stdio.h proporciona una serie de funciones y una estructura tipo FILE que contiene toda la información necesaria para utilizar un archivo.

La estructura es la siguiente:

```
typedef struct {
    short level;           //nivel de ocupacion del buffer
    unsigned flag;         //indicadores decontrol
    char fd;               //descriptor del archivo
    char hold;             // carácter de ungetc()
    short bsize;           //tamaño del buffer
    unsigned char *buffer; //puntero al buffer
    unsigned char *curp;   //posicion en curso
    short token;           //se emplea para control
} FILE;                   // tipo definido con el nombre FILE
```

Esta estructura se crea al momento en que es creado o abierto un archivo, el contenido es manipulado por las funciones de archivo que proporciona la biblioteca stdio.h. La estructura se destruye cuando se cierra el archivo.

En realidad, una variable de tipo FILE * representa un flujo de datos que se asocia con un dispositivo físico de entrada/salida (el archivo “real” estará almacenado en disco).



Para declarar un archivo se utiliza un puntero al tipo **FILE**:

```
FILE *archi;
```

La definición de la estructura FILE depende del compilador, pero en general mantiene un campo con la posición actual de lectura/escritura, un buffer para mejorar las prestaciones de acceso al archivo y algunos campos para uso interno, al programador le interesa su uso.

Operaciones básicas en el manejo de archivos

Para trabajar con archivos desde un programa se debe crear o abrir, según sea el caso. Para ambas situaciones, se crea un manejador lógico o un vínculo entre la variable tipo archivo y el archivo físico.

Para hacer referencia a una componente del archivo el lenguaje proporciona un apuntador de componente o un indicador de componente comúnmente llamado puntero. Al crear el archivo, está vacío y por lo tanto el puntero está en cero, a medida que se van guardando componentes, el archivo va creciendo en tamaño.

Mediante el puntero se puede acceder a distintas componentes, así si su valor es cero se apunta al comienzo de la primera componente, se puede también ubicar en las posiciones intermedias o al final del archivo, esto es al final de la última componente.

Las operaciones habituales en archivos son:

Grabar información: Consiste en guardar una componente en el archivo físico abierto.

Cerrar: Consiste en liberar el vínculo o manejador lógico que une el programa a través de la variable tipo archivo correspondiente, con el archivo físico vinculado.

Recuperar información: Para recuperar información almacenada en un archivo, éste debe primero ser abierto y luego a través de la operación de lectura, almacenar en memoria la componente para su manipulación.

Modificar información: Consiste en alterar una componente en un archivo físico.

Eliminar información: Es el proceso de quitar componentes de un archivo.

Creación y/o apertura de archivos en lenguaje C

Función fopen: Para procesar un archivo, primero se debe vincular una variable del programa con el archivo físico en el dispositivo externo, esto es lo que realiza la función fopen.

Formato

```
FILE * fopen (char *nombre, char *modo);
```

Esta función es utilizada para abrir y/o crear archivos en el dispositivo secundario y retorna un puntero a FILE.

La función fopen tiene dos parámetros que son cadenas de caracteres, el primero es el nombre del archivo físico que contiene los datos y el segundo corresponde al modo de acceso a éste.

nombre: es una cadena de caracteres que contiene un nombre de archivo válido.

modo: especifica la *forma* en que se va utilizar el archivo, esto es si es de lectura, escritura o para agregar información, y *el tipo* de valores permitidos a cada byte, de texto o binarios.



Este parámetro puede tomar los siguientes valores:

Modo	Apertura
“r”	Sólo lectura: el archivo debe existir
“w”	Escritura: se crea uno nuevo o se sobrescribe si ya existe.
“a”	Añadir: el archivo se abre para escritura situándose el puntero al final de éste. Si el archivo no existe, es creado.
“rb”	Lectura: abre un archivo binario
“wb”	Escritura: se crea un archivo binario nuevo o se sobrescribe si ya existe.
“ab”	Añadir: el archivo binario se abre para escritura situándose el puntero al final de éste. Si el archivo no existe, es creado.
“r+”	Lectura y/o escritura: debiendo asegurar la existencia del mismo.
“w+”	Lectura y escritura: se crea un archivo nuevo o se sobrescribe si ya existe.
“a+”	Añadir, lectura y/o escritura: el puntero se sitúa al final del archivo. Si el archivo no existe, es creado, además se dispone para lectura y escritura.

NOTA

Los tipos de archivos son de texto (modo t) o binario (modo b), si no se especifica el tipo, la opción por defecto depende del compilador utilizado, pero en general es modo texto.

El valor que retorna la función `fopen` es un puntero a `FILE` si la operación de la función es exitosa, caso contrario retorna `NULL`.

Un ejemplo es:

```
FILE *archi ;           /*declaración de la variable puntero a FILE*/
archi = fopen("Datos.dat", "w"); /*apertura del archivo Datos para escritura*/
```

Cierre de un archivo

Es importante cerrar los archivos antes de abandonar una aplicación. Esto implica almacenar datos que aún están en el buffer de memoria y actualizar aquellos datos de la cabecera del archivo que mantienen el sistema operativo. Además permite que otros programas puedan abrir el archivo para su uso. A menudo, los archivos no pueden ser compartidos por varios programas.

Función `fclose`

Formato

`int fclose(FILE *archivo);`

El parámetro de esta función es un puntero a la estructura `FILE` del archivo que se quiere cerrar, o `NULL` si se quiere cerrar todos los archivos abiertos.

El valor de retorno cero indica que el archivo ha sido correctamente cerrado, caso contrario indicaría que hubo un error.



Archivos organizados secuencialmente

Lenguaje C, permite que los archivos organizados en forma secuencial, puedan accederse secuencialmente o en forma directa. No obstante, hay archivos que se comportan siempre como secuenciales, por ejemplo los de entrada y salida estándar: stdin, stdout, stderr y stderr, estos son flujos o canales que se abren automáticamente.

En el caso de stdin, que suele estar asociado al teclado, sólo se podrá abrir como de lectura, leer los caracteres en el mismo orden en que fueron ingresados y a medida que estén disponibles. Lo mismo se aplica para stdout, asociado al manejo de la pantalla, sólo puede usarse para escritura y el orden en que se muestra la información es el mismo en que es enviada.

Cuando se accede secuencialmente a un archivo secuencial, hay que tener en cuenta algunas características para este tipo de acceso:

- La escritura de nuevos datos se realiza al final del archivo.
- Para leer una componente concreta hay que hacerlo secuencialmente desde el lugar donde está posicionado el puntero. Por lo tanto, es necesario "posicionarse siempre al comienzo" del archivo y recorrerlo secuencialmente hasta encontrar la componente buscada, dado que ella podría encontrarse en una posición anterior a la de la componente la cual esté apuntando en es momento el puntero.
- Los archivos secuenciales sólo se pueden abrir para lectura o escritura, nunca de los dos modos a la vez.



Se analizarán los siguientes tipos de archivos:

- Archivos de Caracteres
- Archivos de Textos
- Archivos sin formato.



Archivos de caracteres

Los archivos de caracteres son una secuencia de caracteres almacenados en un dispositivo de almacenamiento secundario e identificado por un nombre de archivo.

Grabar información

Para grabar una componente en un archivo de caracteres se utiliza la función **fputc**. En este tipo de archivo la componente es un carácter

Función fputc: Esta función permite guardar un carácter en el archivo.

Formato

```
int fputc(int caracter, FILE *archivo);
```

Los parámetros de entrada de esta función son el carácter a escribir, convertido a int y un puntero al FILE del archivo en donde se realizará la escritura.

El valor de retorno: es el carácter convertido a int escrito, si la operación fue completada con éxito, caso contrario será EOF.

NOTA: EOF es una constante simbólica entera, definida en el archivo de cabecera <stdio.h> que indica si el final de archivo ha sido o no alcanzado. Toma el valor cero si no se ha llegado al final, y distinto de cero si se ha llegado.

Ejemplo

```
#include <stdio.h>
#include <conio.h>
void main(void)
{ FILE *archi ;                               /*declaración de la variable tipo archivo*/
  archi=fopen("archivo.dat", "w");             /*apertura del archivo para escritura*/
  char c;                                       /*declaración de la variable caracter*/
  c ='Z';                                       /*asignación del carácter 'Z' a la variable*/
  fputc(c,archi);                             /*escritura en el archivo */
}
```

Recuperar información

Para recuperar una componente de un archivo de caracteres se utiliza la función fgetc.

Función fgetc: Esta función lee un carácter desde un archivo.

Formato

int fgetc(FILE *archivo);

Parámetro: un puntero a una estructura FILE del archivo del que se hará la lectura.

El valor de retorno es el carácter leído como un unsigned char convertido a int. Si no hay carácter disponible, el valor de retorno es EOF.

Ejemplo

```
#include <stdio.h>
#include <conio.h>
void main(void)
{FILE *archi ;                               /*declaración de la variable tipo archivo*/
  archi=fopen("archivo.dat", "r");           /*apertura del archivo para lectura*/
  char c ;                                    /*declaración de la variable caracter*/
  c=fgetc(archi);                             /*lectura del caracter del archivo */
  printf(" %c",c);
}
```

Ejemplo

El siguiente programa permite mostrar en la pantalla el código fuente del archivo ejer_escribe programa.cpp.

En este caso, se lee el archivo fuente como archivo de caracteres utilizando la función fgetc, y a medida que se lee se genera un archivo de caracteres de salida en la pantalla utilizando la función fputc.

```
#include <stdio.h>
#include <conio.h>
void main(void)
{char c;
  FILE *archivo;                             // declara la variable archivo de tipo file
  archivo = fopen("ejercicio_escribe programa.cpp", "r"); // abre el archivo ejercicio_escribe programa.cpp
  printf(" listado de mi programa fuente \n");           // y se posiciona al comienzo
  while (!feof(archivo))                                //itera el archivo hasta que llegue al final y envía
  { c=fgetc(archivo);                                   // los Caracteres leídos a la salida standard
    fputc(c, stdout);                                   // stdout, es el archivo de salida
  }                                                      // por defecto es el monitor
  fclose(archivo);
}
```

Archivos de textos

Un archivo de texto contiene un conjunto de “líneas” de caracteres de longitud variable y están separadas por un salto de línea.

El salto de línea está formado por dos caracteres especiales: avance de línea (line feed - carácter ASCII 10) y retorno de carro (CR carriage return - carácter ASCII 13)²¹.

El final del archivo se indica mediante el carácter ASCII 26, que también se expresa como ^Z o EOF.

Grabar información

La función que se utiliza para guardar una componente en un archivo de textos es **fputs**. Esta función permite grabar una componente (línea) en un archivo de cadenas caracteres.

Función fputs: Esta función graba una cadena de caracteres especificada como parámetro, es decir, fputs copia la cadena terminada en null en el archivo; (No se añade el carácter de retorno de línea ni el carácter nulo final).

Formato

int fputs(const char *s, FILE *archivo);

Los parámetros son:

s: la cadena que se va a grabar en el archivo.

archivo: un puntero a FILE identificando el archivo donde se guardará la cadena.

El valor de retorno de la función es un *número no negativo*, si la escritura en el archivo fue exitosa, caso contrario, si hubo algún error devuelve la constante EOF.

Ejemplo

El siguiente ejemplo genera un archivo de textos Mi_texto.txt. Para esto, se ingresa un texto por teclado y se supone que cada línea ingresada no supera los 200 caracteres.

```
#include <stdio.h>
#include <string.h>
void main ()
{ char texto[200];
  FILE *archi;
  archi=fopen ("Mi_Texto.txt","w");
  printf("Ingrese el texto a almacenar en el archivo\n");
  if (archi!=NULL)
  {
    gets(texto);
    while(strcmp(texto,"FIN"))
    { fputs (texto,archi);
      gets(texto);
    }
    fclose(archi);
  }
}
```

NOTA: Un archivo de texto se puede visualizar utilizando un procesador de texto como Word, bloc de notas. etc.

²¹ En una máquina de escribir convencional, para pasar a la siguiente línea se necesita dos movimientos: uno para desplazar el carro hacia la izquierda, y otro para mover el papel una línea hacia arriba. En los ordenadores se siguió esta analogía y se definieron los dos caracteres: CR para el retorno de carro, y LF para el salto de línea. Entre los dos (CRLF), se conseguía que una impresora hiciera estos movimientos para poder escribir una nueva línea de texto

Recuperar información:

La función que se utiliza para recuperar una componente en un archivo de textos es: **fgets** permite recuperar una componente (línea) de un archivo de cadenas de caracteres.

Función fgets

Esta función lee caracteres desde un archivo y los coloca en un arreglo de caracteres (variable tipo cadena). La lectura de caracteres termina cuando la cantidad de caracteres leídos alcanzó el número de caracteres especificado, o bien cuando se lea un carácter salto de línea o bien cuando no hayan más caracteres para leer, en este caso retorna NULL

Formato

char *fgets(char *s, int n, FILE * archivo);

Los parámetros son:

s: un puntero a la variable donde se guardará en memoria la cadena leída. **n**: la cantidad de caracteres que se desea leer.

archivo: un puntero a un FILE del archivo a leer.

El valor de retorno de la función es un puntero a la cadena leída o NULL si hubo algún error.

Ejemplo

```
#include <stdio.h>
#include <string.h>
void main ()
{ char texto[200];
  FILE *archi;
  archi=fopen ("Mi_Texto.txt","r");
  if (archi!=NULL)
  {
    printf("\n TEXTO INGRESADO \n ");
    while(!feof(archi))
    {
      fgets(texto,200, archi);
      puts(texto);
    }
    fclose(archi);
  }
}
```

Otras funciones para leer y grabar archivos de texto**fscanf**

La función fscanf funciona igual que scanf en cuanto a parámetros, pero la entrada se toma de un archivo de texto en lugar del teclado. Lee la cadena de caracteres hasta el primer espacio o salto de línea que encuentra. Permite especificar de qué tipo es el dato que se está leyendo. Una vez llevada a cabo la lectura, el puntero del archivo se sitúa en el carácter inmediatamente próximo al último espacio en blanco encontrado.

Formato

int fscanf(FILE *fichero, const char *formato, argumento, ...);

El prototipo correspondiente de fscanf es:

int fscanf(archivo, formato, argumento, ...);

Donde:

archivo: es la variable archivo de tipo FILE;

formato: es un especificador de formato (%d, %f,%4f,%s)

argumento: es la variable que recibe la información leída del archivo de texto, es del tipo indicado por el especificador de formato.

Ejemplo

Abrir el documento "fichero.txt" en modo lectura y mostrar su contenido en pantalla.

```
#include <stdio.h>
void lectura (FILE *fp)
{ char buffer[50];
  fp=fopen("fichero.txt","r");
  if (fp == NULL)
    printf ("Error de apertura de archivo");
  else{
    while (!feof(fp) )
    {
      fscanf(fp, "%s" ,buffer);
      puts(buffer);
    }
    fclose(fp);
  }
}
```

En el ejemplo se lee una cadena 50 caracteres y se muestra en pantalla, se puede en el mismo momento del `fscanf` hacer un cast, para esto, antes se deben declarar variables de los tipos de datos que se quiere obtener: modificando el ejemplo anterior donde en vez de mostrar en pantalla se cargue un arreglo de 40 componentes se obtiene el siguiente código:

```
#include <stdio.h>
typedef struct
{
  int día;
  char nombre[30];
  float sueldo;
} datos;

int lectura (FILE *fp, datos d[40])
{ int i=0;
  fp=fopen("fichero.txt","r");
  if (fp == NULL)
    printf ("Error de apertura de archivo");
  else{
    while ((!feof(fp))&&(i!=40))
    {
      fscanf(fp, "%d", &d[i].dia);
      fscanf(fp, "%s", d[i].nombre);
      fscanf(fp, "%f", &d[i].sueldo);
      i++;
    }
    fclose(fp);
    return (i);
  }
}
```

fprintf

La función `fprintf` funciona igual que `printf` en cuanto a parámetros, pero la salida se dirige a un archivo de texto en lugar de la pantalla.

Formato

```
int fprintf(FILE *archivo, const char *formato, argumento, ...);
```

El prototipo correspondiente de fprintf es:

```
int fprintf(archivo, formato, argumento, ...);
```

Donde:

archivo: es la variable archivo de tipo FILE;

formato: es un especificador de formato (%d, %f,%4f,%s)

argumento: es la variable que contiene la información que se va a escribir en el archivo de texto, es del tipo indicado por el especificador de formato.

Ejemplo

Abrir el documento "fichero.txt" en modo lectura/escritura, leer de teclado escribir en él.

```
#include <stdio.h>
```

```
void escritura (FILE *fp)
```

```
{
    char buffer[50];
    fp=fopen("fichero.txt","r+");
    if (fp == NULL)
        printf ("Error de apertura de archivo");
    else{
        gets(buffer);
        while (strcmp(buffer,"fin"))
        {
            fprintf (fp, "%s" ,buffer);
            gets(buffer);
        }
        fclose(fp);
    }
}
```

En el ejemplo se graban cadenas 50 caracteres. Se pueden usar para guardar en el archivo datos de cualquier tipo pero al momento de la escritura fprintf hará un cast y convertirá su contenido en tipo char. Modificando el ejemplo anterior donde se escribe en el archivo lo que se lee de teclado, se guardará el contenido de un arreglo de 40 componentes:

```
#include <stdio.h>
```

```
typedef struct
```

```
{
    int día;
    char nombre[30];
    float sueldo;
} datos;
```

```
void lectura (FILE *fp, datos d[40])
```

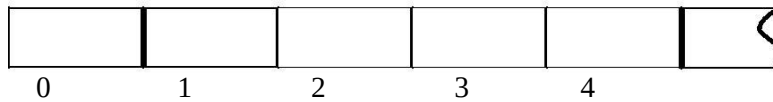
```
{
    fp=fopen("fichero.txt","r");
    if (fp == NULL)
        printf ("Error de apertura de archivo");
    else {
        for (int i=0; i < 40; i++)
        {
            fprintf (fp,"%d" , &d[i].dia);
            fprintf (fp,"%s", d[i].nombre);
            fprintf (fp,"%f", & d[i].sueldo);
        }
        fclose(fp);
    }
}
```

La información grabada en el archivo será solo de tipo char es decir solo texto.

Archivos sin formato

Estos archivos son tratados como un bloque de datos binarios (en bytes) que tan solo tienen significado si se conoce la distribución y los tipos de datos de los mismos, es decir si se conoce la estructura.

Respecto a la generación, es importante tener en cuenta que cuando se crea un *archivo secuencial*, a través de lenguaje C, los bloques se guardan uno a continuación de otro, ocupando posiciones contiguas en el almacenamiento. Estas posiciones se numeran a partir de 0.



En este tipo de archivo el acceso puede ser secuencial o directo. El acceso directo a los datos de un archivo se hace mediante su posición, es decir el lugar que ocupa en el archivo.

Por medio de un ejemplo se estudiarán las funciones a utilizar para realizar las operaciones básicas con archivos sin formato.

Ejemplo

A través de un programa en lenguaje C, generar un archivo de nombre “alumnos.dat”, que contenga la siguiente información correspondiente a los alumnos que cursan la materia Programación Procedural: Nombre y Número de registro y resultado de un parcial- A aprobado- R Reprobado.

Construir un nuevo programa que utilice el archivo “alumnos.dat” que permita:

- 1- Realizar un listado que contenga el número de registro y nombre de todos los alumnos aprobados.
- 2- Dado el nombre de un alumno, indicar si está aprobado.

El problema puede ser resuelto codificando tres funciones:

- a) **Cargar**, que permita generar el archivo. Para ello se lo debe abrir para escritura y grabar en él las componentes.
- b) **Listar**, para generar el listado de aprobados. Para esto, se debe abrir el archivo en modo lectura y recuperar su información. Esto es, se debe recorrerlo secuencialmente, mostrando de entre las componente leídas los datos de los alumnos aprobados
- c) **Buscar**, para la búsqueda de la información correspondiente a un alumno. En esta situación, se debe abrir el archivo en modo lectura y recorrerlo secuencialmente hasta encontrar el alumno buscado o el final del archivo en caso que el mismo no se encuentre

A continuación se muestran los formatos de las funciones provistas por el lenguaje para realizar las operaciones mencionadas:

Grabar información en el archivo

Función fwrite: Esta función permite trabajar con bloques de bytes de longitud constante. Es capaz de escribir en un archivo uno o varios bloques de la misma longitud almacenados a partir de una dirección de memoria determinada.

Formato

```
size_t fwrite(void *puntero, size_t tamaño, size_t nregistros, FILE *archivo);
```

Los parámetros de la función son:

- Un puntero a la zona de memoria donde están almacenados los datos a grabar.
- El tamaño de cada bloque.
- El número de bloques a grabar.
- Un puntero a FILE del archivo correspondiente.
- El valor de retorno: Es el número de bloques escritos.

NOTA: `size_t` es un tipo de dato definido por el lenguaje que se utiliza con la función `sizeof`, el valor que puede tomar la variable del tipo `size_t` pertenece al rango de entero sin signo.

El siguiente programa resuelve el apartado a) del ejemplo.

```
#include <stdio.h>
#include <conio.h>
#include <string.h>

typedef struct
{
    char nombre[30];
    int reg;
    char nota;
} alumno;

void cargar (FILE * archi)
{ alumno c;
  printf("\n ingrese el nombre del alumno (termina con FIN): ");
  gets(c.nombre);
  while( strcmp((c.nombre),"FIN"))
  {
      printf("\n ingrese el número de registro: ");
      scanf("%d",&c.reg);          /* Guarda en la variable tipo componente */
      printf("\n ingrese La Nota (A=aprobado R=reprobado) : ");
      fflush(stdin);
      c.nota=getch();              /* Guarda en la variable tipo componente */
      fwrite(&c,sizeof(c),1,archi); /* Guarda información en el archivo */
      printf("\n ingrese el nombre del alumno (termina con FIN): ");
      fflush(stdin);
      gets(c.nombre);
  }
}

void main(void)
{
    FILE *archi;
    char nom[30];

    if ((archi=fopen("alumnos.dat","w"))= =NULL)          /* Se abre para escritura */
        printf("hay error1 \n");
    else
        { cargar(archi);          //Apartado a)
          fclose(archi);
        }
}
```

Recuperar de información desde el archivo

Función fread

Esta función está pensada para trabajar con bloques de longitud constante. Es capaz de leer desde un archivo uno o varios bloques de la misma longitud, y guardarlos en una dirección de memoria determinada.

Formato**size_t fread(void *puntero, size_t tamaño, size_t numero, FILE *archivo);**

Los parámetros son:

- un puntero a la zona de memoria donde se almacenarán los datos a recuperar del archivo
- el tamaño de cada bloque.
- el número de bloques a leer.
- un puntero al FILE correspondiente.

El valor de retorno es el número de bloques leídos.

Función feof

Para recorrer de manera secuencial el archivo es necesario saber cuando se ha procesado la totalidad del mismo, para ello se cuenta con la función feof.

Formato**int feof(FILE *archivo);**

El parámetro de la función es un puntero al FILE del archivo.

El valor de retorno es cero si no se ha alcanzado el fin de archivo y distinto de cero en caso contrario.

Función fflush

Para mejorar las prestaciones del manejo de archivos se utilizan buffers, almacenes temporales de datos en memoria. Las operaciones de salida se hacen a través del buffer, y sólo cuando éste se llena, se realiza la escritura en el disco efectivamente.

En ocasiones hace falta vaciar el buffer manualmente, para eso se usa la función fflush.

Esta función permite descargar los datos acumulados en el buffer del archivo.

Formato**int fflush(FILE *archivo);**

El parámetro es un puntero a FILE del archivo. Si es NULL se hará el vaciado de todos los buffers asociados a los archivos abiertos.

El valor de retorno es cero si la función se ejecutó con éxito y distinto de cero si hubo error.

El siguiente es el código que permite resolver los ítems b) y c) del ejemplo.

```
#include <stdio.h>
#include <conio.h>
#include <string.h>
typedef struct
{
    char nombre[30];
    int reg;
    char nota;
} alumno;

void mostrar(FILE* xarchi)
{
    alumno c;
    rewind(xarchi);
    fread(&c,sizeof(c),1,xarchi);           //Se debe hacer una lectura anticipada
    while(feof(xarchi)==0 )
    {
        if(c.nota=='A')
            printf("\n Nombre Alumno %s Registro %d nota %c",c.nombre,c.reg,c.nota);
        fread(&c,sizeof(c),1,xarchi);
    }
}
```

```

void buscar(FILE *xarchi, char nom[30])
{
    alumno c; int b=0;
    rewind(xarchi);
    fread(&c, sizeof(c), 1, xarchi);
    while((!feof(xarchi)) && (b==0))
    {
        if (strcmp(c.nombre, nom) == 0)
        {
            printf("\n Alumno %s Encontrado Registro %d nota %c", c.nombre, c.reg, c.nota);
            b=1;
        }
        else fread(&c, sizeof(c), 1, xarchi);
    }
    if(b == 0)
        printf("\n Alumno %s NO se encontro", nom);
}

void main(void)
{
    FILE *archi;
    char nom[30];
    if ((archi = fopen("alumnos.dat", "r")) == NULL)           /* Se abre para lectura */
        printf ("hay error1 \n");
    else
    {
        mostrar(archi);           //Apartado 1
        printf("\n ingrese nombre del alumno a buscar: "); gets(nom);
        buscar(archi, nom);
        fclose(archi);
    }
}

```

Nota 1: Es posible posicionarse al comienzo del archivo a través de la función `rewind`, cuyo prototipo es: `void rewind(FILE *fp);`

Nota 2: Como la función `fread` retorna un valor distinto de cero cuando se ha leído un bloque, y cero cuando no hubo más bloques para leer (o no pudo leer), se puede recorrer el archivo secuencialmente sin usar la función `feof`, como se muestra a continuación:

```

void mostrar(FILE* xarchi )
{
    alumno c;
    rewind(xarchi);
    while(fread(&c, sizeof(c), 1, xarchi))
    {
        if(c.nota == 'A')
            printf("\n Nombre Alumno %s Registro %d nota %c", c.nombre, c.reg, c.nota);
    }
}

```

¿Cómo optimizaría el código anterior?

Para optimizar el código realizado y no recorrer dos veces el archivo en busca de los alumnos aprobados, se puede recorrer una vez el archivo para generar una lista con los alumnos aprobados y luego recorrerla para resolver los ítems b) y c)

```
#include <stdio.h>
#include <conio.h>
#include <string.h>
#include <malloc.h>
typedef char nombre[30];

typedef struct
{
    nombre nom;
    int reg;
    char nota;
} alumno;

struct nodo
{
    alumno datos;
    struct nodo * sig ;
};

typedef struct nodo * puntero;

void mostrar(FILE* xarchi )
{
    alumno c;
    rewind(xarchi);
    while(fread(&c,sizeof(c),1,xarchi)!=0 )
        printf("\n %5s %10d %10c",c.nom,c.reg,c.nota);
}

void generalista (FILE * xarchi, puntero &cab)
{
    alumno c;
    puntero nuevo;
    rewind(xarchi);
    while(fread(&c,sizeof(c),1,xarchi)!=0 )
    {
        if(c.nota=='A')
        {
            nuevo=( puntero) malloc(sizeof(struct nodo));
            nuevo-> datos = c;
            nuevo->sig = cab;
            cab = nuevo;
        }
    }
}

void mostrarApobados( puntero xp)
{
    while (xp != NULL)
    {
        printf("\n %5s %10d % 10c",xp->datos.nom, xp->datos.reg, xp->datos.nota);
        xp = xp->sig;
    }
}
```

int buscar(puntero xp, nombre nomb)

```

{ alumno c;
  while((xp!= NULL) && (strcmp(xp->datos.nom,nomb)!=0))
    xp = xp->sig;
  if (xp != NULL)
    return 1 ;
  else return 0;
}

```

void main(void)

```

{ FILE *archi;
  puntero cab;
  nombre nom;
  int b;
  if ((archi=fopen("alumnos.dat","r"))==NULL)          /* Se abre para lectura */
    printf("hay error1 \n");
  else
  { printf("\n LISTADO COMPLETO ALUMNOS, DESDE ARCHIVO \n");
    printf("\n NOMBRE REGISTRO NOTA");
    mostrar(archi);
    cab=NULL;
    generalista( archi,cab);
    printf("\n LISTADO ALUMNOS APROBADOS, DESDE LISTA\n");
    printf("\n NOMBRE REGISTRO NOTA");
    mostrarAprobados (cab);
    printf("\n ingrese nombre del alumno a buscar: "); gets(nom);
    b= buscar(cab,nom);
    if(b==1)
      printf("\n El Alumno ESTA APROBADO ");
    else printf("\n El Alumno NO está Aprobado ");
    fclose(archi);
  }
}

```

Lectura de varias componentes de un archivo

El lenguaje C permite en la función `fread` la lectura de varias componentes del archivo. El valor de retorno de `fread` es el número de componentes efectivamente leídas, el primer parámetro es la zona de memoria donde se ubicarán los datos leídos y el tercer parámetro es la cantidad de componentes que intenta leer. En el ejemplo anterior se ha realizado el procesamiento de una componente por vez. Se puede leer un grupo de componentes de una sola vez por ejemplo a un arreglo para luego ser procesadas

Ejemplo

Utilizando el archivo "alumnos.dat, mostrar en forma ordenada alfabéticamente los 20 primeros alumnos.

```

#include <stdio.h>
#include <conio.h>
#include <string.h>
const int m=20;
typedef struct
{ char nombre[30];
  int reg;
  char nota;
} alumno;

```

```
void ordena(alumno c[m], int n)    //ordenamiento en un arreglo
{
    alumno auxi;
    int i,cota,k ;
    cota=n-1;
    k=1;
    while (k!=-1)
    {
        k=-1;
        for(i=0;i<cota;i++)
            if(strcmp(c[i].nombre,c[i+1].nombre)>0)
            {
                auxi=c[i];
                c[i]=c[i+1];
                c[i+1]=auxi;
                k=i;
            }
        cota=k;
    }
}
```

```
void muestra( alumno c[m], int n)
{
    int i;
    for (i=0;i<n; i++) //muestra el arreglo ordenado
        printf("/n",          c[i].nombre,
        c[i].reg);
}
```

```
void listar_primeros(FILE* xarchi )
{
    alumno c[m];
    int cant, i;
    rewind(xarchi);
    cant= fread(c, sizeof(alumno), m, xarchi);    /* Se realiza la lectura de 20 Alumnos*/
    printf("Se leyeron %d registros \n",cant);
    ordena(c,cant);    //Se invoca la función para ordenar el arreglo
    printf(" LISTADO %d Alumnos ordenados alfabéticamente \n");
    printf("Nombre          Registro   \n");
    muestra(c,cant);
}
```

```
void main(void)
{
    FILE *archi;
    if ((archi=fopen("alumnos.dat","r+"))==NULL)    /* Se abre para lectura y escritura */
        printf("hay error1 \n");
    else
    {
        listar_primeros(archi);
        fclose(archi);
    }
}
```

Archivos organizados secuencialmente con acceso directo

Lenguaje C, permite manipular algunos archivos organizados en forma secuencial con acceso directo. Este es el caso de archivos con organización secuencial, que tienen en la componente un elemento llamado clave, que está en relación con la posición física del almacenamiento. Esto permite la ubicación precisa de la componente en el archivo.

Cuando se accede en forma directa a un archivo de acceso directo, hay que tener en cuenta algunas características:

- La escritura de nuevos datos se realiza al final del archivo sin que se pierda la relación de la clave con la posición física de la componente.
- Para leer una componente concreta se utiliza la clave para ubicar dicha componente

Funciones que permiten acceder en forma directa a un archivo organizado secuencialmente

Las siguientes son algunas funciones que posee el lenguaje C para gestionar el acceso a distintas posiciones de un archivo:

Función fseek

Esta función sirve para ubicar el puntero en el archivo, para lectura o escritura en el lugar deseado.

Formato

int fseek(FILE *archivo, long int desplazamiento, int origen);

Los parámetros son:

un puntero al FILE del archivo que se quiere modificar,

el valor del desplazamiento, expresado en bytes. Puede ser positivo o negativo.

el punto de origen desde el que se calculará el desplazamiento.

El valor de retorno: es cero si fue exitosa la función, y un valor distinto de cero si hubo error.

El parámetro *origen* puede tomar tres posibles valores:

Nombre	Valor	Descripción
SEEK_SET	0	el desplazamiento se cuenta desde el principio del archivo. El primer byte del archivo tiene un desplazamiento cero.
SEEK_CUR	1	el desplazamiento se cuenta desde la posición actual del puntero.
SEEK_END	2	el desplazamiento se cuenta desde el final del archivo.

Por ejemplo, si se quiere posicionar el puntero en el registro anterior se puede usar la función FSEEK, en la cual el desplazamiento se calcula así :

$\text{Desplazamiento} = \text{posición_actual_puntero_archivo} - \text{tamaño_bloque}$

Función fgetpos

Esta función se utiliza para obtener la posición actual del puntero del archivo. Dicha posición está dada en bytes.

Formato:

int fgetpos(FILE *archivo, fpos_t *posicion);

Los parámetros son, un puntero a FILE del archivo y la ubicación del registro dentro del archivo expresada en bytes.

El valor de retorno es cero si la función se ejecutó con éxito, y distinto de cero si hubo error.

NOTA: **fpos_t** es un tipo de dato definido por el lenguaje que se utiliza para hacer referencia a las posiciones de los punteros de archivos.

Acceso a una componente cuya posición en el archivo es conocida**Ejemplo**

El siguiente código genera un archivo de productos con los siguientes datos: código de producto, descripción, precio unitario y stock. (El código de producto está ordenado en forma ascendente y consecutiva a partir de 100).

```
#include <stdio.h>
#include <conio.h>
#include <string.h>
typedef struct
{
    int cod;
    char descrip[30];
    float pu;
    int stock;
} prod;
```

void cargar (FILE * xarchi)

```
{
    prod p;
    fpos_t x;
    printf("\n ingrese el nombre del producto (termina con FIN): "); fflush(stdin);
    gets(p.descrip);
    while( strcmp((p.descrip),"FIN"))
    {
        fseek(xarchi,0,
        SEEK_END);
        fgetpos(xarchi,&x);
        int cod=(int)(x/sizeof(prod))+100;
        printf("El codigo del nuevo producto es: %d",cod);  p.cod=cod;
        printf("\n ingrese el stock : ");  scanf("%d",&p.stock);
        printf("\n ingrese el precio unitario : ");  scanf("%f",&p.pu);
        fwrite(&p,sizeof(p),1,xarchi);
        printf("\n ingrese el nombre del producto (termina con FIN): "); fflush(stdin);
        gets(p.descrip);
    }
}
```

void mostrar(FILE* xarchi)

```
{
    prod p;
    if ((xarchi=fopen("producto.dat","rb"))==NULL)
        printf(" Error al abrir el archivo \n");
    else
        { while( fread(&p,sizeof(p),1,xarchi))
            printf("\n Nombre %s codigo %d Stock %d precio %f",p.descrip,p.cod,p.stock,p.pu);
        }
}
```

```
void main(void)
{ FILE *archi;
  int cod;
  if ((archi=fopen("producto.dat","w"))==NULL)          /* Se abre para escritura */
    printf("hay error1 \n");
  else
    { cargar(archi);
      fclose(archi);
      mostrar(archi);
      fclose(archi);
    }
}
```

Ejemplo

A partir del archivo de productos antes generado se pide, realizar un algoritmo en C que permita:

- a) Agregar un nuevo producto (el código se debe generar en forma automática).
- b) Dado el código de producto mostrar su stock.
- c) Mostrar la información de todos los productos.

```
#include <stdio.h>
#include <conio.h>
#include <string.h>
typedef struct
{  int cod;
   char descrip[30];
   float pu;
   int stock;
} prod;

void mostrar(FILE * xarchi)
{  prod p;
   rewind(xarchi);
   printf("\n Nombre      codigo   Stock      precio ");
   while( fread(&p,sizeof(p),1,xarchi))
     printf("\n %20s  %3d  %10d  %18.2f ", p.descrip, p.cod, p.stock, p.pu);
}
```

```
void agregar (FILE * xarchi)
{
  prod p;
  fpos_t x;
  printf("\n ingrese el nombre del producto: "); fflush(stdin);
  gets(p.descrip);
  fseek(xarchi,0, SEEK_END);    // se posiciona al final del archivo
  fgetpos(xarchi,&x);
  int cod=(int)(x/sizeof(prod))+100;
  printf("El codigo del nuevo producto es: %d",cod);  p.cod=cod;
  printf("\n ingrese el stock : ");  scanf("%d",&p.stock);
  printf("\n ingrese el precio unitario : "); scanf("%f",&p.pu);
  fwrite(&p,sizeof(p),1,xarchi);
}
```


void mostrarstock(FILE *xarchi, int pos)

```
{prod p;
  rewind(xarchi);
  fseek(xarchi, pos*sizeof(prod),SEEK_SET);
  fread(&p,sizeof(p),1,xarchi);
  printf("\n Nombre producto %s stock %d",p.descrip,p.stock);
}
```

void main(void)

```
{ FILE *archi;
  int cod, opcion;
  if ((archi=fopen("producto.dat","a+"))==NULL) // lectura con posibilidad de agregar
      printf("hay error1 \n");
  else
  { do {
      printf("\n 1. agregar un producto");
      printf("\n 2. mostrar stock de un producto");
      printf("\n 3. mostrar todos los productos");
      printf("\n 4. salir\n");
      scanf("%d", &opcion);
      switch(opcion)
      { case 1: agregar(archi); break;
        case 2:
            { printf("\n Ingrese codigo de producto para mostrar su stock "); //Apartado b
              scanf("%d",&cod);
              mostrarstock(archi,cod-100);
              break;
            } ;
        case 3: mostrar(archi);
        case 4: break;
      } // fin del switch
    } while (opcion !=4); // fin del do
    fclose(archi);
  }
}
```

Modificación de una componente del archivo**Primer caso: Cuando se ingresa la ubicación de una componente**

En este caso el archivo está organizado secuencialmente y está ordenado en forma ascendente y consecutiva a partir de un número natural (por ejemplo, si trabajamos con el archivo “producto.dat” antes generado).

La modificación de una componente requiere un proceso que comprende

1. leer directamente la componente, a partir de la posición de la misma
2. modificar la información en la variable
3. ubicar el puntero en la posición anterior
4. grabar la variable en el archivo.

¿Por qué debe realizarse el paso 3?

Después de cada operación de lectura o escritura, el puntero del archivo se actualiza automáticamente a la siguiente posición. Por lo tanto si se necesita grabar una componente en el archivo que ha sido modificada en memoria, como el puntero está posicionado en la componente siguiente, hay que retroceder tantos bytes como el tamaño del registro. De esta manera, la componente modificada se graba en la posición que le corresponde.

Ejemplo

A continuación se muestra la función que actualiza el stock de un producto del archivo “producto.dat” cuyo código se ingresa por teclado.

void modistock(FILE *xarchi,int cod)

```
{ prod p;
  fseek(xarchi,(cod-100)*sizeof(prod), SEEK_SET); // Se ubica el puntero en la posición deseada
  fread(&p,sizeof(p),1,xarchi);
  printf("\n Nombre producto %s stock %d", p.descrip, p.stock);
  printf("\n Ingrese el nuevo stock");
  scanf("%d",&p.stock); //Se modifica el valor de la componente
  fseek(xarchi,(cod-100)*sizeof(prod), SEEK_SET); //Se ubica el puntero de nuevo
  fwrite(&p,sizeof(p),1,xarchi); //Se graba la componente en el archivo
}
```

Segundo caso: Cuando se ingresa un dato específico de una componente (no la ubicación)

El archivo está organizado secuencialmente y puede o no presentar un orden específico.

La modificación de una componente requiere un proceso que comprende

1. leer el archivo hasta encontrar la componente buscada
2. modificar la información en la variable
3. ubicar el puntero en la posición anterior
4. grabar la variable en el archivo.

Ejemplo

A continuación se muestra la función que actualiza el stock de un producto del archivo “producto.dat” cuyo nombre se ingresa por teclado.

```
#include <stdio.h>
```

```
#include <string.h>
```

```
typedef struct
```

```
{ int cod;
  char descrip[30];
  float pu;
  int stock;
} prod;
```

void modistock_pr_nombre(FILE *xarchi,char *n)

```
{ prod p; fpos_t x;
  rewind(xarchi);
  fread(&p,sizeof(p),1,xarchi);
  while (!feof(xarchi) && strcmp(p.descrip,n)) fread(&p,sizeof(p),1,xarchi);
  if(!feof(xarchi))
  { printf("\n Nombre producto %s stock %d",p.descrip,p.stock);
    printf("\n Ingrese el nuevo stock");
    fgetpos(xarchi,&x); //Obtiene la posición actual del puntero
    scanf("%d",&p.stock); //Se modifica el valor de la componente
    fseek(xarchi,x- sizeof(p),SEEK_SET); //Se ubica en la posición anterior
    fwrite(&p,sizeof(p),1,xarchi); //Se graba la componente en el archivo
    fclose(xarchi);
    printf("\n Cambio realizado");
  }
  else printf("\ el producto no esta en el archivo");
}
```

void main(void)

```
{ FILE *archi;
  char n[30];
  if ((archi=fopen("producto.dat","r+"))==NULL)      /* Se abre para lectura y escritura */
    printf("hay error \n");
  else {
    printf("\n Ingrese nombre del producto para Actualizar su stock "); gets(n);
    modistock_pr_nombre(archi,n);
    fclose(archi);
  }
}
```

Eliminación de información del archivo

La eliminación de registros de un archivo secuencial es complicada, debido a que no existe función de librería estándar que lo haga.

Para eliminar registros, se debe ubicar en el registro que se desea eliminar, leerlo, marcar dicha componente y grabarlo nuevamente. Luego se deberá crear un archivo auxiliar donde se graben los registros sin marcas. Finalmente se borra el archivo original y se renombra el creado recientemente.

rename

Cambia el nombre a un archivo físico de un dispositivo de almacenamiento. Cambia el nombre actual por un nuevo nombre

Formato

int rename(const char* viejo_nombre , const char* nuevo_nombre);

Los parámetros son dos cadenas, el nombre que el archivo tiene actualmente y otra cadena con el nuevo nombre para dicho archivo físico.

El valor de retorno es cero si la función se ejecutó con éxito, y distinto de cero si hubo error.

remove

Borra o elimina un archivo físico de un dispositivo de almacenamiento.

Formato

int remove(const char *nombre_archivo);

El parámetro es, una cadena nombre_archivo que contiene el nombre del archivo físico que se quiere eliminar.

El valor de retorno es cero si la función se ejecutó con éxito, y distinto de cero si hubo error.

Ejemplo

Se tiene un archivo **clientes.dat** con los siguientes datos: nombre del cliente y número de cuenta. Se pide eliminar del archivo los clientes cuyo número de cuenta se ingresa por teclado, el ingreso finaliza con número de cuenta igual a cero.

```
#include <stdio.h>
```

```
typedef struct
```

```
{   char nombre[30];
    int  cuen-
    ta;
} cliente;
```

```
void marcar (FILE *xarchi);
```

```
void compactar (FILE *xarchi, FILE *xauxi);
```

void main(void)

```
{
FILE *archi, *auxi;
if ((archi=fopen("clientes.dat","r+"))==NULL)    // Se abre para lectura y escritura
    printf("hay error \n");
else
{
    marcar(archi);                                /* Marca los registros a borrar */
    auxi=fopen("auxiliar.dat","wb");              /* crea un archivo auxiliar para grabar */
    compactar(archi,auxi);                        /* pasa los registros válidos a un archivo auxiliar */
    fclose(archi);                                /* cierra el archivo */
    fclose(auxi);                                 /* cierra el archivo */
    remove("clientes.dat");                       /* Borra el archivo físico clientes.dat */
    rename("auxiliar.dat","clientes.dat");        /*cambia el nombre */
}
}
```

void marcar (FILE *xarchi)

```
{ cliente c;
  int nro,b;
  printf("\n Ingrese un número de cuenta a eliminar: ");
  scanf("%d",&nro);
  while(nro!=0)
  {   b=0;
      fseek(xarchi, 0, SEEK_SET);
      while((b==0)&&
fread(&c,sizeof(c),1,xarchi))
          if(nro==c.cuenta) b=1;
      if(b==1)
      {   fseek(xarchi,- sizeof(c),SEEK_CUR);
          c.cuenta= -1;                // coloca -1 en el número de cuenta a eliminar
          fwrite(&c,sizeof(c),1,xarchi);
          printf("\n El Cliente %s ha sido Marcado para Borrarse", c.nombre);
      }
      else printf("\n La cuenta no existe");
      printf("\n Ingrese un número de cuenta a eliminar: "); scanf("%d",&nro);
  }
}
```

void compactar(FILE *xarchi,FILE *xauxi)

```
{ cliente c;
  fseek(xarchi, 0, SEEK_SET);
  fread(&c,sizeof(c),1,xarchi);
  while( !feof(xarchi))
  {   if (c.cuenta!=-1)
      fwrite(&c,sizeof(c),1,xauxi);
      fread(&c,sizeof(c),1,xarchi);
  }
}
```

Nota: Estos pasos se pueden optimizar cuando son secuenciales, ya que en vez de marcar, se copian los registros a conservar en el nuevo archivo y luego este se renombra.

Cálculo de la longitud de un archivo

Ejemplo

A continuación se muestra un algoritmo para el cálculo de la longitud de un archivo.

```
#include <stdio.h>
typedef struct
{ char nombre[30];
  int cuenta;
} cliente;

void main(void)
{ long nRegistros;
  fpos_t nBytes;
  cliente cli;
  FILE *archivo;
  archivo=fopen("clientes.dat", rb)           //se abre para lectura
  fseek(archivo, 0, SEEK_END);               // Coloca el puntero al final del archivo
  fgetpos(archivo,&nBytes);                   // Tamaño en bytes
  nRegistros = nBytes/sizeof(cli);           // Tamaño en registros
  printf("La cantidad de registro en el archivo es : %d", nRegistros);
}
```

Bibliografía

- Apuntes de Cátedra Programación Procedural
- Aho Alfred V., John E. Hopcroft, Jeffrey D. Ullman (1988) "Estructura de datos y algoritmos publicación". Addison-Wesley Iberoamericana: Sistemas Técnicos de Edición. México, DF.
- Braunstein y Gioia (1987) "Introducción a la Programación y a la Estructura de Datos" - Eudeba
- Cairó Osvaldo (2006) "Metodología de la Programación. Algoritmos, diagramas de flujo y programas". 3ª Edición Alfaomega. México.
- Criado Clavero, Mª Asunción.(2006) "Programación en Lenguajes Estructurados". Alfaomega. Rama. México.
- Deitel, H M y Deitel, P J (2003) "Como programar en C/C++" Pearson Educación – Hispanoamericana
- Joyanes Aguilar Luis (2004) "Algoritmos y Estructuras de Datos. Una Perspectiva en C". Segunda Edición Mc Graw Hill.

Anexo I

Conceptos de diseño y programación relevantes para una práctica eficiente

La visión clásica de la programación estructurada: la programación sin goto

La visión clásica de la programación estructurada se refiere al control de ejecución. El control de su ejecución es una de las cuestiones más importantes que hay que tener en cuenta al construir un programa en un lenguaje de alto nivel. La regla general es que las instrucciones se ejecuten sucesivamente una tras otra, pero diversas partes del programa se ejecutan o no dependiendo de que se cumpla alguna condición. Además, hay instrucciones (los bucles) que deben ejecutarse varias veces, ya sea en número fijo o hasta que se cumpla una condición determinada.

Sin embargo, algunos lenguajes de programación más antiguos (como Fortran) se apoyaban en una sola instrucción para modificar la secuencia de ejecución de las instrucciones mediante una transferencia incondicional de su control (con la instrucción *goto*, del inglés "go to", que significa "ir a"). Pero estas transferencias arbitrarias del control de ejecución hacen los programas muy poco legibles y difíciles de comprender. A finales de los años sesenta, surgió una nueva forma de programar que reduce a la mínima expresión el uso de la instrucción *goto* y la sustituye por otras más comprensibles.

Esta forma de programar se basa en un famoso teorema, desarrollado por Edsger Dijkstra, que demuestra que todo programa puede escribirse utilizando únicamente las tres estructuras básicas de control siguientes:

- **Secuencia:** el bloque secuencial de instrucciones, instrucciones ejecutadas sucesivamente, una detrás de otra.
- **Selección:** la instrucción condicional con doble alternativa, de la forma "*if* condición *then* instrucción-1 *else* instrucción-2".
- **Iteración:** el bucle condicional "*while* condición *do* instrucción", que ejecuta la instrucción repetidamente mientras la condición se cumpla.

Los programas que utilizan sólo estas tres instrucciones de control básicas o sus variantes (como los bucles *for*, *repeat* o la instrucción condicional *switch-case*), pero no la instrucción *goto*, se llaman **estructurados**.

Ésta es la noción clásica de lo que se entiende por **programación estructurada** (llamada también **programación sin goto**) que hasta la aparición de la programación orientada a objetos se convirtió en la forma de programar más extendida. Esta última se enfoca hacia la reducción de la cantidad de estructuras de control para reemplazarlas por otros elementos que hacen uso del concepto de polimorfismo. Aun así los programadores todavía utilizan las estructuras de control (*if*, *while*, *for*, etc.) para implementar sus algoritmos porque en muchos casos es la forma más natural de hacerlo.

Una característica importante en un programa estructurado es que puede ser leído en secuencia, desde el comienzo hasta el final sin perder la continuidad de la tarea que cumple el programa, lo contrario de lo que ocurre con otros estilos de programación.

Este hecho es importante debido a que es mucho más fácil comprender completamente el trabajo que realiza una función determinada si todas las instrucciones que influyen en su acción están físicamente contiguas y encerradas por un bloque. La facilidad de lectura, de comienzo a fin, es una consecuencia de utilizar solamente tres estructuras de control, y de eliminar la instrucción de transferencia de control *goto*.

Cuando en la actualidad se habla de programación estructurada, nos solemos referir a la división de un programa en partes más manejables (usualmente denominadas **segmentos** o **módulos**). Una regla práctica para lograr este propósito es establecer que cada segmento del programa no exceda, en longitud, de una página de codificación, o sea, alrededor de 50 líneas.

Así, la visión moderna de un programa estructurado es un compuesto de segmentos, los cuales puedan estar constituidos por unas pocas instrucciones o por una página o más de código. Cada segmento tiene solamente una entrada y una salida, asumiendo que no poseen bucles infinitos y no tienen instrucciones que jamás se ejecuten.

Encontramos la relación entre ambas visiones en el hecho de que los segmentos se combinan utilizando las tres estructuras básicas de control mencionadas anteriormente y, por tanto, el resultado es también un programa estructurado.

Cada una de estas partes englobará funciones y datos íntimamente relacionados semántica o funcionalmente. En una correcta partición del programa deberá resultar fácil e intuitivo comprender lo que debe hacer cada módulo.

En una segmentación bien realizada, la comunicación entre segmentos se lleva a cabo de una manera cuidadosamente controlada. Así, una correcta partición del problema producirá una nula o casi nula dependencia entre los módulos, pudiéndose entonces trabajar con cada uno de estos módulos de forma independiente. Este hecho garantizará que los cambios que se efectúen a una parte del programa, durante la programación original o su mantenimiento, no afecten al resto del programa que no ha sufrido cambios.

Esta técnica de programación conlleva como ventaja que el coste de resolver varios subproblemas de forma aislada es con frecuencia menor que el de abordar el problema global.

La visión moderna de la programación estructurada: La Segmentación

La realización de un programa sin seguir una técnica de programación produce frecuentemente un conjunto enorme de sentencias cuya ejecución es compleja de seguir, y de entender, pudiendo hacer casi imposible la depuración de errores y la introducción de mejoras. Se puede incluso llegar al caso de tener que abandonar el código preexistente porque resulte más fácil empezar de nuevo.

Cuando en la actualidad se habla de programación estructurada, nos solemos referir a la división de un programa en partes más manejables (usualmente denominadas **segmentos** o **módulos**).

Así, la visión moderna de un programa estructurado es un compuesto de segmentos, los cuales puedan estar constituidos por unas pocas instrucciones o por una página o más de código.

Cada segmento tiene solamente una entrada y una salida, asumiendo que *no poseen bucles infinitos y no tienen instrucciones que jamás se ejecuten*. Encontramos la relación entre ambas visiones en el hecho de que los segmentos se combinan utilizando las tres estructuras básicas de control mencionadas anteriormente y, por tanto, el resultado es también un programa estructurado.

Cada una de estas partes englobará funciones y datos íntimamente relacionados semántica o funcionalmente. En una correcta partición del programa deberá resultar fácil e intuitivo comprender lo que debe hacer cada módulo.

En una segmentación bien realizada, la comunicación entre segmentos se lleva a cabo de una manera cuidadosamente controlada. Así, una correcta partición del problema producirá una nula o casi nula dependencia entre los módulos, pudiéndose entonces trabajar con cada uno de estos módulos de forma independiente. Este hecho garantizará que los cambios que se efectúen a una parte del programa, durante la programación original o su mantenimiento, no afecten al resto del programa que no ha sufrido cambios.

Esta técnica de programación conlleva las siguientes ventajas:

- El coste de resolver varios subproblemas de forma aislada es con frecuencia menor que el de abordar el problema global.
- Facilita el trabajo simultáneo en paralelo de distintos grupos de programadores.
- Posibilita en mayor grado la reutilización del código (especialmente de alguno de los módulos) en futuras aplicaciones.

Aunque no puede fijarse de antemano el número y tamaño de estos módulos, debe intentarse un equilibrio entre ambos factores.

Programación Estructurada

La programación estructurada es un paradigma de programación que se basa en la división de un programa en bloques de código más pequeños y fáciles de entender, con el objetivo de facilitar su comprensión, mantenimiento y depuración. Se enfoca en el uso de estructuras de control como secuencias, selecciones y repeticiones para organizar el flujo de ejecución de un programa de manera lógica y ordenada. Este enfoque ayuda a mejorar la legibilidad y la eficiencia del código, así como a reducir la posibilidad de errores.

La programación estructurada se basa en 3 técnicas básicas:

- **Diseño descendente:** consiste en dividir el problema y hacer la segmentación en diferentes niveles, el programa es complejo se divide en subprocesos (divide y vencerás).
- **Recursos abstractos:** consiste en el proceso de realización de diferentes pasos hasta encontrar la solución de un problema.
- **Estructura básica de control:** consiste en que el programa cuenta con un único punto de entrada y diferentes tipos de salida.

La programación estructurada es una teoría de programación que consiste en construir programas de fácil comprensión. Es especialmente útil, cuando se necesitan realizar correcciones o modificaciones después de haber concluido un programa o aplicación. Al haberse utilizado la programación estructurada, es mucho más sencillo entender la codificación del programa, que se habrá hecho en diferentes secciones.

Se basa en una metodología de desarrollo de programas llamada refinamientos sucesivos: Se plantea una operación como un todo y se divide en segmentos más sencillos o de menor complejidad (subprocesos - subprogramas). Una vez terminado todos los segmentos del programa, se procede a unificar las aplicaciones realizadas. Si se ha utilizado adecuadamente la programación estructurada, esta integración debe ser sencilla y no presentar problemas al integrar la misma, y de presentar algún problema, será rápidamente detectable para su corrección.

La representación gráfica de la programación estructurada se realiza a través de diagramas que representen el programa con sus entradas, procesos y salidas.

La programación estructurada propone separar los procesos en estructuras lo más simple posibles, las cuales se conocen como secuencia, selección e interacción. Ellas están disponibles en todos los lenguajes modernos de *programación imperativa* en forma de sentencias.

Combinando esquemas sencillos se pueden llegar a construir sistemas amplios y complejos, pero de fácil entendimiento.

Ventajas:

- El programa se puede dividir (segmentar) en varias partes.
- Se puede detectar rápidamente los errores (si se hizo bien la división).
- A partir de esquemas sencillos se puede construir un sistema amplio y complejo pero amigable

Desventajas:

- Si se hacen mal la división no se podrán detectar los errores

Conceptos de la Programación Modular

Se presenta históricamente como una evolución de la programación estructurada para solucionar problemas de programación más grandes y complejos de lo que ésta puede resolver.

Al aplicar la programación modular, un problema complejo debe ser dividido en varios subproblemas más simples, y estos a su vez en otros subproblemas más simples. Esto debe hacerse hasta obtener subproblemas lo suficientemente simples como para poder ser resueltos fácilmente con algún lenguaje de programación.

Esta técnica se llama refinamiento sucesivo, divide y vencerás o análisis descendente (Top-Down).

Un módulo es cada una de las partes de un programa que resuelve uno de los subproblemas en que se divide el problema complejo original.

Cada uno de estos módulos tiene una tarea bien definida y algunos necesitan de otros para poder operar.

La programación modular es uno de los métodos de diseño más flexibles y potentes para mejorar la productividad de un programa.

En programación modular el programa se divide en módulos (partes independientes), cada una de las cuales ejecuta una única función o actividad y se codifican independientemente de otros módulos.

Cada uno de estos módulos se analiza, codifica y pone a punto por separado.

Cada programa contiene un módulo denominado programa principal que controla todo lo que sucede; se transfiere el control a los submódulos o subprogramas. Estos ejecutan su función y una vez completada su tarea, devuelven el control al módulo principal.

Por definición la programación modular es un paradigma de programación que consiste en dividir un programa en módulos o subprogramas con el fin de hacerlo más legible y manejable.

Descomposición Modular

El diseño modular propone dividir el sistema en partes diferenciadas y definir sus interfaces, sus ventajas: claridad, reducción de costos y reutilización.

Los pasos a seguir son:

- Identificar los módulos
- Describir cada módulo
- Describir las relaciones entre módulos

Una descomposición modular debe poseer ciertas cualidades mínimas para que se pueda considerar suficientemente válida.

a) Independencia funcional

a.1) Acoplamiento

a.2) Cohesión

b) Comprensibilidad

c) Adaptabilidad

a) Independencia funcional

Cada módulo debe realizar una tarea concreta o un conjunto de tareas muy afines. Es recomendable reducir las relaciones entre módulos al mínimo.

Para medir la independencia funcional hay dos criterios: acoplamiento y cohesión.

a.1) Acoplamiento

El acoplamiento es una medida de la interconexión entre módulos en la estructura del programa. Se tiende a que el acoplamiento sea lo menor posible, esto es, a reducir las interconexiones entre los distintos módulos en que se structure nuestra aplicación.

Alta (peor)

- **Por contenido**, cuando desde un módulo se puede cambiar datos locales de otro.
- **Común**, se emplea una zona común de datos a la que tienen acceso varios módulos.

Media

- **De control**, la zona común es un dispositivo externo al que están ligados los módulos, esto implica que un cambio en el formato de datos los afecta a todos.

Baja (mejor)

- **De datos**, viene dado por los datos que intercambian los módulos.
- **Sin acoplamiento directo**, no se comparten datos.

a.2) Cohesión

Un módulo coherente ejecuta una tarea sencilla y requiere poca interacción con tareas que se ejecutan en otras partes de un programa.

Podemos decir que un módulo coherente es aquel que intenta realizar solamente una sola actividad.

Para que el número de módulos no sea demasiado elevado y complique el diseño se tratan de agrupar tareas o actividades afines y relacionadas en un mismo módulo.

Alta (Mejor)

- **Cohesión Funcional:** un módulo realiza una única actividad muy específica.

Media

- **Cohesión Secuencial:** un módulo contiene actividades que han de realizarse en un orden particular donde la salida de una actividad es la entrada de la siguiente.
- **Cohesión de Comunicación:** un módulo contiene un conjunto de actividades que se realizan sobre los mismos datos.
- **Cohesión Temporal:** las actividades se incluyen en un módulo porque han de realizarse al mismo tiempo; p.ej. inicialización.

Baja (peor)

- **Cohesión Procedural:** un módulo contiene actividades que se realizan en un orden concreto, aunque sean independientes.
- **Cohesión Lógica:** cuando un módulo contiene actividades cuya ejecución depende de un parámetro que indique cuál de ellas se realizará.
- **Cohesión por Coincidencia:** cuando las actividades de un módulo no guardan ninguna relación observable entre ellas.

b) Comprensibilidad

Para facilitar los cambios, el mantenimiento y la reutilización de módulos es necesario que cada uno sea comprensible de forma aislada.

Para ello es bueno que posea independencia funcional, pero además es deseable:

- **Identificación,** el nombre debe ser adecuado y descriptivo
- **Documentación,** debe aclarar todos los detalles de diseño e implementación que no queden de manifiesto en el propio código
- **Simplicidad,** las soluciones sencillas son siempre las mejores.

c) Adaptabilidad

La adaptación de un sistema resulta más difícil cuando no hay independencia funcional, es decir, con alto acoplamiento y baja cohesión, y cuando el diseño es poco comprensible.

Factores para facilitar la adaptabilidad:

- **Previsión,** es necesario prever que aspectos del sistema pueden ser susceptibles de cambios en el futuro, y poner estos elementos en módulos independientes, de manera que su modificación afecte al menor número de módulos posibles
- **Accesibilidad,** debe resultar sencillo el acceso a los documentos de especificación, diseño, e implementación para obtener un conocimiento suficiente del sistema antes de proceder a su adaptación
- **Consistencia,** después de cualquier adaptación se debe mantener la consistencia del sistema, incluidos los documentos afectados

Programación monolítica

Una arquitectura monolítica es un modelo tradicional de un programa de software que se compila como una unidad unificada y que es autónoma e independiente de otras aplicaciones. Al desarrollar con una arquitectura monolítica, la ventaja principal es la velocidad de desarrollo, gracias a la simplicidad de tener una aplicación basada en una única base de código.

Ventajas de una arquitectura monolítica

- **Implementación sencilla:** un único archivo ejecutable facilita la implementación.
- **Desarrollo:** desarrollar una aplicación compilada con una única base de código es más sencillo.
- **Rendimiento:** en una base de código y un repositorio centralizados suele realizar la misma función que muchas pequeñas aplicaciones.
- **Pruebas simplificadas:** una aplicación monolítica es una unidad única y centralizada, por lo que las pruebas integrales se pueden hacer más rápido que con una aplicación distribuida.
- **Depuración sencilla:** con todo el código ubicado en un solo lugar, es más fácil rastrear las solicitudes y localizar incidencias.

Desventajas de una arquitectura monolítica

Las aplicaciones monolíticas pueden ser bastante efectivas, hasta que crecen demasiado y la escalabilidad se convierte en un desafío. Para hacer un pequeño cambio en una sola función, hay que compilar y probar todo el software, lo que va en contra del concepto ágil que prefieren los desarrolladores de hoy en día.

Estas son algunas desventajas de una arquitectura monolítica:

- **Velocidad de desarrollo más lenta:** con una aplicación grande y monolítica, el desarrollo es más complejo y lento.
- **Escalabilidad:** no se pueden escalar componentes individuales.
- **Fiabilidad:** si hay un error en algún módulo, puede afectar a la disponibilidad de toda la aplicación.
- **Barrera para la adopción de tecnología:** cualquier cambio en el marco o el lenguaje afecta a toda la aplicación, lo que hace que los cambios suelen ser costosos y lentos.
- **Falta de flexibilidad:** un monolito está limitado por las tecnologías que se utilizan en él.
- **Implementación:** un pequeño cambio en una aplicación monolítica requiere una nueva implementación de todo el monolito.

Aplicación de conceptos

Para la elaboración de programas, nos basaremos en los conceptos de la programación estructurada y los de programación modular, teniendo en cuenta los siguientes detalles:

Etapa 1: Análisis de enunciado

Aplicar los conceptos de descomposición modular. Se interpreta el enunciado detectando cuales son los datos de entrada y cuales los datos de salida, paralelamente se deducen las actividades que se deben realizar para la gestión de estos.

El análisis del enunciado implica saber lo que se quiere obtener y con que trabajar para lograrlo.

En esta etapa se debe identificar con detalle todos los datos de entrada, que en general se expresan en la primera parte del texto donde se describe la realidad sobre la cual se trabaja.

Para la detección de los datos de salida es convenientes observar las peticiones, éstas permiten identificar los requisitos de salidas y también identificar actividades a realizar.

El resultado de esta etapa es:

- Datos de Entrada
- Datos de Salida
- Listado de actividades a realizar (una descripción general de **qué hacer**)

Etapa 2: Prediseño

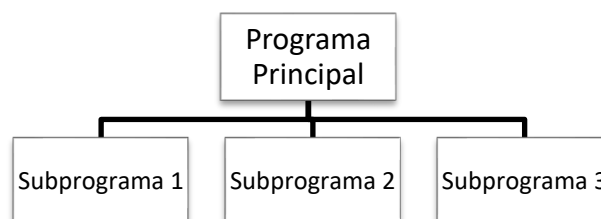
Se debe establecer las relaciones que existen entre las actividades, a partir de esto se organiza una estructura de diseño descendente (árbol) donde se puede visualizar el programa principal y los subprogramas (segmento o módulo).

En este punto se deben diseñar con más detalles los subprogramas a partir del listado de actividades, estas se distribuyen y se agrupan en función de los conceptos de cohesión y acoplamiento.

- Representar Los procesos como un diagrama. Esto para cada actividad es necesario tener en claro que recibe y que devuelve (lo que elabora o produce como salida).



- Estructurar el sistema como jerarquías de procesos. Este paso implica identificar la secuencia en que se ejecutarán los subprogramas.



Etapa 3: Diseño

Se debe aplicar el concepto de divide y vencerás sobre cada subprograma identificado en la etapa anterior, se determina con detalle que hacen (que actividad/es resolverán), sus entradas y salidas con precisión. Tener en cuenta para esto los conceptos de cohesión, el objetivo es la independencia funcional, obtener subprogramas que respondan a la cohesión funcional, secuencial o de comunicación que se encuentran entre las mejores opciones de desarrollo.

- Descomposición de procesos realizando refinamiento sucesivo.
- Verificar el prediseño y reestructurar. La verificación se realiza analizado si están correctamente aplicados los conceptos de Cohesión y Acoplamiento entre otros.

Etapas 4: Codificación y Depuración

Se debe codificar cada subprograma en lenguaje C estándar, teniendo en claro para esto todos los conceptos vistos.

En esta etapa se trabaja en detalle el **cómo hacer**.

Tener mucho cuidado de no caer en el concepto de ruptura de secuencia, es decir no usar sentencias que imiten el funcionamiento de goto como *break* para salir de estructuras de control, o *return* para finalizar opcionalmente funciones.

- Realizar la implementación (codificación).

Etapas 5: Documentación

Esta etapa es transversal en las diferentes etapas. Si bien, no se pretende la formalidad del desarrollo de manuales, gestar como un hábito de trabajo la inserción de comentarios en el código que expliquen que se hace en diferentes partes del mismo.

Para los mensajes que debe leer el usuario, mantener un lenguaje claro y preciso ya que, la carga de datos debe ser correcta y de ello depende que los resultados sean precisos.

Las salidas en pantalla deben ser diseñadas con cuidado, pensar como se leerá la información que se muestre es muy importante, tanto en la cantidad de información que se muestra como así también en los momentos que se limpia la pantalla.

Ejemplo**Enunciado**

Codifique en C funciones óptimas que le permitan resolver la siguiente problemática.

Una industria comercializa 50 productos que están codificados de 1 a 50. Por cada producto se registra el nombre y precio.

Se cuenta además con las ventas realizadas. Por cada una se ingresa código de producto y cantidad de unidades, finalizando el ingreso con código de producto igual a cero.

Se pide realizar un programa que usando funciones óptimas permita lo siguiente:

- 1) Para cada producto mostrar su código, la cantidad y el importe total vendido.
- 2) Mostrar la información de los productos cuya cantidad vendida sea superior a 1000.
- 3) Indicar el total recaudado (función recursiva).

Etapas 1: Análisis de enunciado

Datos de entrada:

Productos: código, nombre y precio unitario de 50 productos. El código va de 1 a 50

Ventas: código de producto y cantidad de unidades de varias ventas (cantidad no determinada)

Salida:

Petición 1: Código, Cantidad total, Importe total de los 50 productos

Petición 2: Código, Nombre y Cantidad total de 50 productos o menos

Petición 3: Total recaudado

Listado de actividades a realizar

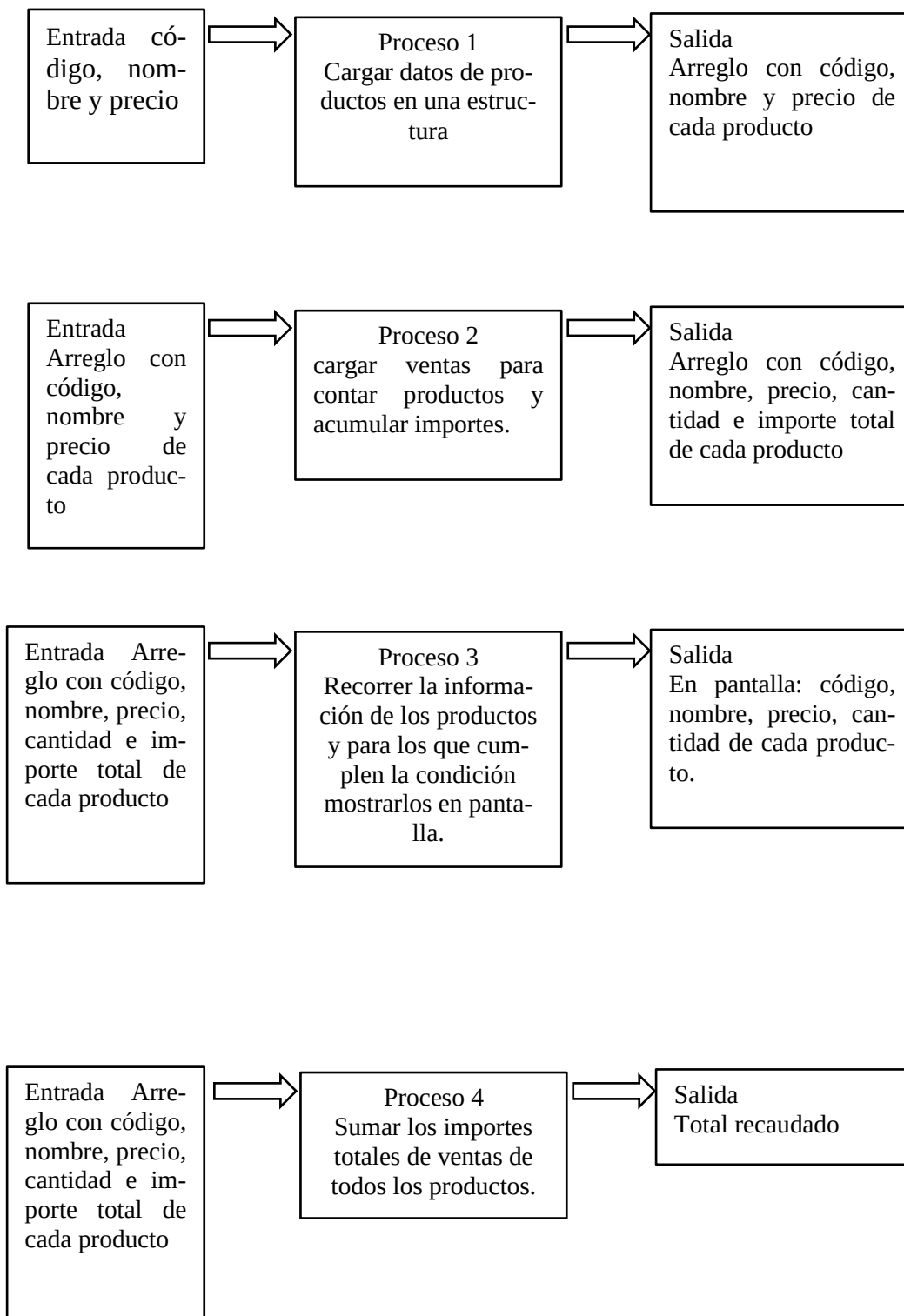
Petición 1: Cargar datos de productos en una estructura, cargar ventas para contar productos y acumular importes.

Petición 2: Recorrer la información de los productos y para los que cumplen la condición mostrarlos en pantalla.

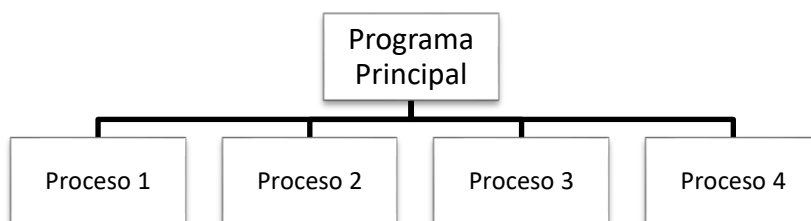
Petición 3: Sumar los importes totales de ventas de todos los productos.

Etapa 2: Prediseño

Representar Los procesos como un diagrama. Esto para cada actividad es necesario tener en claro que recibe y que devuelve (lo que elabora o produce como salida).

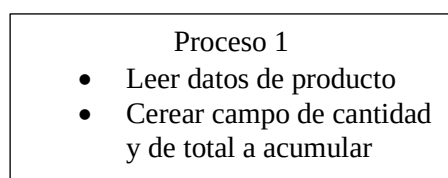


- Estructurar el sistema como jerarquías de procesos. Este paso implica identificar la secuencia en que se ejecutarán los subprogramas.



Etapla 3: Diseño

En esta etapa hay que ver cuanto mas se va a segmentar (divide y vencerás)
 Delo realizado hasta ahora se podría segmentar un poco mas el Proceso 1



Acá vemos como se aumenta el detalle y se evidencian dos actividades a realizar.

Del mismo modo se debería inferir y analizar los demás procesos

Etapla 4: Codificación y Depuración

Se realiza el programa principal sin mucho detalle y luego se lo irá completando:
 // PP

```

struct producto
{
char nombre [20],
float precio, importe;
int código, cantidad;
}
Main()
{ producto Prod[50];
  Inicia(Prod); // Proceso 1
  CargaVentas(Prod); // Proceso 2
  MostrarMayor1000(Prod); // Proceso 3
  printf("\n Total recaudado es %f", TotalRec(Prod)); // Proceso 4
}
  
```

En el programa principal se puede ver el flujo de control que señala el orden en que se ejecutan las funciones que resuelven el problema.

Se debe realizar la codificación de las funciones, tener en cuenta los criterios de cohesión para asegurarnos que estemos programando una buena opción.

Etapla 5: Documentación

Se puede observar en el código del programa principal lo propuesto en esta etapa, el texto de los comentarios está vinculado en este caso al análisis realizado, pero debería ser mas detallado y específico.